CS 225

Data Structures and Algorithms

Spring 2025

Anirudh Konidala

Table of Contents

1 Introduction to CS 225 4
2 C++ Review 6
3 List ADT and Linked List Implementation 9
4 ArrayList: An Array Implementation of a List 22
5 Amortized Analysis 27
6 Stacks and Queues 32
7 Iterators 38
8 Trees 43
9 Tree Traversals 48
10 Tree Searching and Dictionaries 55
11 BST and BST Implementation 59
12 BST Balance 72
13 BST Rotations 76
14 Adelson-Velsky Landis (AVL) Trees 79
15 Lambdas 93
16 BBST Range Search and KD Trees 96
17 BTrees 104
18 BTree Analysis 111
19 Priority Queues 114
20 Heaps 124
21 Disjoint Sets 130
22 Graphs 144
23 Graph Implementations 148
24 Graph Traversals 158
25 MST: Minimum Spanning Tree (Kruskal's Algorithm) 171
26 MST: Minimum Spanning Tree (Prim's Algorithm) 182
27 Dijkstra's Algorithm 192
28 Floyd Warshall's Algorithm 203
29 Probability in Computer Science 214
30 Hashing, Hash Collisions, and Hash Tables 215

31 Bloom Filters 238
32 Cardinality 253
33 MinHash Sketch Strategy 262
34 Wrapping up CS 225! 271
Appendix A: Midterm Reviews 278
A.1 Midterm 0: Review of Prerequisite Material 278
A.2 Midterm 1: Lists 305

1 Introduction to CS 225

> **CS 225** - Covers the fundamental **data structures** (list, stack, queue, tree, etc.) and their implementations using C++ and object oriented programming along with an **introductory analysis to algorithms**

> **Prereqs** - Must have credit in both **CS 173/MATH 213** (Discrete Math) and **CS 128** (Introduction to Computer Science II in C++)

> **ECE Students** - Electrical/Computer Engineering (ECE) students may substitute CS 128 credit for **ECE 220** (Intro to Computing Systems), as both earn the same credit

As stated in the prereqs, **a strong knowledge in C++ is required** to be successful in this course! In other words, please ensure you have a **solid grasp** in CS 128/ECE 220 course material. Please see **Course Info and Resources** below for more info

1.1 Background

This course is crucial as it provides the foundation for writing efficient software, a skill highly sought after companies like **Google**, **Amazon**, and **Meta**. Mastery in this course will open pathways into working at such companies as they and many other employers rigorously evaluate your skill in writing software before prior to hiring you as a software developer.

1.2 Grading

- **36%** - Programming Assignments (MPs) (each MP is worth 6%)
- **12%** - Laboratory Assignments (each Lab is worth 10%)
- **36%** - Exams/Midterms (each Midterm is worth 6%)
- **16%** - Final Exam

1.3 Course Info and Resources

- CS 225 Lectures
- Full Syllabus is available here
- Review C++ Here
- C++ Reference Links
  - Online C++ Reference
  - Tutorial Based Approach of C++

- Textbook C++ Reference

2 C++ Review

Contacting the CS 225 course - cs225admin@lists.cs.illinois.edu

2.1 Encapsulation

Encapsulation in C++ is primarily distinguished via two things: **header** and **code**. In C/C++, code is separated into two files, one which we call the **header** and one which we call the **code segment**

- **header files** - Describes how the code is used, or how people should use this specific data structure
  - has a `.h` or `.hpp` extension. Mostly `.h`, but `.hpp` is used for **templates**, which help write **generic** code that supports any data type
- **code files** - Describes how the code works, or how the data structure is implemented!
  - has a `.cpp` file extension

Object Oriented languages also provide with two main things, a **constructor**, which initializes the necessary data for us in a object and a **destructor** which gets rid of any necessary data for us in an object

2.2 Memory Management - Ownership

Someone or some code owns everything, which simply means that **one part of the program is responsible** for

- Creating an object in memory (**Constructor**)
- Keeping track of it
- Deleting (freeing) it when it's no longer needed (**Destructor**)

This essentially means that the programmer is responsible for managing memory, which can both be powerful and dangerous. Forgetting to free allocated memory can cause **memory leaks**, or program crashes. In contrast, **freeing** memory too early can cause **dangling pointers** and **undefined behavior (segfaults)**. You can see both mistakes in **Figure 2.1**, where `h1` is leaked on the left and `p` is left dangling on the right.

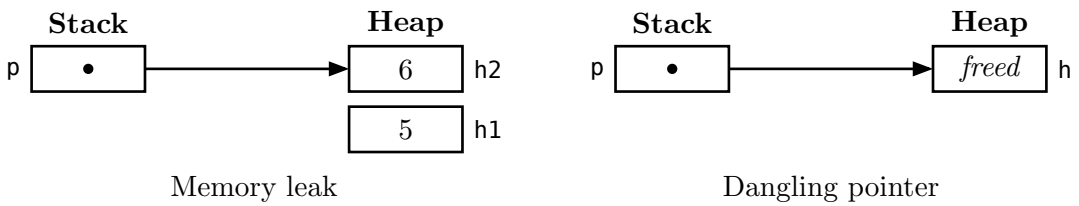


Figure 2.1: Example: after `p = new int(6);` the old block `h1` is unreachable (left); after `delete p;` the pointer still holds the freed address (right)

Ownership is tracked carefully using `new` and `delete`, or use smart pointers like `std::unique_ptr`, `std::shared_ptr`, `std::weak_ptr`

2.3 Rule of Zero/Three/Five

When your class manages resources (**dynamic memory**, file handles, etc.), you need to define how it is **copied**, **moved**, and **destroyed** in order to avoid issues like memory leaks or double deletes

The rule of zero/three/five are rules widely adopted in C++ related to memory/resource management and ownership, specifically about when and how you should define or avoid defining special member functions in a class

Rule of Zero - If your class doesn't manage resources directly, do not define the **big five**

- Destructor
- Copy Constructor
- Copy Assignment
- Move Constructor
- Move Assignment

Rule of Three - If you define one of a **destructor**, **copy constructor**, or **copy assignment operator**, **you should define ALL three of them!**

- This is because handling one implies your class is managing a resource that needs careful copying and cleanup
- Notice in **Figure 2.2** what goes wrong when only the destructor is defined: `a` and `b` point to the same block, so it gets deleted twice

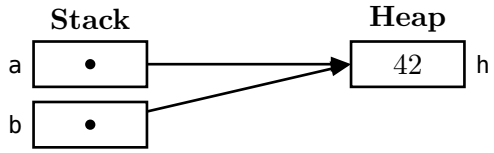


Figure 2.2: Example: with a destructor but the implicit copy constructor, `Foo b = a;` copies only the `data` pointer, so both destructors delete the same block

Rule of Five - Applies to modern C++, where if you plan on managing resources with move semantics, define **all 5** (this is the opposite of the **rule of zero**)

- Destructor
- Copy Constructor
- Copy Assignment
- Move Constructor
- Move Assignment

Use this rule when you need to **avoid deep copying** and allow your class to be efficiently moved, like the move in **Figure 2.3**, where `b` takes over the heap block of `a` and nothing gets copied.

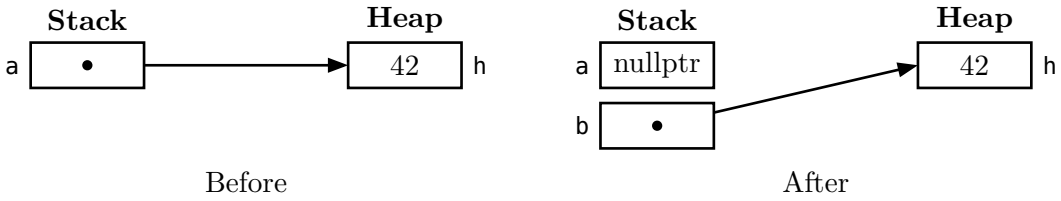


Figure 2.3: Example: `Foo b = std::move(a);` hands the heap block of `a` to `b` without copying it and leaves `a.data` as `nullptr`

3 List ADT and Linked List Implementation

ADT - Abstract Data Type
- ADTs are conceptual models that define a set of operations and behaviors for a data structure

List ADT Goals
- `insert` - Done via front/back, at index, or at pointer
- `delete` - Done via front/back, at index, or at pointer
- `getData` - Done via front/back, at index or at pointer
- `isEmpty`
- `size`
- **Store any necessary constructor, destructors, copy constructors, etc. so that it behaves like any basic variable**

The goal is to have every data structure behave like a fancy interface, but at a fundamental level act exactly the same as a basic data type! They should also behave like **templates** so that they can store anything (integers, pictures, imaginary numbers, etc.). We just want one generic data structure!

This is done via generics and templates!

There are two main implementations of a list
1. vector or array (store a list by allocating a chunk of data!)
2. linked list (allocate every element in a list separately and link them together via pointers or references)

Data structures can be best shown as diagrams, like the linked list in **Figure 3.1**

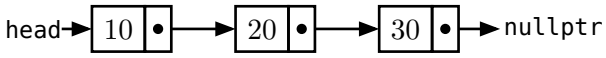


Figure 3.1: A linked list holding 10, 20, 30

Looking at **Figure 3.1**, the **head** is the **first node** in the linked list, and is in fact a **pointer** that points to the first node in the linked list.

Similarly, the **tail** is the **last node** in the linked list and it points to **nothing** (null pointer)!

This means that our List ADT must contain
- `head` - a pointer to a `ListNode`

Intuitively, this also shows that the linked list that contains several nodes, which begin with the value `10`, where then points to the next node of the linked list with value `20`, and so on

- From a C++ standpoint, this means that our List ADT must chain nodes together via **pointers**
  - Each node contains its own data along with a **pointer** that points to the next node in the List!

This is shown in the code below, which represents a generic `ListNode`, `ListNode<T>`. In this `ListNode` representation, each `ListNode` holds a generic type of data notated as `T`

Additionally, there is a pointer to the next `ListNode` in the List along with a copy constructor that can be used to copy data into the List!

```
#pragma once

template <typename T>
struct ListNode {
    // the current data in the list
    T data;

    // a pointer to the next node in the linked list
    ListNode* next;

    // a copy constructor for the list
    ListNode(const T& data) : data(data), next(nullptr) { }
};
```

3.1 `insertAtFront`

The `.hpp` extension is used to define functionality of templates defined in a `.h`. It typically contains the full definitions of template functions, enabling the compiler to generate code for each type instantiation based on the defined type

One of the functions that a good List data structure must implement is `insertAtFront`, which as the name suggests inserts a node to the front of the List!

To understand this better, let's use a concrete example, where `3` will be inserted at the front of the List in **Figure 3.2**

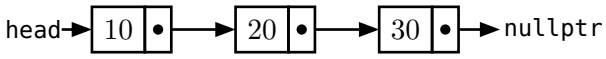


Figure 3.2: The list before inserting `3` at the front

1. In order to insert `3` at the front of the List, we need to create a new `ListNode` type! However, the data cannot get destroyed after the function ends because the List type needs to store this value for as long as it needs to
   1. Therefore, we must dynamically allocate this newly created `ListNode` via the heap, where we will receive a pointer to that data on the stack!
2. Once we dynamically allocate the new `ListNode`, we must reassign the `next` pointer to the head of the List because it will be inserted at the front!

1. In other words, the pointer to the next node is no longer null, like it was when newly created. Now it will point to the `head`!

3. Make `head` point to the newly created `ListNode`

After these steps, the List looks like **Figure 3.3**, where the new node (shaded) now sits in front of `10`

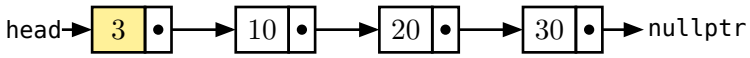


Figure 3.3: The list after `insertAtFront(3)`

In C++, this is what the `insertAtFront` algorithm would look like

```
// List.hpp

#include "List.h"

/** Inserts data at the front of a List
  * Runtime: O(1)
 */
template <typename T>
void List<T>::insertAtFront(const T& data) {
    // Dynamically allocate the data to the heap
    ListNode<T>* newNode = new ListNode<T>(data);

    // Reassign the next pointer to the head
    newNode->next = head;

    // Reassign the head pointer to the newly created node
    head = newNode;
}
```

This algorithm has a runtime of $O(1)$, due to three simple operations and doesn't loop or scale through a list size!

Recall what `->` (arrow operator) does in C++. It's a nice operator that dereferences the pointer and then calls an attribute/method on that pointer! `newNode->next` is the mere equivalent of `(*newNode).next`

3.2 `insert`

The previous algorithm only inserted nodes at the front of the List. However a more general insert algorithm is recommended to insert nodes in more places than just the front!

To solve this problem, the best thing to do is to create a `insert` algorithm that is able to insert data anywhere at a given index of the list. This is shown in our updated `LinkedList` ADT code below

```
// List.h

#pragma once

template <typename T>
struct ListNode {
    // the current data in the list
    T data;

    // a pointer to the next node in the linked list
    ListNode* next;

    // a copy constructor for the list
    ListNode(const T& data) : data(data), next(nullptr) { }
};

template <typename T>
class List {
private:
    // a pointer to the head (front) of the linked list
    ListNode<T>* head = nullptr;

public:
    /** Inserts data at the front of a List
    * Runtime: O(1)
    */
    void insertAtFront(const T& data);

    /** Inserts data in the given index of a list
    *  Runtime: O(n)
    */
    void insert(const T& data, int index);
};
```

When the index is 0, `insert` would behave exactly how `insertAtFront` works

To understand this much trickier algorithm, we can use the same concrete List example from `insertAtFront`, with example function call `insert(3, 1)`

To make this call take place, we need to find the node at the first index and insert the newly created node before that. To do this, our insert algorithm must do two following steps

1. Allocate the given data onto the heap
2. Find the place to insert the newly created node (typically the code semantics are before)
    1. Therefore, we need a helper `find` function to return the node by reference at the given index. Our updated `LinkedList` ADT is represented in the code below

```
// List.h
```

```
#pragma once

template <typename T>
struct ListNode {
    // the current data in the list
    T data;

    // a pointer to the next node in the linked list
    ListNode* next;

    // a copy constructor for the list
    ListNode(const T& data) : data(data), next(nullptr) { }
};

template <typename T>
class List {
private:
    // a pointer to the head of the linked list
    ListNode<T>* head = nullptr;

    /** Helper for the recursive version of `find`
    *  Runtime: O(n)
    */
    ListNode<T>*& find(int index, ListNode<T>*& head);

public:
    /** Inserts data at the front of a List
    * Runtime: O(1)
    */
    void insertAtFront(const T& data);

    /** Inserts data in the given index of a list
    *  Runtime: O(n)
    */
    void insert(const T& data, int index);

    /** Finds the `ListNode` at the given index!
    *  Runtime: O(n)
    */
    ListNode<T>*& find(int index);
};
```

3.2.1 `find`

1. To implement `find`, it should return a `ListNode*&`, which is **a `ListNode` pointer by reference**
    1. Recall that with return type semantics in C++, returning just a `ListNode*` won't actually allow us to change the original `ListNode*`. Doing so will create a copy of the `ListNode*` and instead modify the **copy** instead of the original pointer. Therefore, we need to return a **reference** to the original pointer, which is a type `ListNode*&`

2. Recall that `insertAtFront` changed our `head` member variable. Therefore, `head` needs to be the pointer pointing at the given index needed to be inserted
3. Implement this **iteratively**. Implementing **recursively** will cause a **Stack Overflow Error**, where no more stack frames can be placed on the call stack, resulting in **stack memory overflow**
    1. Regardless, thinking about the recursive solution can still be a good thing to do
        1. When `index == 0`, we know that this is the `head` of the List, or simply the first node, otherwise we return a recursive function call, which subtracts the index by 1, and then passes in the next node of the List. This recursive algorithm is shown in the first code snippet below
    2. To implement find iteratively, we can follow a fixed algorithmic step
        1. Return the `head` ListNode if the index is `0`. We can do this because we know the `head` is always at index `0`. We also return `head` if the List is empty, since there's nothing to walk through
            1. Recall that this method returns a `ListNode*&`, which is a **pointer reference**. `head` is in fact a **reference to a pointer**!
        2. Create a temporary `ListNode`, `curr` that is assigned to head. This represents the current node being referenced in the `LinkedList`
        3. Otherwise, loop while `x` is less than `index - 1`, and reassign `curr` to the next node in the `LinkedList`
            1. We also stop early if `curr->next` is null, so `curr` never walks off the end of the List and becomes null. An index past the end just lands at the end of the List
        4. Then return `curr->next` because we know that once `x` becomes `index - 1`, we can stop and just return the a reference to the next node in the `LinkedList`, hence `curr->next`
            1. Recall that `curr` is a **copy** of head! We don't want to return a copy `ListNode*`, but rather a reference `ListNode*`, which is simply a type `ListNode*&`
            2. Therefore, we return `curr->next` and loop from `x` to `index - 1`, instead of looping from `x` to `index` and returning just `curr`
        5. This process is shown in the second code snippet below. You can trace it for `find(2)` on the List from **Figure 3.2** in **Figure 3.4**, where `curr` stops at the shaded node and returns its `next`

The code below shows implementing `find` **recursively**

```
// List.hpp

#include "List.h"

// RECURSIVE IMPLEMENTATION OF FIND!

template <typename T>
ListNode<T>*& List<T>::find(int index) {
    return find(index, head);
}

template <typename T>
```

```
ListNode<T>*& List<T>::find(int index, ListNode<T>*& head) {
    // we know that the head is always the starting point at index 0 of the list,
so we can just simply return the head
    if(index == 0) { return head; }

    // Return a recursive method call passing in the next node and decrement the
given index!
    return find(index - 1, head->next);
}
```

The code below shows implementing `find` **iteratively**

```
// List.hpp

#include "List.h"

// ITERATIVE IMPLEMENTATION OF FIND!

/** Finds the ListNode at the given index!
 *  Runtime: O(n)
 */
template <typename T>
ListNode<T>*& List<T>::find(int index) {
    // Return the head if the index is 0
    // We know that the first node in the list is always the head!
    if(index == 0 || head == nullptr) {
        return head;
    }

    // Create a copy pointer of the head to traverse through the list!
    ListNode<T>* curr = head;

    // Traverse through the list to find the node right before the given index
    // Stop early at the last node so `curr` never becomes null
    int x = 0;
    while(x < index - 1 && curr->next != nullptr) {
        curr = curr->next;
        x += 1;
    }

    // Return the next node in the list, since that matches the node at the given
index
    return curr->next;
}
```

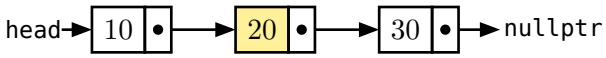


Figure 3.4: `find(2)` stops with `curr` at the shaded node (index 1, holding `20`) and returns its `next` pointer by reference, which points to the node at index 2 (holding `30`)

We can also update our current implementation of the `LinkedList` ADT, shown below

```
// List.h

#pragma once

template <typename T>
struct ListNode {
    // the current data in the list
    T data;

    // a pointer to the next node in the linked list
    ListNode* next;

    // a copy constructor for the list
    ListNode(const T& data) : data(data), next(nullptr) { }
};

template<typename T>
class List {
private:
    // a pointer to the head of the linked list
    ListNode<T>* head = nullptr;

public:
    /** Inserts data at the front of a List
    * Runtime: O(1)
    */
    void insertAtFront(const T& data);

    /** Inserts data in the given index of a list
    *  Runtime: O(n)
    */
    void insert(const T& data, int index);

    /** Finds the `ListNode` at the given index!
    *  Runtime: O(n)
    */
    ListNode<T>*& find(int index);
};
```

Runtime for our `find` function is $O(n)$ because we have to traverse up to n nodes in the `LinkedList` to find the node at the given `index`

Now that the `find` function has been coded, we can continue with finishing our `insert` function. Recall that this is a generic function, which will insert data into the `LinkedList` at a given index. Recall that this algorithm is done in three steps

1. Call `find` to find the `ListNode` at the given index
    1. Recall that `find` returns a reference of a pointer to a `ListNode`, written as a type `ListNode*&`
2. Allocate the given data onto the heap
    1. Returned pointer should not be a reference pointer! Make just a regular pointer
    2. Recall that a reference pointer refers to an existing pointer! There is no existing pointer here, we make a new one when allocating onto the heap
3. Make the newly created node point to the found `ListNode` at the given index
4. Set the node found at the given index's pointer to the newly created node

This algorithm is shown in the code below

```
// List.hpp

#include "List.h"

template <typename T>
void List<T>::insert(const T& data, int index) {
    // Call `find` to find the `ListNode` at the given index
    ListNode<T>*& nodeAtIndex = find(index);

    // Dynamically allocate the data to the heap
    ListNode<T>* newNode = new ListNode<T>(data);

    // Reassign the next node of the newly created node to be the `ListNode` found
at the given index
    newNode->next = nodeAtIndex;

    // Reassign the found `ListNode` at the given index's pointer to the newly
created node from the given data
    nodeAtIndex = newNode;
}
```

Let's go back to the List from **Figure 3.2** and call `insert(3, 1)`. As you can see in **Figure 3.5**, the shaded node holding `20` is the node at index 1 before the insert. Afterwards, the new node holding `3` (shaded) takes over index 1 and points to `20`, and the `next` pointer returned by `find` (the one inside `10`) now points to the new node

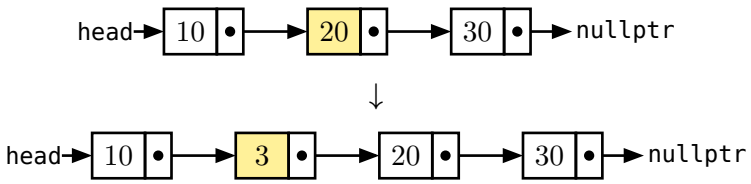


Figure 3.5: Inserting `3` at index 1 of the List from **Figure 3.2**

`insert` has a $O(n)$ runtime, since it needs to traverse to the given position/index and then insert the newly created node of the given data

3.3 `operator[]`

The next function our `LinkedList` ADT must have is the operator `[ ]`. Recall that in C++ we can operator overload with a function that changes the behavior of the `[ ]` operator.

The brackets operator will behave very similarly to the `find` function, but rather return the `data` by reference that the `ListNode` at the given index contains. This updates our `LinkedList` class like this (shown below)

```
// List.h

#pragma once

template <typename T>
struct ListNode {
    // the current data in the list
    T data;

    // a pointer to the next node in the linked list
    ListNode* next;

    // a copy constructor for the list
    ListNode(const T& data) : data(data), next(nullptr) { }
};

template <typename T>
class List {
private:
    // a pointer to the head (front) of the linked list
    ListNode<T>* head = nullptr;

public:
    /** Inserts data at the front of a List
    * Runtime: O(1)
    */
    void insertAtFront(const T& data);

    /** Inserts data in the given index of a list
    *  Runtime: O(n)
    */
    void insert(const T& data, int index);

    /** Finds the `ListNode` at the given index!
    *  Runtime: O(n)
    */
    ListNode<T>*& find(int index);
```

```
    /** Returns the data by reference of the `ListNode` at the given index
    *   Runtime: O(n)
    */
    T& operator[](int index);
};
```

This algorithm, therefore, works very simple

1. Find the `ListNode` at the given index (we already did this with the `find` function!)
2. Return that `ListNode`'s data (Every `ListNode` instance holds its own data!)

And the code for this function is below

```
// List.hpp

#include "List.h"

/** Returns the data by reference of the `ListNode` at the given index
 *  Runtime: O(n)
 */
template <typename T>
T& List<T>::operator[](int index) {
    // Call `find` to find the `ListNode` at the given index
    ListNode<T>* loc = find(index);

    // Return the data that the `ListNode` at the given index holds
    return loc->data;
}
```

The runtime for `operator[]` is also $O(n)$ because it takes n times to traverse through the list to find the `ListNode` at the given index to then return its data!

3.3.1 `remove`

The next function is to remove data from our `LinkedList` at a given index. This updates our `LinkedList` class structure, as shown below!

```
// List.h

#pragma once

template <typename T>
struct ListNode {
    T data; // the current data in the linked list
    ListNode<T>* next; // a pointer to the next node in the linked list
    ListNode(const T& data) : data(data), next(nullptr) { }
};
```

```
template<typename T>
class List {
private:
    ListNode<T>* head = nullptr; // a pointer to the head of the linked list

public:
    void insertAtFront(const T& data);
    void insert(const T& data, int index);
    ListNode<T>*& find(int index);
    T& operator[](int index);
    T remove(int index);
};
```

The algorithm for this is once again simple

1. Find the `ListNode` at the given index, with our helper `find` function, let's call it `toRemove`
2. Create a `temp` `ListNode` containing the `ListNode` to be removed, let's call it `temp`
3. Create a variable containing `temp`'s `data`, can call this `val`
4. Set toRemove to the next `ListNode` in the `LinkedList`
5. delete the `temp` `ListNode` from memory (`temp` was what was removed)
6. return `val`

The code below shows this algorithm in C++

```
// List.hpp

#include "List.h"

/** Removes a `ListNode` by index and returns its associated data!
 *  Runtime: O(n)
 */
template <typename T>
T List<T>::remove(int index) {
    // Find the `ListNode` to be removed
    ListNode<T>*& toRemove = find(index);

    // Create a temporary `ListNode` containing what should be removed
    ListNode<T>* temp = toRemove;

    // Create a `val` variable to store temp's data
    T val = temp->data;

    // Reassign `toRemove` to the next node in the `ListNode`
    toRemove = toRemove->next;

    // Delete `temp` from memory since we no longer need it
    delete temp;
```

```
    return val;
}
```

If we call `remove(1)` on that same List from **Figure 3.2**, we get **Figure 3.6**. Notice how the node holding `10` now points past the shaded node holding `20` to `30`, the node holding `20` is deleted, and `20` is returned

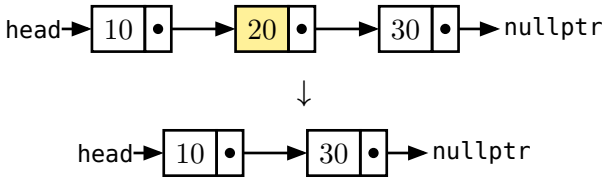


Figure 3.6: `remove(1)` removes the node holding `20`

A couple things about this algorithm

- You need a pointer to the node you're removing so you can later delete it, which is what we call `toRemove`
- Creates a copy pointer to the node being removed called `temp`. We keep a separate reference so that we can return its data in the function
  - Without `temp`, once `toRemove = toRemove->next`, the previous node that `toRemove` stored is now lost
  - We need the original node that was stored in `toRemove` in order to be able to return the data that was removed from the `LinkedList`, which is why we store it in `temp`!
- `delete temp`, ensures the memory allocated to `temp` is freed. We do this call because we know that the `LinkedList` no longer needs this `temp ListNode` (**`LinkedList` no longer needs to track it!**)

This wraps up the first implementation of a List, often referred to as a `LinkedList`

4 ArrayList: An Array Implementation of a List

There is another way to implement a `List`, which is by using an Array. This is simply implementing a C++ `std::vector`, or in broader programming terms, an `ArrayList`

The primitive foundation of an `ArrayList` is based on the idea of storing elements in a chunk of contiguous memory, allowing efficient access and manipulation. The list starts with a raw array and builds abstraction over it to support dynamic resizing, safe memory handling, and user-friendly operations, like the ones covered in `LinkedList`

We begin by figuring out what we need to store the list. In the `LinkedList`, we stored individual `ListNodes`, where they each held data along with a pointer to the next node, which allowed for easy chaining access between the nodes

However, in an `ArrayList` approach, we need to simply store a chunk of memory somehow, while this is typically a `std::vector` in C++, we are actually implementing this, so we need a primitive array, as mentioned previously. Therefore, a chunk of memory is allocated using a **pointer**, where we can call this a generic type `T`. This is reflected in our `ArrayList` class below

```
// ArrayList.h

#pragma once

template<typename T>
class ArrayList {
public:

private:
    // A pointer to the start of a chunk of memory
    T* data = nullptr;
};
```

`data` is a pointer to a chunk of memory that will be allocated to the heap

In addition to this, we need to store the **size** and **capacity**, where the **size** represents how many items are actually in the list and the **capacity** represents how much space is available in the memory block

These both could be `ints`, or they could be pointers, where `size` represents a pointer to the end of the used portion of the array and `capacity` represents a pointer to the end of the allocated chunk of memory, or buffer. This approach is reflected in our code below,

```
// ArrayList.h

#pragma once

template<typename T>
class ArrayList {
public:

private:
    // a pointer to the start of the buffer
    T* data = nullptr;

    // a pointer one past the last element in use
    T* size = nullptr;

    // a pointer one past the end of the allocated buffer
    T* capacity = nullptr;
};
```

For example, the list in **Figure 4.1** holds 3, 4, 5, so it has room for three more elements before `size` catches up to `capacity`.

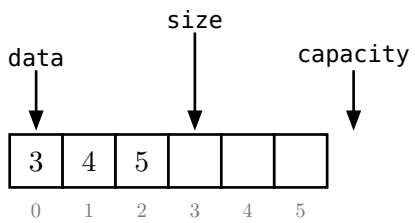


Figure 4.1: Example: an `ArrayList` holding 3, 4, 5 in a buffer of 6 slots. `size` points one past the last element and `capacity` points one past the end of the buffer

Now, the next idea to think about this `ArrayList` implementation is how do we add more space to the buffer? If the capacity is full, meaning that size and capacity both point to the same area in the buffer, then how would I resize the buffer to add additional data into the buffer?

The solution is to allocate some new buffer somewhere with a bigger space. Then, copy the data from the previous buffer along with its pointers (`size`, `capacity`, and `data`). This approach will be done with a private method, `addSpace()`. Our `ArrayList` ADT is updated below. Recall that this is a helper method that resizes the buffer, so therefore, it should be private.

```
// ArrayList.h

#pragma once

template<typename T>
```

```
class ArrayList {
public:

private:
    T* data = nullptr;
    T* size = nullptr;
    T* capacity = nullptr;

    /** Copies the elements into a bigger buffer when it is full
     * Runtime: O(n)
     */
    void addSpace();
};
```

Notice that this is very expensive, which is why the `LinkedList` approach is often much more recommended

Our helper `addSpace()` function must add data to the end of the allocated chunk of memory. However, this causes issues when the allocated chunk of memory is full with data. Recall that this occurs when `size == capacity`.

- To solve this problem, we simply need to allocate a new buffer with a bigger capacity and then copy over the previous buffer's data along with the respective pointers to the buffer including `data`, `size`, and `capacity`
  - You can see this copy in **Figure 4.2**, where the four elements of the full buffer (highlighted) move into a buffer twice as big

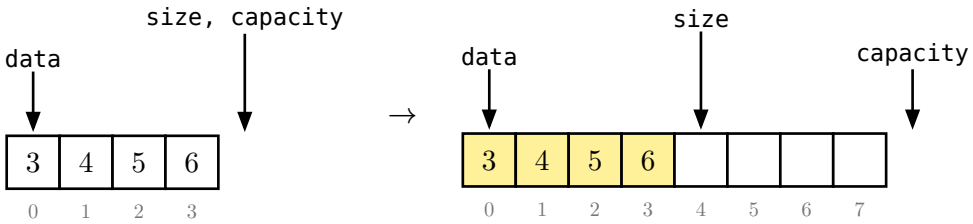


Figure 4.2: Example: a full buffer (`size == capacity`) is copied into a bigger buffer. Highlighted cells are the copied elements

We will come back to this `addSpace()` helper function, but for now we will talk about the main public function that will use `addSpace()` called `pushBack()`, which is one very common function in a `std::vector`! Recall, that it simply inserts an element at the end of the list

We can update our `ArrayList` ADT like this,

```
// ArrayList.h

#pragma once
```

```
template<typename T>
class ArrayList {
public:
    /** Inserts data to the back of this `ArrayList`
     * Runtime: amortized O(1)
     */
    void pushBack(const T& value);
private:
    T* data = nullptr;
    T* size = nullptr;
    T* capacity = nullptr;

    /** Copies the elements into a bigger buffer when it is full
     * Runtime: O(n)
     */
    void addSpace();
};
```

4.1 `pushBack()`

The algorithm for `pushBack()` becomes simple with our helper `addSpace()` function

1. Check if `size == capacity`. If so, then call helper function `addSpace()` to add more space into the function
2. Then, dereference the `size` pointer and set it equal to the new `value` parameter variable
   - we do this because this is how we place the given element at the end of the current end of the buffer
   - Remember that `size` tracks the end of filled data in the buffer and `capacity` tracks the end of the entire buffer!
3. Add 1 to the `size` pointer to set the pointer to the new current end of the buffer

The code for this algorithm is shown below

```
// ArrayList.hpp

#include "ArrayList.h"

/** Inserts data to the back of this `ArrayList`
 *  Runtime: O(1)
 */
template <typename T>
void ArrayList<T>::pushBack(const T& value) {
    // Add additional space to the buffer if `size` == `capacity`
    if(size == capacity) {
        addSpace();
    }

    // Place the new element at the current end
```

```
    *size = value;

    // Increment the `size` pointer to now point to the new end of the filled
buffer
    ++size;
}
```

Let's trace a single call in **Figure 4.3**. There's still room in the buffer, so `pushBack(6)` skips `addSpace()` and just drops 6 into the highlighted slot.

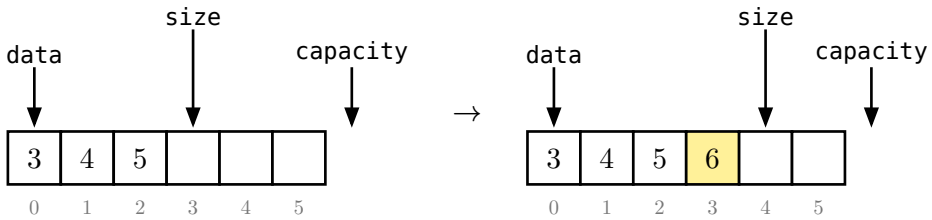


Figure 4.3: Example: `pushBack(6)` writes 6 at `*size`, then moves `size` forward one slot

5 Amortized Analysis

Amortized Analysis is a technique commonly used in Computer Science to analyze the **average running time** per operation over a sequence of operations in an algorithm. Rather than focusing on the worst case cost for a single operation (this is simply **Big-O**), amortized analysis evaluates the overall cost when spread out evenly across all operations performed.

This is useful because even though some operations in an algorithm can cost a lot more than others, the average cost per operation is low when many operations are performed consecutively.

5.1 Resizing by adding 2 elements every time

One way to implement our `addSpace()` function is to resize the buffer by increasing the `capacity` by 2 each time the array is full. This means that when more space is needed, `addSpace()` allocates a new array with a capacity of `capacity + 2`. Recall that after this, `addSpace()` then needs to copy all the data from the previous buffer to the new buffer and then update the buffer's internal pointers.

Since `addSpace()` needs to copy all the data from the previous buffer to the new buffer when updating its capacity by `capacity + 2`, this means that the algorithm has a runtime of $O(n)$, where n represents the number of elements currently in the array. This is because each element must be copied one by one into the new buffer, as you can see from the highlighted cells in **Figure 5.1**, where each resize copies 2 more elements than the last.

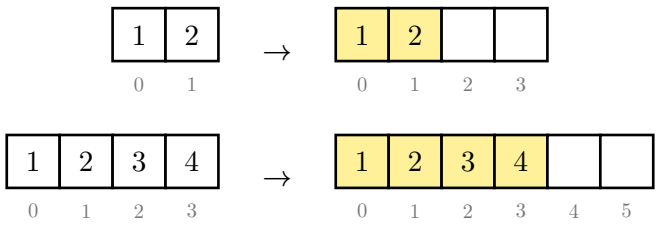


Figure 5.1: Example: pushing 1, 2, 3, 4, … with `capacity + 2` resizing. Highlighted cells are copied: 2 copies, then 4, then 6, and so on

Let r represent the number of resizes, where a resize occurs when the ArrayList is full (or simply every time two elements are added).

Then, the total runtime cost of this algorithm, where there are 2 copies made every 2 iterations, can be represented through the summation

$$\sum_{k=1}^{r} 2k$$

Since r represents the number of resize operations every two iterations, the summation is equivalent to $2 \times \frac{r(r+1)}{2} = r(r+1) = r^2 + r$. Therefore,

$$\sum_{k=1}^{r} 2k = r^2 + r$$

However, r is the number of resize operations every two iterations. When analyzing algorithmic runtime, it is important to express it in terms of n rather than r because an algorithm is expressed in terms of its input size, which is simply n.

If there were 5 resizes that occurred every 2 elements, then the total number of elements, n, would be 10, which can be mathematically expressed as $n = 2r$, or simply $r = \frac{n}{2}$

Substituting n for r becomes

$$= \left(\frac{n}{2}\right)^2 + \frac{n}{2}$$
$$= \frac{n^2 + 2n}{4}$$

Therefore, the total time for n inserts is $\frac{n^2+2n}{4}$.

The total amortized cost is then calculated by dividing by n, the total number of operations performed on an algorithm

$$\frac{\frac{n^2+2n}{4}}{n} = \frac{n^2 + 2n}{4n} = \frac{n(n+2)}{4n} = \frac{n+2}{4} = \frac{1}{4}n + \frac{1}{2}$$

making the total amortized cost $O(n)$ (since constants are dropped)

5.2 Resizing by doubling the elements every time

Now suppose, we resized the `ArrayList` by doubling the capacity every time. Each time we resize, the time it takes to do so doubles from 1, 2, 4, 8, 16, etc. This is because each resize involves copying all the elements from the old array into a new array that is twice as large.

Let r represent the number of resizes, where a resize occurs when the `ArrayList` is full (or simply every time two elements are added)

When the `ArrayList` is full, its capacity will resize by double the original capacity.

Recall that each time we resize, the time it takes to do so doubles from 1, 2, 4, 8, 16, etc. This is because each resize involves copying all the elements from the old array into a new array that is twice as large. Therefore, this results in 2^r copies per resize. Notice in **Figure 5.2** how the highlighted copies go 1, 2, 4 as the number of resizes grows

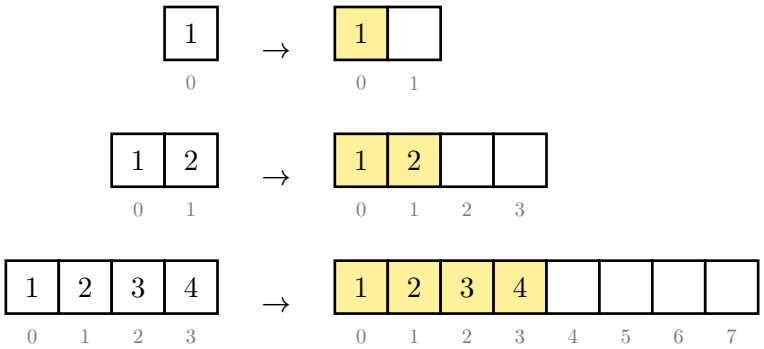


Figure 5.2: Example: pushing 1, 2, 3, 4, … with doubling. Highlighted cells are copied: 1, then 2, then 4, so each resize buys as many free slots as it copied

Therefore, the summation that represents the runtime of this algorithm is

$$\sum_{k=0}^{r} 2^k$$

This summation is equivalent to the sequence $1 + 2 + 4 + \cdots + 2^{r-1} = 2^r - 1$

However, in an amortized sense, we need to multiply this by the number of operations we are doing. We are doubling the capacity and inserting a new element into our buffer. This means we have to multiply this runtime cost by our insertions. The $+1$ at the end represents the initial operation and constant cost that isn't captured by the summation. Therefore,

$$\sum_{k=0}^{r} 2^k = 2(2^r - 1) + 1$$

where $2^r = n$ and $r = \log_2 n$. Therefore,

$$\sum_{k=0}^{r} 2^k = 2\big(2^{\log_2 n} - 1\big) + 1 = 2(n-1) + 1 = 2n - 1$$

where $2n - 1$ represents the total copies with n inserts, meaning that the amortized cost is represented by

$$\frac{2n-1}{n}$$

which simplifies to $2 - \frac{1}{n}$. When analyzing this behavior's function, the

$$\lim_{n \to \infty} \left(2 - \frac{1}{n}\right) = 2$$

which means that it approaches a constant value of 2. Therefore, its amortized cost is $O(1)$

Since the amortized cost when doubling the capacity each resize is $O(1)$, it makes more sense to implement the `addSpace()` function using this resizing method

Now to implement our `addSpace()` function into our data buffer, we need to do a series of steps following our resize approach

1. Reallocate the data buffer, typically done by doubling its capacity (the current capacity is `capacity - data`, since both are pointers)
2. Check if the original capacity is 0. If so, set the new capacity to 1. This is to ensure memory efficiency
3. Allocate a new data buffer equivalent to the new capacity (which was double the original capacity)
4. Calculate the number of elements currently in the `ArrayList` (recall that this can be done with our `size` and `data` pointers by doing `size - data`)
5. Copy all contents from the old data buffer into the new data buffer
6. Free all the contents from the old data buffer and update the pointer of the new data buffer
7. Update the `capacity` pointer to point to the end of the new buffer and update the size pointer to point to the correct position in the new buffer

The code for this algorithm that reflects these series of steps is shown below,

```
// ArrayList.hpp

#include "ArrayList.h"

/** Allocates additional space to the buffer in this `ArrayList`
 *  Runtime: O(n)
 */
template <typename T>
void ArrayList<T>::addSpace() {
    /* Reallocate the capacity of the buffer (similar to realloc in C)
     * Typically that of double the original capacity
     */
    int newCapacity = (capacity - data) * 2;

    // If the original capacity is 0, set it to 1 to be minimal and memory
efficient
    if(newCapacity == 0) {
        newCapacity = 1;
    }

    // Allocate a new data buffer that is set to the updated capacity
    T* newData = new T[newCapacity];

    // Calculate the number of elements currently in the `ArrayList`
    int currentSize = size - data;

    // Copy all contents from the old data buffer into the new data buffer
    for(int i = 0; i < currentSize; i++) {
        newData[i] = data[i];
    }

    // Free the contents from the old buffer from memory
    delete[] data;
```

```
    // Update the data pointer to point to the newly allocated buffer
    data = newData;

    // Point `capacity` to the end of the new buffer
    capacity = data + newCapacity;

    // Update the size pointer to point to the correct position in the new buffer
    size = data + currentSize;
}
```

This will effectively add more space to our buffer (double that the original) when its capacity is full

5.3 Amortized Costs of various operations on various `List` implementations

Operation	**Singly Linked List**	**Array**
Insert/Remove at front/back	`O(1)`	`O(1)`* (amortized)
Insert/Remove given a pointer	`O(1)`	`O(n)`
Insert/Remove at a given index	`O(n)`	`O(n)`
Find a value in a list	`O(n)`	`O(n)`
Find a value in a sorted list	`O(n)`	`O(log n)`

The * represents **amortized** cost, which means the average cost per operation over a sequence of operations. Whether to use the amortized case depends on the application you are building. Amortized analysis isn't ideal for explaining the cost of a single, individual operation, but rather useful for understanding the average cost spread across many operations.

6 Stacks and Queues

One of the biggest problems with both `ArrayList` and `LinkedList` is that accessing elements can sometimes be slow. Recall that the runtime complexity is `O(1)` to remove an element at the beginning and end and `O(n)` for elements not at the beginning/end.

Therefore, if we want `O(1)` insertion and removal, we need a faster data structure, which introduces Stacks and Queues.

This lecture will introduce Stacks, but will only fully cover the Queue implementation. The next lecture will cover the Stack implementation in depth along with introducing another new topic

6.1 Queues

Queues are everywhere and are used in an enormous amount of computer hardware. A Queue is like a list, but with a fixed order of **FIFO (First In First Out)**.

FIFO is a good mnemonic that helps remember how queues fundamentally work. **This means that the first element in the queue will be the first element out of the queue**. Some may also refer the FIFO mnemonic as LILO - Last In Last Out, but the idea behind remembering what a queue does is still fundamentally the same.

A queue is like a **line of people** (funnily enough a line in British English is called a queue!). Suppose people would line up to ride on the scariest rollercoaster in town. The first person who is in line will be the first person who will leave the line to go on the rollercoaster. Similarly, if someone joins the line for the rollercoaster they will be placed at the end of the line.

Queues have a simple implementation because they only really need two core functionalities, `enqueue()` and `dequeue()`.

- `enqueue()` will add elements to the end of the queue
- `dequeue()` will remove elements at the front of the queue

These functions are both `O(1)`, meaning adding and removing is constant time. This is because these functionalities happen at the back and front of the queue for `enqueue()` and `dequeue()` respectively, meaning we don't have to deal with the other elements in the queue. In **Figure 6.1**, for example, 5 joins at the back and 3 leaves from the front, and 4 never moves.

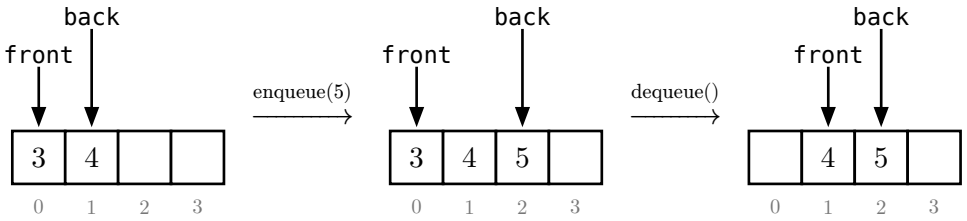


Figure 6.1: Example: `enqueue(5)` joins at the back, then `dequeue()` removes 3 from the front

6.2 Stacks

Like a Queue, a Stack also has a fixed order, but it works a little bit differently than a Queue. This order is referred to as **LIFO (Last In First Out)**.

LIFO is a good mnemonic that helps remember how stacks fundamentally work. **The last item put into the stack will be the first item out of the stack**. Some may also refer to the LIFO mnemonic as FILO - First In Last Out, but the idea behind remembering what a stack does/will do is still fundamentally the same.

A Stack is best described when thinking about a stack of plates in the kitchen. You add plates to the top, but remove plates from the top. You can't touch any other plate underneath until it appears at the top (well you could but the plates on top will topple over and break!).

Stacks also a simple implementation because they also only really need two core functionalities, `push()` and `pop()`.

- `push()` will add elements to the top of the stack
- `pop()` will remove elements from the top of the stack

If we store a stack in an array, both operations happen at `top`, as you can see in **Figure 6.2**: `push(5)` puts 5 on top and `pop()` takes that same 5 right back off.

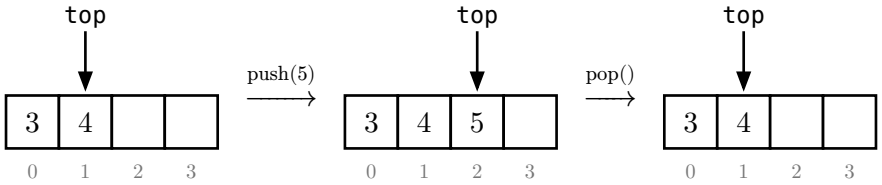


Figure 6.2: Example: a stack stored in an array. `push(5)` and `pop()` both work at `top`, so `pop()` returns 5, the last element pushed

These functions are also both `O(1)`, meaning adding and removing is constant time. This is because these functionalities both happen at the top of the stack, meaning we don't have to deal with the other elements in the queue.

6.3 Stacks vs. Queues

- Stacks operate upon the mnemonic LIFO - **Last In First Out** (last element in is the first element out), while Queues operate upon the mnemonic FIFO - **First In First Out** (first element in is the first element out)
- Insertion/Removal on a Stack happens at the same end (the top), while Insertion/Removal on a Queue happens at different ends, which are the back and front respectively.
  - Regardless of this, insertion/removal are `O(1)` because they happen at the ends of the Stack/Queue

6.4 `CircularQueue` Implementation - `enqueue` and `dequeue`

Take a look at the following `CircularQueue` implementation below

```
// CircularQueue.h

#pragma once

template <typename T>
class CircularQueue {
public:
    /** Adds an element to the rear of the queue
     * Runtime: amortized O(1)
     */
    void enqueue(T e);

    /** Removes and returns the element at the front of the queue
     * Runtime: O(1)
     */
    T dequeue();

    /** Returns `true` if the queue is empty; `false` otherwise
     * Runtime: O(1)
     */
    bool isEmpty();
private:
    // Pointer to the dynamically allocated array holding the queue elements
    T* items;

    // Total capacity of the queue (number of elements it can hold)
    unsigned int capacity;

    // Current number of elements in the queue
    unsigned int size;
};
```

This queue is an Array implementation, where data is stored in an array that `items` points to.

Before implementing this queue, we need to understand how the functionality of `enqueue()` and `dequeue()` fully work.

- Recall that `enqueue()` adds elements to the back of a queue. From a code standpoint, we insert elements by writing `items[size]`, inserting at the next most empty place in our `items` array
  - Recall that `capacity` represents the total length of the `items` array, while `size` represents the total number of elements the array has
  - Suppose we called `enqueue(3)`. This inserts 3 at the back of the array, where `items` now has a size of 1
    - This means that if we were to call `enqueue(4)`, this means that `items[1]` will in fact represent the proper position to place the next element in the queue, which in this case is 4.
  - After both calls, `items` looks like **Figure 6.3**, with 3 at index 0, 4 at index 1, and two empty slots left over

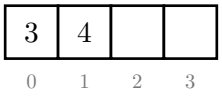


Figure 6.3: `items` after `enqueue(3)` and `enqueue(4)`, with a capacity of 4

- `dequeue()` removes elements from the front of a queue. From a code standpoint, we return the element at the front of a queue and then return it.
  - However, in order to properly implement this, we need to track where to remove elements from. This means adding a new variable, called `start`, where `items[start]` represents the element to remove from a queue. `size` would then decrement by 1 since an element was removed.
  - Since `items[start]` removes elements from a queue, `items[size]` will no longer work to insert additional elements from a queue.
    - This is because `items[size]` represents the last element in `items`, not the next index in the array to insert the new element.
    - To fix this issue, we can write `items[size + start]` to properly insert new elements into the queue. This is because start represents the index in `items` to remove an element, and size represents the number of elements `items` contains, so the next available spot in memory would be `start + size`
  - Remember that this doesn't free the data from memory, the data is still occupied in memory, it's just that the queue no longer considers the dequeued element to be part of the queue, which is done programmatically by simply updating `start`
  - Notice in **Figure 6.4** how `start` moves from 0 to 1 after a `dequeue()`, even though the 3 is still sitting in `items[0]`

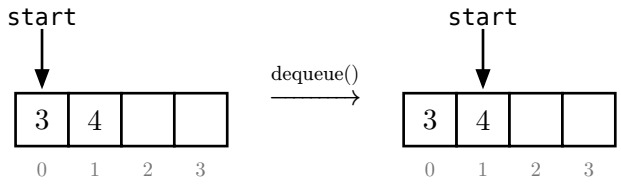


Figure 6.4: Example: `dequeue()` returns `items[0]` (3) and moves `start` to 1. The 3 is still in memory but is no longer part of the queue, and the next `enqueue` writes to `items[start + size]`, index 2

This is great, `enqueue()` and `dequeue()` work perfectly, until we come across another core issue. What if `items` is full? Since `items` is full and we were to insert elements using `items[start + size]`, our index would simply be out of bounds, since `start + size` exceeds the total size of `items`

- To fix this problem, we need a way to loop back from the beginning of `items`, and in programming this is referred to as the modulo operator `%`.
  - If we modulo `start + size` by capacity, we ensure that once the index reaches the end of the array, it wraps around back to the beginning, which effectively prevents an index out of bounds access into `items`
  - Therefore, `items[(size + start) % capacity]` will insert elements into `items` and `items[start % capacity]` will remove elements from `items`
    - This works because when we call `items[(size + start) % capacity]` and `items[start % capacity]`, `start` ensures that we overwrite memory that has already been removed by the queue.
  - In **Figure 6.5**, for example, `enqueue(7)` wraps around to index 0, right where the dequeued 3 used to be

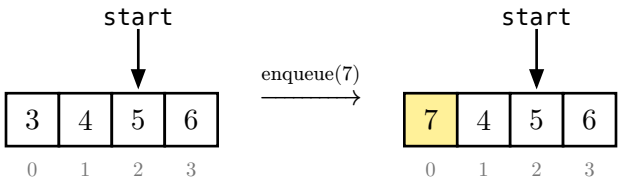


Figure 6.5: Example: with `start = 2`, `size = 2`, and `capacity = 4`, the queue is 5, 6. `enqueue(7)` writes to index `(2 + 2) % 4 = 0`, wrapping around over the already dequeued 3

So great, this queue implementation is amazing now, but there is still one more problem. What if the queue is actually full, where `size == capacity`? (recall that this typically happens when you enqueue a bunch of elements to the point where there is no more available allocated memory in `items` to place another element)

- To solve this problem, we use the same approach as the `ArrayList`.
  - Create a new container by doubling the `capacity` and then copy all elements from `items` into the new container
  - Update the `items` pointer to point to the newly made container.

- However, this implementation poses another problem because the `start` of the queue is no longer the `0`th element.
  - To solve this, when copying, we set the elements in the new container to correspond to the order of the queue by copying `items[start]`, then `items[start + 1]`, `items[start + 2]`, and so on, wrapping around as needed
  - You can see this copy in **Figure 6.6**: the wrapped queue 5, 6, 7, 8 starts at index 2, but after copying it starts back at index 0

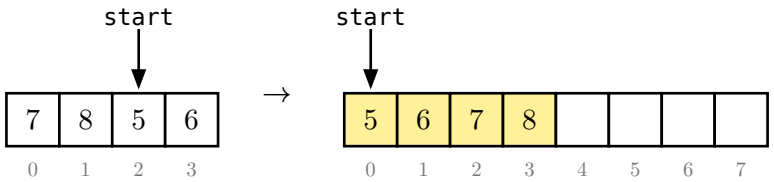


Figure 6.6: Example: a full queue 5, 6, 7, 8 that has wrapped around. Copying from `start` in queue order puts it back at index 0 of the doubled array

This effectively describes implementing a queue in depth via a `CircularQueue`. In a `CircularQueue`, we use a dynamically allocated array, like `items` that circles around when inserting an element goes outside the array's dynamically allocated memory!

Typically from a hardware standpoint, we don't resize a queue. In hardware, memory is already preallocated and doesn't change at runtime.

7 Iterators

One concept that is often in the scope of DSA is the **iterator**. An iterator is an object that traverses through all the elements/items in a data structure.

Suppose we had 3 data structures, `Array`, `LinkedList`, and a `Hypercube` (also known as a tesseract and is a 4D cube). How would we be able to traverse through every element in this data structure?

The solution is simply an iterator, which provides encapsulated access to our data in a data structure. Each data structure, `Array`, `LinkedList`, and `Hypercube` has its own way of iterating through its data

- For example, an `Array` typically uses an index to iterate, where `index++` traverses through each element in an array
- Similarly, a `LinkedList` uses a pointer to iterate, where `cur->next` represents a pointer to the next `ListNode` in a `LinkedList`. This is shown in the table below

7.1 Iterate through data in each data structure

Data Structure	Current Location	Current Data	Next
`LinkedList`	`ListNode*` (named `cur`)	`cur->data`	`cur = cur->next`
`Array`	`index`	`data[index]`	`++index`
`Hypercube`	`(x, y, z)` (a list or tuple of dimensions)	?? (cannot be determined)	?? (cannot be determined)

These intuitively make sense because we know how these data structures work. With an array, we can use its index to access an element, a `LinkedList` we can hold a pointer to the next node, etc.

But if we don't know how the data structure works, it will be very difficult to implement an iterator for it because we wouldn't know how to iterate through its elements.

7.2 Iterators with respect to Classes

Every class that implements an iterator must have the following **two pieces**.

1. Have two methods
   - `begin()` - Return iterator to the start item in a container (index 0)
   - `end()` - Returns iterator to one past the end item in a container
     - `end()` can come in handy a lot to check if an item exists in an iterator.
     - If I want to find string `milk` in a `std::vector<std::string>` named `groceries`, I can write `milk != groceries.end()` to see if `groceries` contains milk.

    - If `milk` is found, `end()` will return an iterator to what it found (in this case is `milk`) or will return an iterator to `end()` saying that there is no milk in the list.
    - This is a useful tool because we get an extra feature in our data structures to say that an element does/does not exist
2. Must have the base class `std::iterator`, which requires us to minimally implement
    - `operator*()` to get the data, where `*it` is the data at iterator `it`
    - `operator++()` to move/traverse through the data in a data structure
        - This standardized approach is what allows us to iterate via a data structure regardless of whether or not we iterate via a pointer (`LinkedList`) or index (`Array`)
    - `operator!=()`, which compares a constant reference to an iterator (`const Iterator& i`) to an element in the data structure and check whether they are equal or not

There are also some other nice features that can be implemented in an iterator, one of which is the arrow operator `->` which accesses members of a pointer.

- Recall that this is the shorthand for dereferencing a pointer and then calling its member, where `(*item).data` is equivalent to `item->data`.

A bidirectional iterator could also be implemented in a data structure, where it can traverse through elements **forwards** and **backwards**.

With these requirements, how would we implement `begin()` and `end()` with the example data structure mentioned earlier in this module?

7.3 Implementing `begin()` and `end()` for `Array` and `LinkedList`

Data Structure	`::begin`	`::end`
`LinkedList`	`Iterator(head_)`	`Iterator(nullptr)`
`Array`	`Iterator() { index = 0; }` or `Iterator(0)`	`Iterator(size)`

Let's explain the table above for an `Array` and `LinkedList`

- `LinkedList`
    - We would implement `begin()` by having our `Iterator` constructor take in the head of the `LinkedList`, which is `this->head_`, or simply `head_`
        - The reason why this intuitively makes sense is because the beginning of our `LinkedList` is a pointer to the first element of the `LinkedList`, which is simply `head_`
    - We would implement `end()` by returning our `Iterator` object, where it takes in a `nullptr`
        - This also intuitively makes sense because the end of a `LinkedList` occurs when a node no longer holds a pointer to its next node, when this happens its pointer is set to `nullptr` (see **Figure 7.2**)
- `Array`
    - We would implement begin by having our constructor set our `index` instance variable to `0`. This makes sense because the 0th index is always the first element in an array
        - You could also have the `Iterator` constructor take in a 0

- We would implement `end()` by returning `Iterator(size)`. This makes sense because size is the last element plus 1 in an array!
  - In **Figure 7.1**, for example, `end()` sits at index 3, one slot past the 5

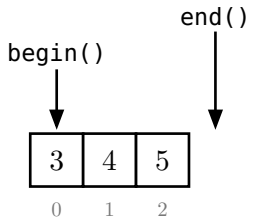


Figure 7.1: Example: an `Array` holding 3, 4, 5. `begin()` is index 0 and `end()` is index 3 (`size`), one past the last element, so it is never dereferenced

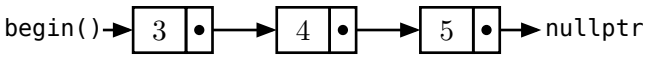


Figure 7.2: Example: a `LinkedList` holding 3, 4, 5. `begin()` wraps `head_` and `end()` wraps `nullptr`, which `++it` reaches after the last node

7.3.1 Iterator Usage Example with `std::vector`

Take a look at the following `Animal` class below

```
#include <iostream>
#include <string>
#include <vector>

struct Animal  {
    std::string name, food;
    bool big;
    Animal(std::string name = "blob", std::string food =  "you", bool big =
true) : name(name), food(food), big(big)  {  }
};

int main()  {
    Animal g("giraffe", "leaves", true), p("penguin", "fish", false), b("bear");
    std::vector<Animal> zoo;

    zoo.push_back(g);
    zoo.push_back(p);
    zoo.push_back(b);

    for(std::vector<Animal>::iterator it = zoo.begin();  it != zoo.end(); it++)  {
        std::cout << it->name << " " << it->food << std::endl;
    }
    return 0;
}
```

1. `Animal` stores 3 things, `name`, `food`, which describe the animal's name, food, and a boolean `big` that determines whether an `Animal` is big or not.
2. The `main` function creates 3 `Animal` objects and stores them in a `std::vector`. We then use an iterator to loop through the objects in the `std::vector`.

However, take a look at a snippet of the code above, specifically the part that utilizes an iterator

```
for(std::vector<Animal>::iterator it = zoo.begin();  it != zoo.end(); it++)  {
    std::cout << (*it).name << " " << (*it).food << std::endl;
}
```

one of the problems with this code is using the post-increment operator over the pre-increment operator. Luckily, `std::vector` implements this operator to work for its iterator, so this isn't an issue in our example code.

However, if we wanted to implement an iterator minimally, we would use the pre-increment operator (`++it`)

- If I were to do `it++`, then post increment would make a copy of the original iterator, move the original iterator forward, and then return the value of the original iterator that will appear there, which isn't what we want!

The rest of the code in our `for` loop prints the fields of the objects being traversed by the iterator.

- Notice that we access the members of `it` by dereferencing it and then calling a member via `(*it).name`. Recall that in C++, `(*it).name` is equivalent to `it->name`. Couldn't we use the arrow operator (`->`) instead as a shorthand of calling a member on a pointer?
  - To simply answer this question, `->` is typically implemented manually by several containers, but it is **not required**.
  - `std::vector` and many other containers do implement the `->`, so we could replace that snippet with `->`, as in the loop below, but the code snippet assumes that `->` is not implemented by the container

    ```
    for(std::vector<Animal>::iterator it = zoo.begin();  it != zoo.end(); ++it)  {
        std::cout << it->name << " " << it->food << std::endl;
    }
    ```

This code is now great, we utilize a `std::iterator` with a `std::vector`. We don't worry about how `std::vector` implemented the iterator.

We could add some other changes to this code. One is by using the `auto` keyword to define the iterator

```
auto it = zoo.begin();
```

`auto` is nice because the compiler infers the type of our iterator. This is good when we don't have to really remember what container we are working with. Whether to use `auto` is in fact debatable among the community.

Another way to write the code is by using a range based for loop to loop through constant `Animal` references, `Animal&`, in a `std::vector`

```
for(const Animal& animal : zoo) {
    std::cout << animal.name << " " << animal.food << std::endl;
}
```

In this for loop, I am introducing a constant `Animal` reference, `Animal&`, and this reference is going to take the value of every element in `zoo` one at a time using the iterator interface.

- The `animal` variable is the dereferenced iterator, so I would have to use `.` instead of `->`
- This approach completely abstracts/hides the fact I am even using an iterator. This code assumes that all the minimum iterator requirements are implemented.

And this brings us to the two approaches to use iterators

1. Using a for each loop, like the range-based `for` loop above

```
std::vector<Animal> zoo;

for(const Animal& animal : zoo) {
    std::cout << animal.name << " " << animal.food << std::endl;
}
```

2. Using a manually implemented iterator, like the loop in the `Animal` example above

```
std::vector<Animal> zoo;

for(auto it = zoo.begin(); it != zoo.end(); ++it) {
    std::cout << (*it).name << " " << (*it).food << std::endl;
}
```

Recall that the code above assumes that `->` is not implemented for the iterator

8 Trees

A **tree** comprises of two things:

1. It's an acyclic graph (has no cycles)
2. It is connected, meaning that there is a unique path from the root to every other node

A **binary tree**, T, is either

1. A tree composed of a tuple containing the **root**, **left subtree**, and **right subtree**, $T = (r, T_L, T_R)$, where r, T_L, and T_R represent the root, left subtree, and right subtree respectively
2. A **null** tree (there are no nodes in the tree)

A data structure should be able to contain nothing because that is how we tend to start evolving and adding elements/nodes into it. In the context of a tree, we would like to be able to start it off with nothing. Therefore, it makes sense to define a binary tree as a **null** tree, where it has no vertices

A tree's **height**, height(T), is the longest path from the root to a leaf. However, this changes a little bit for a binary tree

In a binary tree, it is equal to maximum subtree of the left and right plus 1, or simply

$$\text{height}(T) = \max(\text{height}(T_L), \text{height}(T_R)) + 1$$

This makes sense because if $T_L = 5$ and $T_R = 3$, then the whole tree can go at most 5 levels deep through the left. The maximum of the two tells you the **deepest path** from the current node to a leaf. We add 1 because we add the **edge of the root node**. Let's try this on the tree in **Figure 8.1**, where the left subtree rooted at B is the taller one, so it decides the height of the whole tree.

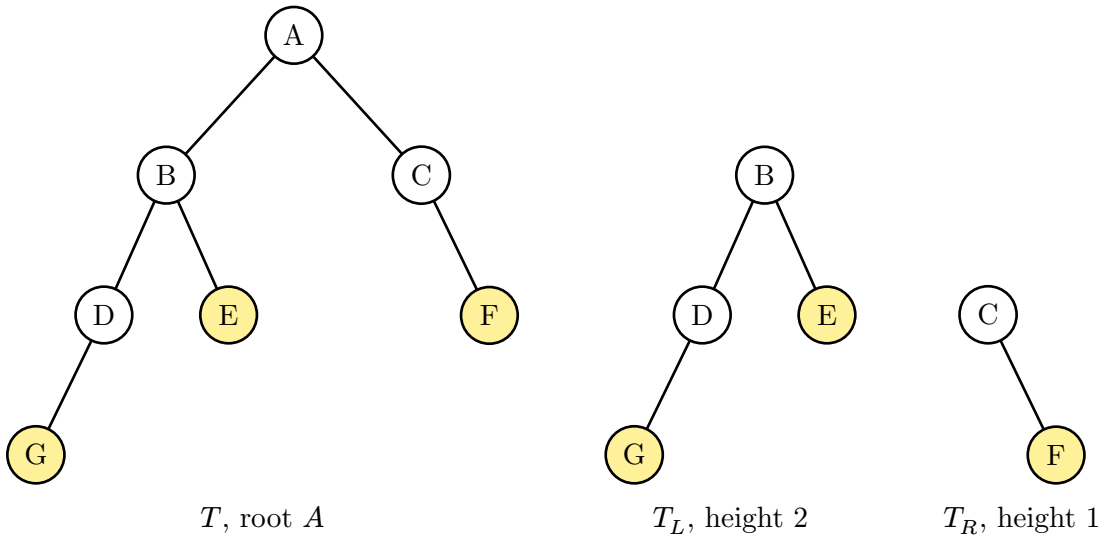


Figure 8.1: Example: the leaves (no children) are highlighted. The longest path from A is A, B, D, G, so $\text{height}(T) = \max(2, 1) + 1 = 3$

The height of a null tree is simply -1, because all the math and recursion will work out. Most of the time, the base case in tree recursion occurs when there are no nodes to choose from

A tree is **full** if and only if

1. The tree is empty (has no nodes, or simply $F = \varnothing$)
2. $F = (r, F_L, F_R)$ where either $F_L = F_R = \varnothing$ or $F_L \neq \varnothing$ and $F_R \neq \varnothing$
   - in code, this simply means checking whether the two nodes both have null pointers or neither have null pointers

A **perfect** tree, P, is defined in terms of a tree's height (simply because either everything exists at the tree's height or nothing exists).

If P_h represents a perfect tree with height h, then

1. $P_{-1} = \varnothing$
2. $P_h = (r, P_{h-1}, P_{h-1})$, meaning that the height of the left and right subtrees are **equal**

However, if I wanted to build a perfect tree with only 4 nodes, that wouldn't be possible since we have an odd number of nodes remaining after expending 1 for the root node

A **complete** tree is a binary tree where all levels are completely filled except for the last level, which is filled from left to right

More specially, complete trees

1. A perfect tree for every level except the last, where the last level is "pushed to the left"
2. For all levels k in $[0, h-1]$, k has 2^k nodes. For level h, all nodes are "pushed to the left"

A complete tree of height h, C_h

1. $C_{-1} = \{\}$
2. C_h (where $h \geq 0$) $= \{r, T_L, T_R\}$ and either:

- T_L is C_{h-1} and T_R is P_{h-2} - given a complete tree with a height h, its left subtree must be complete with height $h-1$ and its right subtree must be perfect with height $h-2$
- T_L is P_{h-1} and T_R is C_{h-1} - given a complete tree with a height h, its left subtree must be perfect with height $h-1$ and its right subtree must be complete with height $h-1$

This just mathematically explains the definition of a complete tree, which is simply a perfect tree where every level is filled except the last, which is filled with nodes towards its left subtree

If you compare the three trees in **Figure 8.2**, the complete tree is just the perfect tree with its rightmost leaf (7) missing.

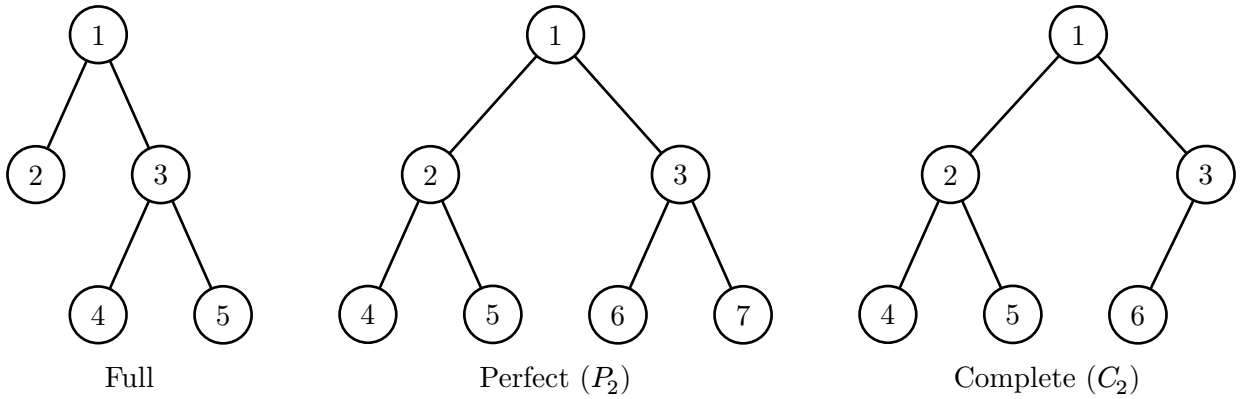


Figure 8.2: A full, a perfect, and a complete binary tree

8.1 Tree ADT

Tree ADT involves 3 primary functionalities

1. `insert`, inserts an element to the tree - into a location in structure
2. `remove`, removes an element from a tree - from a location in structure
3. `access`, access elements from a tree - from a location in structure

A list has a fixed structure and order, but a tree has varied structure depending on how it is used. The tree's structure may not matter because many algorithms may still work the same regardless of the tree's shape, as long as the structure still preserves the necessary relationships.

Therefore, let's think about how we code a `BinaryTree`. An example header file that defines a `BinaryTree` is shown below

```
// BinaryTree.h

#pragma once
template <class T>
class BinaryTree {
public:

private:
```

```
};
```

The first thing we need is a way to store the tree's root. In a `LinkedList`, the way we did it was by having a `ListNode` class to store each node of a list. Trees are similar to lists as well. They also have data stored in nodes, but are in hierarchical order rather than sequential

Therefore, we will also want to use a `struct` here to store each individual `TreeNode`, which will have 3 fields

1. `data` - the data the `TreeNode` holds
2. `left` - a **pointer** to another `TreeNode` that represents the left child (or `nullptr` if no left child)
3. `right` - a **pointer** to another `TreeNode` that represents the right child (or `nullptr` if no right child)

The code for a `TreeNode` is reflected below

```
struct TreeNode {
    // the data each tree node holds
    T data;

    // a pointer to the left child `TreeNode` (or `nullptr` if non-existent)
    TreeNode* left;

    // a pointer to the right child `TreeNode` (or `nullptr` if non-existent)
    TreeNode* right;
};
```

With this, it makes sense for our root to also be a **pointer** to a `TreeNode` as well. By making the left/right children nodes and root nodes pointers, it allows for efficient memory access and avoids copying data unnecessarily, allowing the tree to grow dynamically in memory without using too much overhead. The `BinaryTree` class is reflected below

```
// BinaryTree.h

template <class T>
class BinaryTree {
private:
    struct TreeNode {
        // the data each tree node holds
        T data;

        // a pointer to the left child `TreeNode` (or `nullptr` if non-existent)
        TreeNode* left;

        // a pointer to the right child `TreeNode` (or `nullptr` if non-existent)
        TreeNode* right;
    };
```

```
    // a pointer to the root node of a tree
    TreeNode* root;
};
```

In memory, each tree node is created on the heap, and the `root` pointer (on the stack) points to the first node. Each node's `left` and `right` fields store pointers to other nodes on the heap, linking them together without copying any data. This allows the tree to grow dynamically, using memory only when needed.

When we create a tree in memory, there is often wasted space because child pointers in many nodes can point to `null`, meaning they don't reference any actual node. Knowing **how many nodes store these null pointers** helps us understand the **memory overhead** and measure how **efficient** our tree data structure is.

In contrast, with a `LinkedList`, we only had to worry about one pointer per node possibly being null (the `next` pointer), so the overhead was much simpler and easier to quantify (it was always 1)

Therefore, it makes sense to **prove how many null pointers exist in a tree** to analyze and optimize its memory usage and better understand the structure's efficiency.

There will always be $n + 1$ `NULL` pointers for n nodes in our binary tree. But why?

Prove that if there are n data items in our representation of a binary tree, then there are $n + 1$ `NULL` pointers (Hint: use induction on n)

1. Let NULLS(n) be defined as the number of null pointers in a tree of size n
2. **Base Case(s)**
   - $n = 0 : 1$ null pointer; $(n + 1 = 0 + 1 = 1)$ (the tree is empty so the `root` pointer is null)
   - $n = 1 : 2$ null pointers; $(n + 1 = 1 + 1 = 2)$ (there is only the root node and both `left` and `right` child pointers are null)
   - $n = 2 : 3$ null pointers; $(n + 1 = 2 + 1 = 3)$ (One node has two children - one child node and one null child - and the child node has two null pointers, totaling 3)
3. **Induction Hypothesis**: Suppose NULLS(n) $= n + 1$ for all $0 \leq n < k$
4. **Inductive Step**: Prove for $n = k + 1$ nodes
   - Consider any binary tree with $k + 1$ nodes
   - Remove a leaf node (or any node that can be removed while keeping a valid binary tree structure), resulting in a tree with k nodes
   - By the inductive hypothesis, this smaller tree has $k + 1$ null pointers
   - When adding back the removed node:
     - One null pointer (where the node was attached) is replaced by a new node, so we lose 1 null pointer
     - The newly added node has two child pointers, both null, so we add two null pointers
   - Therefore, the **total null pointers** $= (k + 1) - 1 + 2 = k + 2$
   - Which equals: $(k + 1) + 1$
   - Therefore, by induction this statement holds true for $n \geq 0$

9 Tree Traversals

Previously, we discussed a new data structure: trees, where we discussed that there is a $O(n)$, linear space, overhead cost because n-space is required to store the data.

Here, we will continue discussing Trees and the Tree ADT. Previously, we failed to discuss how the implementation of `insert`, `remove`, and `access` would work. This is because the way they work is different for every tree.

Recall that previously we discussed that every data structure should be able to have its own way of getting everything it contains, which was done through an iterator. This means that if we want to achieve this functionality, we would need to implement an iterator interface. But **how**?

We need to know how to find everything in our tree. Otherwise, how would we know that we found everything. The simple answer to these questions is by implementing **tree traversals**.

Tree traversals are algorithms that will follow the structure of a tree in some fixed order that **visits** every node. They access **all the nodes** in a tree. So every data point is stored in some sort, and we are going to visit all of the nodes.

- By visit, we typically mean whatever the algorithm wants to do with the data, which for demonstration purposes, we just simplify to printing the data because that is an easy way to test what is going on
- So when a node is visited, it will then be printed and then at the end check whether we went through every node in the tree

While it makes sense that the goal of a tree traversal is to visit every node, but **how**? Should a tree visit a node **before** I traverse into the subtree, **in between** the subtree, or **after** the subtree?

The simple answer is that **it's up to you**. It depends on **when** you choose to visit a node in relation to its children. These three choices correspond to the three well-known tree traversal techniques:

- **Pre-order traversal** — visit the node **before** its subtrees
- **In-order traversal** — visit the node **between** its subtrees
- **Post-order traversal** — visit the node **after** its subtrees

By understanding how these traversals work, it makes coding them much more simple and straightforward.

1. Start from the root and do one of three of the following options (the order in which you do the tree depends on the traversal type)
   - Visit
   - Descend into the left subtree
   - Descend into the right subtree

2. Repeat step 1 for the **next** subtree (depending on its left or right) until there is no subtree available

The order in which you do the three options above depends on the traversal type

- **Preorder** - visit, descend left, descend right
- **Inorder** - descend left, visit, descend right
- **Postorder** - descend left, descend right, visit

We'll run every traversal in this section on the tree in **Figure 9.1**.

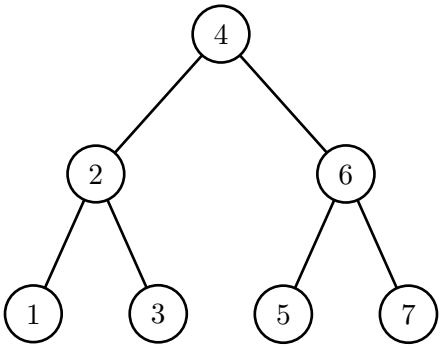


Figure 9.1: Example tree used for every traversal below

Traversals are **recursive** by nature because trees are recursive data structures. When we visit a node, we then descend into its left and right subtrees, and for each of those, we repeat the same process: visiting the node and then exploring its subtrees. This pattern continues until we reach nodes with no children, making recursion the best way to mimic this behavior.

9.0.1 Pre Order traversal

Preorder traversal visits a tree node, and then descends into the left and right subtrees. It's recursive since trees are recursive data structures in nature.

This makes coding pre order traversal very simple. This function will take in the current `TreeNode` being processed (`curr`): `void preOrderTraversal(TreeNode* curr);`

The code for `preOrderTraversal` is shown below

```
// BinaryTree.hpp

/** Prints every node in pre-order (node, left subtree, right subtree)
 * Runtime: O(n)
 */
template <typename T>
void BinaryTree<T>::preOrderTraversal(TreeNode* curr) {
    // If the current node is a null pointer (`nullptr`), halt the recursion
    if(curr == nullptr) { return; }
```

```
    // Visit the current `TreeNode`
    std::cout << curr->data << " ";

    // Repeat the process for the left subtree
    preOrderTraversal(curr->left);

    // Repeat the process for the right subtree
    preOrderTraversal(curr->right);
}
```

Our **base case** occurs when the current node pointer (`curr`) is `nullptr`. This means we've reached a point in the tree where no further nodes exist (e.g. a leaf node's child) and there's nothing to process or recurse on, so we simply return.

If the node is not `nullptr`, we proceed with the pre-order traversal steps:

1. **Visit the current node** — we get its data (e.g., print it).
2. **Traverse the left subtree** — recursively call `preOrderTraversal` on `curr->left`.
3. **Traverse the right subtree** — recursively call `preOrderTraversal` on `curr->right`.

Each of these recursive calls follows the same pattern: check if the node is `nullptr`, visit it, and recurse on its children. This process naturally continues until all nodes have been visited, and both left and right child pointers are `nullptr`, causing the recursion to unwind.

Running pre-order on the tree in **Figure 9.1** visits 4, 2, 1, 3, 6, 5, 7.

9.0.2 In Order traversal

Inorder traversal descends into the left subtree, visits a tree node, and then descends into the right subtree

This also makes coding in order traversal very simple. This function will take in the current `TreeNode` being processed (`curr`): `void inOrderTraversal(TreeNode* curr)`

The code for `inOrderTraversal` is shown below

```
// BinaryTree.hpp

/** Prints every node in in-order (left subtree, node, right subtree)
 * Runtime: O(n)
 */
template <typename T>
void BinaryTree<T>::inOrderTraversal(TreeNode* curr) {
    // If the current node is a null pointer (`nullptr`), halt the recursion
    if(curr == nullptr) { return; }

    // Repeat the process for the left subtree
    inOrderTraversal(curr->left);

    // Visit the current `TreeNode`
```

```
    std::cout << curr->data << " ";

    // Repeat the process for the right subtree
    inOrderTraversal(curr->right);
}
```

Our **base case** is the same as pre order, when the current node pointer (`curr`) is `nullptr`. No further nodes exist anymore (e.g. a leaf node's child) and there's nothing to process or recurse on, so we simply return.

If the node is not `nullptr`, we proceed with the in-order traversal steps:

1. **Traverse the left subtree** — recursively call `inOrderTraversal` on `curr->left`.
2. **Visit the current node** — we get its data (e.g., print it).
3. **Traverse the right subtree** — recursively call `inOrderTraversal` on `curr->right`.

If we apply in-order traversal to the tree in **Figure 9.1**, we get 1, 2, 3, 4, 5, 6, 7 (sorted, because the example is a BST).

9.0.3 Post Order Traversal

Post Order traversal descends into the left subtree, descends into the right subtree, and then visits the current tree node.

This also makes coding post order traversal very simple, since we know its process. This function will take in the current `TreeNode` being processed (`curr`): `void postOrderTraversal(TreeNode* curr)`

The code for `postOrderTraversal` is shown below

```
// BinaryTree.hpp

/** Prints every node in post-order (left subtree, right subtree, node)
 * Runtime: O(n)
 */
template <typename T>
void BinaryTree<T>::postOrderTraversal(TreeNode* curr) {
    // If the current node is a null pointer (`nullptr`), halt the recursion
    if(curr == nullptr) { return; }

    // Repeat the process for the left subtree
    postOrderTraversal(curr->left);

    // Repeat the process for the right subtree
    postOrderTraversal(curr->right);

    // Visit the current `TreeNode`
    std::cout << curr->data << " ";
}
```

The base case is equivalent to pre and in order traversals.

If the node is not `nullptr`, we proceed with the post-order traversal steps:

1. **Traverse the left subtree** — recursively call `postOrderTraversal` on `curr->left`.
2. **Traverse the right subtree** — recursively call `postOrderTraversal` on `curr->right`.
3. **Visit the current node** — we get its data (e.g., print it).

Back on **Figure 9.1**, post-order visits 1, 3, 2, 5, 7, 6, 4, so the root 4 comes last.

The ideas behind these traversals are very similar. It's just that the order a `TreeNode` is visited and when to descend into subtrees varies based on the type of traversal

A good way to remember the traversals are the three mnemonics

- **NLR** - pre order traversal (node, left, right)
- **LNR** - in order traversal (left, node, right)
- **LRN** - post order traversal (left, right, node)

More specifically, these tree traversals fall under a broader category known as **depth first search (DFS)**. In DFS, we start at the root and recursively descend as far down each branch as possible before backtracking, where nodes are then visited in a way that explores one entire path before moving to another.

9.1 A different type of traversal - Level Order Traversal

There is another type of tree traversal that behaves differently than the three DFS traversals and it is known as a **Level Order Traversal**.

In this traversal, we visit each node **layer by layer**, or level by level, starting from the root node. This will help find nodes at each level.

This level order traversal method is more commonly known as **breadth first search** or simply **BFS**, which involves visiting the nodes of a tree level by level

In the three DFS traversals we used recursion on the **tree** itself to implement them. In BFS, we can use recursion, but it wouldn't make sense to do it on the **tree** itself since we don't need to recursively descend into each subtree.

Instead, we would need to use another data structure that the recursion would work with. Since BFS visits the nodes in horizontal, level order, a queue would make sense here. This is because queues are **First-In First-Out** and preserve the level ordering rule that BFS needs.

However, using recursion with BFS is not intuitive and not preferred, since it requires visiting all of a node's siblings before its children, which goes against the natural depth-first behavior of recursion.

We can still implement it iteratively with the help of a queue.

1. Create a queue of `TreeNode` pointers (`TreeNode*`)
2. Enqueue `root` to the queue
3. While the queue isn't empty, dequeue the front node and check if its pointer is `null`. If not, then print its data, and then enqueue its left and right child

Level-order on **Figure 9.1** goes one level at a time: 4, then 2, 6, then 1, 3, 5, 7.

The code for this process, which uses a queue, is shown below

```
// BinaryTree.hpp

/** Prints every node level by level, from left to right
 * Runtime: O(n)
 */
template <typename T>
void BinaryTree<T>::levelOrderTraversal() {
    // Create a queue to store `TreeNode` pointers at every level
    std::queue<TreeNode*> q;

    // Add the root `TreeNode`, `root` to the queue
    q.push(root);

    while(!q.empty()) {
        // Get the `TreeNode` at the front of the queue and remove it
        TreeNode* n = q.front();
        q.pop();

        // Check if the front `TreeNode` is not null
        if(n != nullptr) {
            // visit the node
            std::cout << n->data << " ";

            // add the node's left and right children to the queue
            q.push(n->left);
            q.push(n->right);
        }
    }
}
```

9.2 Traversals as Iterators

So far, every traversal has been a recursive function that visits the whole tree at once. Sometimes we want to **iterate** over a tree instead, using `begin()` and `end()` like we did for lists, so that each `++` moves to the next node in pre-order, in-order, or post-order

An iterator can't pause in the middle of a recursive call, so it has to remember where it is in the traversal some other way. Instead of relying on the call stack that recursion uses implicitly, the iterator keeps its **own** explicit stack of the nodes it still has to come back to. This is the same idea as the iterator in MP Lists needing to store a little more than just a pointer

A clean way to support all three orders is **inheritance**

- A base class, `TreeTraversal`, holds the shared work. It acts as an **iterator generator**: it provides `begin()` and `end()`, which create the iterator that walks the tree

- `InorderTraversal`, `PreorderTraversal`, and `PostorderTraversal` inherit from it and provide only the part that differs between them, which is **when** a node is visited relative to its children, the same way the recursive versions only differ in where the print goes
- Code that takes a `TreeTraversal*` works with any of the three. If the pointer is concretely an `InorderTraversal`, we get an in-order traversal, and so on

Try to put almost all of the work in the base class and only the order-specific step in the derived classes. Working through this carefully matters, since this way of reasoning about recursion done iteratively comes back in later MPs and exams

10 Tree Searching and Dictionaries

A **traversal** is an algorithm that goes through every single node in a tree and visits it ($O(n)$ time)

A **search** is an algorithm that finds a node in a tree of value v if it exists (worst case is when $v \notin T$, where T represents any tree; this involves running the full traversal, so it is in $O(n)$ time)

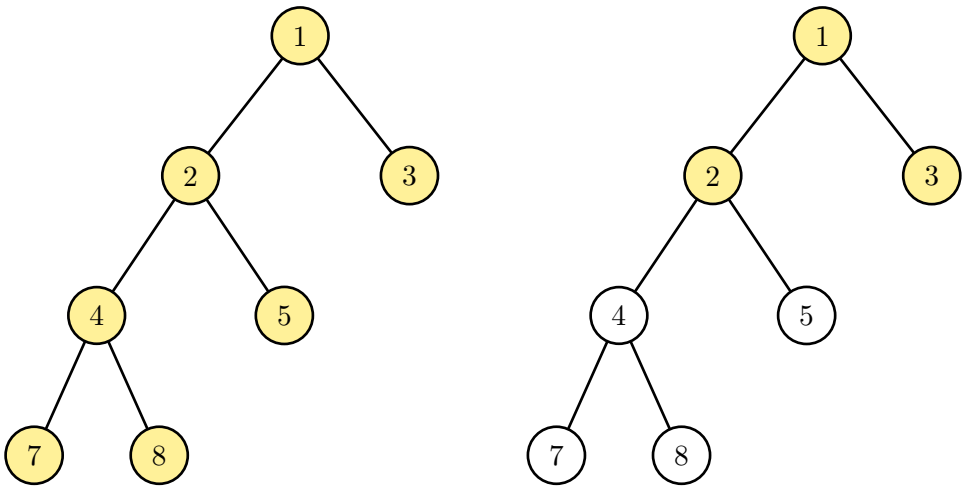
Previously, we discussed two types of search **Breadth First** and **Depth First** (**BFS** and **DFS**)

BFS is the same as level order traversal, where its overhead (or space complexity) is the maximum number of nodes in a queue (can also be seen as the width of a tree)

DFS involves any of the three traversals that descend into the tree's children: pre order traversal, in order traversal, and post order traversal. Its overhead is the maximum depth in the tree (or simply height)

BFS is more **reliable** than DFS. This is because it will eventually find a solution to a valid search if one exists, even if the tree is infinite in depth. Since BFS explores nodes level by level, this guarantees that all nodes at depth d are visited before any node at depth $d + 1$. So if the solution lies at a shallow depth, BFS will find it eventually.

In contrast, DFS can get **"stuck"** going down an infinite branch of a tree, especially if the solution lies on a different, shallower path. Therefore, DFS might never return a proper solution, even if it does exist. You can already see this cost on a finite tree in **Figure 10.1**: when searching for 3, DFS wanders through the whole left branch while BFS finds it after one level.



DFS (pre-order): 1, 2, 4, 7, 8, 5, 3 BFS: 1, 2, 3

Figure 10.1: Example: searching for 3, which sits at depth 1. DFS exhausts the whole left branch first; BFS reaches 3 after visiting only one level

BFS will always find a solution if one exists, even in infinite-depth trees, but it may require an infinite amount of memory. This is because BFS stores all nodes at a given depth (one full level of the tree) in a queue before moving on to the next level. If the tree is very wide, this means the queue can grow extremely large, consuming a lot of memory.

In contrast, DFS typically uses less memory, since it only needs to store the current path from the root to a leaf, which depends on the depth of the tree, not its width.

Therefore, in any practical application, memory is going to be the forefront problem. Since DFS uses less memory, that will often be the best approach. But, if there are infinite trees, DFS wouldn't work. There is a solution that solves this problem and it is known as **iterative deepening**

Iterative Deepening runs DFS with a **maximum depth limit** of k, meaning it will not explore any nodes beyond depth k (starting at $k = 0$). When DFS reaches that depth, it stops descending and backtracks up the tree. This approach is useful because the maximum memory used when doing DFS is proportional to k (simply the overhead cost - $O(k)$). If no solution is found at depth k, the algorithm restarts DFS with a new depth limit $k + 1$, and this process repeats until a solution is found.

This effectively simulates BFS for DFS and will always find the solution because it explores each node at every level by using bounded memory (as deep as depth k with overhead cost of $O(k)$).

These are the two basic searching strategies, but they aren't that effective, since they can be slow in both time and memory. But what if there was a data structure that put searching at its forefront, which is essentially what a **map** or **dictionary** does

10.1 Dictionaries

Dictionaries store **key-value** pairs, just like a regular dictionary, which stores words to definitions. If I had a word, I can use a dictionary to look up its definition! This could be done most optimally using binary search, where I start from the middle and then slowly go left or right based on whether the word I'm looking for comes before or after the middle word alphabetically.

This is just one example of a dictionary though, we can use it to look up students using their unique ID numbers (UIUC UIN) or by looking at a Flight number (**key**) for its departure time/ arrival time (**value**).

In fact, an array really is just a dictionary, because we use an index (**key**) to lookup the value of an element that is present in the array (**value**)

10.1.1 Dictionary Interface (ADT)

- `insert(key, value)` - insert a key and value into the dictionary
- `remove(key)` - remove a key value mapping from the dictionary via passing in the key
- `std::optional<value> find(key)` - return the value associated with the key (C++ has an `std::optional` type that you could use to return nothing if no value is found for the key!)

And then there's the obvious things like a constructor, destructor, and an iterator, but at the end of the day, this is what we want with a dictionary.

And with this, we solve our problems with searching in Computer Science. A lot of other data structures, like Hash Tables and Tries, will rely on dictionaries to provide fast and efficient key-based access, which is essential for everything from caching to autocomplete and spell checking

The code below is a sample header file for our dictionary, backed by a `std::unordered_map`

```
// Dictionary.h

#pragma once
#include <unordered_map>

template <class K, class V>
class Dictionary {
public:
    using Iterator = typename std::unordered_map<K, V>::iterator;

    /** Inserts a key-value pair into the dictionary
     * Runtime: expected O(1)
     */
    void insert(const K key, const V value);

    /** Removes the mapping for `key` and returns its value
     * Runtime: expected O(1)
     */
    V remove(const K& key);

    /** Returns an iterator to the pair with `key`, or `end()` if it is missing
     * Runtime: expected O(1)
     */
    Iterator find(const K& key);

    /** Returns an iterator to the first key-value pair
     * Runtime: O(1)
     */
    Iterator begin();

    /** Returns an iterator one past the last key-value pair
     * Runtime: O(1)
     */
    Iterator end();

private:
    // the underlying map that stores every key-value pair
    std::unordered_map<K, V> data;
};
```

Our custom dictionary implements an iterator by leveraging C++'s standard library iterator from the underlying data structure of `std::unordered_map`. This allows us to avoid writing a custom iterator from scratch, while still providing full iterator functionality like `begin()`, `end()`, and `find()`.

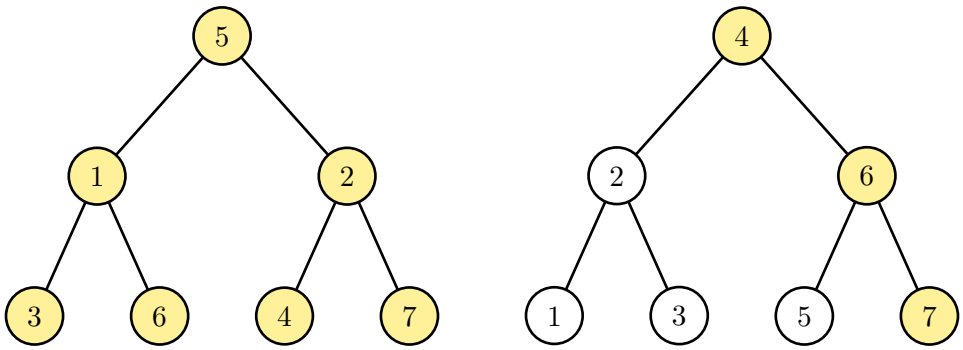
10.2 Using Binary Trees as a Search Structure

In the context of a tree, these three functionalities of a dictionary, `insert`, `remove`, and `find`, are reflected below

Dictionary Operation	Runtime	Explanation
`insert`	$O(1)$	Similar to a LinkedList; put a new pointer to head and point the tree's head to this new node, which is constant time
`remove`	$O(n)$	Need to first find the item by scanning the tree, and then shift elements to fill the gap
`find`	$O(n)$	Searches nodes one by one, traversing the tree until it finds the key or uses all nodes

This sets up the foundation for many other types of trees including, BSTs, BBSTs, and AVL trees, which will be covered later on

To preview why ordering the keys matters for `find`, compare the two trees in **Figure 10.2**: the plain binary tree visits all 7 nodes to find 7, while the BST only visits 3.



Plain binary tree: 7 nodes visited BST: 3 nodes visited

Figure 10.2: Example: finding 7 among the same keys. A plain binary tree gives no hint where 7 is, so the search scans every node; a BST orders its keys so the search follows a single path 4, 6, 7

11 BST and BST Implementation

Recall that all trees have **structure**. Fundamentally, a tree gives us **positions** that when we put nodes in a tree, we put them in a position. Previously, the trees we discussed were in a position that had no meaning whatsoever. Is there a way for us to put nodes in a specific position that has meaning?

For this to take place, we need to discuss a different type of tree: the **Binary Search Tree**, or BST. The BST has a property, in which a node's left subtree contains nodes with keys less than the node's key and a node's right subtree contains nodes with keys greater than the node's key.

When it comes to equality, we typically don't define it for a BST. We assume that there will not be any duplicates (every node is unique in value) in a BST, and therefore we don't place it anywhere. It still can be extended in a BST definition, but it ends up with odd decisions and semantics, so it's just better to assume that all nodes in a BST are **unique**.

Mathematically speaking, for tree T to be a BST:

- $T = \varnothing$
- $T = (r, T_L, T_R)$, where $\forall_x \in T_L$, $x < r$ and $\forall_x \in T_R$, $x > r$

x is a key in T_L, where $x < r$ and a key in T_R, where $x > r$

In **Figure 11.1**, the tree on the left is a valid BST, but the one on the right breaks this property because of the highlighted 12.

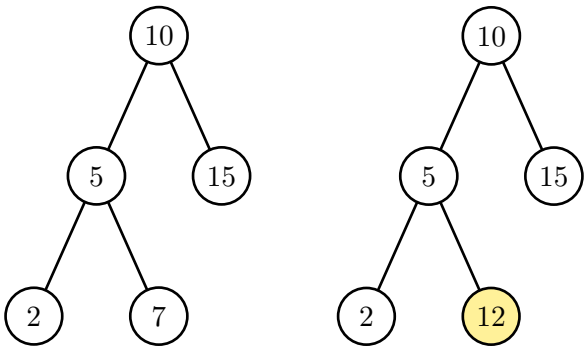


Figure 11.1: Example: on the right, 12 is a valid right child of 5, but it lies in the left subtree of 10 while $12 > 10$. The property must hold for the whole subtree, not just the parent

This creates structure in the tree, where if I wanted to find a node like 21, I would start searching from the root node, and then check if 21 is greater than or less than the root node and then descend to the right or left subtree respectively and repeat the process until I find the node of 21.

In this process, we rule out half one of the subtrees. Notice in **Figure 11.2** how the search for 21 only touches the highlighted path 15, 25, 21.

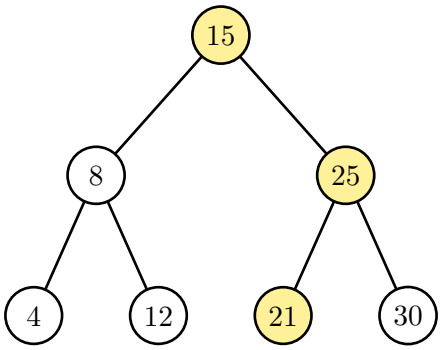


Figure 11.2: Example: finding 21 visits 15, 25, 21 ($21 > 15$, go right; $21 < 25$, go left). The left subtree of 15 is never touched

This is the unified process of searching that we have in a BST, which gives the tree a proper structure, giving each node's position a meaning.

Simply speaking, a BST is a binary tree that is organized in a way where all the nodes in its left subtree are smaller than it and all of the nodes in the right subtree are bigger than it.

Finding a node in a BST is $O(h)$, where h is the height of the tree. This is because the worst case occurs when we have to traverse all the way to the root of a leaf, which takes h steps. This means that $O(h) \equiv O(n)$ because the height h can be as large as n in the worst case.

Below is a good concrete implementation of a BST

```
// BST.h

#pragma once
#include <unordered_map>

template <class K, class V>
class BST {
public:
    using TreeIterator = typename std::unordered_map<K, V>::iterator;

    BST() : root(nullptr) { }

    /** Inserts a key-value pair into the BST
     * Runtime: O(h)
     */
    void insert(const K key, const V value);

    /** Removes the node with the given key from the BST
     * Runtime: O(h)
     */
```

```
    void remove(const K& key);

    /** Returns an iterator to the pair with `key` (looked up in `node_map`)
     * Runtime: expected O(1)
     */
    TreeIterator find(const K& key);

    /** Returns an iterator to the first key-value pair
     * Runtime: O(1)
     */
    TreeIterator begin();

    /** Returns an iterator one past the last key-value pair
     * Runtime: O(1)
     */
    TreeIterator end();

private:
    struct TreeNode {
        TreeNode* left;
        TreeNode* right;
        K key;
        V value;

        TreeNode(const K& key, const V& value) : left(nullptr), right(nullptr),
key(key), value(value) { }
    };

    TreeNode* root;

    // a map that lets us use iterators for fast access without needing to
traverse the tree
    std::unordered_map<K, V> node_map;

    // internal helpers

    /** Returns the pointer to `key`'s node, or the null spot where it goes
     * Runtime: O(h)
     */
    TreeNode*& _find(TreeNode*& root, const K& key);

    /** Inserts a new node at the null pointer `_find` stops on
     * Runtime: O(h)
     */
    void _insert(TreeNode*& root, const K& key, const V& value);

    /** Returns the pointer to the in-order predecessor of `root`
     * Runtime: O(h)
     */
```

```
    TreeNode*& _iop(TreeNode*& root);

    /** Swaps the key and value stored in two nodes
     * Runtime: O(1)
     */
    void _swap(TreeNode* a, TreeNode* b);

    /** Removes the node with `key` from the subtree rooted at `node`
     * Runtime: O(h)
     */
    void _remove(TreeNode*& node, const K& key);
};
```

This looks great, it shows the exact functionalities we need for our BST

Notice how we added an `std::unordered_map`, `node_map`. This helps us use the unordered map's iterator for fast access of our tree nodes, without having to constantly traverse the tree. It's a fast access cache alongside our BST

- `find` - Will be used to find a node in a BST. Returns a custom implemented `TreeIterator` that behaves like the STL iterator of a template library. This is because we want to return the iterator's `.end()` behavior when not found.
- `insert` - Will be used to insert a node into a BST
- `remove` - Will be used to remove a node from a BST

Let's first start with the simplest implementation, `find`

11.1 `find`

To implement find in a BST, the process is pretty straightforward

1. Visit the root node; if it's greater than the root node, descend into the right subtree and visit its node. otherwise, descend into the left subtree and visit its node.
2. Repeat step one for each node as you descend until you find the node and return it
3. Otherwise, return the iterator's `.end()` behavior if the node was not found in the BST

First we will begin with the internal find (`_find`). The external find function will just wrap the internal find with `root`, and then dealing with the case when the pointer is null and returning an iterator to the `end` rather than an iterator to that node. The signature of `_find` is shown below.

```
/** Returns the pointer to `key`'s node, or the null spot where it goes
 * Runtime: O(h)
 */
template <class K, class V>
typename BST<K, V>::TreeNode*& BST<K, V>::_find(TreeNode*& root, const K& key) {

}
```

This internal `_find` is a lot like `find` from a `LinkedList`. It returns a reference to a pointer to a `TreeNode`. This returns the pointer that points to the node that has the key that exists somewhere in my tree data structure. Or if not, the last pointer I found while trying to find it. And then we take in the `root`, which is our recursion and the `key`

Since this is a recursive problem we first need to figure out our base case, which is just checking if our root's key is equal to the key we want to find. If it is, we should just return the root, since it is a pointer as a reference to the root node!

However, this code will crash if `root` is null, so we need to add another base case to check if `root` is null.

Then, we need to check if the root's key is greater than the given key, and if so, we call `_find` and return the root's left node and the key

Otherwise, we can just call `_find` and then return the root's right node and the key

This full code for internal find, `_find`, is shown below

```
// BST.hpp

/** Returns the pointer to `key`'s node, or the null spot where it goes
 * Runtime: O(h)
 */
template <typename K, typename V>
typename BST<K, V>::TreeNode*& BST<K, V>::_find(TreeNode*& root, const K& key) {
    // if the root's pointer is null, return the root or
    // if the root's key equals the given key, return the root
    if(root == nullptr || root->key == key) { return root; }

    // if the root's key is greater than the given key, call find on the left
subtree
    if(root->key > key) {
        return _find(root->left, key);
    }

    // if the root's key is less than the given key, call find on the right
subtree
    return _find(root->right, key);
}
```

To implement the external find, `find`, we can just simply return `node_map.find(key)` as an iterator, since `node_map` always stores the same key-value pairs as the BST and supports fast lookup:

```
// BST.hpp

/** Returns an iterator to the pair with `key` (looked up in `node_map`)
 * Runtime: expected O(1)
 */
template <typename K, typename V>
```

```cpp
typename BST<K, V>::TreeIterator BST<K, V>::find(const K& key) {
    // use `node_map` to quickly find the value without traversing the tree
    return node_map.find(key);
}
```

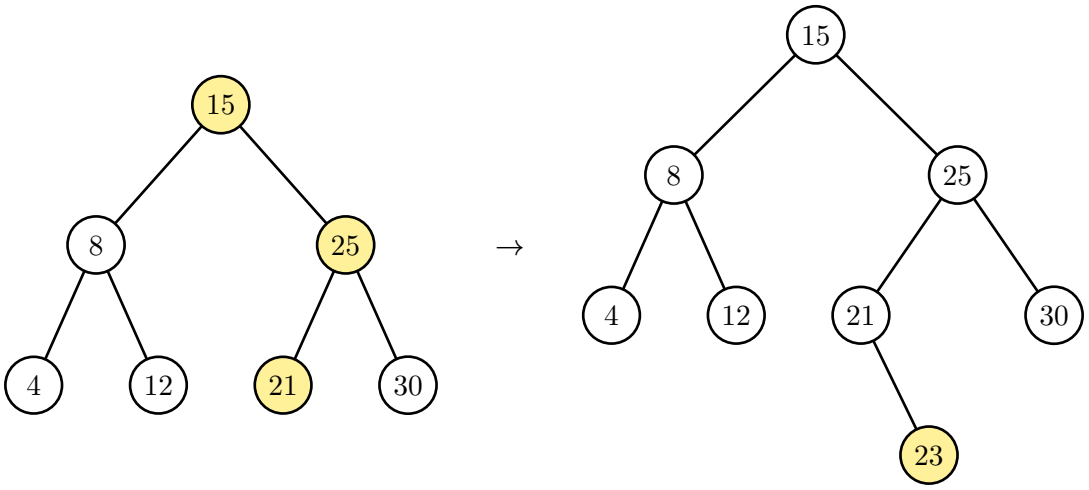
However, there are some questions to ask, especially since this was a recursive approach.

1. What if the tree is really tall? Because in a list, we don't want to write things recursively because the nodes will expend a lot of memory on the call stack.
   - The only valid solution would be to implement this iteratively, but I wouldn't want to do this because they work in a naturally recursive format. For now we can safely ignore this because there isn't much of a use case, but this is still a problem.

11.2 `insert`

We will now implement an internal insert function called `_insert`. The process is simple: to insert a node into a Binary Search Tree (BST), we search for the position where the new node should go, and then assign a new `TreeNode` with the given key and value at that location.

This works because, when we search for the correct position in the BST, we eventually reach a `null` pointer, since the key does not already exist in the tree. The `_find` function returns a **reference to that pointer**, giving us access to the exact spot where the new node should be inserted. With this reference, we can directly assign a new `TreeNode` to that location, effectively inserting the node into the tree. For example, inserting 23 in **Figure 11.3** follows the same path as a search (15, 25, 21), and 23 lands as the right child of 21.



Search visits 15, 25, 21 — 23 becomes the right child of 21

Figure 11.3: Example: inserting 23. `_find` stops at the null `right` pointer of 21, and the new node is assigned there

The code for internal insert, `_insert` is shown below

```
// BST.hpp

/** Inserts a new node at the null pointer `_find` stops on
 * Runtime: O(h)
 */
template <class K, class V>
void BST<K, V>::_insert(TreeNode*& root, const K& key, const V& value) {
    // Create a new `TreeNode` at the null pointer location returned by `_find`
    _find(root, key) = new TreeNode(key, value);
}
```

This code looks a little bit weird, but it makes sense

1. Recall that `_find(root, key)` returns a **reference to a pointer** (`TreeNode*&`), specifically the pointer in the tree where the node with `key` should be located
2. When the key doesn't exist, that pointer is **null**
3. Because `_find` returns a **reference to that pointer**, you can assign to it **directly**

This code can be done more clearly to understand what's going on, by storing the result of `_find` in an explicit `loc` reference

```
// BST.hpp

/** Inserts a new node at the null pointer `_find` stops on
 * Runtime: O(h)
 */
template <class K, class V>
void BST<K, V>::_insert(TreeNode*& root, const K& key, const V& value) {
    // grab the reference/location to the pointer where `key` should be inserted
    TreeNode*& loc = _find(root, key);

    // assign `loc` to point to a new `TreeNode` with the given `key` and `value`
    loc = new TreeNode(key, value);
}
```

There is a bug if you don't add the ampersand to the `TreeNode*` type when creating `loc`, since that then creates a copy pointer which changing it only changes the copy

It's a preference as to which method is better for writing `_insert`; the first one is slightly more clear as it expresses the entire insertion action in a single, direct statement without introducing intermediate variables or extra steps.

To then code the external `insert`, we just have to simply call the internal insert, `_insert` and then add the key to our `node_map`, which is an `std::unordered_map` that contains our `TreeNodes`

```
// BST.hpp

/** Inserts a key-value pair into the BST
```

```
 * Runtime: O(h)
 */
template <typename K, typename V>
void BST<K, V>::insert(const K key, const V value) {
    _insert(root, key, value);

    // also insert the key-value pair into the hashmap for fast lookup and
iterator support
    node_map[key] = value;
}
```

11.3 `remove`

The final step is to implement `remove` for a BST. This seems simple since I just need to find the location of the `TreeNode` that I want to remove and then set its pointer to null.

However, this actually will only work if the node is a **leaf node**. There are two other cases we have to consider when removing a node, which rely on the node having at least one child

1. **Removing a leaf node** - If the node is a leaf node (has no children), its parent's pointer just becomes null (see **Figure 11.4**)
2. **Removing a node with one child (left or right)** - If the node has exactly either one left or right child, you remove the node by connecting its parent directly to that child. This bypasses the node being removed, preserving the BST structure without losing any nodes. In **Figure 11.5**, for example, removing 8 lets 15 point straight at 4.
3. **Removing a node with two children** - If the node has two children, meaning it has both a left and right child, we need to instead find a replacement node to take the removed node's place in order to maintain the property of a BST. This is either:
   - The **inorder predecessor** (IOP) - the largest node in the left subtree
   - The **inorder successor** (IOS) - the smallest node in the right subtree
   - Swap the target node's key and value with the replacement node's, then recursively remove the replacement node, which then falls into the simpler cases of leaf or single child. You can see this in **Figure 11.6**, where the IOP of 15 is 12 (highlighted), so the two swap and 15 is then removed with the one child case

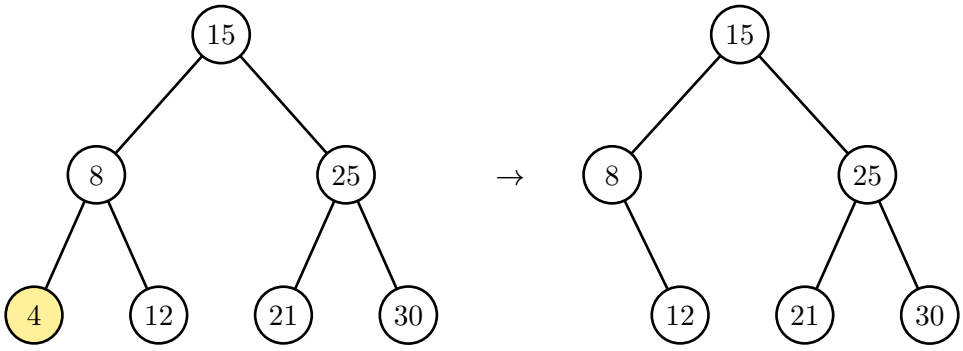


Figure 11.4: Example: removing the leaf 4. Its parent's `left` pointer simply becomes null

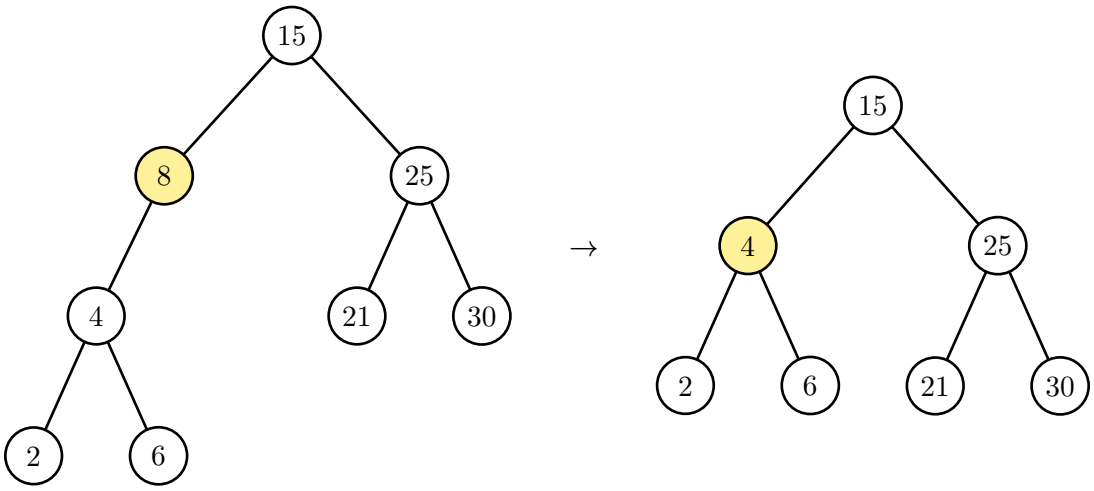


Figure 11.5: Example: removing 8, which has only a left child. 15 now points directly at 4, and the whole subtree under 4 moves up with it

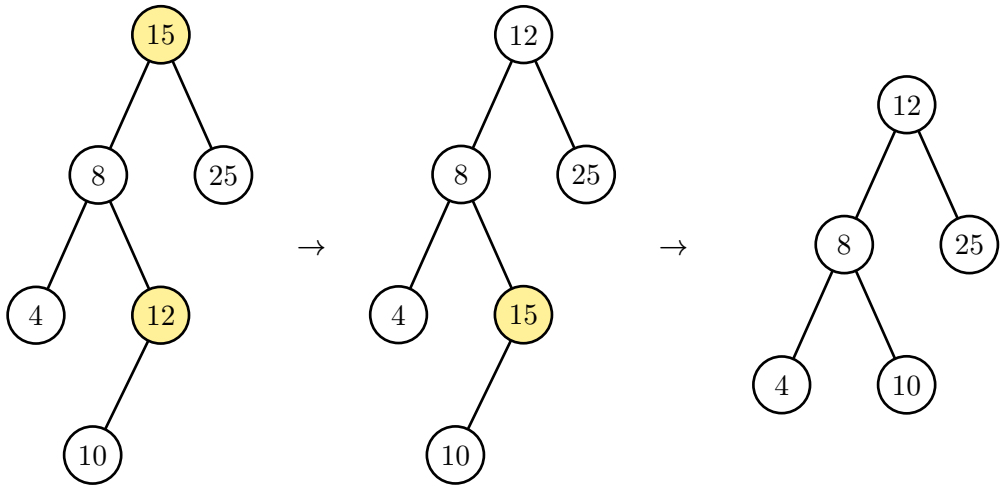


Figure 11.6: Example: removing 15, which has two children. Its IOP is 12, the rightmost node of the left subtree (8, then right to 12). After the swap, 15 has only a left child, so it is removed with the one child case

Finding either IOP and IOS (it doesn't matter which one) is a very effective approach when removing a node with both a left and right child because it always runs in $O(h)$ time, where h is the height of a tree

This is because when a left or right node follows a single path down a tree, the worst case occurs when the node travels through the entire path of h

First, we need a helper function that helps find either the IOP or IOS (will do IOP in this example). Recall that the IOP finds the largest node in the left subtree.

Therefore, to implement this, we need to find the rightmost value in the left subtree, since this will always be the largest value according to the BST property. To find the largest value, we just need to iterate through each node's right child until a node's right child no longer exists (null pointer). The code for finding the IOP is reflected below

```
// BST.hpp

/** Returns the pointer to the in-order predecessor of `root`
 * Runtime: O(h)
 */
template <typename K, typename V>
typename BST<K, V>::TreeNode*& BST<K, V>::_iop(TreeNode*& root) {
    // grab a pointer to the left subtree pointer of the root
    // (a pointer to the pointer, so moving `curr` never overwrites a link in the
tree)
    TreeNode** curr = &root->left;

    // iterate through each right child of the left subtree to find the rightmost
node
    // return the rightmost node (since that always contains the greatest value)
    while((*curr)->right != nullptr) {
        curr = &(*curr)->right;
    }
    return *curr;
}
```

The next thing we need to code is a helper swap function that swaps the internal data between two nodes. This is simple, it's just swapping the key and value of two nodes. We can use C++ STL swap function to achieve this. The code is shown below

```
// BST.hpp

/** Swaps the key and value stored in two nodes
 * Runtime: O(1)
 */
template <typename K, typename V>
void BST<K, V>::_swap(TreeNode* a, TreeNode* b) {
    // swap the internal data of nodes `a` and `b`
    std::swap(a->key, b->key);
    std::swap(a->value, b->value);
}
```

Finally, we can top it off by coding the functionality of `remove`. We first need a helper to handle `remove`, `_remove`, for all kinds of nodes, and then we can call the external remove with `root`.

Recall that `_remove` involves handling the cases that vary based on the number of children (leaf node - no children, either a left or right child, both a left or right child). The code for `_remove` is shown below

```cpp
// BST.hpp

/** Removes the node with `key` from the subtree rooted at `node`
 * Runtime: O(h)
 */
template <typename K, typename V>
void BST<K, V>::_remove(TreeNode*& node, const K& key) {
    if(node == nullptr) { return; }

    if(key < node->key) {
        // descend down the left subtree because left subtree contains the smaller
values
        _remove(node->left, key);
    } else if(key > node->key) {
        // descend down the right subtree because right subtree contains the
larger values
        _remove(node->right, key);
    } else {
        // case 1: the leaf node case - free the node and set it to null
        if(node->left == nullptr && node->right == nullptr) {
            delete node;
            node = nullptr;
        }

        // case 2: the one child node case (either left or right child)
        else if(node->left == nullptr) {
            // remove the node by connecting its parent directly to the node's
only child (the right child)
            TreeNode* tmp = node;
            node = node->right;
            delete tmp;
        }
        else if(node->right == nullptr) {
            // remove the node by connecting its parent directly to the node's
only child (the left child)
            TreeNode* tmp = node;
            node = node->left;
            delete tmp;
        }

        // case 3: the two child node case (both left and right child)
        // swap the removed node with the iop and recursively call the function on
the left subtree
        else {
            K iop_key = node->key;
            TreeNode* iop = _iop(node);
            _swap(iop, node);

            // recursively call on the left subtree because the iop (the node to
```

```
remove) is located there
            _remove(node->left, iop_key);
        }
    }
}
```

A lot of things happen in this code, but most of it has already been previously explained. A couple things to clarify however,

1. If the key we need to remove is less than the node's key, it makes sense to descend down the left subtree, since that contains keys less than the key we need to remove
2. If the key we need to remove is greater than the node's key, it makes sense to descend down the right subtree, since that contains keys greater than the key we need to remove
3. We have to check which child is null for the one child case.
   - When we get to the else if statement, we know that at least one child has to point to some node since we already checked if both were null, and obviously wasn't the case
   - If only the right child is present (the left child is null), then we can simply remove the node and connect its parent directly to this right child. The same thing works with the left child: if only the left child is present, remove the node and connect its parent directly to this left child.
4. For the two child case, once we found the iop and swapped it with the current node, we have to recursively call `_remove` on the left subtree, since that is where the IOP will always be located. That is because the IOP is the greatest node in the left subtree, but smaller than its root (meaning it is the rightmost node of the left subtree)

This makes coding the exposed (external) version of `remove` simple. We just call `_remove` and pass in the root

```
// BST.hpp

/** Removes the node with the given key from the BST
 * Runtime: O(h)
 */
template <typename K, typename V>
void BST<K, V>::remove(const K& key) {
    _remove(root, key);

    // remove the key from the map
    node_map.erase(key);
}
```

`begin()` and `end()` work by simply returning the begin and end iterators of `node_map`, our unordered map

```
// BST.hpp
```

```
/** Returns an iterator to the first key-value pair
 * Runtime: O(1)
 */
template <typename K, typename V>
typename BST<K, V>::TreeIterator BST<K, V>::begin() {
    return node_map.begin();
}

/** Returns an iterator one past the last key-value pair
 * Runtime: O(1)
 */
template <typename K, typename V>
typename BST<K, V>::TreeIterator BST<K, V>::end() {
    return node_map.end();
}
```

11.4 BST-Analysis

The table below summarizes the runtime for all our important BST algorithms

Operation	BST Worst Case
`find`	$O(h) \equiv O(n)$
`insert`	$O(h) \equiv O(n)$
`remove`	$O(h) \equiv O(n)$
`traverse`	$O(n)$

For the operations `find`, `insert`, and `remove` on a BST, the worst-case time complexity depends on the height h of the tree. These operations require traversing from the root down to a specific node, which involves up to h recursive or iterative steps. Therefore, their worst-case time complexity is $O(h)$.

If the height h of a Binary Search Tree (BST) can be as large as the number of nodes n in the worst case, then any operation that takes $O(h)$ time can take up to $O(n)$ time. The worst-case scenario holds when $O(h) \equiv O(n)$

The height h of a BST is the length of the longest path from the root to a leaf, measured in edges

1. The worst case scenario occurs when the BST weakens into a `LinkedList`, where each node has only one child
2. The height of this worst case scenario BST, h, then becomes $n-1$, where $h = n-1$, since the longest path must visit all nodes except the root

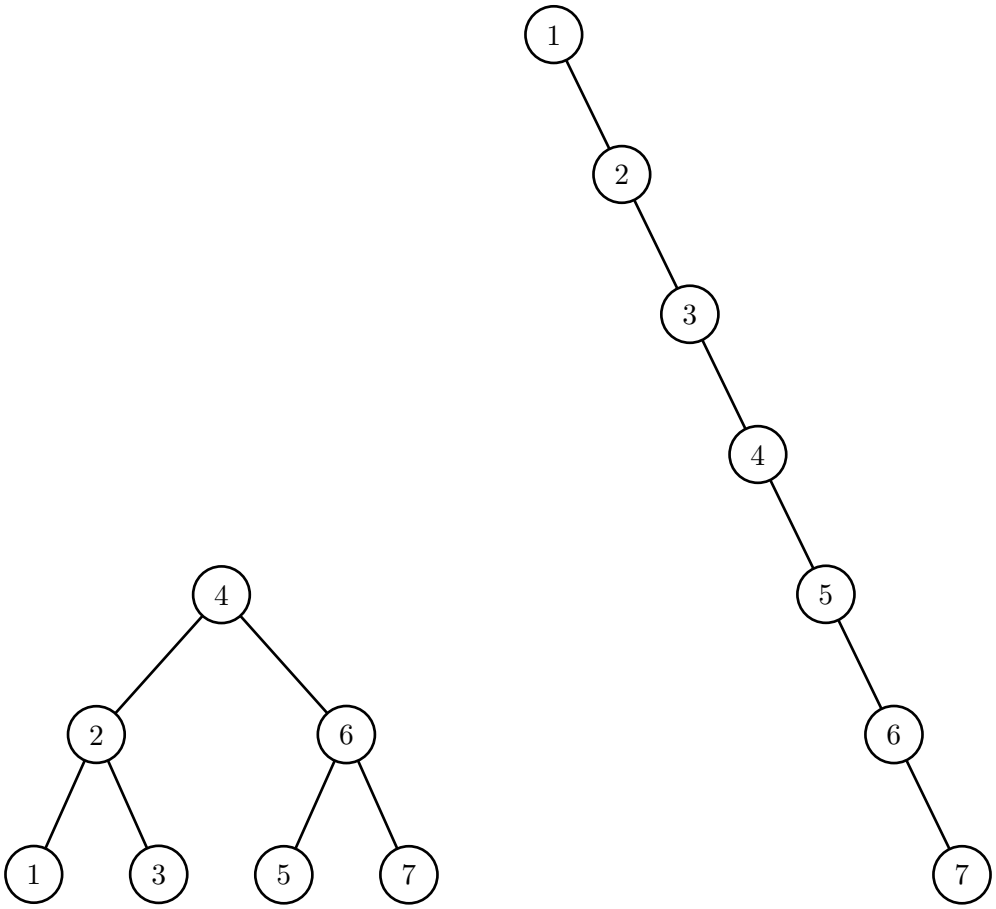
$h = n-1$, so that means $O(h) = O(n-1) = O(n)$. Since constants are insignificant in measuring an algorithm's runtime, meaning that $O(h) \equiv O(n)$

12 BST Balance

Previously, we discussed that $O(h) \equiv O(n)$ in the worst case scenario. When comparing our BST runtime analysis to our runtime analysis of a `LinkedList`, it's not actually any better, and is in fact almost the same.

The height of a BST depends on the order in which its nodes were inserted into it

Take a look at **Figure 12.1**, where the same seven keys are inserted in two different orders: the mixed order gives a tree of height 2, but the sorted order gives a stick of height 6.



Insert 4, 2, 6, 1, 3, 5, 7: height 2 Insert 1, 2, 3, 4, 5, 6, 7: height 6

Figure 12.1: Example: the same seven keys. Inserted in a mixed order they form a balanced tree of height $\approx \log_2 n$; inserted in sorted order every node gets only a right child, giving a "stick" of height $n - 1$ that behaves like a linked list

Therefore, a BST can still outperform a linked list because its best-case scenario occurs when the tree is **balanced**, resulting in a height that grows **logarithmically** with the number of nodes. In contrast, a `LinkedList`'s best-case scenario still involves traversing a length that grows **linearly** with the number of elements

This means BST operations can be done in $O(\log(n))$ time only when **balanced**, whereas a linked list requires $O(n)$ time even in the best case.

For all BST's:

- Its **lower bound** (best case scenario) occurs when $h \geq O(\log n) \equiv h \geq O(\log_2 n)$
- Its **upper bound** (worst case scenario) occurs when $h \leq O(n)$

Previously, we discussed why the upper bound must be at most $O(n)$. But, why is the lower bound at least $O(\log n)$?

To explain this, we must understand the structure of a perfect BST, which occurs when each level has twice the number of nodes as the previous level

- Level 0 (root) has 1 node
- Level 1 has 2 nodes
- Level 2 has 4 nodes
- $\vdots$
- Level h has 2^h nodes

This also means that level l has 2^l nodes

This forms a geometric series, where the number of nodes at level l, n_l, occurs as a sum of the geometric series of all nodes up to level l, where

$$n_l = 1 + 2 + 4 + \cdots + 2^l = \sum_{i=0}^{l} 2^i$$

Since $h = l$ at the maximum level of a tree, the total number of nodes in a BST, n, is 2^h, which can also be represented as a sum of the geometric series of all nodes up to level h, where

$$n = 1 + 2 + 4 + \cdots + 2^h = \sum_{i=0}^{h} 2^i$$

The sum of a geometric series is

$$1 + r + r^2 + \cdots + r^n = \sum_{i=0}^{n} r^i = \frac{r^{n+1} - 1}{r - 1}$$

Since $r = 2$,

$$\frac{2^{n+1} - 1}{2 - 1} = 2^{n+1} - 1$$

Solving for h,

$$
\begin{aligned}
n &= 2^{h+1} - 1 \\
n + 1 &= 2^{h+1} \\
\log_2(n+1) &= h + 1 \\
h &= \log_2(n+1) - 1
\end{aligned}
$$

Ignoring constants, the best case scenario of a BST occurs at a runtime of $O(\log n)$

Therefore, BSTs can run faster, only if their height remains close to $O(\log n)$

As discussed earlier, a BST's height depends on the number of nodes inserted into it.

If we assume that we have random numbers being inserted into our BST, and choose those random numbers, how many different ways are there?

The simple answer is that there is a lot. Mathematically speaking, if we put n random numbers into our tree, that means we have n choices to choose from for the first number to insert, $n-1$ for the second (since the first number is now gone), and so on. This means that we have $n!$ sequences to insert into our BST, which results in n unique numbers.

If we looked at every possible sequence and computed its height, its average is $O(\log n)$

This is because inserting keys in a random order splits the tree roughly in half at each step, where the tree's height grows the number of times you can halve the keys, which is about $\log_2(n) \equiv \log(n)$

This is delving into the average case runtime of a BST because the time complexity depends on the tree's height, and since the average height of a BST built from random insertions is about $O(\log n)$, the average case runtime for these operations is also $O(\log n)$

The table below summarizes BST analysis with the average case

Operation	**BST Average Case**	**BST Worst Case**	**Sorted List**	**Unsorted List**
`find`	$O(\log n)$	$O(h) \equiv O(n)$	$O(\log n)$	$O(n)$
`insert`	$O(\log n)$	$O(h) \equiv O(n)$	$O(n)$	$O(1)$
`remove`	$O(\log n)$	$O(h) \equiv O(n)$	$O(n)$	$O(n)$
`traverse`	$O(n)$	$O(n)$	$O(n)$	$O(n)$

We discussed the BST average cases, which make sense when thinking about it in context of permutations.

Some other things to note

1. `find` - Sorted List `find` is $O(\log n)$. This is basically performing binary search, since binary search is a more optimal search when a list is sorted. Similarly, unsorted list `find` is $O(n)$, which is simply linear search, which the worst case occurs when the element is found after traversing n times
2. `insert` - Unsorted is $O(1)$, since we can just append an element to the back of a list without maintaining an order, while sorted is $O(n)$ since we have to first insert the element to the back and then shift elements up to n times to make space for the new element

3. `remove` and `traverse` are $O(n)$ for both sorted and unsorted lists since the worst case scenario occurs when having to traverse a maximum of n times

This explains the notion of **balancing** a BST. We want to balance a BST so that its operations come with a cheaper cost of $O(\log n)$

To achieve this, we aim for a tree where the number of nodes in the left and right subtrees of every node is roughly equal. This ensures that the height of the tree remains logarithmic in terms of the number of nodes, preventing degeneration into a `LinkedList`

But how??

To **find the balance of a BST node**, you use the concept of a **balance factor** (b), where

$$b = \text{height}(T_R) - \text{height}(T_L)$$

When $b < 0$, the left subtree is taller; when $b > 0$, the right subtree is taller; and when $b = 0$, the tree is perfectly balanced

If $|b| > 1$, then the tree is unbalanced

Therefore, a tree is height balanced if $|b| \leq 1$ or $|b| < 2$ at every node. Comparing the two trees in **Figure 12.2**, every node on the left passes this check, but the highlighted 8 on the right has $b = -2$.

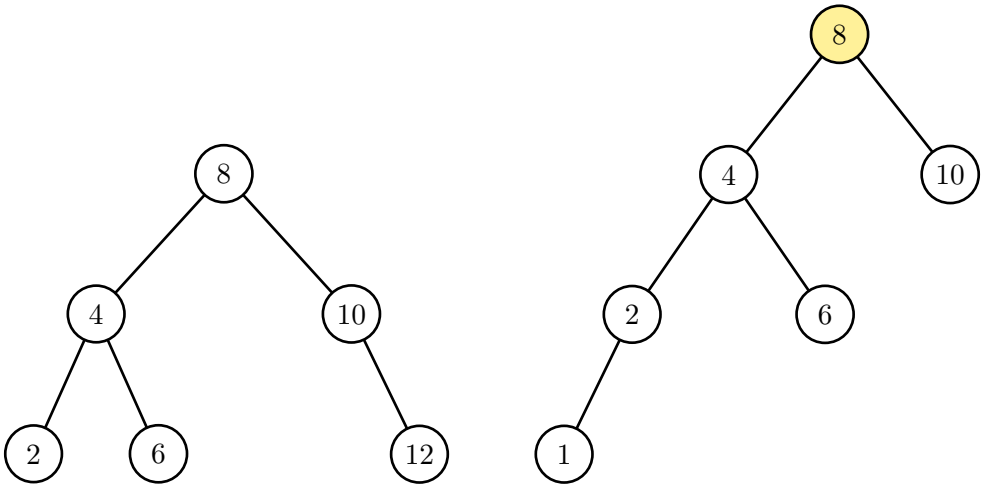


Figure 12.2: Example: on the left, 8 has $b = 1 - 1 = 0$ and 10 has $b = 0 - (-1) = 1$, so every $|b| \leq 1$. On the right, 8 has $b = 0 - 2 = -2$, so the tree is unbalanced even though every other node is fine

13 BST Rotations

A **BST rotation** is a strategy to help change the structure of a BST to help **rebalance** the tree without violating the BST property.

Two key details:

1. Maintain the BST property
   - `node->left < root`
   - `node->right > root`
2. Change a "stick" (every node basically has only one child) into a "mountain" (roughly equal subtrees)

When a BST becomes very imbalanced, where there are plenty of more nodes on one side than another, you can perform **rotations** to rebalance it by redistributing nodes so that the left and right subtrees are more evenly balanced (this is how "sticks" turn into "mountains")

Rotations:

- **Rotate nodes around a pivot** to "move" some nodes from the heavier side with more nodes to the lighter side with less nodes
- After rotation, the tree height decreases or the balancing improves
- The goal is to **evenly split nodes** (or get as close as possible) between left and right subtrees at that pivot node

For example, if the left subtree is much taller than the right, a right rotation can lift the left child up and move the original root down to the right subtree, balancing nodes between the two sides, like the R rotation in **Figure 13.1**, where 4 becomes the new root and its old right child 5 moves over to 6.

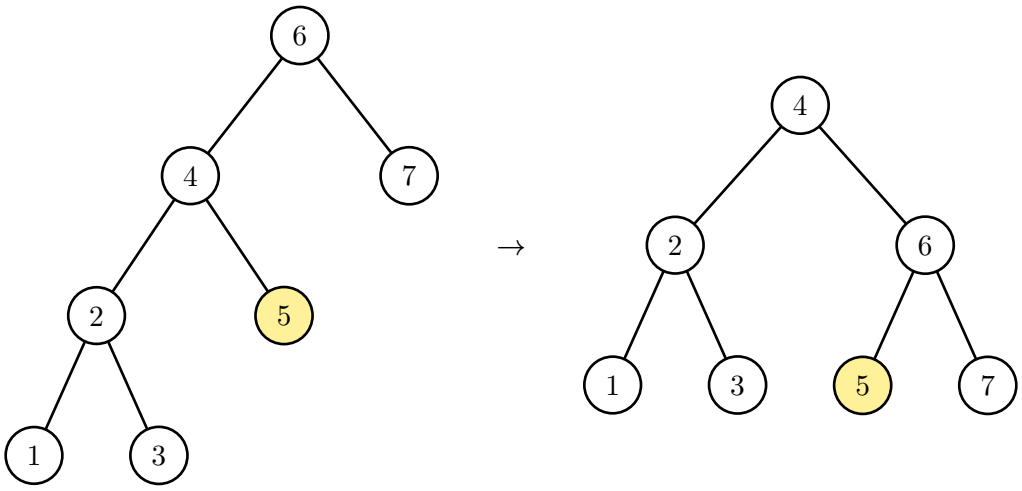


Figure 13.1: Example: an R rotation about 6. Node 4 moves up, 6 moves down to the right, and 4′s right subtree (5) is handed over to become 6′s left subtree

There are a total of **four** rotations: L (left), R (right), LR (left-right), RL (right-left)

- **L** (Left Rotation) - used when the right subtree is too tall and forms a **"right stick"** (due to its shape). This is known formally as the Right-Right case
- **R** (Right Rotation) - used when the left subtree is too tall and forms a **"left stick"** (due to its shape). This is known formally as the Left-Left case
- **LR** (Left-Right Rotation) - Used when there's imbalance in the left-right direction and forms a **"left elbow"** (due to its shape). This is known formally as the Left-Right case, where the left child has a taller right subtree
  - Perform a **left** and then **right** rotation
- **RL** (Right-Left Rotation) - Used when there's imbalance in the right-left direction and forms a **right elbow** (due to its shape). This is known formally as the Right-Left case, where the right child has a taller left subtree
  - Perform a **right** and then **left** rotation

If you're having trouble picturing these shapes, **Figure 13.2** draws all four with just three nodes, labeled with the rotation that fixes each one.

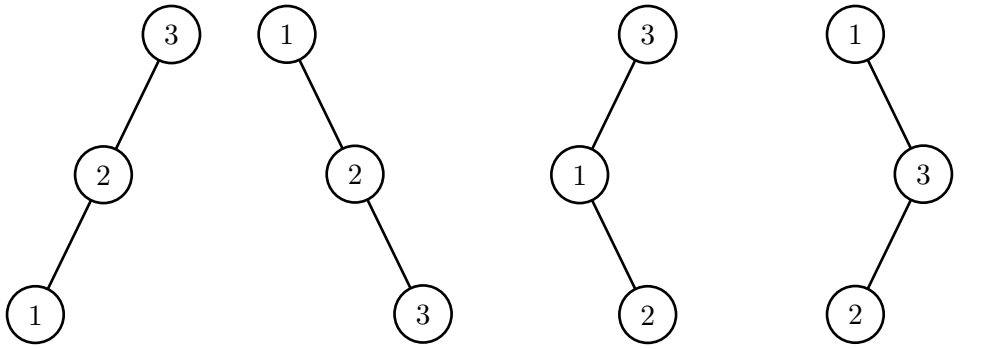


Figure 13.2: The four imbalanced shapes and the rotation that fixes each

Other things to know about rotations:

- All rotations are local, meaning subtrees aren't impacted
- All rotations are constant time, $O(1)$ because they only rearrange a **fixed, small number of pointers** (either a node, its child, and their subtrees); the amount of work doesn't grow with the size of the tree
- The BST property is maintained

The goal is to get $h \equiv O(\log n)$. These types of trees are called **Adelson-Velsky Landis trees** (or AVL trees for short), which are a specific type of **BBST** (Balanced Binary Search Tree)

14 Adelson-Velsky Landis (AVL) Trees

Adelson-Velsky Landis trees, or simply **AVL trees** are **self-balancing binary search trees** that automatically keep themselves balanced through rotations after insertions and deletions, ensuring efficient operations

This means that we have to consider 3 things:

- Rotations
- Maintaining Height
- Detecting Imbalance - check subtree height in $O(1)$ time and compute balance in $O(1)$ time

We can compute balance by having each subtree store its height, which means that I can retrieve it in constant time

Recognize the patterns to figure out which rotation to use

If insertion occurred on a subtree, t, where a **"right stick"** formed (two consecutive right children and no left children), meaning that its right subtree was heavier, we would need to do a **L** rotation about t to restore the balance of the tree. Notice in **Figure 14.1** how 2 becomes the new root once we rotate left about 1.

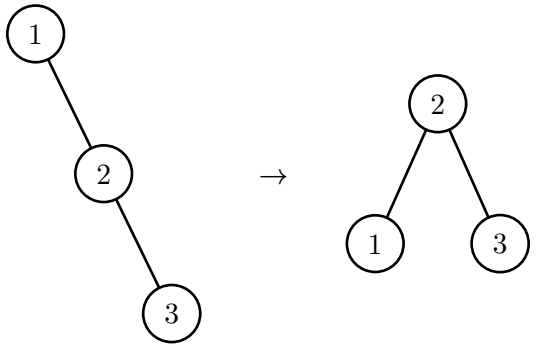


Figure 14.1: A right stick fixed by an L rotation about 1

$h = x$ prior to the left rotation and is now $h = x + 1$, where x is the height of the right subtree

We leverage this by noting that the balance factor, b of t is $+2$ and the balance factor of `t->right` is 1, which is greater than 0 (`t->right > 0`)

If insertion occurred on a subtree, t, where a **"right elbow"** formed (right child has taller left subtree), we would need to do a **RL** rotation about t to restore the balance of the tree (see **Figure 14.2**, where the R rotation about 3 first turns the elbow into a right stick).

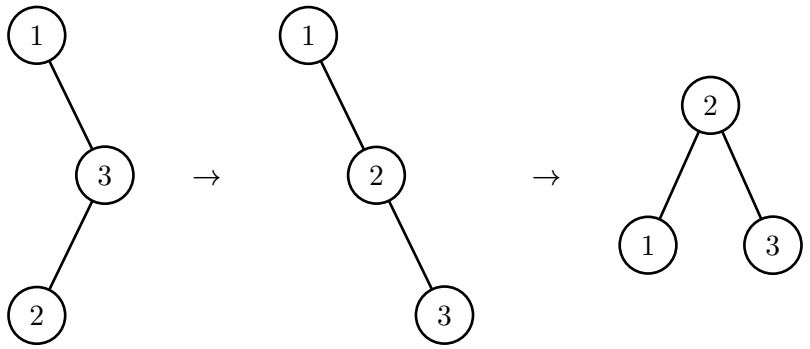


Figure 14.2: A right elbow fixed by an RL rotation: an R rotation about 3, then an L rotation about 1

We leverage this by noting that the balance factor, b of `t->right` is -1, which is less than 0 (`t->right < 0`)

The opposites are true for a **"left stick"** and **"left elbow"**, where we would need **R** and **LR** rotations respectively. If you compare **Figure 14.3** and **Figure 14.4** with the right side versions above, they're just mirror images

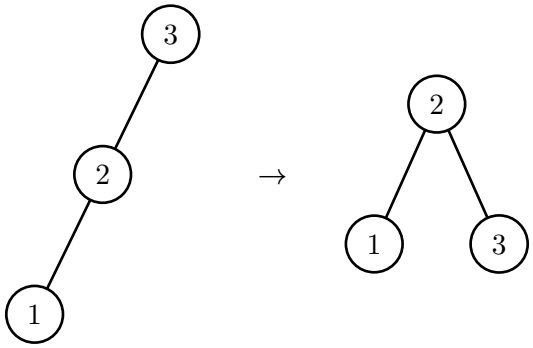


Figure 14.3: A left stick fixed by an R rotation about 3

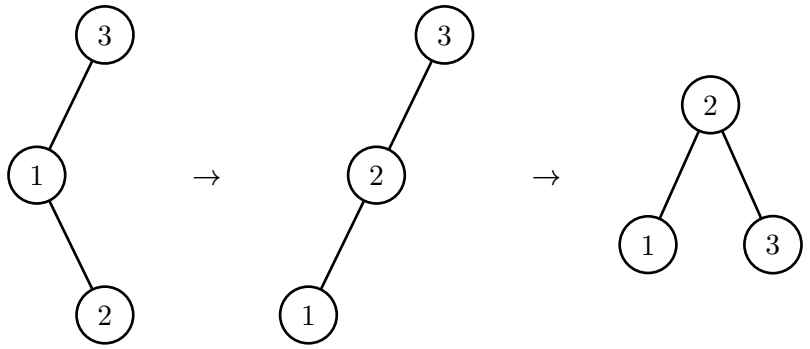


Figure 14.4: A left elbow fixed by an LR rotation: an L rotation about 1, then an R rotation about 3

14.1 Insertion into an AVL tree

With the knowledge in BST rotations, to insert a node into an AVL tree, we know we have to do the following steps

1. Insert at proper place
2. Check for imbalance
3. Rotate, if necessary
4. Update height

We already established that we need each `TreeNode` in an AVL tree to track its height. Therefore, we add a `height` member to each `TreeNode`

```
struct TreeNode {
    K key;
    V value;

    // have each `TreeNode` store the height in order to self balance
    unsigned int height;

    TreeNode* left;
    TreeNode* right;

    // a new node is a leaf, so it has height 0 and no children
    TreeNode(const K& key, const V& value)
        : key(key), value(value), height(0), left(nullptr), right(nullptr) {}
};
```

Inserting a node into an AVL tree is very similar to a BST (since it is in fact a type of BST!), as shown below

```
// AVL.hpp

/** Inserts a node as in a BST, without rebalancing yet
 * Runtime: O(h)
 */
template <typename K, typename V>
void AVL<K, V>::_insert(const K& key, const V& value, TreeNode*& curr) {
    // found the empty spot where the new node belongs
    if(curr == nullptr) {
        curr = new TreeNode(key, value);
        return;
    }

    // maintain BST property
    if(curr->key > key) {
        _insert(key, value, curr->left);
    } else if(curr->key < key) {
        _insert(key, value, curr->right);
```

```
    }

    // we still need to ensure the balance of the AVL tree here
}

/** Inserts a key-value pair into the AVL tree
 * Runtime: O(h)
 */
template <typename K, typename V>
void AVL<K, V>::insert(K key, V value) {
    _insert(key, value, root);
    node_map[key] = value;
}
```

The code above ensures the AVL tree maintains BST properties by descending into the left subtree if the key is smaller and descending into the right subtree if the key is greater

However, we are missing one crucial detail: ensuring that the AVL tree is **balanced**. To do this, we need to calculate the **balance factor** and then based on the balance factor, perform the appropriate rotation

The balancing function is outlined below

```
// AVL.hpp

/** Rotates `curr`'s subtree if it is unbalanced (rotations not filled in yet)
 * Runtime: O(1)
 */
template <typename K, typename V>
void AVL<K, V>::_ensureBalance(TreeNode*& curr) {
    // calculate the balance factor
    int balance = height(curr->right) - height(curr->left);

    // check if the node is currently not in balance
    if(balance == -2) { // heavy left ("left stick" or "left elbow" shaped
subtree)
        int leftBalance = height(curr->left->right) - height(curr->left->left);
        if(leftBalance <= 0) {
            // perform right rotation
        } else {
            // perform left-right rotation
        }
    } else if(balance == 2) { // heavy right ("right stick" or "right elbow"
shaped subtree)
        int rightBalance = height(curr->right->right) - height(curr->right->left);
        if(rightBalance >= 0) {
            // perform left rotation
        } else {
            // perform right-left rotation
```

```
        }
    }

    _updateHeight(curr);
}
```

This is a lot of code, but what we are essentially doing is calculating the balance factor so we know which rotation to perform!

Compute the balance of the tree to determine the shape of a stick

- If the balance is -2, we know that the shape of the left subtree is either a **"left stick"** or **"left elbow"**
  - We then need to compute the balance of the left subtree to figure out specifically which one it is
  - A left subtree balance of -1 means that its a **"left stick"**, so we need to do a **right rotation**
  - A left subtree balance of $+1$ means that its a **"left elbow"**, so we need to do a **left-right rotation**
- Similarly, if the balance is $+2$, we know that the shape of the right subtree is either a **"right stick"** or **"right elbow"**
  - We then need to compute the balance of the right subtree to figure out specifically which one it is
  - A right subtree balance of $+1$ means that its a **"right stick"**, so we need to do a **left rotation**
  - A right subtree balance of -1 means that its a **"right elbow"**, so we need to do a **right-left** rotation
  - After a `remove`, the child's balance can also be 0. That case is fixed with a single rotation too, which is why the code checks `rightBalance >= 0` (and `leftBalance <= 0` on the left side) instead of exactly ± 1

We need a helper height function in order to avoid any crashes. While `curr->left->height` and `curr->right->height` do work, they may crash if `curr->left` and `curr->right` are null.

Therefore, a helper function to get the height of a node can help prevent any unnecessary crashes. It simply checks if the node is not null before calling its height property, and returns -1 for a null node, since the height of a null tree is -1. The code is shown below

```
// AVL.hpp

/** Returns the height of a node, or -1 for a null node
 * Runtime: O(1)
 */
template <typename K, typename V>
int AVL<K, V>::height(TreeNode* node) {
    return node != nullptr ? node->height : -1;
}
```

We also need a helper function to update the height after performing necessary rotations

The updated node's height is 1 plus the max height of its left and right subtrees

Height represents the number of edges in the longest path from that node down to a leaf. When calculating this longest path, you need to consider that it will go through either the left subtree or the right subtree, depending on whichever is taller. This is performed by taking the maximum height between the two subtrees

However, to get the total path length from the current node, you must add one additional edge, because that accounts for descending into the taller subtree, which `_updateHeight` does below

```
// AVL.hpp

/** Recomputes the height of `curr` from its children
 * Runtime: O(1)
 */
template <typename K, typename V>
void AVL<K, V>::_updateHeight(TreeNode* curr) {
    // calculate height as 1 + maximum height of children
    // the +1 accounts for the extra edge needed to descend into the subtree with
a greater height
    // this works after any tree modification because height depends on the
current tree structure
    if(curr != nullptr) {
        curr->height = 1 + std::max(height(curr->left), height(curr->right));
    }
}
```

From here, the rest is simple. We just have to simply code each rotation. The rotations intuitively make sense to code if you know what's going on

1. **Left Rotation**
   - The right child becomes the new root of the subtree
   - The new root's left subtree becomes the old root's right child
   - The old root becomes the left child of the new root of the subtree
   - Update the heights of the old root and then the new root, since only those two nodes moved
2. **Right Rotation**
   - The left child becomes the new root of the subtree
   - The new root's right subtree becomes the old root's left child
   - The old root becomes the right child of the new root of the subtree
   - Update the heights of the old root and then the new root, since only those two nodes moved
3. **Left-Right Rotation**
   - Perform a left rotation about the left child
   - Perform a right rotation about the root
4. **Right-Left Rotation**
   - Perform a right rotation about the right child
   - Perform a left rotation about the root

Coding these then becomes easy. Below are `_rotateLeft`, `_rotateRight`, `_rotateLeftRight`, and `_rotateRightLeft`, in that order

```
// AVL.hpp

/** Rotates the subtree at `root` to the left
 * Runtime: O(1)
 */
template <typename K, typename V>
void AVL<K, V>::_rotateLeft(TreeNode*& root) {
    // the right child becomes the new root
    TreeNode* newRoot = root->right;

    // move `newRoot`'s left subtree to old root's right subtree
    root->right = newRoot->left;

    // the old root becomes the left child of the new root
    newRoot->left = root;

    // the old root is now below the new root, so update its height first
    _updateHeight(root);
    _updateHeight(newRoot);

    // update pointer to point to the new root
    root = newRoot;
}
```

```
// AVL.hpp

/** Rotates the subtree at `root` to the right
 * Runtime: O(1)
 */
template <typename K, typename V>
void AVL<K, V>::_rotateRight(TreeNode*& root) {
    // the left child becomes the new root
    TreeNode* newRoot = root->left;

    // move `newRoot`'s right subtree to old root's left subtree
    root->left = newRoot->right;

    // the old root becomes the right child of the new root
    newRoot->right = root;

    // the old root is now below the new root, so update its height first
    _updateHeight(root);
    _updateHeight(newRoot);

    // update pointer to point to the new root
```

```
    root = newRoot;
}
```

```
// AVL.hpp

/** Rotates left about the left child, then right about `root`
 * Runtime: O(1)
 */
template <typename K, typename V>
void AVL<K, V>::_rotateLeftRight(TreeNode*& root) {
    // rotate left about the left child and then rotate right about the root
    _rotateLeft(root->left);
    _rotateRight(root);
}
```

```
// AVL.hpp

/** Rotates right about the right child, then left about `root`
 * Runtime: O(1)
 */
template <typename K, typename V>
void AVL<K, V>::_rotateRightLeft(TreeNode*& root) {
    // rotate right about the right child and then rotate left about the root
    _rotateRight(root->right);
    _rotateLeft(root);
}
```

This updates our balance function as follows

```
// AVL.hpp

/** Rotates the subtree at `curr` if it is out of balance
 * Runtime: O(1)
 */
template <typename K, typename V>
void AVL<K, V>::_ensureBalance(TreeNode*& curr) {
    // calculate the balance factor
    int balance = height(curr->right) - height(curr->left);

    // check if the node is currently not in balance
    if(balance == -2) {
        int leftBalance = height(curr->left->right) - height(curr->left->left);
        if(leftBalance <= 0) { // heavy left ("left stick" shaped subtree)
            _rotateRight(curr);
        } else { // "left elbow" shaped subtree
            _rotateLeftRight(curr);
        }
```

```
    } else if(balance == 2) {
        int rightBalance = height(curr->right->right) - height(curr->right->left);
        if(rightBalance >= 0) { // heavy right ("right stick" shaped subtree)
            _rotateLeft(curr);
        } else { // "right elbow" shaped subtree
            _rotateRightLeft(curr);
        }
    }

    // update the height of `curr`, which is the root of this subtree after any
rotation
    _updateHeight(curr);
}
```

Finally, `_insert` calls `_ensureBalance` on every node along the path back up from the new node, so any imbalance is fixed right where it appears

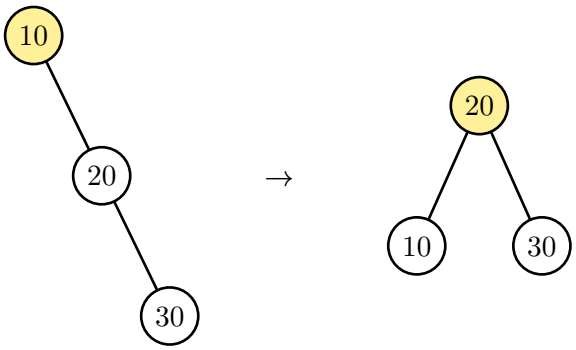
```
// AVL.hpp

/** Inserts a node and rebalances every node on the way back up
 * Runtime: O(h) = O(log n)
 */
template <typename K, typename V>
void AVL<K, V>::_insert(const K& key, const V& value, TreeNode*& curr) {
    // found the empty spot where the new node belongs
    if(curr == nullptr) {
        curr = new TreeNode(key, value);
        return;
    }

    // maintain BST property
    if(curr->key > key) {
        _insert(key, value, curr->left);
    } else if(curr->key < key) {
        _insert(key, value, curr->right);
    }

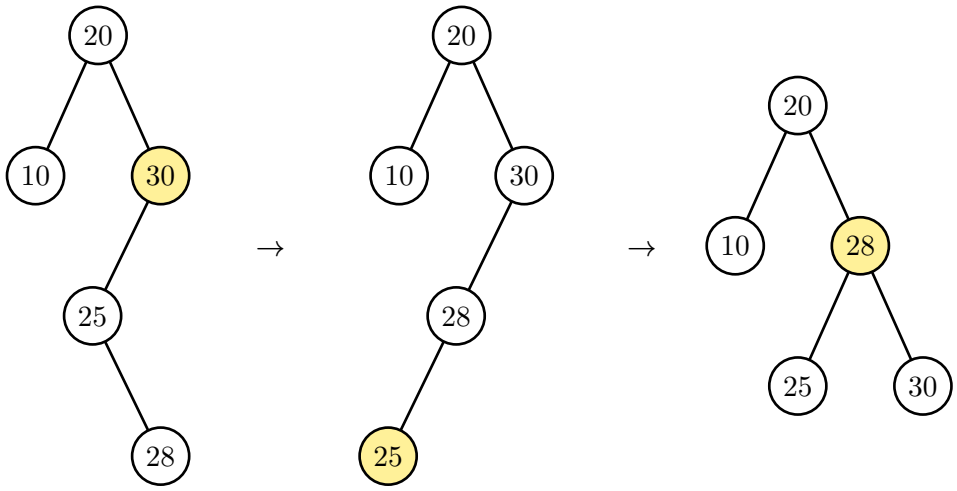
    // ensure the balance of the AVL tree (this also updates the height of `curr`)
    _ensureBalance(curr);
}
```

For example, inserting the keys 10, 20, 30, 25, 28 in that order triggers two rebalances. The first happens right after 30 goes in, when 10 becomes a right stick and gets an L rotation (see **Figure 14.5**). The second happens after 28, when 30 becomes a left elbow and needs an LR rotation, which you can follow step by step in **Figure 14.6**.



Insert 10, 20, 30 L rotation about 10

Figure 14.5: Example: inserting 10, 20, 30 forms a right stick at 10 (balance +2), fixed by an L rotation



Insert 25, then 28 L rotation about 25 R rotation about 30

Figure 14.6: Example: inserting 25 and then 28 forms a left elbow at 30 (balance −2, left child balance +1), fixed by an LR rotation

I will not code the remaining functionalities, including `find` and `remove`. `find` is identical to a regular BST, so please see the BST implementation for it. `remove` starts out exactly like BST `remove`, but it is **not** identical: removing a node can unbalance every ancestor on the path back up to the root. Just like `_insert`, `_remove` must call `_ensureBalance` on each node as the recursion unwinds, and a single `remove` may need a rotation at several of those nodes (up to $O(\log n)$ rotations), while an `insert` needs at most one.

14.2 AVL Tree Analysis

`insert`, `remove`, and `find` also all run in $O(h)$. Recall that this is because in the worst case, we may need to traverse from the root down to a leaf node, visiting $h + 1$ nodes.

However, we will now argue that $h = O(\log n)$, using the balance property to prove this

Recall from discrete math, the definition of big-O states that $f(n)$ is $O(g(n))$ iff $\exists_{c,k}$ such that $f(n) \leq c \cdot g(n)$ for all $n > k$

Or simply, the worst-case growth rate of a function as the input gets very large. When I give it more data, how does it slow down?

Let $h(n)$ be the height of an AVL tree as a function of n, the number of nodes. However, we can't directly compute this function, since tree height depends on not only the number of nodes, but also on the structure of the tree.

In other words, 2 AVL trees can have 4 nodes, but be fundamentally different in structure

Since we can't compute the exact height function, $h(n)$, we analyze the worst-case scenario: which is the tallest possible tree with n nodes, or simply $h_{\max}(n)$

But directly computing $h_{\max}(n)$ is also difficult. There are too many possible tree shapes to check them all. Instead, we approach the problem indirectly by taking the inverse and finding the **minimum number of nodes** required to build an AVL tree of height h

This makes sense because when I find the lower bound of the inverse, $h_{\min}(n)$, I essentially get the upper bound on $h_{\max}(n)$

The number of nodes in the tree, $f^{-1}(h)$ will always be **greater than** $c \times g^{-1}(h)$ for all values where $n > k$

This is because if one function grows slower than another, its inverse must grow faster, so to reach the same output, the slower-growing function requires a larger input, making its inverse larger (up to a constant factor) for sufficiently large values

To find the lower bound on n given h, we can begin by defining a function given h, which describes the **smallest number of nodes in an AVL tree** given height h

NODES(h) is the minimum number of nodes in an AVL tree of height h.

- When $h = -1$, NODES(h) $= 0$ because the tree is **empty**
- When $h = 0$, only one tree exists because only the root node exists in such tree, so NODES(h) $= 1$
- When $h = 1$, there are three possible trees: a tree with only a **left child**, a tree with only a **right child**, and a tree with **both** a **left and right** child. NODES(h) $= 2$ since the minimum number of nodes required for height 1 is 2
- When we get to $h = 2$, however, I get a lot of possible trees and not all of them are balanced.
  - However, there is one possible balanced tree that can be formed with a minimum of four nodes, where the **root's left child also contains a right child**
  - This forms a minimum of 4 nodes, where NODES(h) $= 4$, as in **Figure 14.7** (the root's left child 1 has the right child 2)

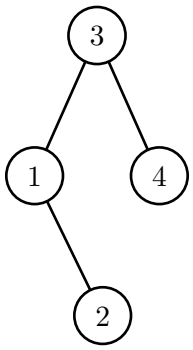


Figure 14.7: A minimum-size AVL tree of height 2

Assuming that $N(h)$ is an alias for NODES(h), we build a recurrence relation, where

$$N(h) = 1 + N(h-1) + N(h-2)$$

If we build the first few minimum-size AVL trees with this recurrence, we get **Figure 14.8**. Notice how the tree for $N(3)$ is just a root with the $N(2)$ tree on its left and the $N(1)$ tree on its right.

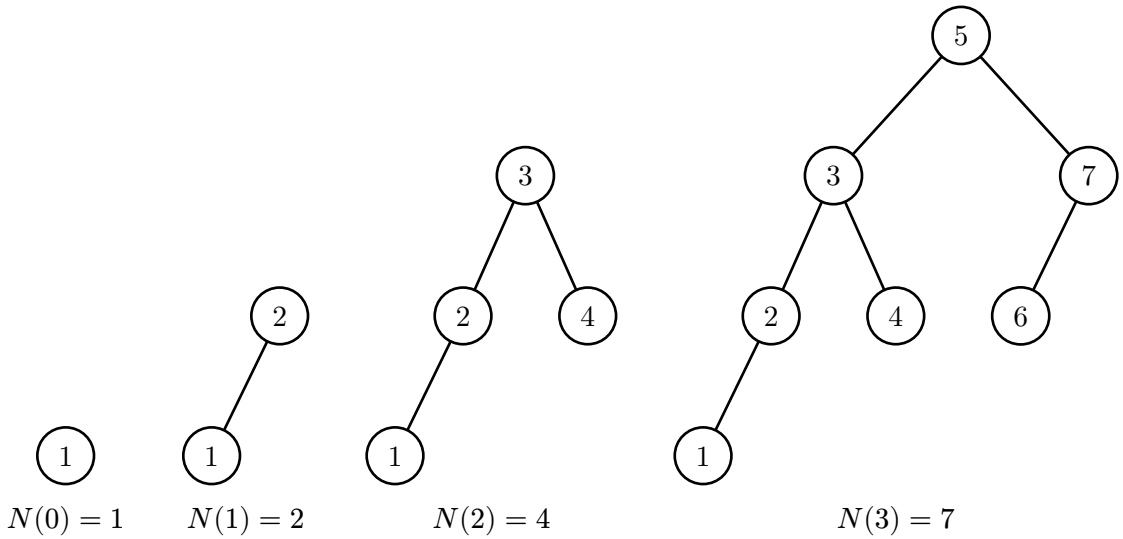


Figure 14.8: Minimum-size AVL trees of heights 0 to 3: each is a root, a minimum tree of height $h-1$ on the left, and a minimum tree of height $h-2$ on the right

We can first argue that $N(h) = 1 + N(h-1) + N(h-2)$, which means that $N(h) > 2N(h-2)$. This is important so that we establish a lower bound on the minimum required number of nodes for height h

We can argue that the minimum number of nodes $N(h)$ in an AVL tree of height h is equal to the sum of one root node plus the minimum nodes of its two subtrees: $N(h) = 1 + N(h-1) + N(h-2)$.

Since $N(h-1) > N(h-2)$, this implies $N(h) > 1 + 2N(h-2)$ because $N(h-1)$ is strictly greater than $N(h-2)$

For larger h, the constant 1 becomes insignificant, so we simplify this to $N(h) > 2N(h-2)$, showing that $N(h)$ grows exponentially as the height increases

With this, we can prove a closed form of $2N(h-2)$

Prove that an AVL tree of height h has at least $2^{\frac{h}{2}}$ nodes.

Consider an AVL tree and let h denote its height. Start with $h = 1$

$$N(1) = 2^1 = 2$$

$$2^{\frac{1}{2}} = \sqrt{2}$$

and $2 \geq \sqrt{2}$

This means that an AVL tree with height 1 has at least $\sqrt{2}$ nodes

Suppose $h = 2$

$$\begin{aligned} N(2) &= 1 + N(2-2) + N(2-1) \\ &= 1 + 1 + 2 \\ &= 4 \end{aligned}$$

$$2^{\frac{2}{2}} = 2^1 = 2$$

and $4 \geq 2$, which means that an AVL tree with height 2 has at least 2 nodes

By inductive hypothesis

$\forall_{j<h}$, $N(j) \geq 2^{\frac{j}{2}}$, where we will prove via inductive step that

$$N(h) = N(h-1) + N(h-2) + 1 > 2N(h-2)$$

$$2N(h-2) \geq 2 \times 2^{\frac{h-2}{2}} = 2^{\frac{h}{2}}$$

An AVL tree of height h has at least $2^{\frac{h}{2}}$ nodes.

via induction, we have shown that $n \geq N(h) \geq 2^{\frac{h}{2}}$, where inverting this results in

$$h \leq 2 \log n$$

meaning that

$$h \equiv O(\log n)$$

14.3 Summarizing AVL and BBST

Red-Black Trees

- BBST's where max height is equal to $2 \cdot \log n$
- Constant number of rotations on `insert`, `remove`, and `find`, $O(1)$

AVL Trees

- **Max Height**: $1.44 \cdot \log n$
- **Rotations**: $O(h)$

Red-Black trees outperform AVL trees in rotations. They are also laid out well in memory that improves cache performance.

Red-Black trees are much more complicated to understand how they work under the hood

Previously, when we were constructing BST's, we were using `std::unordered_map` to track our nodes. This was so that we had access to a fast iterator. But there is actually something even more interesting about this.

C++ STL maps in fact provide us with a balanced BST that is in fact a red-black tree. When we use `operator[]` on a map, what we are essentially doing is searching for the key in a tree and

- **If the key exists**, it returns a reference to the corresponding value.
- **If the key doesn't exist**, it inserts the key with a default-constructed value and then returns a reference to that value.

But from a big-O standpoint, all operations are $O(\log n)$, considering the trees are **balanced**, which is a huge improvement from $O(n)$ of a regular BST

This is great for specific applications when looking for close matches, such as range queries, nearest-neighbor lookups, or finding the next largest or smallest key, all of which benefit from the in-order structure and efficient traversal of balanced binary search trees like red-black trees

14.4 Runtime Analysis for all Data Structures Covered

	Unsorted Array	Sorted Array	Unsorted List	Sorted List	Binary Tree	BST	AVL
`find`	$O(n)$	$O(\log n)$	$O(n)$	$O(n)$	$O(n)$	$O(h)$	$O(\log n)$
`insert`	$O(1)$	$O(n)$	$O(1)$	$O(n)$	$O(n)$	$O(h)$	$O(\log n)$
`remove`	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(h)$	$O(\log n)$
`traverse`	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$

15 Lambdas

> NOTE: This is a simple lesson reviewing iterators and lambda functions from CS 128!

15.1 Iterators

From 128, we know that there are **three** types of iterators:

1. **Forward**
   - Only moves in the **forward** direction,
   - **Operations**: `++it` (increment), `*it` (dereference), `==` and `!=` (comparison)
   - **Example**: Singly `LinkedList` iterator, where you can go through the list once from start to end, but can't go backwards
2. **Bidirectional**
   - Can move in **both directions** (forward and backward)
   - **Operations**: Everything from Forward, plus `--it` (decrement)
   - **Example**: Doubly linked list iterator and `std::set` iterator
   - You can traverse forward with `++` or backward with `--`
3. **Random Access**
   - Can **jump to any position** in constant time
   - **Operations**: Everything from Bidirectional, plus `it + n`, `it - n`, `it[n]`, `<`, `>`, etc.
   - **Example**: `std::vector` iterator, array pointers
   - You can do `it + 5` to jump 5 positions ahead, or `it[10]` to access element 10

15.2 Lambdas

In C++, a **lambda function** is an **anonymous function** you can define on the fly and inline. This means that I define and write the function right when I use it, instead of somewhere else

This is typically used for short snippets of code like callbacks, custom sorting, or quick operations that don't necessarily need a separate named function.

Lambda functions are so useful because they allow us to use functions as **data**. This means that the function itself is a value that can be stored, passed around, and manipulated!

Lambdas consist of 3 things

1. Capture, notated in `[ ]` - specifies which variables from the surrounding scope the lambda can access and whether it captures them by value (copy) or by reference
2. Parameter, notated in `( )` - The list of parameters with a type and name specified and separated with commas
3. Body, notated in `{ }` - The body of the lambda

For example,

```
auto isBig = [big](int num) { return num >= big; };
```

- `[ ]` contains the variables within scope that the lambda "captures" for it to use. In this example, this variable is `big`
- `( )` contains the parameters needed to pass into the function
- `{ }` contains the body of the function

Take a look at a popular function used in excel, `COUNTIF`. This function counts the number of cells in a range that meet a specific criteria

Suppose the criteria is that any integer, x, is less than 5. To code this in C++ we would do,

```cpp
bool lessThan5(int x) {
    return x < 5;
}

/** Counts the values in `data` that are less than 5
 * Runtime: O(n)
 */
int countIf(std::vector<int>& data) {
    return std::count_if(data.begin(), data.end(), lessThan5);
}
```

In **Figure 15.1**, for example, `count_if` calls `lessThan5` on every value from `begin()` up to `end()`, and only 3, 1, and 4 (highlighted) pass.

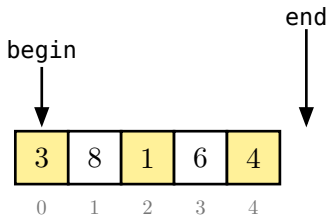


Figure 15.1: Example: `count_if` walks from `begin()` up to, but not including, `end()` and counts the highlighted values less than 5, returning 3

However, `lessThan5` is an unnecessary function when you can achieve the same result more concisely and directly using a lambda expression inline, avoiding extra named functions and keeping the code simpler and easier to read.

To solve this problem, we can use a **lambda**, like the one below

```cpp
auto lessThan5 = [](int x) { return x < 5; };
```

this makes our `countIf` function much more readable

```cpp
/** Counts the values in `data` that are less than 5
 * Runtime: O(n)
 */
int countIf(std::vector<int>& data) {
    auto lessThan5 = [](int x) { return x < 5; };
    return std::count_if(data.begin(), data.end(), lessThan5);
}
```

16 BBST Range Search and KD Trees

So far, we've used balanced BSTs as dictionaries that answer one question: is this key in the dictionary? But what makes a balanced BST special is that it also finds things that are **near** a key, not just the key itself

That makes BBSTs useful for **range-based** searches (e.g. finding every key that falls between $[11, 42]$) and **nearest-neighbor** searches (finding the closest point to some query point)

16.1 Range Searches in a BBST

Range queries are a search to find all keys that fall within a given range, typically written as $[x, y]$, where x is the lowest possible value (lower bound) and y is the highest possible value (upper bound)

Since a BBST keeps its keys in sorted order, a range query is just a `find` followed by an in-order traversal

1. Search the tree for x. If x is in the tree, we stop at it. Otherwise, we stop at the last node we visited before falling off the tree
2. Do an in-order traversal starting from that node, collecting every key that is in $[x, y]$
3. Stop as soon as the traversal reaches the first key larger than y

Take the tree in **Figure 16.1** and the range $[11, 42]$. We search down and find `11`, then walk the tree in order: 11, 19, 25, 33, 42. The next key is 44, which is past the range, so we stop. If we searched for $[10, 42]$ instead, the search for `10` would end at `6`, since 10 is not in the tree. We'd start the traversal there, just leave `6` out because it's less than 10, and get the exact same answer

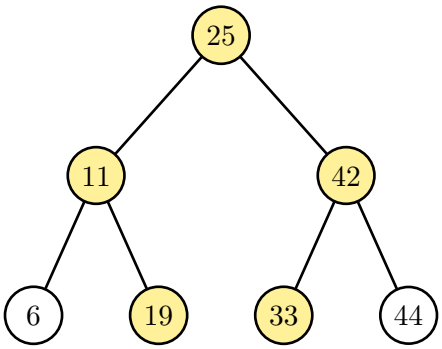


Figure 16.1: Example: a range search for $[11, 42]$ returns the highlighted keys. The traversal stops at 44, the first key past the range

The runtime is $O(\log n + k)$, where n is the number of keys in the tree and k is the number of keys in the answer. The $\log n$ is the initial search down the tree, and the k is the traversal. We can never do better than k anyways, since we have to actually return every key in the answer!

16.1.1 BBSTs vs. Sorted Lists

A sorted list and a balanced BST are very, very similar things. We could do the same range search on a sorted list with binary search followed by a linear scan

If we only build the structure once and then query it, they cost the same: sorting a list takes $O(n \log n)$ time, and inserting n things into a BBST takes $O(\log n)$ time each, or $O(n \log n)$ total

The difference shows up when the data keeps changing. A sorted list may need to be re-sorted before every query, whereas a BBST maintains its sorted order with only $O(\log n)$ work per change

16.2 `lower_bound` and `upper_bound`

Previously, we discussed how C++'s STL map is implemented via a red-black tree under the hood. So, in one dimension, `std::map` already does range searches for us!

`std::map` includes a `lower_bound` and an `upper_bound` function, which allow efficient logarithmic-time range queries by returning iterators

- `lower_bound` - The smallest element greater than or equal to (not less than) the argument
- `upper_bound` - The smallest element strictly greater than the argument

For the keys in **Figure 16.1**, `lower_bound(11)` returns 11, since 11 is the smallest key not less than 11. `lower_bound(10)` also returns 11, because the next smallest key, 6, is less than 10

`upper_bound(42)` returns 44, the key **after** 42. `upper_bound(41)` returns 42. So `lower_bound` includes the key if it's in the map, but `upper_bound` never does

That's exactly what a standard range-based loop wants: `lower_bound` is our `begin`, and `upper_bound` is our `end`, which is one past the last key we care about. Looping from `lower_bound(x)` to `upper_bound(y)` covers $[x, y]$, inclusive at the bottom and exclusive at the top, just like `begin()` and `end()`. You can see this in **Figure 16.2**, where the loop for $[10, 42]$ runs from index 1 up to, but not including, index 6

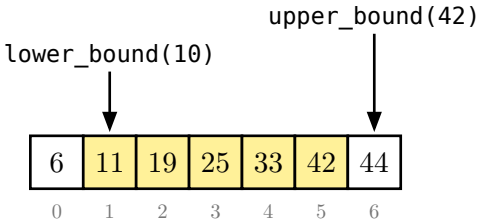


Figure 16.2: The sorted keys of a `std::map`. A loop from `lower_bound(10)` to `upper_bound(42)` visits exactly the highlighted keys

In code, this looks like

```cpp
std::map<int, std::string> m = {
    {6, "f"}, {11, "k"}, {19, "s"}, {25, "y"}, {33, "g"}, {42, "p"}, {44, "r"}
};

// every key in [10, 42]
for(auto it = m.lower_bound(10); it != m.upper_bound(42); ++it) {
    std::cout << it->first << " ";
}
// prints 11 19 25 33 42
```

16.3 Range Searches in Two Dimensions

One dimension is easy because we can sort. But what if our data is points in two dimensions, and we want to find every point inside a rectangle, like the one in **Figure 16.3**?

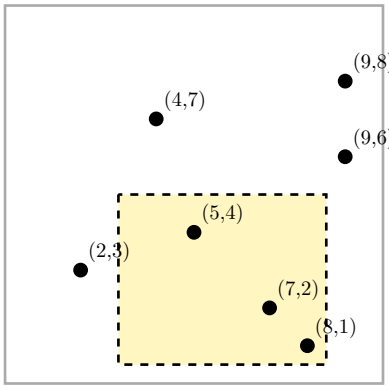


Figure 16.3: Example: which points are inside the dashed rectangle?

If we sort the points by x, we can tell whether a point is between the left and right sides of the rectangle, but not whether it's between the top and bottom. If we sort by y, we get the opposite problem. We could do both and then look at which points show up in both answers, but that means sorting the data twice (or building two complete trees), which is inefficient

This kind of query happens all the time in real code

- **Computer graphics** - We only want to draw the things in front, but to skip drawing what's hidden, we first have to detect what's behind
- **Collision detection** - Only check objects that are near each other
- **Computer vision** and **sensor data** - Only process the points in a region of interest

The other common question is **nearest neighbor**: given some point, which point in our data set is closest to it? The naive way is to check every point, because a point can be farther away in one dimension but so much closer in the other that it more than makes up for it

16.4 KD Trees

A ***k*-dimensional tree**, or **KD Tree**, is a binary tree that stores points in k-dimensional space, organizing them so that large portions of space can be skipped entirely during range and nearest-neighbor queries

The idea is to split the world in half, one dimension at a time. The root splits the points by x, its children split their halves by y, their children split by x again, and so on. Every level of the tree cuts the space into smaller and smaller regions

16.4.1 Building a KD Tree

To build a KD tree from k-D space

1. Start with one dimension (doesn't matter which one)
2. Find the **median** point along that dimension. It becomes the current root
3. Split the data set into two parts: points less than the median in that dimension go in the left subtree, and points greater than the median go in the right subtree
4. Recursively build the left and right subtrees, moving to the next dimension (e.g., if you split by x at this level, split by y at the next)
5. When a subtree has only one point left, that point is a leaf

Instead of fully sorting at every level, we find the median with **quickselect**, which you can think of as a "one-handed quicksort". It partitions the array just like quicksort does, but only recurses into the side that contains the index we care about. When it finishes, the median is at the middle index, everything smaller in that dimension is to its left (in no particular order), and everything larger is to its right. That takes linear time per level instead of $O(n \log n)$

Quickselect takes a comparison function, so we call it with one that compares points only by x at the first level, only by y at the next, and so on. We have to cycle through the dimensions evenly ($x, y, x, y, ...$ in 2D, or $x, y, z, x, y, z, ...$ in 3D). If we split by x twice in a row, the second split might not shrink the regions at all

Figure 16.4 shows the KD tree built from seven points. $(7, 2)$ is the median by x, so it splits the whole plane with a vertical line. On the left, $(5, 4)$ is the median of the remaining points by y, so it splits that half with a horizontal line, and $(9, 6)$ does the same on the right

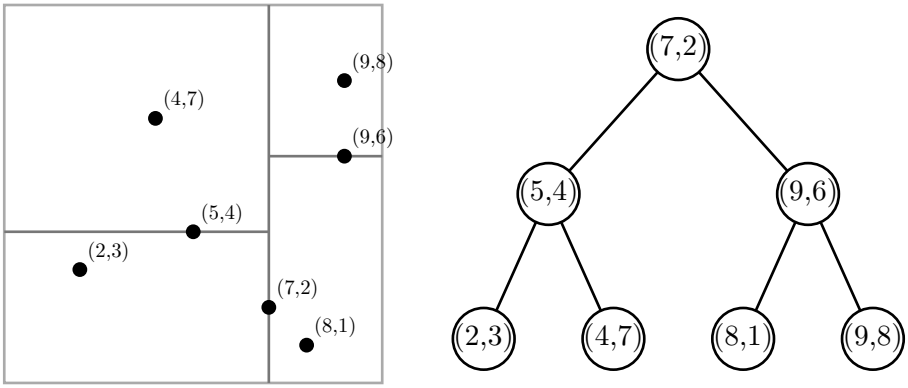


Figure 16.4: Example: the KD tree built from (2,3), (5,4), (9,6), (4,7), (8,1), (7,2), (9,8). The root splits on x and its children split on y

A KD tree stays balanced simply because of the median: at each step we split the data set using the median point along the current dimension, ensuring that half the points go to the left subtree and half go to the right subtree

The same idea works in any number of dimensions. That might seem ridiculous for physical space, but machine learning often describes data points with hundreds or thousands of **features** (color, shape, size, …), and each feature is just another dimension

16.4.2 Range Search in a KD Tree

To find every point inside a rectangle, we start at the root and work our way down

1. If the current point is inside the rectangle, add it to the answer
2. If the rectangle extends to the **smaller** side of the current split, search the left subtree
3. If the rectangle extends to the **larger** side of the current split, search the right subtree

In **Figure 16.5**, the rectangle crosses the root's split at $x = 7$, so we search both sides, and $(7, 2)$ itself is in the box. On the left, the rectangle crosses $y = 4$, so we check both $(2, 3)$ and $(4, 7)$. On the right, though, the rectangle is entirely below $y = 6$, so we only check $(8, 1)$ and never even look at $(9, 8)$

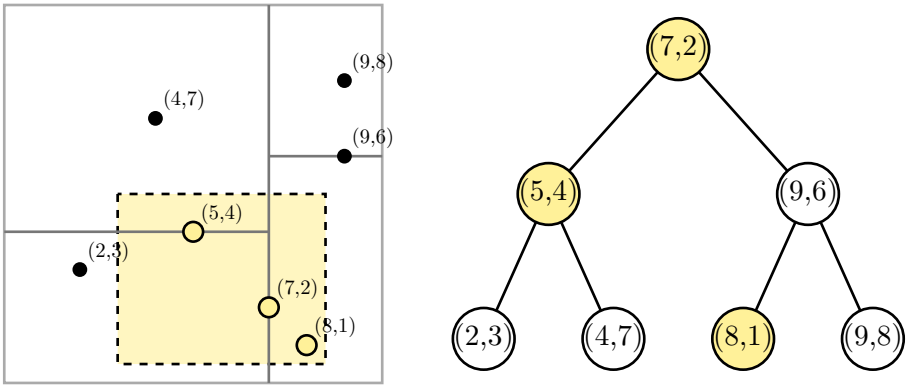


Figure 16.5: Example: a range search for the dashed rectangle finds the highlighted points. The rectangle never reaches above $y = 6$ on the right side, so the subtree at $(9, 8)$ is skipped

With a small tree, the savings don't look like much, but imagine a tree with thousands of points and a box that is small compared to the whole space. We'd only have to search a small number of the points

16.5 Nearest Neighbor Search

Given a query point q, we want the point in the tree that is closest to it, using the standard (Euclidean) sense of distance. The query point doesn't have to be in the tree, and usually isn't

We never actually take the square root when computing distances. The square root preserves order, so comparing squared distances gives the same answer, and skipping it has two benefits: square roots take work, and floating point math has errors, so we want to do as little of it as possible. Squaring integers doesn't even need floating point at all

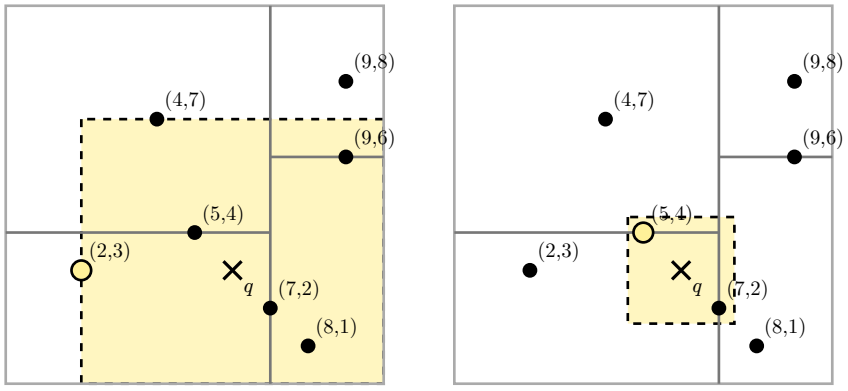
To find the nearest neighbor of q

1. Start at the root and move down the tree, choosing the child subtree that would contain q based on the current splitting dimension (just like `find`)
2. When we can't descend any further, the point we stopped at becomes our first **best guess**
3. Backtrack up the tree. At each node on the way back up
   1. If the node is closer to q than our best guess, it becomes the new best guess
   2. If the **other** side of the node's split could still hold a closer point, search that subtree too
4. Return the best guess once we're back at the root

How do we know whether the other side could hold a closer point? Any closer point has to be inside the circle around q whose radius is the distance to our best guess. Checking a circle (or a **hypersphere** in more than 3 dimensions) against a split is annoying, so we use the square around q instead (a **hyperrectangle**). The square contains the whole circle, so we never miss a point, and the check becomes a single comparison: does the square cross the splitting line in the current dimension? In other words, is $(q_d - s_d)^2$ no more than the best squared distance so far, where s is the node's point and d is its splitting dimension?

Let's find the nearest neighbor of $q = (6, 3)$ in the tree from **Figure 16.4**

1. At the root $(7, 2)$, we split by x. Since $6 < 7$, we go left
2. At $(5, 4)$, we split by y. Since $3 < 4$, we go left
3. $(2, 3)$ is a leaf, so it's our first best guess, with a squared distance of $(6-2)^2 + (3-3)^2 = 16$. The big square on the left of **Figure 16.6** shows every point that could still be closer



First guess $(2, 3)$ Better guess $(5, 4)$

Figure 16.6: Searching for the nearest neighbor of $q = (6, 3)$. Each dashed square holds every point that could still beat the current best guess, so it shrinks as the guess improves

4. Back up at $(5, 4)$, the squared distance is $1 + 1 = 2$, which beats 16, so $(5, 4)$ becomes our best guess and the square shrinks (right of **Figure 16.6**). The square still crosses the split at $y = 4$, so we have to check the other side, $(4, 7)$. Its squared distance is 20, so no luck
5. Back up at the root $(7, 2)$, the squared distance is also 2: a tie with $(5, 4)$. Ties are broken by a fixed tie-break rule (the MP tells you which), and here we keep $(5, 4)$. The square crosses $x = 7$, so we have to search the right subtree
6. On the right, q is below $y = 6$, so we first descend to $(8, 1)$ (squared distance 8), then check $(9, 6)$ (squared distance 18). Neither beats 2. The square never reaches $y = 6$, so we skip $(9, 8)$ entirely

The nearest neighbor is $(5, 4)$, and we never looked at $(9, 8)$. In a bigger tree, every subtree we can rule out like this saves a lot of work

16.5.1 Ties

When two points are the same distance away, we need some way to choose. Analysis usually assumes no two distances are ever equal, which just makes the math easier. In code, we add a tie-break rule instead, which acts like adding a tiny ε to one of the distances so there's never really a tie. Checking both on a tie would never find anything better anyways, so we just need to choose consistently

16.5.2 Implementation Tips

The search is the hardest part of MP Mosaics, mostly because of knowing what you have to backtrack into and what you don't

- One way to start is with an exhaustive search of the whole tree. It always finds the right point, but it will fail the tests, since some of the points are booby-trapped: if you ever look at them, you didn't follow the algorithm. In **Figure 16.6**, $(9, 8)$ would be one of those points. You have to look at $(9, 6)$, but only to figure out whether you need to look at $(9, 8)$
- Your first guess **must** be a point that actually exists in the tree. It's really easy to accidentally initialize the best guess to $(0, 0, ...)$ before descending. That point usually isn't in the tree, and it might even be closer to q than any real point. Your code will work most of the time, but a few points of your mosaic will come out the wrong color

16.5.3 Analysis

The full analysis of KD trees is beyond the scope of this course. We usually think of a search as $O(\log n)$, but that would require a fixed amount of backtracking work, and there's no guarantee of that. If someone carefully arranges the points (for example, in a narrow spiral where every point is almost the same distance from q), the splits end up as very inconvenient slices, and a search can end up checking every point

Even so, it is the best data structure and algorithm we know for nearest neighbor search in an arbitrary number of dimensions, on a query point that isn't necessarily in the data set

16.6 k-Nearest Neighbor (k-NN)

***k*-Nearest Neighbors (k-NN)** is a simple algorithm used in machine learning and data analysis to find the k closest points to a given query point in a dataset

It is used very often in **Machine Learning**, specifically classification and regression

- **Classification** - Assign the query point the most common class among its k nearest neighbors.
- **Regression** - Predict the query point's value as the average of its k nearest neighbors' values.

It becomes very efficient with a constructed KD Tree because it avoids brute force checking each point. Instead, it narrows the search by recursively exploring only those regions of space that could contain points closer than the current best candidates, ignoring irrelevant subtrees, to make nearest neighbor searches much faster

17 BTrees

17.1 Why AVL Trees Fall Short

In Big-O, we assume that we have uniform time for all operations, but this is not always the case

AVL trees (and red-black trees) assume that following a pointer, or doing a rotation, takes a constant and **small** amount of time. That's true when the whole tree lives in memory, since memory is fast. We can wrap it in a Big-O and it correctly describes what's going on

But what if following a pointer was really, really slow? For example, what if the tree was stored on a hard disk, or spread across a bunch of computers instead of a single one, like a huge database? Finding something on a spinning disk takes a lot of time, and two nodes on a disk aren't necessarily anywhere near each other

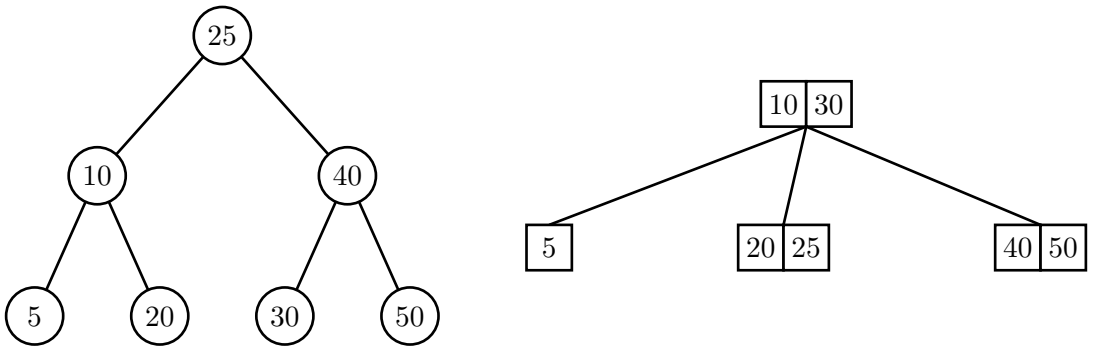
Where the next node lives	**Rough cost to reach it**
Memory	~100 instructions
SSD	~1,000 instructions
Spinning disk seek, or another server	~40 ms, or ~40 million instructions

Following one of these slow pointers is called a **seek**. The name comes from when B-trees were first designed for spinning hard drives: the read head had to physically move to the right spot on the disk, and that time was called the **seek time**

Technically, Big-O still says this is $O(1)$ per node, since 40 million is a constant. But instead of 6 or 7, our constant is now 40 million! $\log n$ is really small, until every one of those $\log n$ steps costs 40 milliseconds. The height of the tree starts to cost a lot

So we want the tree to be **really** short, but we still want the best part of a BST: when we search, we only go down, so we only ever look at the relevant parts of the tree

A binary tree divides the data we're looking at in half at every step. But you know what's better than dividing by 2? Dividing by 4, or 16, or 1024. Pointers are expensive, but nodes are okay, so we pack **more than one key** into each node and have fewer pointers. Compare the two trees in **Figure 17.1**, which hold the same seven keys



Binary: height 2 Multiple keys per node: height 1

Figure 17.1: The same seven keys in a perfect BST and in a tree with up to two keys per node. Packing more keys into each node makes the tree shorter, so a search follows fewer slow pointers

17.2 B-Trees

A **B-Tree** is a self-balancing search tree designed for efficient storage and retrieval on disks. Unlike a binary search tree that stores only one key per node, each B-Tree node can hold multiple keys and have multiple children

- The keys inside a node are kept in **sorted order**, and the children split the key ranges between them
- All leaves appear at the same depth, so the tree remains balanced
- Because each node can hold many keys (often filling a disk block), the tree height is very small, which reduces disk accesses
- Operations like search, insertion, and deletion still take $\boldsymbol{O(\log n)}$ time, but with a much smaller constant factor, making B-Trees ideal for databases and file systems

A **B-tree of order $\boldsymbol{m}$** is an **$\boldsymbol{m}$-way** (or **$\boldsymbol{m}$-ary**) tree, meaning every node has between 0 and m children

- Every node stores **at most $\boldsymbol{m-1}$ keys**, one less than the maximum number of children
- The keys in a node are stored in a sorted array that is **preallocated** to hold all $m-1$ keys, so there is no amortized resizing to worry about
- An internal node with k keys has $k+1$ children: one to the left of each key, plus one to the right of the last key

Take the small B-tree in **Figure 17.2**: its root holds two keys, 20 and 40, which split its three children by range.

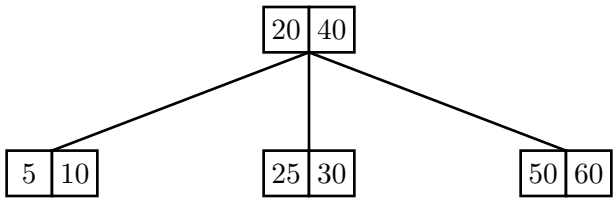


Figure 17.2: Example: a B-tree of order 3. Each node holds at most 2 sorted keys, the keys 20 and 40 split the children into < 20, between 20 and 40, and > 40, and every leaf is at the same depth

Searching or inserting inside a single node takes up to $O(m)$ time, since the keys are in a sorted array. That's fine! m is a constant, and work inside a node is basically free compared to a seek. When we analyze a B-tree, the only thing we count is **seeks**, since each one costs millions of instructions and completely dominates everything else

17.2.1 Choosing m

m is a design choice: we pick it so that one node fits exactly in whatever we get back from a single seek

- **Network** - Data is sent in packets of up to the **MTU** (maximum transmission unit), which is 1,500 bytes on basically everything modern. So we choose m so that a node's keys fit in 1,500 bytes
- **Disk** - Disks read and write whole **blocks**, traditionally 512 bytes and nowadays often 4,096 bytes. Writing even 2 bytes uses up a whole block, so we choose m so that a node fills one block

This works because almost all the time of a seek is spent **finding** the start of the data (something like 100 times longer than actually reading it). Once we've paid for the seek, reading a whole node full of keys costs almost nothing extra

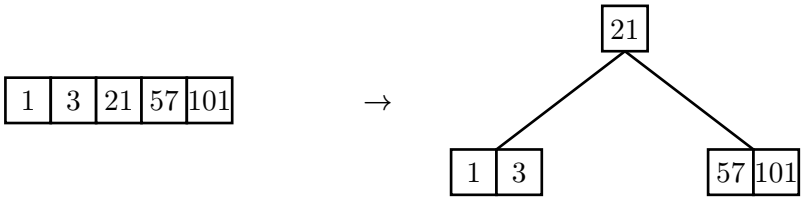
17.3 Inserting into a B-Tree

New keys are always inserted into a **leaf**. The only way a key ever ends up in an internal node is by being pushed up out of a leaf that got too full

To insert a key into a B-tree of order m

1. Starting at the root, find the child to go down into, just like a search: go to the left of the first key that is larger than the new key. An empty key slot acts like ∞, so if the new key is larger than every key in the node, go to the right of the last key
2. Repeat until we reach a leaf, and insert the key into the leaf's sorted array
3. If the leaf now has m keys, it is **overfilled**, so we **split** it
   1. Take the **middle** key and throw it up into the parent. If there is no parent, it becomes a brand new root
   2. The keys smaller than the middle key become a node to its left, and the keys larger than it become a node to its right
4. If throwing the key up overfills the parent, split the parent the same way, and keep going up the tree as long as nodes overflow

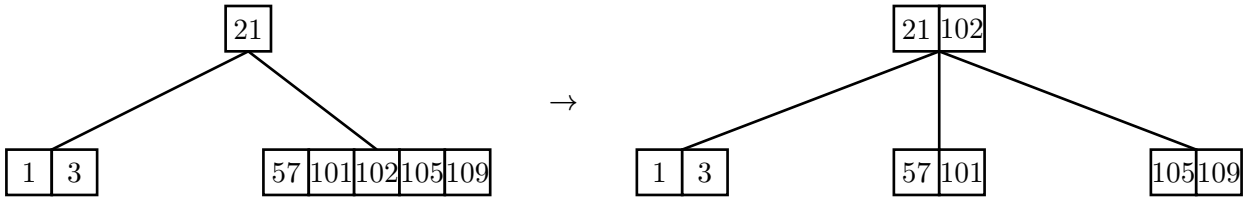
Let's build a B-tree of order $m = 5$, so each node holds at most 4 keys. Inserting 1, 57, 3, and 101 just fills up the root: we insert each one into the sorted array, shifting keys over as needed (inserting 3 shifts 57 over). Inserting 21 makes the root overflow with 5 keys, so we split it: 21, the middle key, gets thrown up into a new root, with the smaller keys on its left and the larger keys on its right, like in **Figure 17.3**



Insert 21: 5 keys is too many Split: 21 goes up

Figure 17.3: Example: inserting 1, 57, 3, 101, then 21 into a B-tree of order 5. The fifth key overfills the root, so the middle key becomes a new root and the rest splits into two leaves

Now we insert 102. The root isn't a leaf, so we don't insert there. 102 is not less than 21, but it is less than the empty slot after 21 (∞), so we go to the right of 21 and insert into that leaf. Inserting 105 also fits. Inserting 109 overfills the leaf again, so we split it and throw 102 up into the root, giving us three leaves in **Figure 17.4**



Insert 102, 105, 109 Split: 102 goes up

Figure 17.4: Example: inserting 102, 105, and 109. The right leaf overfills on 109, so 102 is thrown up into the root, which now has two keys and three children

A split can cascade all the way up. In the B-tree of order 3 at the top of **Figure 17.5**, each node holds at most 2 keys. To insert 79, we go right of 42 and land in the leaf with 43 and 55, which overfills. 55 gets thrown up into the root, but that overfills the root too, so we split the root as well: 42 becomes the new root, and 55 keeps the two halves of the old leaf as its children

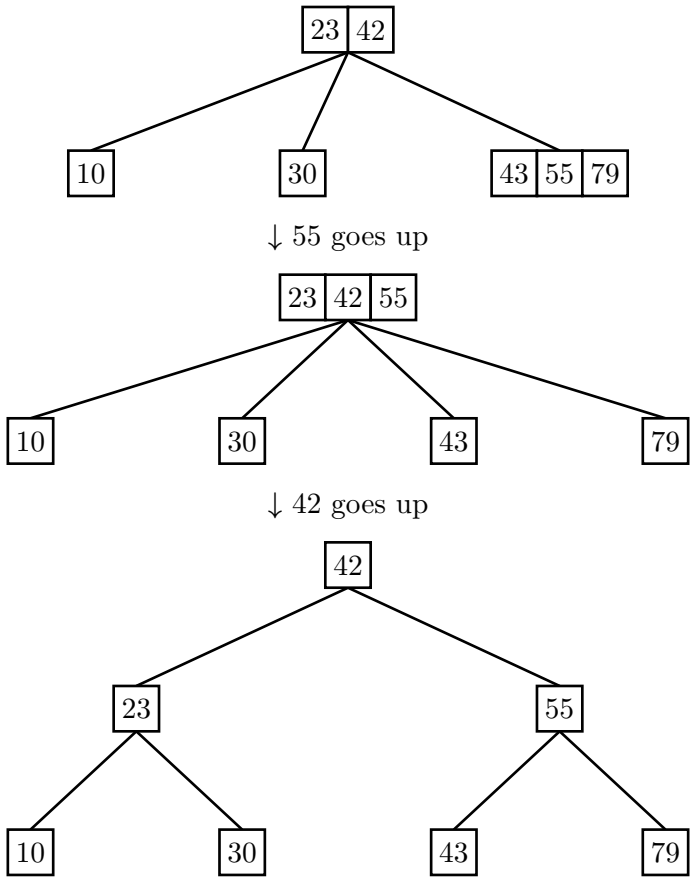


Figure 17.5: Example: inserting 79 into a B-tree of order 3. The leaf overfills, so 55 moves up, which overfills the root, so 42 moves up into a new root, making the tree one level taller

If m is even, an overfilled node has an even number of keys, so there is no single middle key. Pick either one (taking the floor or the ceiling of the middle index); it just biases the tree slightly to the left or right and doesn't change the analysis. In general, we choose an odd m because it makes the middle obvious

We won't cover removing from a B-tree. It makes the code a lot more complicated without changing any of the interesting ideas, so our B-trees only support `insert` and `find`

17.4 Properties of B-Trees

These are true of every B-tree of order m

- **All leaves are at the same depth.** The tree only ever grows taller when the root splits and a new root is created above it, which pushes every leaf down by one level at the same time
- **The root can have as few as 1 key**
- **Every non-root node has at least $\lceil \frac{m}{2} \rceil - 1$ keys.** A non-root node can only come into existence by splitting an overfilled node of m keys. One key goes up, and the other $m-1$ are split into two halves, so the smaller half has $\lfloor \frac{m-1}{2} \rfloor = \lceil \frac{m}{2} \rceil - 1$ keys. For $m = 5$, both halves get 2 keys. For $m = 4$, one half gets 1 key and the other gets 2

- **Every non-root internal node has at least $\lceil \frac{m}{2} \rceil$ children**, since a node with k keys has $k+1$ children

These properties also let us figure out what order a B-tree could be. A single node can't tell us much: a leaf with 2 keys could come from $m = 5$ or $m = 6$, since splitting an overfilled node of 6 keys leaves 3 and 2. But the structure of the whole tree can rule options out, since each part had to be produced by splits of the same order

17.5 Searching a B-Tree

Searching works the same way as finding where to insert

1. Find the first index i in the node whose key is **not smaller** than the key we're looking for. If we run out of keys, i is one past the last key
2. If i is a valid index and the key there is the one we want, we found it
3. Otherwise, if the node is a leaf, the key isn't in the tree
4. Otherwise, fetch child i (the child to the left of key i) and search it recursively

For example, to find 101 in the tree from **Figure 17.4**, we stop at index 1 of the root, since 102 is the first key not smaller than 101. 101 isn't 102, and the root isn't a leaf, so we fetch child 1, the leaf with 57 and 101, and find it there

The code for `_exists` is shown below. It is intentionally written tersely, so it's worth working through carefully

```
// BTree.hpp

/** Returns whether `key` is in the subtree rooted at `node`
 * Runtime: O(log_m n) seeks, plus O(m) work inside each node
 */
template <typename K>
bool BTree<K>::_exists(BTreeNode& node, const K& key) {
    unsigned i;
    // find the index of the key, or the first index with a larger key
    for(i = 0; i < node.keys_ct_ && key > node.keys_[i]; i++) { }

    // the key is in this node
    if(i < node.keys_ct_ && key == node.keys_[i]) {
        return true;
    }

    // a leaf has no children left to search, so the key isn't in the tree
    if(node.isLeaf()) {
        return false;
    } else {
        // the expensive part: a seek to fetch the next node from disk or another
server
        BTreeNode nextChild = node._fetchChild(i);
        return _exists(nextChild, key);
```

```
    }
}
```

- The `for` loop has an empty body. It keeps incrementing `i` as long as there are keys left in the node **and** our key is larger than the key at `i`. When it stops, `i` is either the index of our key, or the first index with a larger key
- `i < node.keys_ct_` is checked first, so if `i` ran past the last key, the `&&` short-circuits and we never read past the end of the array. In practice, we'd return the value stored at index `i` instead of just `true`
- A node is a leaf if it has no children. If a node has any children at all, it has a child to the left of key 0, so `isLeaf` just checks whether that first child pointer is null
- `_fetchChild` is where the seek happens: it asks the disk or the database for the next node. Every other line runs on a node we already have, so the cost of a search is the number of seeks, which is the height of the tree

18 BTree Analysis

In the last chapter, we saw how a B-tree works: we insert more than one key into each node, and when a node ends up with one more key than it's allowed to hold, we split it into two nodes and push the middle key up to the parent (or up into a brand new root)

Search is a direct extension of a BST. Every key in a non-leaf node has a child before it and a child after it, so if the key we want isn't in the current node, it has to be before all of its keys, between two of them, or after all of them, and there is a child there to go down into. A BST does exactly the same thing with one key and two children; a B-tree just has up to $m-1$ keys and m children

The choice to grow the tree by kicking keys **up** is a little unobvious, but it means all of the leaves are always at the same level. So in a sense, a B-tree is always approaching a **perfect** m-ary tree. That suggests a B-tree is pretty good at staying short, and this chapter shows exactly how short

18.1 Properties of a B-Tree

A B-tree of order m is an m-way (or m-ary) tree, so every node has between 0 and m children

- All keys within a node are kept **sorted** in a fixed-size array, so we can binary search them
- Every node contains **no more than $m-1$ keys**. That's just the design: with $m-1$ keys we need m children, and m is the most children an m-ary tree can have
- Every internal node has **exactly one more child than it has keys**. This comes directly from how search works: one child before each key, plus one after the last key
- The root is either a leaf (when the tree has at most $m-1$ keys) or has between 2 and m children
- Every **non-root internal node** has **at least $\lceil \frac{m}{2} \rceil$ children**

The last property is the one that matters for the analysis, so let's make sure it's right. A non-root node only comes into existence when a full node splits. If $m = 6$, an overfilled node has 6 keys: one goes up, and the remaining 5 split into 3 and 2. The smaller node has 2 keys, so it has 3 children, and $\lceil \frac{6}{2} \rceil = 3$. If $m = 5$, an overfilled node has 5 keys: one goes up, and the other 4 split into 2 and 2, each with 3 children, and $\lceil \frac{5}{2} \rceil = \lceil 2.5 \rceil = 3$

When we think about a B-tree, we tend to think about the keys, since that's what we split. But we state the minimum in terms of **children**, because the minimum number of children is what describes the minimum **structure** of the tree

18.2 What We Want to Show

The whole reason to use a B-tree is that fetching a node (a **seek**) is somewhere between 100,000 and 100 million times more expensive than anything else we do. So the only thing we count is

seeks. Every search and insert follows a single path from the root to a leaf, so the number of seeks is the **height** of the tree

Our goal is to show that the height is about $\log_m n$, where n is the number of keys

How good is that? If $m = 10$, then $\log_{10} n$ is just the number of digits in n. A million keys means a height of about 6. In practice m is often around 1000, and then the height is the number of **commas** in n. Storing a trillion records (1,000,000,000,000) gives a tree of height 4! Four seeks to find anything in a trillion records is an enormous improvement over a binary tree's 40

The work **inside** each node isn't free, but it's cheap: we binary search the node's sorted keys in $O(\log m)$ time at each of the $O(\log_m n)$ levels. As m gets bigger, the tree gets shorter but each node takes longer to search, so that is the trade-off. In practice, n is millions of times bigger than m, and a seek costs millions of times more than a comparison, so the number of seeks dominates

18.3 Minimum Number of Nodes

Just like with AVL trees, we can't directly figure out the height from the number of keys, since that gets incredibly convoluted. Instead, we go the other way around: given a height h, what is the **fewest** keys a B-tree of order m can have? If even the emptiest possible tree of height h has a lot of keys, then a tree with n keys can't be very tall

We find the minimum number of keys by first finding the minimum number of **nodes**, which is defined by the minimum branching. Let $b = \lceil \frac{m}{2} \rceil$, the minimum number of children of a non-root internal node

- **Level 0** - The root, which is just 1 node
- **Level 1** - The root only needs 1 key, so it can have as few as **2** children
- **Level 2** - Each of the 2 nodes at level 1 is a non-root internal node, so each has at least b children, for $2b$ nodes
- **Level 3** - Each of those $2b$ nodes has at least b children, for $2b^2$ nodes
- **Level k** - In general, $2b^{k-1}$ nodes

Figure 18.1 shows the smallest possible B-tree of order 5 with height 2: the root has one key and two children, and every other node has the minimum $b = 3$ children and $b - 1 = 2$ keys

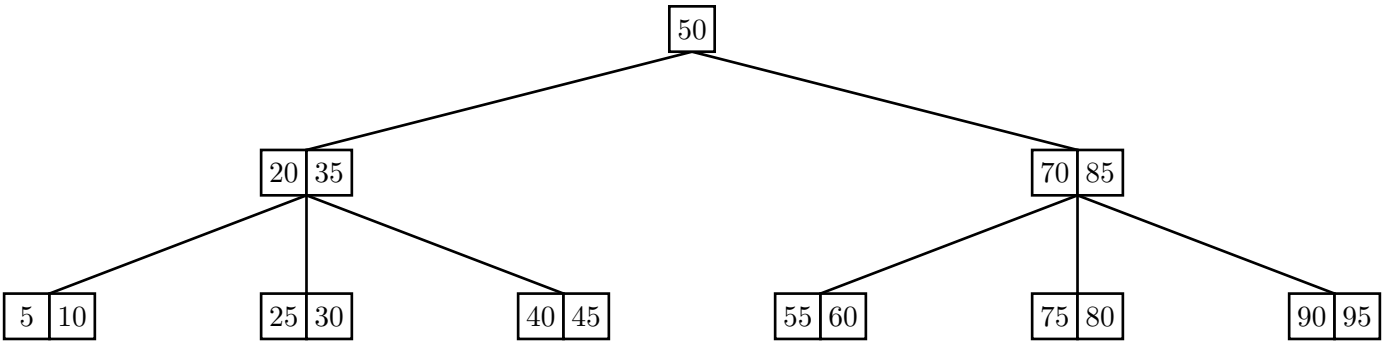


Figure 18.1: The smallest B-tree of order 5 with height 2. With $b = \lceil \frac{5}{2} \rceil = 3$, it has $1 + 2 + 2 \cdot 3 = 9$ nodes and $1 + 8 \cdot 2 = 17$ keys

Adding up every level gives the minimum number of nodes in a B-tree of height h

$$\text{min nodes} = 1 + \sum_{k=1}^{h} 2b^{k-1} = 1 + 2\sum_{k=0}^{h-1} b^k$$

Using the closed form of a geometric series (the same kind of identity we used for AVL trees), $\sum_{k=0}^{h-1} b^k = \frac{b^h - 1}{b-1}$, so

$$\text{min nodes} = 1 + 2 \cdot \frac{b^h - 1}{b - 1}$$

18.4 Minimum Number of Keys

Nodes aren't what we want, though. n is the number of **keys**, so we need to turn nodes into keys

- The root can have just **1** key
- Every other node has at least b children (or is a leaf made by a split), so it has at least $b - 1$ keys

So we multiply only the non-root part by $b - 1$, and the $b - 1$ cancels out

$$\text{min keys} = 1 + (b-1) \cdot 2 \cdot \frac{b^h - 1}{b - 1} = 1 + 2\big(b^h - 1\big) = 2b^h - 1$$

For the tree in **Figure 18.1**, $b = 3$ and $h = 2$, so the minimum is $2 \cdot 3^2 - 1 = 17$ keys, which matches

Any B-tree of height h with n keys has at least this many keys, so

$$n \geq 2\left\lceil \frac{m}{2} \right\rceil^h - 1$$

Solving for h gives us what we wanted

$$\left\lceil \frac{m}{2} \right\rceil^h \leq \frac{n+1}{2} \quad \Longrightarrow \quad h \leq \log_{\lceil \frac{m}{2} \rceil}\left(\frac{n+1}{2}\right)$$

Wrapping this in a Big-O, the $\frac{1}{2}$ inside the log and the $\lceil \frac{\cdot}{2} \rceil$ in the base are both constants (changing the base of a log only changes it by a constant factor), so

$$h = O(\log_m n)$$

So the number of seeks for `find` and `insert` is $O(\log_m n)$: count the commas

> We're being a little sloppy here: Big-O with multiple variables (m and n) is a much more complicated beast than Big-O with one variable, and we are hand-waving the math. If you're a math major, it's worth digging into. For engineering purposes, though, the answer clearly comes out to $\log_m n$

19 Priority Queues

We finally get a new abstract data type! Besides creating an empty one, it only has three operations

- `insert` - Adds a value (takes a `const T&`)
- `remove` - Takes no arguments, and removes and returns a value
- `isEmpty` - Checks whether there is anything left to remove

So far, that sounds like a queue or a stack. What makes this one special is **which** value `remove` returns

19.1 What Can We Store?

Unlike most of the data structures we've seen, we can't store just any kind of data in this one. Strings and integers work fine, but a general node, or a pointer, does not

The reason is that everything we store must be **comparable**: given two values, we have to be able to tell which one is smaller. Strings and integers have a `<`, but there's no meaningful way to say whether one node is smaller than another

Pointers are the surprising one, especially if you come from ECE. A pointer is just a memory location, so it feels like we should be able to compare them, and in assembly we could. But formally, C++ doesn't give a meaningful result for comparing two unrelated pointers with `<` (the standard leaves it unspecified). You **can** work around it, by writing your own comparison or by comparing with `std::less`, but the type itself doesn't come with one

So the one requirement is that the type has a **priority operator** that orders its values, generally thought of as `<`. Technically any relationship that produces an ordering works (`>`, `<=`, …), and we can choose whatever priority we want

19.2 The Priority Queue ADT

Suppose we insert 12, 7, 225, and −57, in that order. If we then call `remove`, we don't get 12 (the first in, like a queue) or −57 (the last in, like a stack). We get the **smallest** value, −57. Calling `remove` again returns 7, and again returns 12

If we then insert 42 and 21, the next `remove` returns 21, then 42, and finally 225

This ADT is called a **priority queue**. It stores its elements in some way the user never sees, but `remove` always returns the element with the **smallest** value. Technically, this is a **min priority queue**, which is the standard when theoreticians talk about a priority queue (a max priority queue returns the largest instead)

```cpp
// PriorityQueue.h

template <typename T>
class PriorityQueue {
public:
    void insert(const T& value);
    T remove(); // removes and returns the smallest value
    bool isEmpty() const;
};
```

19.3 Implementations So Far

With the data structures we already know, there are four ways we could implement a priority queue

- **Unsorted array** - `insert` just adds the value to the end in $O(1)$ time, but `remove` has to look through the whole array for the minimum and then shift everything back together, which is $O(n)$
- **Unsorted list** - `insert` is $O(1)$, but `remove` still has to look through the whole list for the minimum, which is $O(n)$
- **Sorted array** - If we sort it from largest to smallest, the minimum is always at the end, so `remove` is $O(1)$. But `insert` has to find the right place and shift everything over to make space for it, which is $O(n)$
- **Sorted list** - The same trade-off: `insert` is $O(n)$ and `remove` is $O(1)$

Figure 19.1 compares the two array versions holding the values from our example



Unsorted: scan everything to find the min

Sorted largest to smallest: the min is at the end

Figure 19.1: Two array-based priority queues holding 12, 7, 225, −57. The unsorted array makes `insert` cheap and `remove` expensive, and the sorted array does the opposite

Implementation	`insert`	`remove`
Unsorted array	$O(1)$	$O(n)$
Unsorted list	$O(1)$	$O(n)$
Sorted array	$O(n)$	$O(1)$
Sorted list	$O(n)$	$O(1)$

Every option makes one operation $O(1)$ and the other $O(n)$. Can we do better?

We actually already have a data structure that does: a **balanced BST**, like an AVL tree. `insert` takes $O(\log n)$ time, and the minimum is just the leftmost node (keep going left until you can't anymore), which we can find and remove in the height of the tree, also $O(\log n)$

But BSTs are fairly complicated, and there is a classic data structure built specifically for this that is a little bit better and shows some interesting ideas

19.4 Heaps

Figure 19.2 shows our new data structure. It's a binary tree, but it is **not** a BST: 14 is smaller than 15, but it isn't to the left of 15

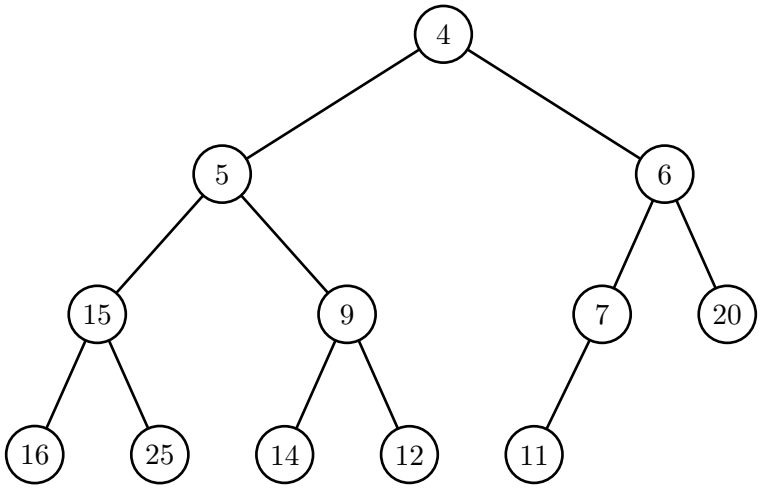


Figure 19.2: Example: a min-heap. Every node is smaller than both of its children, and the tree is complete

So what's true of every node? **The root of every subtree is the minimum of that subtree**, or in other words, every node is smaller than the roots of both of its subtrees. On top of that, it's a **complete** tree: every level is full except the last, which is filled from the left

A binary tree with these two properties is called a **heap**, which is an implementation of the priority queue ADT. Formally, a complete binary tree T is a **min-heap** if

- $T = \varnothing$, or
- $T = (r, T_L, T_R)$, where r is less than the roots of T_L and T_R, and T_L and T_R are min-heaps

The minimum is always at the root, so finding it is trivial

> The heap **data structure** has nothing to do with **heap memory**, where `new` allocates from. Computer science just reuses names in confusing ways

19.5 Storing a Heap in an Array

We'd always know where the minimum is even if the tree weren't complete, so why require it? Because in a complete tree, every level except the last has an **exact** number of nodes: 1, 2, 4, 8, … And even the last level is easy, since if we know the total number of nodes, we can subtract all the other levels

Those powers of two should remind us of the doubling array. So instead of storing a tree with pointers (pointers are a pain), we store the tree in an **array**, one level at a time: the root, then the second level, then the third, and so on. The tree is how we **think** about the heap, and the array is how we actually **store** it, as you can see in **Figure 19.3**

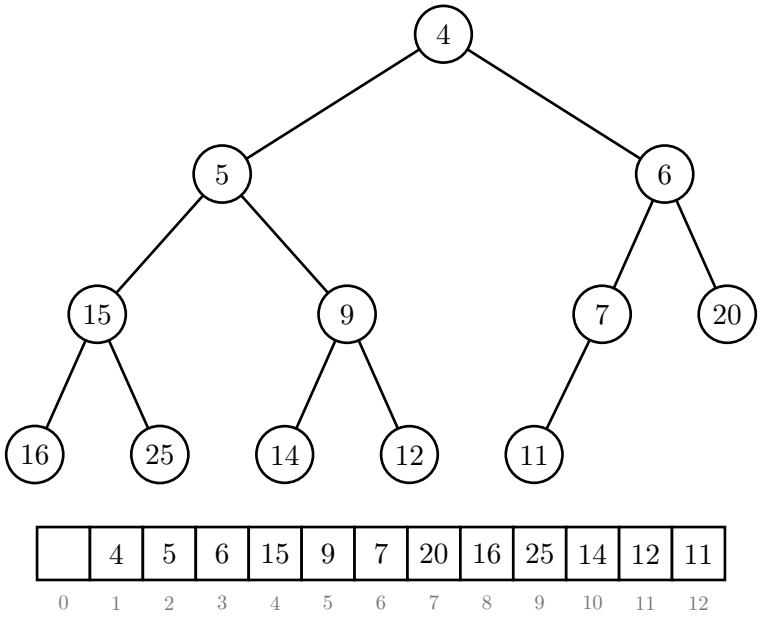


Figure 19.3: The heap from **Figure 19.2** (conceptual storage) and the array that actually stores it, one level at a time (physical storage). Index 0 is left empty

We leave index 0 empty (or store the number of nodes there), so the root is always at index 1. That used to be the standard way to implement a heap, back when heaps mostly stored integers. Nowadays heaps usually store more complicated types, so people tend to store the size separately and start the root at index 0. Either works, but starting at 1 makes the math easy

With the root at index 1, for a node at index i

- **Left child** - $2i$
- **Right child** - $2i + 1$
- **Parent** - $\lfloor \frac{i}{2} \rfloor$, which is just integer division in code
- **Last node** - If the heap has n nodes, the last one is at index n

For example, in **Figure 19.4**, 5 is at index 2: its left child is at index 4 (15), its right child is at index 5 (9), and its parent is at index 1 (4). 20 is at index 7, so its parent is at index $\lfloor \frac{7}{2} \rfloor = 3$, which is 6

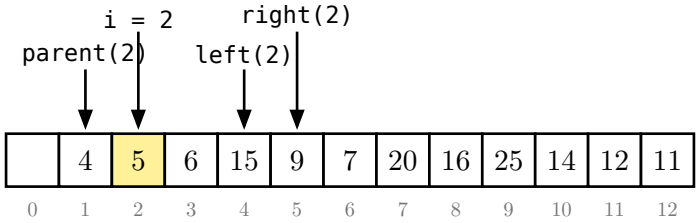


Figure 19.4: Index math for the node at index 2 (holding 5): its parent is at index 1, and its children are at indices 4 and 5

> If the root is at index 0 instead, the formulas become $2i + 1$, $2i + 2$, and $\left\lfloor \frac{i-1}{2} \right\rfloor$. Nothing is wrong with that, the math is just messier (you can't double 0 to find its left child)

That's everything we need: no pointers, just an array and a size, and we can move up and down the tree. A pointer-based heap is possible, but it would need pointers going **up** to parents too, and the array uses a lot less memory

The only question left is how to keep it a heap. There is no rotating an array, and we can't leave holes in it, so the tree must always stay complete. Luckily, only two operations change the heap: `insert` and `removeMin`

19.6 Insert

After inserting one more node, the tree has to stay complete, so we **already know where the new node goes**: the next open spot on the last level, which is index $n + 1$ in the array

1. Put the new value at index $n + 1$
2. If it's smaller than its parent, the heap property is broken, so **heapify up**: swap it with its parent, and repeat from the parent's index until it's no longer smaller than its parent, or it reaches the root

Inserting 21 into the heap from **Figure 19.2** puts it under 7, and since 21 is larger than 7, the heap is still valid with no swaps at all

Inserting 3 next puts it under 20. 3 is smaller than 20, so we swap them. Then 3 is smaller than its new parent 6, so we swap again, and then it's smaller than 4, so we swap one last time and 3 becomes the new root. You can see each swap in **Figure 19.5**, in the array where it actually happens. Everything below 3 is fine after each swap, since 3 is smaller than whatever it swapped with, so we only ever have to keep checking **above**

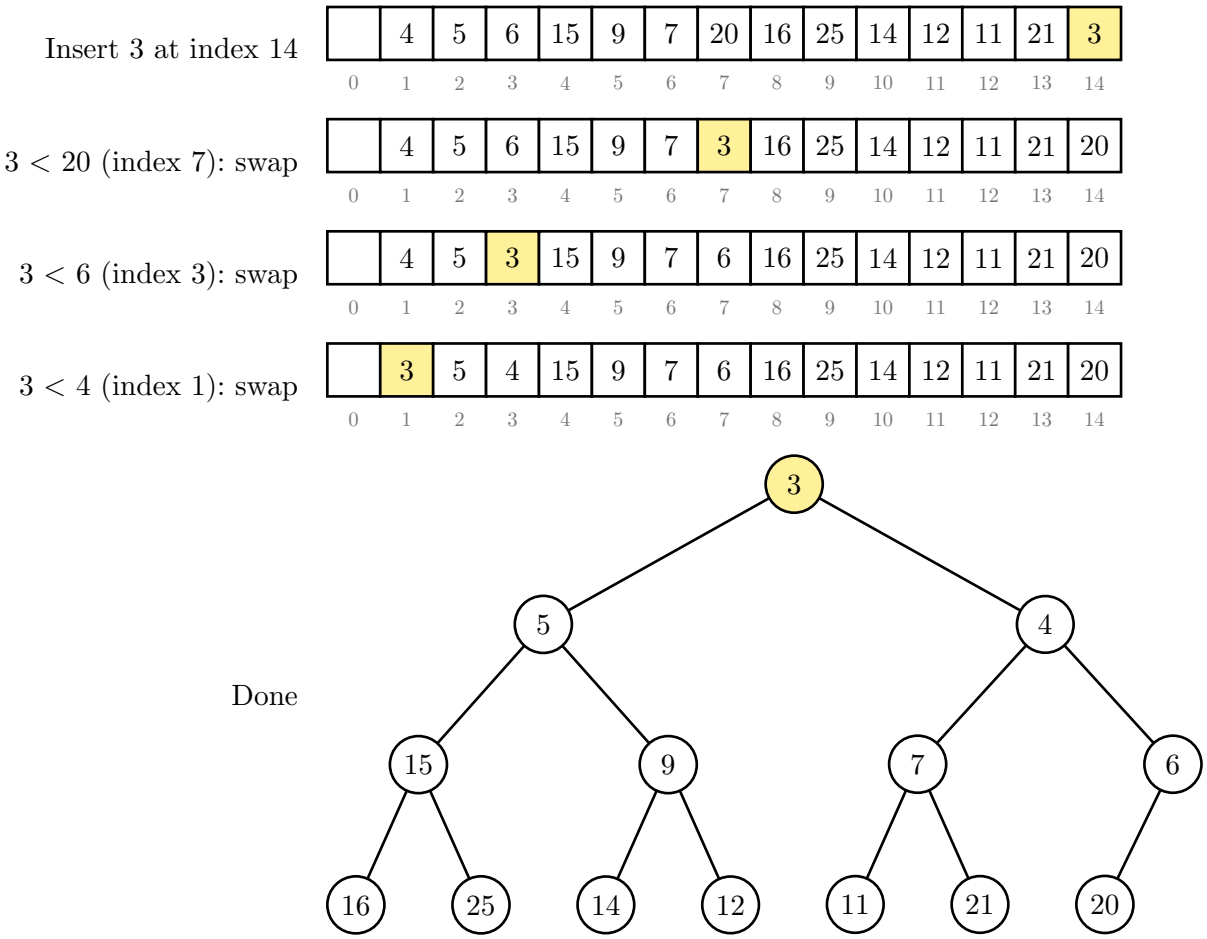


Figure 19.5: Inserting 3 (after 21). It starts at the next open spot, index 14, and heapifies up through its parents at indices 7, 3, and 1 to become the new root

Heapify up isn't constant time: it can go all the way up the tree, which is the height. But the tree is complete, and a complete tree is only a constant amount off of a perfect tree, so its height is $O(\log n)$. So `insert` is $O(\log n)$

If the array runs out of space, we double it, just like `push_back` on a vector, which gives amortized constant time growth. In a heap, doubling also happens to allocate **exactly** the space for the next level of the tree

```
// Heap.hpp

/** Inserts `key` at the end of the array and heapifies it up
 * Runtime: O(log n)
 */
template <class T>
void Heap<T>::_insert(const T& key) {
    // check to ensure there's space to insert an element
    // ...if not, grow the array
```

```
    if(size_ == capacity_) { _growArray(); }

    // insert the new element at the end of the array
    item_[++size_] = key;

    // restore the heap property
    _heapifyUp(size_);
}
```

`++size_` moves the size from the old last position to the new last position, and we put the key there. Then we heapify up starting from the **index** `size_`, not from the key, since that's where we just put it

```
// Heap.hpp

/** Swaps the value at `index` up until it is no smaller than its parent
 * Runtime: O(log n)
 */
template <class T>
void Heap<T>::_heapifyUp(size_t index) {
    // index 1 is the root, which has no parent to swap with
    if(index > 1) {
        // swap with the parent if it is smaller, then keep going up
        if(item_[index] < item_[_parent(index)]) {
            std::swap(item_[index], item_[_parent(index)]);
            _heapifyUp(_parent(index));
        }
    }
}
```

- `index > 1` is the base case: once we reach the root, there is nothing above us, so we're done
- Otherwise, we compare the **value** at `index` with the value at its parent's index, and if ours is smaller, swap the two values and recurse on the parent's index. The confusing part is switching between indexes and values, just like a pointer versus the thing it points to
- Duplicates are fine: if we're not **strictly** smaller, we don't swap. The heap property just becomes "less than or equal to", and the root is still a smallest value
- `std::swap` takes its two arguments by reference, so it swaps the values in the array in place

> `size_t` is an unsigned integer type that is guaranteed to be big enough to hold any array index. C++ only promises that an `int` is no smaller than a `short` and no bigger than a `long`, so the size of an `int` depends on your compiler and computer. When you need an exact size, there are types like `uint32_t` and `int64_t` (unsigned and signed, with that many bits)

19.7 Remove Min

Once again, structurally we already know which node goes away: the tree has to stay complete, so the node in the **last** position (index n) is the one that disappears, even though the value we're removing is at the root

This might feel familiar from removing a node with two children in a BST

1. Save the minimum value at the root
2. Move the last value into the root, and shrink the size by one. The tree now has the correct shape, but the root is probably not the smallest anymore
3. **Heapify down** from the root: compare the node with both of its children, and if it's larger than the **smaller** child, swap with that child. Repeat from the child's index until the node is smaller than both children, or it's a leaf
4. Return the saved minimum

Calling `removeMin` on the heap from **Figure 19.2** removes 4 and moves 11, the last value, to the root. 11 is larger than its smaller child 5, so they swap. Then 11 is larger than its smaller child 9, so they swap again. Now 11 is smaller than both of its children, 14 and 12, so we stop. You can follow it in **Figure 19.6**

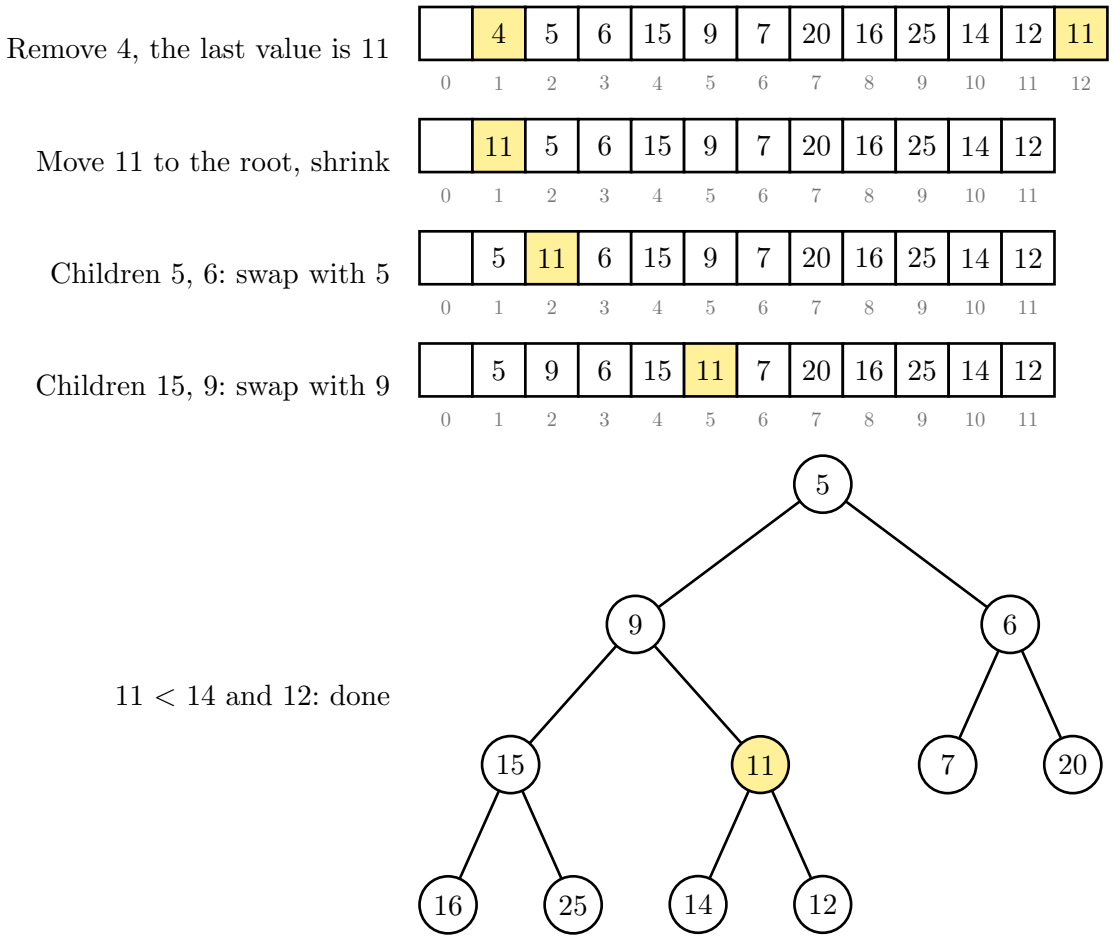


Figure 19.6: `removeMin` on the heap from **Figure 19.2**. The last value moves to the root and heapifies down, always swapping with the smaller child

Swapping with the **smaller** child is what keeps this correct: whatever moves up was smaller than or equal to both children, so it's a valid root for that subtree. It's one swap per level, so `removeMin` is $O(\log n)$ too

```
// Heap.hpp

/** Removes and returns the smallest value (the root)
 * Runtime: O(log n)
 */
template <class T>
T Heap<T>::_removeMin() {
    // swap with the last value
    // the minimum is always the root at index 1
    T minValue = item_[1];
    item_[1] = item_[size_];
    size_--;
```

```
    // restore the heap property
    _heapifyDown(1);

    // return the minimum value
    return minValue;
}
```

```
// Heap.hpp

/** Swaps the value at `index` down until it is no larger than its children
 * Runtime: O(log n)
 */
template <class T>
void Heap<T>::_heapifyDown(size_t index) {
    if(!_isLeaf(index)) {
        // swap with the smaller child so the heap property holds below
        size_t minChildIndex = _minChild(index);
        if(item_[index] > item_[minChildIndex]) {
            std::swap(item_[index], item_[minChildIndex]);
            _heapifyDown(minChildIndex);
        }
    }
}
```

- `_isLeaf(index)` checks whether `2 * index > size_`. If our left child's index would be past the end of the heap, we have no children. We don't need to check the right child, since the tree is complete, so a right child can't exist without a left child
- `_minChild(index)` returns the index of the smaller child (or the left child, if there is no right child)
- If our value is larger than our smaller child's, we swap down and heapify down again from the child's index

19.8 Heap vs. BST

Both `insert` and `removeMin` are $O(\log n)$, the same as a balanced BST. So a heap isn't obviously better yet, but it uses a lot less space because of the array, and in practice it runs faster

The real advantage shows up when turning an existing array into a heap. Building a BST from n values takes $O(n \log n)$ time, since we insert them one at a time, but we can turn an array into a heap in just $O(n)$ time. That, and using a heap to sort, come next

20 Heaps

Recall the priority queue ADT: `insert`, `removeMin`, and `isEmpty`. Our implementation is a **min-heap**: a complete binary tree, stored in an array, where every node is at least as small as both of its children. Since the tree is always complete, we always know its shape after an `insert` (a new rightmost node on the bottom level) or a `removeMin` (the last node disappears), and we fix the heap property with **heapify up** or **heapify down**, each in $O(\log n)$ time

A priority queue doesn't have to put the smallest value first. A **max-heap** keeps the largest value at the root, with every node at least as **large** as its children. In fact, we can build a heap around any comparison we want, and whatever comes first under that comparison comes out first

So far, a heap isn't much better than a balanced BST, which can also `insert` in $O(\log n)$ and find the minimum as its leftmost node. We'd still pick the heap, since storing it in an array is much more compact, but the real advantage is **building** one

20.1 Building a Heap

Given an array of n values, how do we turn it into a heap? The algorithm we'll call `buildHeap` is specifically the **best** way we know to do this, so if you're asked to write `buildHeap`, it needs to be as good as the one we find here. Let's look at three options

20.1.1 Option 1: Sort the Array

If we sort the array (ignoring index 0, where we keep the size) from smallest to largest, it's already a heap! The array stores the tree one level at a time, so after sorting, every value on a higher level is no larger than every value on a lower level, so every node is no larger than its children

But the best possible **comparison sort** (a sort that only compares two values at a time, which is all we know how to do with our values) has been proven to take $O(n \log n)$ time. That's no better than inserting n values into a balanced BST

20.1.2 Option 2: Insert n Times

We could just `insert` each value into an empty heap, one at a time. Each `insert` puts the value at the end and **heapifies up**, so this is the same as calling heapify up on index 2, then 3, then 4, all the way to n. Each heapify up takes $O(\log n)$, so our first guess is $O(n \log n)$

20.1.3 Option 3: Heapify Down From the Bottom Up

This one is more subtle. When we call heapify down on a node, the node itself might be in the wrong place, but its two subtrees are already correct heaps. So we can think of heapify down as **merging two heaps with a new value on top**: if we have two correct heaps (no matter what's in them), stick a new value on top of them, and heapify down, we get one correct heap

So we build the heap from the bottom up

1. Every leaf is already a correct heap on its own, so we skip all of them. The last node with a child is the parent of the last node, at index $\lfloor \frac{n}{2} \rfloor$
2. Call heapify down on index $\lfloor \frac{n}{2} \rfloor$, then $\lfloor \frac{n}{2} \rfloor - 1$, and so on, all the way back to the root at index 1
3. By the time we heapify down any node, both of its subtrees have already been turned into heaps, so each call merges two heaps into a bigger one, and after the root, the whole array is a heap

In **Figure 20.1**, the array has 10 values, so the leaves are at indices 6 through 10, and the first node we heapify down is at index $\lfloor \frac{10}{2} \rfloor = 5$

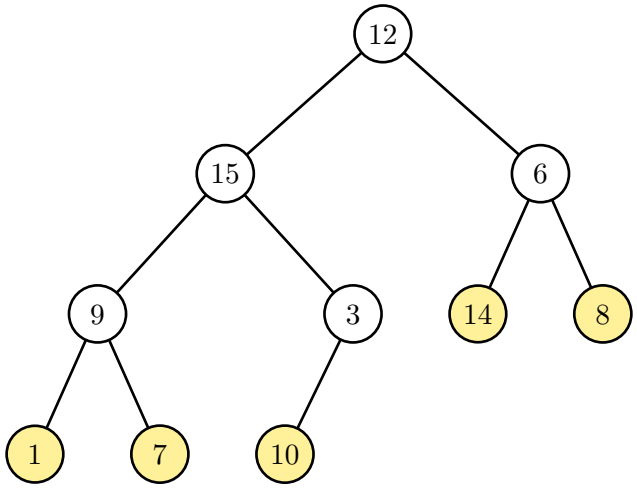


Figure 20.1: Example: the array 12, 15, 6, 9, 3, 14, 8, 1, 7, 10 viewed as a complete tree. The highlighted leaves are already heaps, so `buildHeap` starts at index 5 (holding 3)

Figure 20.2 then heapifies down each node from index 5 back to the root. Some nodes, like 3 and 6, are already smaller than their children, so they don't move at all. Others, like 15 and 12, sink down more than one level

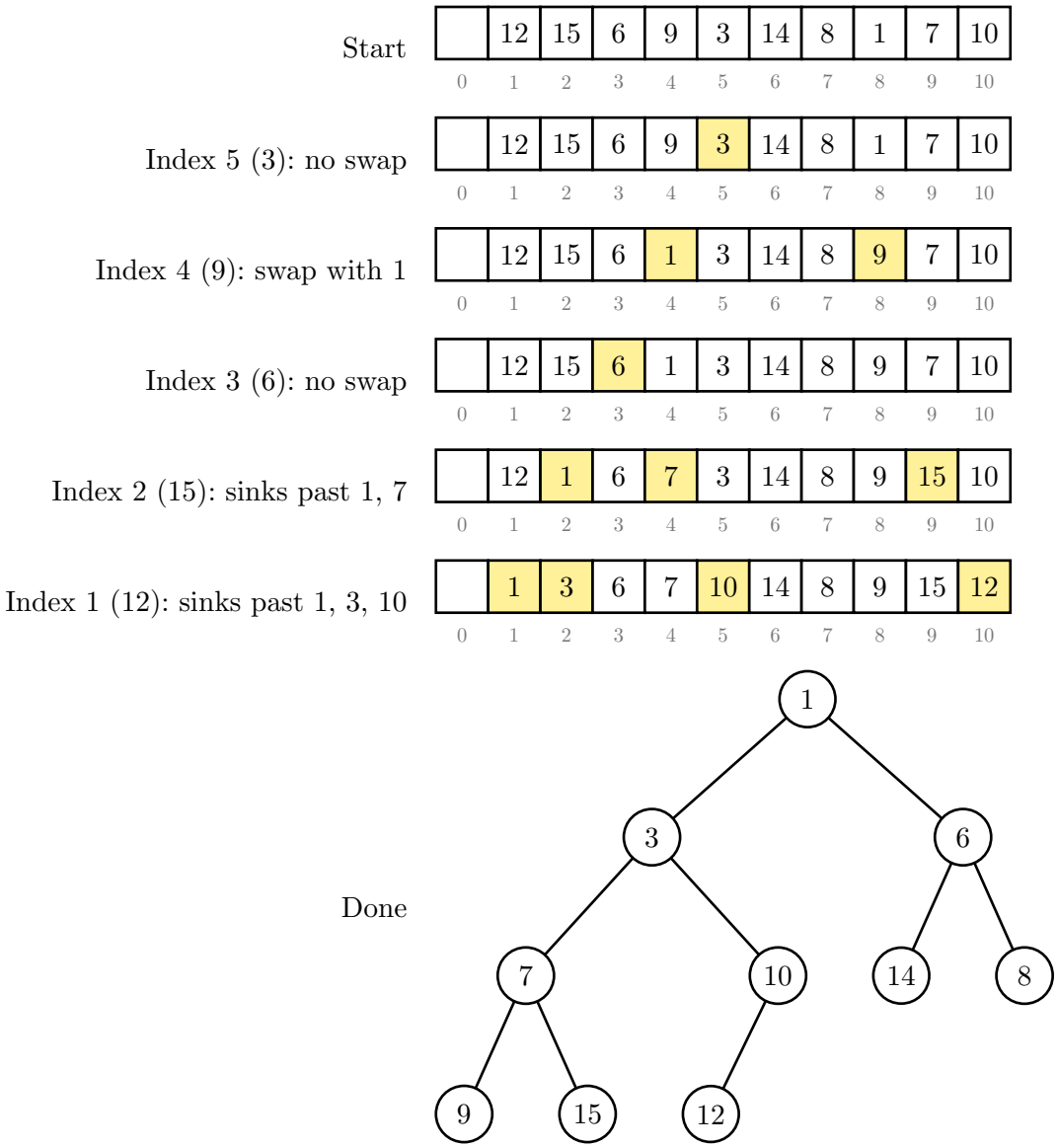


Figure 20.2: `buildHeap` on the array from **Figure 20.1**, heapifying down from index 5 back to the root. The highlighted cells are the ones that changed

Each heapify down can take up to the height of the tree, and we do $\frac{n}{2}$ of them, so our first guess is $O(n \log n)$ again. It turns out that guess is wrong

```
// Heap.hpp

/** Turns the unsorted array into a heap by heapifying down from the bottom up
 * Runtime: O(n)
 */
template <class T>
void Heap<T>::buildHeap() {
    // every leaf is already a heap, so start at the parent of the last node
```

```
    for(size_t i = _parent(size_); i > 0; i--) {
        _heapifyDown(i);
    }
}
```

20.2 Analysis

All three options looked like $O(n \log n)$ at first. Sorting really is: no comparison sort can do better. To compare the other two, we count the number of swaps, assuming the worst case is a **perfect** tree of height h, so $n = 2^{h+1} - 1$ (a perfect tree has more nodes on every level, so it can only take more work)

20.2.1 Inserting n Times

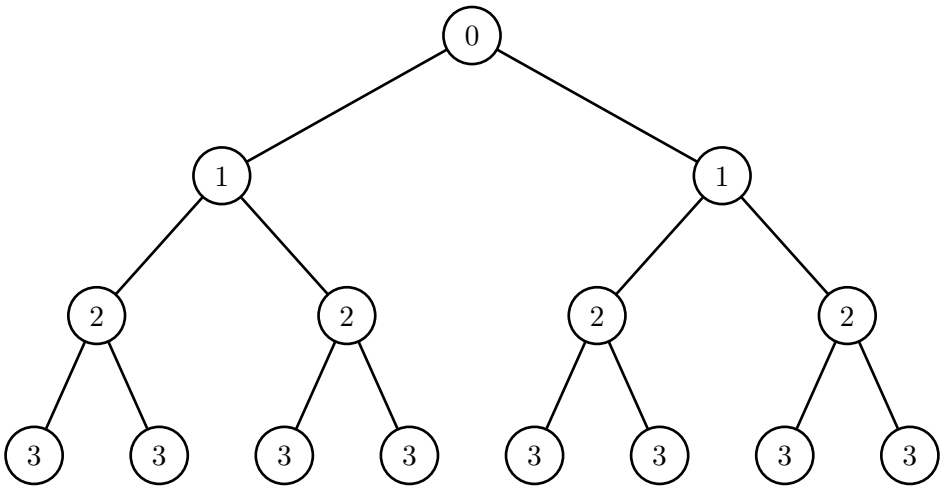
When we heapify up a node, it can travel up as far as its **depth**, its distance from the root. There are 2^d nodes at depth d, so

$$\text{swaps} = \sum_{d=0}^{h} d \cdot 2^d = 0 \cdot 1 + 1 \cdot 2 + 2 \cdot 4 + 3 \cdot 8 + ... + h \cdot 2^h$$

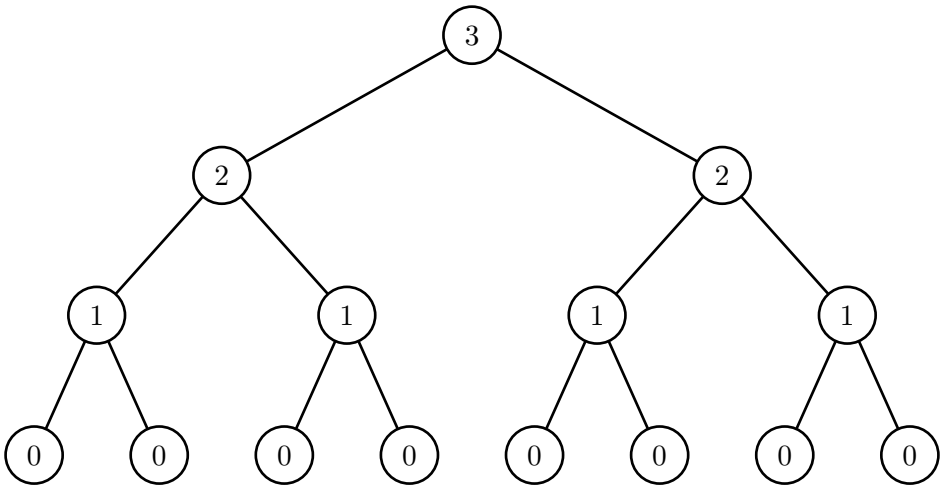
The problem is that the **most** nodes (the bottom level, about half of them) do the **most** work (the full height). The last level alone is $h \cdot 2^h \approx \left(\frac{n}{2}\right) \log n$, so this really is $O(n \log n)$

20.2.2 Heapify Down From the Bottom Up

When we heapify down a node, it can travel down as far as its **height**, its distance to the bottom. The leaves have height 0 and do no work at all, the level above them can do 1 swap each, and only the root can travel the whole height of the tree. **Figure 20.3** labels every node of a perfect tree with how many swaps each approach could need



Insert n times: each node can move up its depth



Heapify down from the bottom up: each node can move down its height

Figure 20.3: The most swaps each node can need in a perfect tree of height 3. Inserting puts the biggest numbers on the biggest level, while bottom-up heapify down puts them on the smallest

There are 2^{h-k} nodes at height k, so the total number of swaps is the sum of the heights of every subtree

$$\text{swaps} = \sum_{k=0}^{h} k \cdot 2^{h-k} = 2^h \sum_{k=0}^{h} \frac{k}{2^k}$$

The sum $\sum_{k=0}^{\infty} \frac{k}{2^k}$ converges to exactly 2, so

$$\text{swaps} < 2^h \cdot 2 = 2^{h+1} = n + 1$$

The exact total for a perfect tree is $2^{h+1} - h - 2 = n - h - 1$ swaps, which is less than n. So `buildHeap` runs in $\boldsymbol{O(n)}$ time!

The intuition: when there are only a few subtrees (near the root), each one does a lot of work, but when there are lots of subtrees (near the bottom), each one does very little. With heapify up, it's the other way around, so the work grows with the number of nodes. This should feel like nonsense at first, since all three options look like $O(n \log n)$, and that's the right intuition to start with. It only comes out to $O(n)$ when you work through the math

Way to build a heap	**Running time**
Sort the array	$O(n \log n)$
Insert n times (heapify up)	$O(n \log n)$
Heapify down from the bottom up (`buildHeap`)	$O(n)$

That's the real advantage of a heap over a BST: building a BST out of n values takes $O(n \log n)$, but we can turn any array into a heap in $O(n)$

21 Disjoint Sets

We're starting to see data structures that ask weird questions. A priority queue is still a queue in the sense that you put things in and get things out, just in a strange order. A **disjoint set** is really different from anything we've seen before

A disjoint set stores a **partition** of a set. Recall that a partition splits a set into subsets so that every element appears in **exactly one** subset. **Figure 21.1** shows a partition of the numbers 0 through 9 into four subsets

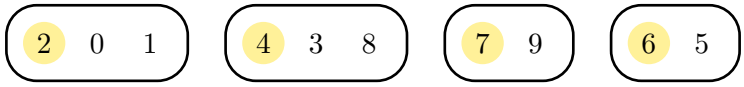


Figure 21.1: Example: a partition of 0 through 9 into four subsets. The shaded element of each subset is its representative

21.1 Representatives

How do we refer to one of these subsets? In a drawing, we could say "the yellow one", but that's only because we colored it. As the partition changes, subsets come and go, so we need a better way to name them

We identify each subset by its **canonical element**, also called its **representative**. "Canonical" is a weird word (it actually comes from religious law), but it just means the **official** element: the one we all agree represents the set. Which element we pick doesn't matter, as long as everyone agrees on it. In **Figure 21.1**, we picked 2, 4, 7, and 6

This makes membership easy to check. If 3 says "I belong to the set represented by 4", and 8 also says "I belong to the set represented by 4", then 3 and 8 are in the same set

21.2 The Disjoint Sets ADT

- `find(k)` - Returns the representative of the subset that contains `k`
- `union(k1, k2)` - Merges the subsets containing `k1` and `k2` into one subset
- `addElements(num)` - Adds `num` new elements, each in a brand new subset all by itself

In **Figure 21.1**, `find(4)` returns 4 and `find(8)` also returns 4. So `find(4) == find(8)` tells us that 4 and 8 are in the same subset, which is the basic question this data structure answers

Calling `union(5, 7)` merges $\{6, 5\}$ and $\{7, 9\}$ into $\{5, 6, 7, 9\}$. The new subset needs a representative, which could be any of its elements, so after a `union`, the representative of at least one of the old subsets has to change

Why is this useful? We're keeping track of groupings, and we want to know whether two elements are in the same group. Mathematically, that's the same as asking whether two elements are related by an **equivalence relation**

We always label the elements 0 through $n-1$. If we want to store something other than integers, we keep a separate table on the side (like a dictionary) that maps each item to its number. Since we can never remove anything, that table never goes out of date

21.3 A First Implementation

The most obvious implementation is an array where index `k` stores the representative of `k`. At the start, every element is in its own subset, so every index stores itself. **Figure 21.2** shows the array after a few unions, with three subsets represented by 1, 2, and 5

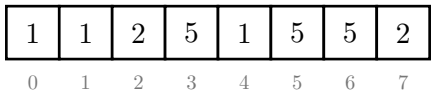


Figure 21.2: Example: the subsets $\{0, 1, 4\}$, $\{2, 7\}$, and $\{3, 5, 6\}$ stored in an array. Each index stores the representative of its subset

`find` is as good as it gets: we just return `data_[k]`, which is $O(1)$

For `union`, we'll assume we're always called with two **representatives**. We can always guarantee that by calling `union(find(k1), find(k2))`, which only adds a constant amount of work, but keeping `find` and `union` separate will matter once we start analyzing this data structure. To merge the two subsets, we go through the whole array and change every element that belongs to `k1` so it belongs to `k2` instead

```
// DisjointSets.cpp

/** Merges set `k1` into set `k2` by relabeling every element of `k1`
 * Runtime: O(n)
 */
void DisjointSets::_union(int k1, int k2) {
    // every element whose representative is `k1` now belongs to `k2`
    for(size_t i = 0; i < data_.size(); i++) {
        if(data_[i] == k1) { data_[i] = k2; }
    }
}
```

That's clearly correct, but it's $O(n)$! We're back to the same problem we started with for heaps: one operation is $O(1)$ and the other is $O(n)$. Will we end up at $O(\log n)$ and $O(\log n)$ like heaps did? As it turns out, no: we're going to end up with something far better

$O(n)$ is almost always terrible for a data structure, since you can do almost anything in $O(n)$. At this point, we're really just writing down which set every element belongs to, which hardly seems like a data structure. But it **is** correct, and it implements the whole ADT. Anything we build has

to be right before it's fast, since otherwise the fastest algorithm is just `return 0;`. It's constant time, and the compiler might even remove it entirely, but it's completely useless

As for why we'd want this at all: it answers "are 3 and 5 in the same set?" with `find(3) == find(5)`. There are a number of algorithms that fundamentally depend on that question, and we'll see a really cool one soon, but they only work because a good disjoint set is a lot better than $O(n)$

21.4 UpTrees

Our new data structure is a **forest of trees**, one tree per subset. The twist is that the trees point **upwards**: every node points to its **parent**, and the **root** of each tree is the representative of the subset. To `find` an element's representative, we just follow parent pointers up until we reach the root

We don't want actual pointers, though. Just like with heaps, we can avoid them, since every element is already named by an index 0 to $n-1$. Instead of storing each element's representative, we store its **parent's index**. A root has no parent, so it stores -1, which is never a valid index. When we add elements, every new element starts as its own root, so we initialize it to -1 (setting a big block of memory to the same value is also very fast for hardware)

Figure 21.3 stores the same three subsets from **Figure 21.2** as uptrees: 0 points to 1 and 1 points to 4, 7 points to 2, and 3 and 6 both point to 5

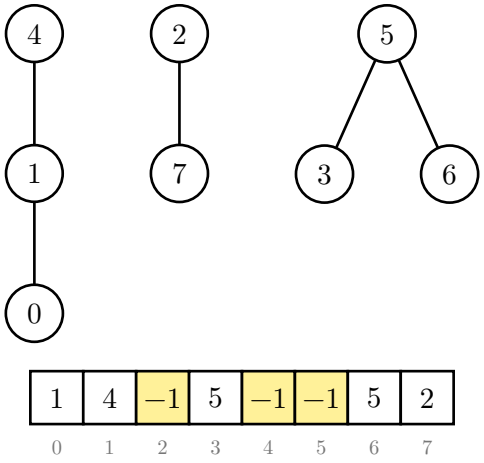


Figure 21.3: Example: the subsets $\{0, 1, 4\}$, $\{2, 7\}$, and $\{3, 5, 6\}$ as uptrees. Every node points up to its parent, and the roots (highlighted) store -1

Which tree we build for a subset (like whether 0 points to 1 or straight to 4) depends on the order of the unions. We're not claiming this is the best set of trees for this data, just that this memory represents these trees. Also notice nothing here requires a **binary** tree: a node can have as many children as point to it

21.4.1 Find

To find the representative of `k`

1. If `data_[k]` is -1, then `k` is a root, so `k` is the representative
2. Otherwise, `data_[k]` is `k`'s parent, which is in the same set, so ask it for the representative instead

```
// DisjointSets.cpp

/** Returns the representative (root) of the set containing `k`
 * Runtime: O(h), up to O(n) without smart union
 */
int DisjointSets::find(int k) {
    // a root stores -1, so `k` is its own representative
    if(data_[k] == -1) { return k; }

    // otherwise ask the parent, which is in the same set
    return find(data_[k]);
}
```

In **Figure 21.3**, `find(7)` looks at 7, which stores 2. 2 stores -1, so the representative of 7 is 2. The running time of `find` is the **height of the uptree**, since we might have to walk all the way up from the bottom

21.4.2 Union

Since we always union two roots, all we have to do is make one root point to the other

```
// DisjointSets.cpp

/** Hangs the uptree rooted at `r1` underneath the root `r2`
 * Runtime: O(1)
 */
void DisjointSets::_union(int r1, int r2) {
    data_[r1] = r2;
}
```

That's $O(1)$! Calling `union(4, 5)` on **Figure 21.3** just stores 5 at index 4, which hangs the whole tree rooted at 4 underneath 5, like in **Figure 21.4**. Now `find(0)` goes from 0 to 1, to 4, to 5, which stores -1, so the representative of 0 is 5

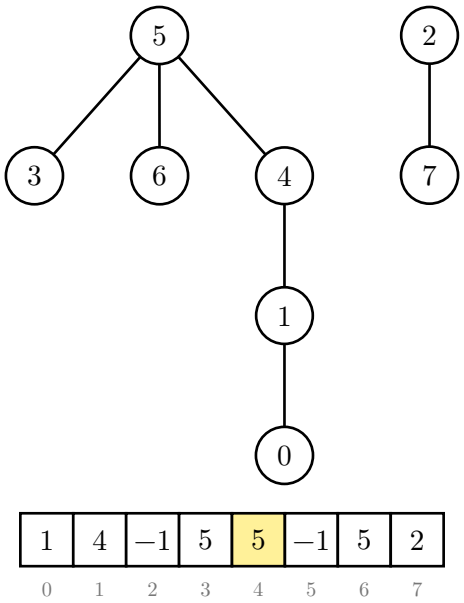


Figure 21.4: After `union(4, 5)` on **Figure 21.3**, the only change in memory is index 4, which now stores its new parent, 5

21.4.3 How Bad Can It Get?

`union` is $O(1)$ and `find` is the height of the tree, so everything depends on how tall the trees get, and that depends on how people call `union`. If we start with every element by itself and call `union(0, 1)`, then `union(1, 2)`, then `union(2, 3)`, and so on, we get a single long line like **Figure 21.5**, and `find(0)` is $O(n)$. This is the usual problem with trees: if we just depend on how people use them, they can end up terrible

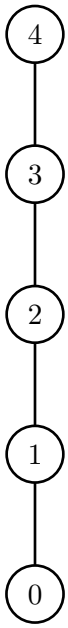


Figure 21.5: The worst case: `union(0, 1)`, `union(1, 2)`, `union(2, 3)`, `union(3, 4)` builds a single long line

21.4.4 The Ideal UpTree

The best possible uptree has every node pointing **directly** at the root, so `find` is $O(1)$. But even if we had two of those, unioning them makes one of them hang one level lower, like in **Figure 21.6**. To make it ideal again, we'd have to update every node of that tree to point at the new root. In an uptree, a node only knows its parent, not its children, so the only way to find every node in a tree is to check every element, which is exactly our first $O(n)$ implementation. That implementation was really just uptrees that were forced to always be ideal

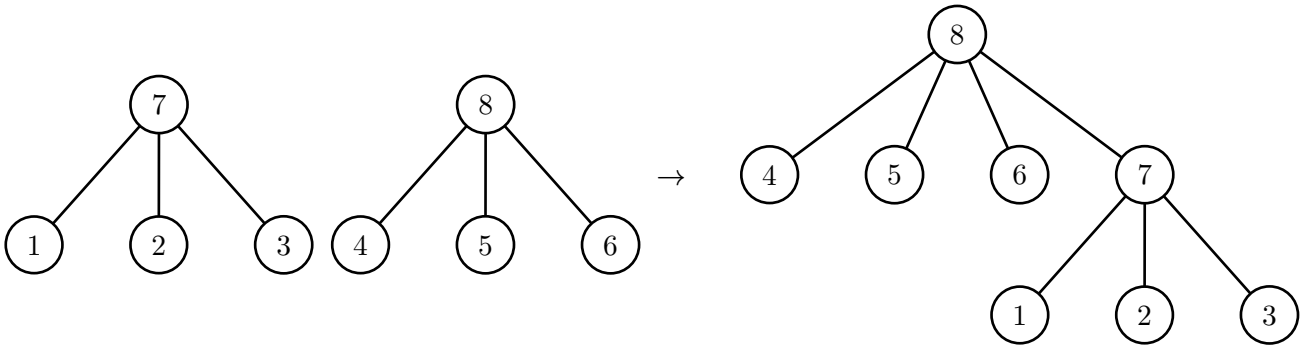
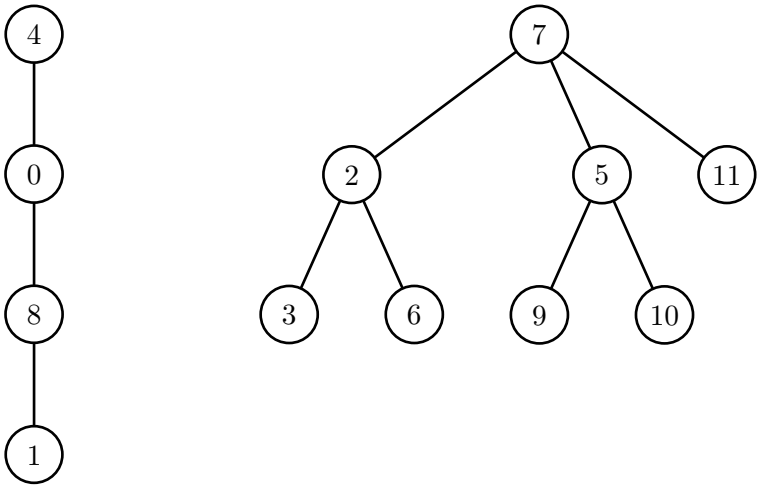


Figure 21.6: Two ideal uptrees, where every node points straight at its root. After a `union`, the nodes of one tree are two steps from the new root instead of one

So we can't keep our trees perfect, but we **can** choose how we union to keep them short and bushy instead of tall. If we unioned 1 into 0, then 2 into 0, then 3 into 0, and so on, we'd get a perfect tree instead of the line in **Figure 21.5**

21.5 Smart Union

When we union two trees, which one should point at the other? Consider the two sets in **Figure 21.7**



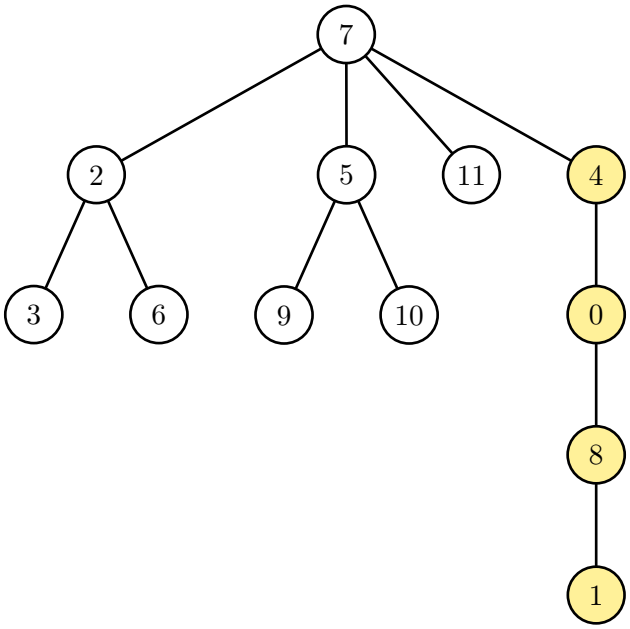
4 nodes, height 3 8 nodes, height 2

Figure 21.7: Example: a tall set with only 4 nodes, and a short, bushy set with 8 nodes

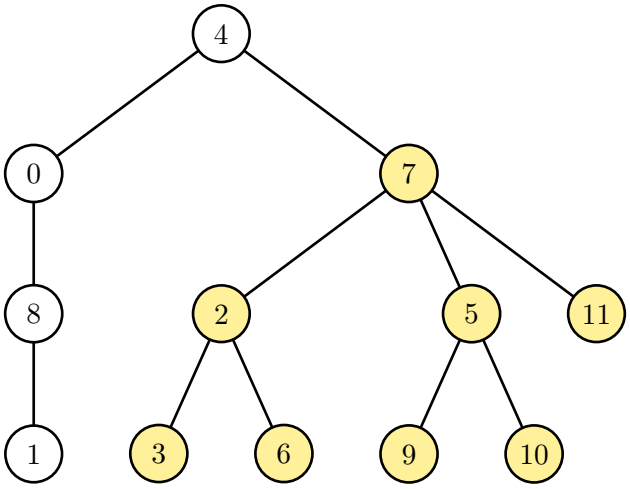
There are two reasonable rules

- **Union by size** - Point the root of the set with **fewer nodes** at the root of the set with more nodes
- **Union by height** - Point the root of the **shorter** tree at the root of the taller tree

Both make the obvious choice in cases like unioning a single node into a big tree, but they disagree in **Figure 21.7**. Union by size points 4 at 7, and union by height points 7 at 4. You can see both results in **Figure 21.8**



Union by size: 4 nodes get one step deeper, but the height grows to 4



Union by height: the height stays 3, but 8 nodes get one step deeper

Figure 21.8: Unioning the sets from **Figure 21.7**. The highlighted nodes are the ones that got farther from the root

Union by size makes the fewest nodes worse: only 4 `finds` get slower, and 8 don't change at all, so on average our `finds` are faster. But union by height never makes the **worst case** worse unless it has to: in **Figure 21.8**, union by size creates a node 4 steps from the root, while union by height keeps the height at 3. So which is better? We'll need to prove something about the heights of these trees to find out

21.5.1 Storing the Size

How do we know the size of each set without counting all of its nodes? Roots store -1, but we can store **any** negative number at a root and still tell that it's a root. So instead of -1, a root stores **$-$size**. No set has size 0, so this always works

In **Figure 21.7**, index 4 stores -4 and index 7 stores -8. To union by size, we compare the two (the **more negative** one is the bigger set), point the smaller root at the bigger one, and store the sum of the two sizes in the new root, which is all $O(1)$. Pointing 4 at 7 stores 7 at index 4 and -12 at index 7

```cpp
// DisjointSets.cpp

/** Points the root of the smaller set at the root of the bigger one
 * Runtime: O(1)
 */
void DisjointSets::unionBySize(int r1, int r2) {
    // sizes are stored negated, so this is the merged set's size, negated
    int newSize = data_[r1] + data_[r2];
    // the more negative root is the bigger set
    if(data_[r1] < data_[r2]) {
        data_[r2] = r1;
        data_[r1] = newSize;
    } else {
        data_[r1] = r2;
        data_[r2] = newSize;
    }
}
```

Since a root can now store any negative number, `find` checks for `data_[k] < 0` instead of `data_[k] == -1`

21.5.2 Storing the Height

We could try the same trick with heights: store $-$height at the root. In **Figure 21.7**, index 4 would store -3 and 7 would store -2. But there's a problem: a brand new element has height 0, and -0 is just 0, which looks like "my parent is index 0"

The fix is a little clunky but straightforward. We only care about the heights **relative to each other**, so store **$-$(height $+1$)** instead. Index 4 stores -4 and index 7 stores -3, and a new element stores -1 just like before

When does the height change? When we hang the shorter tree under the taller tree's root, the shorter tree's nodes all get one step deeper, so the new height is the bigger of the taller height and the shorter height $+1$. If the heights are different, the shorter height $+1$ is at most the taller height, so nothing changes. The height only grows when the two trees are the **same** height, and then it grows by exactly 1. If they're equal, it doesn't matter which root we pick, as long as we subtract 1 from the new root

```cpp
// DisjointSets.cpp

/** Points the root of the shorter uptree at the root of the taller one
 * Runtime: O(1)
 */
void DisjointSets::unionByHeight(int r1, int r2) {
    // the more negative root is the taller tree
    if(data_[r1] < data_[r2]) {
        data_[r2] = r1;
    } else if(data_[r1] > data_[r2]) {
        data_[r1] = r2;
    } else {
        // same height: the new tree is one taller
        data_[r1] = r2;
        data_[r2]--;
    }
}
```

Both of these are smart: union by height only makes a tree taller when the heights are equal, and union by size changes the running time of the fewest nodes. But how good are they really?

21.6 Analysis of Smart Union

We want to show that with either rule, an uptree with n nodes has height $O(\log n)$. Just like with AVL trees, we prove the inverse: a tree of height h must have **at least 2^h nodes**, so $n \geq 2^h$, which means $h \leq \log n$

21.6.1 Union by Size

Union by size doesn't obviously attack the worst case, so it's the trickier one to prove. Instead of inducting on the height, we induct on the number of **links**: the number of times `union` has been called on the nodes in the tree. `union` is the only way a tree changes shape. Linking a tree built with 3 links to a tree built with 2 links makes a tree with $3 + 2 + 1 = 6$ links, so both pieces always have fewer links than the result

Claim: Every uptree built with union by size has $n \geq 2^h$ nodes

Base case: A tree with 0 links has never had `union` called on it, so it's a single node with height 0, and $1 \geq 2^0$

Inductive hypothesis: Every tree built with fewer than i links has $n \geq 2^h$

Inductive step: Any tree R' with i links was made by linking two trees with fewer links: S was hung below the root of R. Since we union by size, R has **at least as many nodes as S**

- **Case 1:** $h(R) > h(S)$. Hanging the shorter S below R's root doesn't make the tree taller, so $h(R') = h(R)$. But we added nodes, so $n(R') > n(R) \geq 2^{h(R)} = 2^{h(R')}$
- **Case 2:** $h(R) \leq h(S)$. Now the tallest path goes through S, so $h(R') = h(S) + 1$. R has at least as many nodes as S, so

$$n(R') = n(R) + n(S) \geq 2 \cdot n(S) \geq 2 \cdot 2^{h(S)} = 2^{h(S)+1} = 2^{h(R')}$$

Either way, $n(R') \geq 2^{h(R')}$. Taking the log of both sides gives $h \leq \log n$, so the height is $O(\log n)$

Intuitively, for a tree to get tall, it needs a lot of unions, which bring in a lot of nodes, and since the bigger tree always goes on top, it was probably already the taller one anyways

21.6.2 Union by Height

This proof is more direct, since we can induct on the height itself

Claim: Every uptree built with union by height has $n \geq 2^h$ nodes

Base case: A tree of height 0 has 1 node, and $1 \geq 2^0$

Inductive hypothesis: Every tree of height less than k has $n \geq 2^h$

Inductive step: With union by height, the only way to get a tree of height k is to link two trees that both have height $k-1$ (if one were shorter, it wouldn't make the result taller). By the inductive hypothesis, each has at least 2^{k-1} nodes, so together

$$n \geq 2^{k-1} + 2^{k-1} = 2 \cdot 2^{k-1} = 2^k$$

So again, $h \leq \log n$

Either rule gives us `find` in $O(\log n)$ and `union` in $O(1)$, which is already one of the best runtimes we've seen for a data structure. But we can do better

21.7 Path Compression

Suppose we call `find(6)` on the uptree on the left of **Figure 21.9** (a tree this tall couldn't actually come from smart union without a lot more nodes, but the idea is the same). `find(6)` calls `find(4)`, which calls `find(2)`, then `find(7)`, `find(9)`, and finally `find(10)`, which is the root. Then 10 gets passed all the way back down

That's correct, but on the way back down, every one of those calls **knows** the root is 10. So before 6 returns, it might as well update itself to point straight at 10, and so might 4, 2, and 7 (9 already points at 10, but updating it doesn't hurt). This technique is called **path compression**, since it compresses the long path into a short, bushy tree, which you can see on the right of **Figure 21.9**

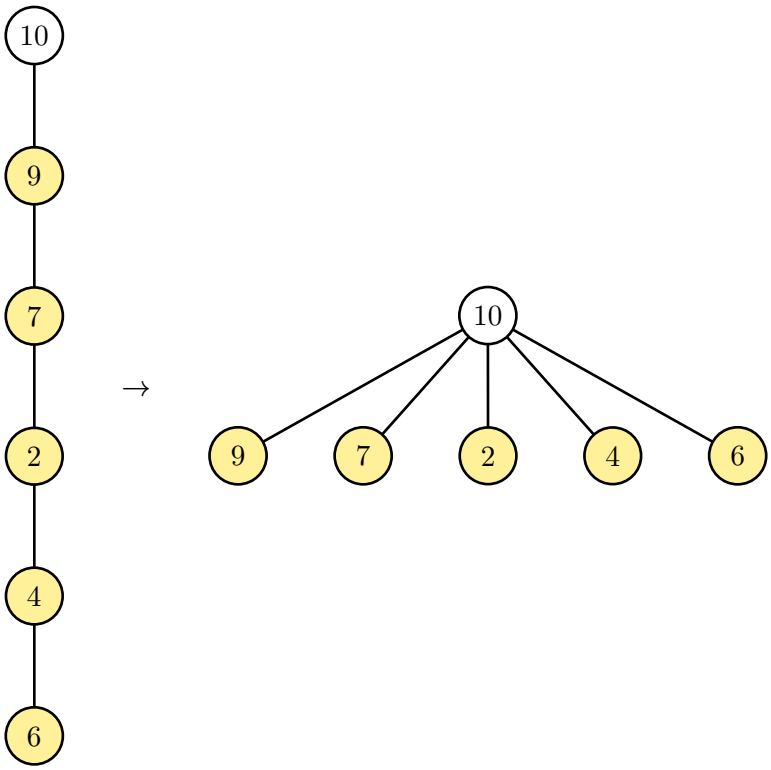


Figure 21.9: Example: calling `find(6)` with path compression. Every node on the path from 6 to the root ends up pointing directly at the root, 10

The code only adds a tiny constant amount of work, since we're already returning through every call on the path. Instead of just returning the result, each call stores it first

```
// DisjointSets.cpp

/** Returns the root of `k` and points every node on the path at it
 * Runtime: amortized O(α(m, n))
 */
int DisjointSets::find(int k) {
    // roots store a negative number (the size or height), so `k` is the root
    if(data_[k] < 0) { return k; }
    // point k straight at the root before returning it
    data_[k] = find(data_[k]);
    return data_[k];
}
```

This doesn't eventually give us a perfect tree: if we keep calling `find(4)`, nothing outside of 4's path ever improves. But every node we `find`, and every node on its path, gets better. It's a bit like a cache: something we never look at never improves, but we never look at it, so who cares?

21.7.1 Union by Rank

Path compression changes the heights of trees, and there is no cheap way to keep track of a tree's height after compressing paths in it. So union by height doesn't work anymore. Instead, we use **union by rank**

- Every new uptree starts with a **rank** of 0
- When we union two roots, the root with the **larger rank** becomes the new root
- If the two ranks are **equal**, pick either one as the new root and increase its rank by 1

Without path compression, this is exactly union by height, so rank and height are the same thing. But rank is tied to the **history** of the unions, not the current shape of the tree, so path compression doesn't change it: making a tree shorter never reduces its rank. We store it the same way as the height, as $-(\text{rank}+1)$ at the root, so the code is the same as `unionByHeight`

Some things are always true about ranks

- The rank of every node is **less** than the rank of its parent, since the root with the larger rank always ends up on top
- Ranks only change at roots, and once a node stops being a root, it never becomes one again. Path compression only moves nodes **up** the tree, and everything below them moves up too, so everything above a node still has a larger rank

The same proof as union by height (with "rank" instead of "height") shows that a tree of rank r has at least 2^r nodes, even with path compression

21.8 Iterated Log

To describe how good this is, we need a new function. $\log n$ counts how many times we have to **divide** n by 2 to get down to 1. The **iterated log**, $\log^* n$ (read "log star"), counts how many times we have to take the **log** to get down to 1

$$\log^* n = \begin{cases} 0 & \text{if } n \le 1 \\ 1 + \log^*(\log n) & \text{otherwise} \end{cases}$$

n	$\log^* n$
1	0
2	1
4	2
16	3
65,536	4
2^{65536}	5

2^{65536} is way more than the number of atoms in the universe, and $\log^*$ is only 5. It's a really slow-growing function

21.8.1 Analysis Sketch

With smart union and path compression, any sequence of m `union` and `find` operations on n elements takes $O(m \log^* n)$ time total, or amortized $O(\log^* n)$ per operation. The full proof takes more than a whole lecture, so here is just the rough idea

- Once a node stops being a root, its rank never changes again
- Group the non-root nodes into **buckets** by rank, where each bucket covers a range of ranks (like [1], [2], $[3, 4]$, $[5, 16]$, $[17, 65536]$, …). There are only $\log^* n$ buckets, and since a tree of rank r has at least 2^r nodes, high-rank buckets can't hold many nodes
- Charge the work of every `find` step by step: steps of distance 0 or 1 get charged to the `union` that created the root, and every other step moves a node up to a parent of higher rank, which gets charged to the node moving through the buckets. Since there are only $\log^* n$ buckets, the total charge works out to $O(m \log^* n)$

The interesting idea here is **charging** work to different places, similar to how we charged copies to inserts in amortized analysis

21.8.2 The Real Bound

$O(m \log^* n)$ is great, but the best bound that has been proven is even better: m operations take $O(m \cdot \alpha(m, n))$ time, where α is the **inverse Ackermann function**

The Ackermann function grows absurdly fast. For example, $A(4, 2) = 2^{65536} - 3$, which is 19,729 digits long. So its inverse grows even slower than $\log^*$: where $\log^*$ reaches 5, α is only around 3. Technically α isn't constant, since it does go to infinity as n goes to infinity, just very, very slowly: for any input that could fit in the universe, it's at most about 4

So a disjoint set implemented with smart union and path compression runs a sequence of m `union` and `find` operations in worst case $O(m \cdot \alpha(m, n))$ time. Dividing by m, that's an amortized $O(\alpha(m, n))$ per operation, which is **effectively constant**. Any single operation might take longer (like a `find` after a long chain of unions), but averaged over all of them, it's practically $O(1)$

Implementation	**`find`**	**`union`**
Array of representatives	$O(1)$	$O(n)$
UpTrees	$O(n)$	$O(1)$
UpTrees + smart union	$O(\log n)$	$O(1)$
UpTrees + smart union + path compression	amortized $O(\alpha(m, n))$, effectively $O(1)$	amortized $O(\alpha(m, n))$, effectively $O(1)$

Disjoint sets turn out to be a very useful tool for a number of algorithms on **graphs**, which are coming up next

22 Graphs

Graphs are our last **deterministic** data structure. Every data structure we've seen so far is completely determined by its inputs: if I told you what operations you were going to do, you could describe exactly what the data structure would look like. After graphs, we'll move on to **randomized** data structures, where that's no longer true

Looking back, we can split everything we've seen into two halves

- **Linked** data structures, built with pointers: doubly linked lists and most of our trees (binary trees, BSTs, AVL trees, KD trees, B-trees)
- **Array** data structures: sorted and unsorted arrays, stacks and queues (which limit how we use an array for efficiency), heaps (a tree stored in an array), and disjoint sets (uptrees stored in an array)

Arrays really are the fundamental data structure. Memory is just a bunch of bytes in a line, and everything else is built on top of that

22.1 Why Graphs?

We can represent pretty much anything as a graph. It's not always the best way to do it, but it is a very useful one

- **The internet** - Routers are vertices and connections are edges. We want short paths between routers, but also **redundancy**, so if someone digs up a cable with 10,000 connections in it, traffic can take another route
- **Control flow graphs** - A compiler breaks code into **basic blocks**: chunks where if the first instruction runs, all of them run. The blocks are vertices, and the edges are where execution can go next (looping back, branching, returning). A **data flow graph** instead has an edge from wherever a value is written to wherever it's read
- **Literature** - In a graph of a Shakespeare play, characters are vertices, there's an edge whenever two characters talk to each other, and a node's size is how many lines that character has. It shows who's important and how the play flows
- **Exam scheduling** - Each class is a vertex, with an edge between two classes if some student is taking both. Coloring the graph so no edge connects two vertices of the same color tells us how many exam time slots we need. Graph coloring is hard: the best algorithm we know amounts to trying all the options
- **Puzzles** - In a sliding car puzzle, every possible configuration is a vertex, with an edge between two configurations if one move turns one into the other. Solving the puzzle is just finding a path from the start to a winning configuration, and a shortest path algorithm finds the fastest solution
- **Prerequisites** - Classes are vertices, with directed edges for prerequisites

- **State machines** - A machine that decides whether a number is divisible by 7 is just a graph you walk through one digit at a time
- **3D meshes** - The triangles covering the surface of a 3D model form a graph, and we can run graph algorithms on the mesh to ask questions about its structure, like whether it has holes

Graphs also show up when we never build one explicitly. In MP Traversals, the image **is** the graph: every pixel is a vertex, and two neighboring pixels are connected if the neighbor's color is within the tolerance. Often we'll have data we care about for some other reason, and we'll just **treat** it as a graph

That's also why there isn't really one graph ADT. Algorithms usually only care about a small set of questions. A traversal only ever asks "what are the vertices?" and "is A connected to B?"

22.2 Graph Vocabulary

A **graph** $G = (V, E)$ is an ordered pair of a set of **vertices** V and a set of **edges** E. The number of vertices is $n = |V|$ and the number of edges is $m = |E|$. Yes, n and m are easy to mix up, but that's what everyone uses

Figure 22.1 shows one graph G that we'll use for the rest of the vocabulary. It has three separate pieces, which we'll call G_1, G_2, and G_3

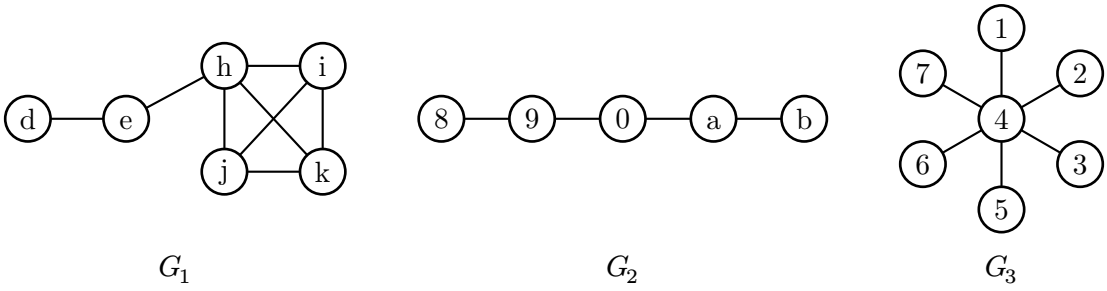


Figure 22.1: Example: one graph G made of three pieces, G_1, G_2, and G_3

- **Incident edges** - The incident edges of a vertex v are the edges that touch it: $I(v) = \{(x, v) \in E\}$. In **Figure 22.1**, the only edge incident on d is the one between d and e, while 4 has six incident edges. Asking for a vertex's incident edges is one of the most common things a graph algorithm does. In MP Traversals, they're just the up, down, left, and right neighbors, if they exist
- **Degree** - The degree of v is the number of incident edges, $\deg(v) = |I(v)|$. So $\deg(d) = 1$ and $\deg(4) = 6$
- **Adjacent vertices** - The vertices connected to v by an edge, $A(v) = \{x : (x, v) \in E\}$

22.2.1 Paths and Cycles

A **path** is a sequence of vertices connected by edges. In theory classes, this is usually called a **walk**, and a path isn't allowed to repeat vertices. In this class, **path** allows repeats: going from 1 to 4 to 2 and back to 4 is a perfectly fine path

A **simple path** is a path that never reuses a vertex. Most of the time, either our paths will be simple just because of the problem, or we won't care

A **cycle** is a path with at least three vertices whose first and last vertices are the same, and that doesn't otherwise reuse any vertices. In **Figure 22.1**, h, i, k, h is a cycle

22.2.2 Simple Graphs

A **simple graph** has **at most one edge** between any two vertices, and no edge that leaves a vertex and comes straight back to itself (a **self-loop**). Unless we say otherwise, every graph we talk about is simple: most graphs can be made simple, and most algorithms are much easier to reason about on simple graphs

22.2.3 Subgraphs

A **subgraph** $G' = (V', E')$ of G is a graph where

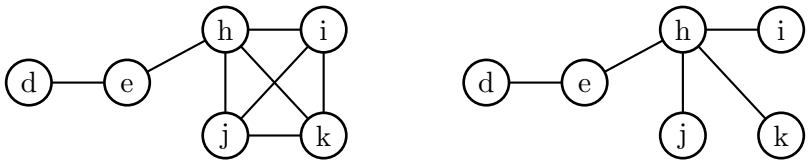
- $V' \subseteq V$ and $E' \subseteq E$, so every vertex and edge of the subgraph is in the original graph
- If $(u, v) \in E'$, then $u \in V'$ and $v \in V'$. This just says every edge in the subgraph has **both** of its endpoints in the subgraph. The single vertex 5 is a fine subgraph, but 5 with its edge to 4 (and without 4) isn't a graph at all, since that edge goes nowhere

Some special kinds of subgraphs

- **Complete subgraph** - Every vertex is connected to every other vertex. In **Figure 22.1**, h, i, j, and k with all of their edges form a complete subgraph, a K_4 hiding inside G_1
- **Connected subgraph** - There's a path between every pair of its vertices. h, i, j, k is connected, and so is the line 8, 9, 0, a, b
- **Connected component** - A connected subgraph that is as big as possible, meaning nothing else could be added to it and keep it connected. G has exactly three connected components: G_1, G_2, and G_3. Just h, i, j, k is connected, but it's not a connected component, since d and e are connected to it too
- **Acyclic subgraph** - A subgraph with no cycles. This is exactly what MP Traversals does: when it reaches a neighbor it has already visited, it doesn't follow that edge, which is the same as removing the edge
- **Spanning tree** - A subgraph that is **connected** and **acyclic** and includes **every** vertex of the graph. It has just enough edges to keep everything connected without any cycles

A graph is **connected** if there's a path from every vertex to every other vertex. A connected graph has one connected component, and a graph with more than one has at least two vertices with no path between them

G isn't connected, so it has no spanning tree at all. But G_3 is already a spanning tree of itself. And starting from any connected graph, we can keep removing edges that are part of a cycle until we're left with a spanning tree, like G_1 in **Figure 22.2**. Spanning trees end up being really useful: network routing is built around maintaining spanning trees over a distributed system



G_1 A spanning tree of G_1

Figure 22.2: Removing the edges ij, ik, and jk, which were all part of cycles, leaves a spanning tree of G_1 with $6 - 1 = 5$ edges

22.3 How Many Edges?

- **Not connected** - The minimum is 0. A graph of isolated vertices with no edges at all is a perfectly fine graph
- **Connected** - The minimum is $n - 1$, since the smallest connected graph is a tree
- **Simple** - The maximum is a **complete graph**, where every vertex is connected to every other vertex. Counting the edges one vertex at a time (without counting any edge twice), the first vertex has $n - 1$ edges, the next has $n - 2$ new ones, and so on, giving $(n - 1) + (n - 2) + ... + 1 + 0 = \frac{n(n-1)}{2}$. **Figure 22.3** shows K_5 with $\frac{5 \cdot 4}{2} = 10$ edges
- **Not simple** - There's no maximum, since we can add as many edges as we want between the same two vertices

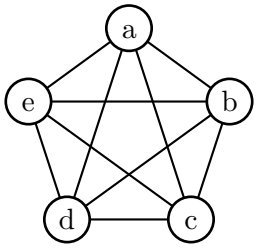


Figure 22.3: The complete graph K_5 has $\frac{5 \cdot 4}{2} = 10$ edges, the most any simple graph with 5 vertices can have

22.3.1 The Handshaking Theorem

$$\sum_{v \in V} \deg(v) = 2m$$

If we add up the degrees of every vertex, we get exactly twice the number of edges, since every edge is incident on two vertices and gets counted once at each end. In G_3, the degrees add up to $6 + 1 + 1 + 1 + 1 + 1 + 1 = 12 = 2 \cdot 6$. This holds for non-simple graphs too

This might seem like a quaint little theorem, but almost every graph algorithm from now on gets its runtime from it. Exploring a graph means starting at a vertex, looking at its neighbors, then their neighbors, and so on, which ends up checking the neighbors of every vertex. That's $\sum_{v \in V} \deg(v) = 2m$ work

23 Graph Implementations

Recall that in this class, a **path** is any sequence of vertices connected by edges, even with repeats, and a **simple path** never repeats a vertex (theory classes call these a **walk** and a **path**). Everything else about graphs is the same as classical graph theory

23.1 The Graph ADT

What does a graph data structure need to do? Its operations fall into two groups

Changing the graph

- `insertVertex` and `removeVertex`
- `insertEdge` and `removeEdge`

Asking about the graph

- `incidentEdges(v)` - The edges that touch `v`. This is basically the same as asking for `v`'s **adjacent vertices**, since from one we can get the other, so which one we use depends on the algorithm
- `areAdjacent(v1, v2)` - Is there an edge between `v1` and `v2`? This is exactly what MP Traversals asks when it checks whether a neighbor is within the tolerance: if it's not, there is no edge, so the traversal doesn't go there
- `origin(e)` and `destination(e)` - Only for **directed** graphs, where an edge from u to v is different from an edge from v to u. A simple directed graph can have both, since there's still at most one edge from the same place to the same place

In practice, applications often don't use a general graph data structure through these operations. Our other data structures (like an AVL tree mid-rotation) are allowed to be invalid in the **middle** of an operation, but if `insertVertex` and friends are an exposed interface, the graph has to be valid after every single one, which controls their runtime. Real code often makes a huge batch of changes at once, where the graph might be nonsense in the middle, as long as it's a proper graph again by the end

23.2 Storing the Vertices

Vertices can have any names, but just like with disjoint sets, our graph data structures always number the vertices 0 to $n-1$. If they have real names, we keep a separate dictionary that maps each name to its number. That way, finding vertex 7 is just going to index 7 of an array, which is $O(1)$

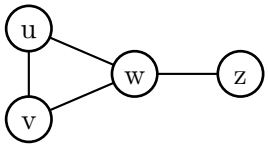
Each vertex can store whatever extra data we want, like a name, a color, or a weight. In MP Traversals, the vertices are pixels, and the extra data is each pixel's color. That data is useful, but it isn't part of the **structure** of the graph

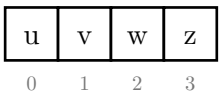
23.3 Edge List

The simplest idea follows the definition of a graph exactly: $G = (V, E)$ is a set of vertices and a set of edges. So we store

- A vector of **vertices**, where vertex i is at index i
- A linked list of **edges**. Each edge can store whatever extra data we want, but it **must** store its two endpoints, by their vertex numbers

Figure 23.1 stores a small graph with four vertices, u, v, w, and z, numbered 0 through 3





Edge	Endpoints
a	0, 1
b	0, 2
c	1, 2
d	2, 3

Figure 23.1: Example: a graph (left) stored as an edge list: a vector of its vertices (middle) and a list of its edges, each storing its two endpoints (right)

```
struct Edge {
    int u;
    int v;
    // plus any extra edge data, like a weight
};

std::vector<Vertex> vertices; // vertex i is stored at index i
std::list<Edge> edges;
```

It clearly holds all of the information about the graph, since it's exactly the theoretical definition. So how fast is it?

- `insertVertex` - Just `push_back` onto the vertex vector, which is $O(1)$
- `removeVertex` - First, we have to deal with the vertex itself. Shifting everything in the vector over would be a mess, so instead we use **tombstoning**: mark the vertex as gone and add its number to a list of available numbers, which `insertVertex` reuses before pushing back. That's $O(1)$, and the only cost is that the vector never shrinks below the most vertices we've ever had. But we're not done: an edge must have both endpoints in the graph, so we also have to remove every edge incident on the vertex. The only way to find them is to check every edge, so `removeVertex` is $O(m)$
- `insertEdge` - Add the edge to the front (or back) of the edge list. The list has no order, so this is $O(1)$
- `removeEdge(v1, v2)` - If all we know is the endpoints, we have to check every edge until we find it, which is $O(m)$. Tombstoning the edge wouldn't help, since **finding** it is the slow part

- `incidentEdges(v)` - Walk through the whole edge list and return every edge that touches `v`, which is $O(m)$
- `areAdjacent(v1, v2)` - Walk through the edge list looking for an edge with both endpoints, which is $O(m)$

So we can build a graph in $O(1)$ per operation, but everything else is $O(m)$, which is terrible. Remember that exploring a graph means looking at the incident edges of every vertex, which here takes $O(nm)$ time even if we do nothing else. That could be $O(n^3)$!

Notice that `insertEdge` doesn't check whether the edge already exists. That's a design decision. With our C++ glasses on, we assume the caller knows what they're doing, just like `vector`'s square brackets don't check whether the index is in bounds. In an edge list, checking would make every `insertEdge` cost $O(m)$, which makes loading a graph slow. If we were building an interface where users click to add edges, checking might be worth it

23.3.1 The Edge List as a File Format

So why talk about such a slow data structure? Because almost every program that uses graphs needs to **save** and **load** them, and for that, the edge list is perfect. To save a graph, write every vertex, then every edge, and that's the file. To load it, add every vertex and every edge, each in $O(1)$ time, which is as fast as just reading the file

Almost every graph data set you'll find is stored this way. Really large graphs are stored in binary, but most graphs are small enough to store as text, with one vertex per line followed by one edge per line

```
0 u red 4.5
1 v blue 2.0
2 w red 1.5
3 z green 3.0
0 1
0 2
1 2
2 3
```

Each vertex line starts with the vertex number and then any data we store with it (like a name, color, and weight), and each edge line is its two endpoints followed by any data we store with it

23.4 Adjacency Matrix

The edge list has all of the information, it's just slow to search. So we keep it and add an **index**: an $n \times n$ **adjacency matrix**, where entry (i, j) says whether there's an edge between vertex i and vertex j

Two improvements make the matrix more useful

- In an undirected graph, entry (i, j) and entry (j, i) are the same edge, so we only need half of the matrix (say, where $i < j$). That doesn't change the $O(n^2)$ space, but it does cut it in half

- Instead of storing just true or false, each entry stores a **pointer** to the edge in the edge list (or `nullptr` if there's no edge). Since the edge list is a linked list, following that pointer lets us remove the edge in $O(1)$

Figure 23.2 shows the adjacency matrix for the graph from **Figure 23.1**

	0 (u)	**1 (v)**	**2 (w)**	**3 (z)**
0 (u)		a	b	
1 (v)			c	
2 (w)				d
3 (z)				

Figure 23.2: The adjacency matrix for the graph in **Figure 23.1**. Each entry points to the edge between those two vertices, empty entries are `nullptr`, and the shaded half is never used in an undirected graph

Now our operations are

- `incidentEdges(v)` - Check every entry in `v`'s row (or, if we only store half the matrix, the bent row and column that belong to `v`). That's $O(n)$ instead of $O(m)$. m can be smaller than n, but in a connected graph, which is what we usually work with, m is at least $n - 1$, so moving from $O(m)$ to $O(n)$ is generally an improvement
- `areAdjacent(v1, v2)` - Check one entry, which is $O(1)$. If we only store the upper half, make sure the smaller index comes first, swapping them if needed
- `insertEdge` - Add the edge to the edge list, then store a pointer to it in the matrix, which is $O(1)$
- `removeEdge(v1, v2)` - Now we can find the edge in $O(1)$ by its endpoints and follow the pointer to remove it, so this is $O(1)$
- `insertVertex` - Every row of the matrix needs a new entry, and we need a whole new row. Even with array doubling, that's $O(n)$ (without doubling, it's $O(n^2)$, so you'll see both runtimes depending on what's assumed)
- `removeVertex` - We still have to go through `v`'s whole row and column to remove its edges, which is $O(n)$

So the matrix makes `areAdjacent` and `removeEdge` constant, but building a graph one vertex at a time is now slow, since we keep resizing the matrix. In practice, an adjacency matrix gets used for **static** sets of vertices: count the vertices, create a matrix that size once, then fill it in. There are real graphs with a fixed set of vertices but very dynamic edges, and the matrix is very effective for those. It's also really easy to write, so it gets used a lot, and the few algorithms that only ever ask `areAdjacent` are very fast with it

The other problem is **space**. If the graph is **sparse**, with m on the order of n, we're using $O(n^2)$ space to store $O(n)$ edges. Whether that's a problem depends on whether the graph could ever fill up and whether we can afford to store the full matrix at all

Both of these get used, but there's a better choice for most graphs, which comes next

23.5 Adjacency List

The adjacency matrix tried to speed up **everything** by indexing every possible edge. Let's be more targeted. Think about what almost every graph algorithm does: exploring a graph, looking for a path, or a traversal like the one in MP Traversals all start at a vertex, look at its neighbors, then at **their** neighbors, and so on. The way we learn the structure of a graph is by asking for the neighbors of a vertex, so the operation we really want to be fast is `incidentEdges(v)`

How fast could `incidentEdges(v)` possibly be? Just **telling** someone the edges incident on v takes $\deg(v)$ time: if v has six incident edges, that's six things to hand back, and if it has 10,000, that's 10,000. So the best we can hope for is $O(\deg(v))$, and the simplest way to get it is to store the answer. We keep the vertex vector and the edge list from before, and give every vertex a linked list of its incident edges, called its **adjacency list**

Figure 23.3 shows the adjacency lists for the graph from **Figure 23.1**, with each edge labelled by its name. Edge a connects u and v, so it shows up in both of their lists, and w has three incident edges, so its list has three entries. The lists are **doubly** linked, so we can remove any entry in $O(1)$ once we have a pointer to it. Linked lists are a solved problem, so in practice we'd just use an STL `list` (which is doubly linked) rather than writing our own

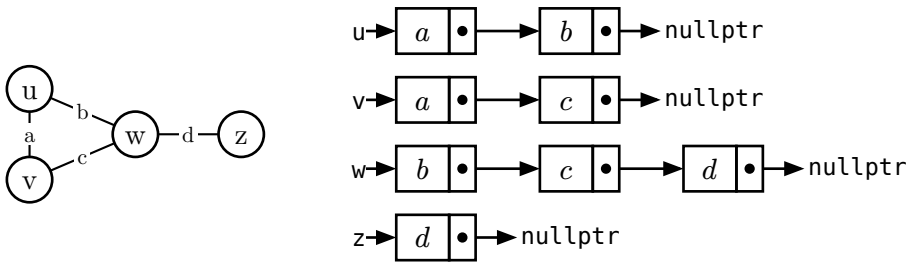


Figure 23.3: The graph from **Figure 23.1** with its edges named (left), and the adjacency list of each vertex (right). The lists are drawn singly linked to keep them readable, but each one is really doubly linked

23.5.1 Connecting the Edges

The adjacency lists make `incidentEdges` fast, but we still can't **find** an edge. Vertices are numbered, so going to vertex 2 is $O(1)$, but an edge's name is its pair of endpoints, and there's nothing to index by that. So, since we love pointers, we add more of them

- Every entry in an adjacency list points to the edge it stands for in the edge list
- Every edge in the edge list points back to its two entries in the adjacency lists, one in each endpoint's list

So edge a points to its entry in u's list and to its entry in v's list, and each of those entries points back at a. That's four pointers for every edge. Here's what each edge in **Figure 23.3** stores

Edge	Endpoints	Points to
a	0, 1	The 1st entry of u's list and the 1st entry of v's list
b	0, 2	The 2nd entry of u's list and the 1st entry of w's list
c	1, 2	The 2nd entry of v's list and the 2nd entry of w's list
d	2, 3	The 3rd entry of w's list and the 1st entry of z's list

That's a lot of pointers to keep straight, and it's exactly the kind of thing that's easy to get subtly wrong. But it's all stuff we've been doing all semester. In C++, an iterator into an STL `list` can play the role of a pointer to one of its elements

```
// Graph.h

struct Edge;
using EdgeIt = std::list<Edge>::iterator; // points to an edge in the edge list

struct Edge {
    int u;
    int v;
    std::list<EdgeIt>::iterator uEntry; // this edge's entry in u's adjacency list
    std::list<EdgeIt>::iterator vEntry; // this edge's entry in v's adjacency list
    // plus any extra edge data, like a weight
};

std::vector<Vertex> vertices;       // vertex i is stored at index i
std::vector<std::list<EdgeIt>> adj; // adj[i] is the adjacency list of vertex i
std::list<Edge> edges;
```

How much space did this cost? Every edge adds two entries to adjacency lists and four pointers, which is a constant amount per edge, and every vertex adds one (possibly empty) list. So the total space is $O(n + m)$, which is as good as it gets, since that's what it takes just to store the vertices and edges at all. Unlike the adjacency matrix, space isn't a concern here

23.5.2 Inserting

- `insertVertex` - Push the vertex onto the vector along with an empty adjacency list, since a new vertex has no edges yet. That's $O(1)$
- `insertEdge(u, v)` - Add the edge to the edge list, then add an entry for it to the end of both u's and v's adjacency lists, and hook up the four pointers. That's a fixed amount of work no matter how big the graph is, so it's $O(1)$

```
// Graph.cpp

/** Adds a vertex with no edges
 * Runtime: O(1)
 */
```

```
void insertVertex(const Vertex& data) {
    vertices.push_back(data);
    // a new vertex starts with an empty adjacency list
    adj.push_back({});
}

/** Adds an edge between u and v
 * Runtime: O(1)
 */
void insertEdge(int u, int v) {
    EdgeIt e = edges.insert(edges.end(), Edge{u, v, {}, {}});
    // add the edge to both endpoints' lists, and remember where it went in each
    e->uEntry = adj[u].insert(adj[u].end(), e);
    e->vEntry = adj[v].insert(adj[v].end(), e);
}
```

That's as good as the edge list, and better than the adjacency matrix, which had to grow a whole new row and column for every new vertex

23.5.3 Removing an Edge

If we already have the edge, removing it is like taking apart anything else built out of pointers. Before we remove the edge itself, we follow its two pointers out to its entries in its endpoints' adjacency lists and remove those, which is $O(1)$ each in a doubly linked list. Then we remove the edge from the edge list, which is also $O(1)$. So with access to the edge, `removeEdge` is $O(1)$

```
// Graph.cpp

/** Removes an edge we already have
 * Runtime: O(1)
 */
void removeEdge(EdgeIt e) {
    // remove the edge's entries from both adjacency lists before the edge itself
    adj[e->u].erase(e->uEntry);
    adj[e->v].erase(e->vEntry);
    edges.erase(e);
}
```

What if all we know is the endpoints, like `removeEdge(v1, v2)`? Then we have to find the edge first: go to `v1` and walk through its adjacency list until we find the edge whose other end is `v2`, which takes $O(\deg(v_1))$ in the worst case. We can do a little better. An STL `list` keeps track of its size, so we can check both degrees in $O(1)$ and search the shorter list, which finds the edge in $O(\min(\deg(v_1), \deg(v_2)))$

In practice, we almost always have the edge already. An algorithm that wants to remove an edge has usually just been **looking at** that edge, which is how it decided to remove it. It doesn't wake up thinking "I bet there's an edge between 7 and 12, let's remove it"

23.5.4 Removing a Vertex

Going to the vertex is still $O(1)$, and we can still tombstone it just like in the edge list. But first we have to remove all of its incident edges, since every edge needs both of its endpoints in the graph. Its adjacency list tells us exactly which edges those are, so

1. For each edge in `v`'s adjacency list, remove that edge
2. Tombstone `v`

```
// Graph.cpp

/** Removes vertex v and every edge incident on it
 * Runtime: O(deg(v))
 */
void removeVertex(int v) {
    // removing an edge erases it from this list, so keep removing the front one
    while (!adj[v].empty()) {
        removeEdge(adj[v].front());
    }
    // then tombstone vertex v, like in the edge list
}
```

Notice we keep removing the **front** edge instead of looping through the list with a range-based `for`, since removing an edge erases its entry from the very list we'd be looping through

Each `removeEdge` is $O(1)$, and we do it $\deg(v)$ times, so `removeVertex` is $O(\deg(v))$. That isn't constant, but it can't be: we **have** to remove every edge connected to `v`, and those are the only things we look at

So every operation that changes the graph is as fast as it could possibly be (other than finding an edge by its endpoints), it just takes a lot of code. That's also why we don't save graphs to disk this way: all of these pointers only exist to make changes fast, and a saved file never changes

23.5.5 Incident Edges

`incidentEdges(v)` just hands back `v`'s adjacency list. Strictly speaking, returning a reference to the list is $O(1)$ and making a copy is $O(\deg(v))$, and you'll see both. Either way, we'll call it $O(\deg(v))$: the reason anyone asks for the incident edges is to go through them next, which takes $O(\deg(v))$ anyway, and $O(\deg(v)) + O(\deg(v))$ is still $O(\deg(v))$

```
// Graph.cpp

/** Returns the edges incident on v
 * Runtime: O(deg(v))
 */
const std::list<EdgeIt>& incidentEdges(int v) {
    return adj[v];
}
```

This is the operation we built the whole thing for, and it's exactly as fast as it can be

23.5.6 Are Adjacent

Here's where the adjacency list loses. To check whether `v1` and `v2` are adjacent

1. Go to whichever of `v1` and `v2` has the smaller degree
2. Check every edge in its adjacency list, and if one of them has the other vertex as its other end, they're adjacent
3. If none of them do, they aren't adjacent

```
// Graph.cpp

/** Checks whether there's an edge between v1 and v2
 * Runtime: O(min(deg(v1), deg(v2)))
 */
bool areAdjacent(int v1, int v2) {
    // make v1 the endpoint with the shorter adjacency list
    if (adj[v1].size() > adj[v2].size()) { std::swap(v1, v2); }
    for (EdgeIt e : adj[v1]) {
        // e's other end is v2 exactly when one of its endpoints is v2
        if (e->u == v2 || e->v == v2) { return true; }
    }
    return false;
}
```

That's $O(\min(\deg(v_1), \deg(v_2)))$, which doesn't sound bad, until we ask how big a degree can get. In a graph where everything is connected to everything, both vertices have degree $n-1$, so `areAdjacent` is $O(n)$ in the worst case, compared to the adjacency matrix's $O(1)$

This is why the adjacency matrix still gets used. If a graph has a fixed set of vertices and lots of edges that keep appearing and disappearing, and the main thing we ask is whether two vertices are adjacent, the matrix is a big win. Think of a city traffic network, where every intersection is a vertex and an edge means the road between them is open. Accidents and construction keep closing and reopening roads, and we keep asking "can I get directly from here to there?" But most graph algorithms rarely ask `areAdjacent`. They explore, which means calling `incidentEdges` over and over

23.5.7 Why Linked Lists?

Why not store each adjacency list as a vector? We could, but removing an entry from the middle of a vector means shifting everything after it over, which is $O(\deg(v))$. Then `removeEdge` would cost $O(\deg(v_1) + \deg(v_2))$ instead of $O(1)$, which is $O(n)$ in a well-connected graph, and `removeVertex`, which removes $\deg(v)$ edges, would go from $O(\deg(v))$ to $O(n^2)$ in the worst case. Worse, our edges point **into** the adjacency lists, and shifting or resizing a vector moves its entries around, so those pointers would stop working

An array is great when we want a block of data. A linked list shines when we keep jumping to arbitrary places inside it, and that's exactly what graph algorithms do. Say we're looking for a path from u to z in **Figure 23.3**. From u, we take edge a to v. In v's list, a is where we came from, so we try c, which takes us to w, and in w's list, d takes us to z. Chasing those pointers is

exactly how a graph algorithm runs. That mess of pointers **is** the graph, just turned from a nice picture into data. That's also why we'll reason about graph algorithms using the nice picture and not the mess, while knowing we could code the mess and get all of these fast operations

23.5.8 The Adjacency List in Practice

The adjacency list is the gold standard general graph format, but in practice it doesn't get used as often as you might expect, because it's a lot of pointer code to get right. The C++ standard library doesn't even have a general graph, for much the same reason. Real programs usually already have their data stored in a structure that makes sense for everything else they're doing, and that data already has connections. A compiler, for example, links instructions and basic blocks to each other in exactly this way, but with structures built for its own data, not a general graph library. So instead of building a graph, we usually treat the data we already have **as** a graph

23.6 Comparing the Implementations

These are the three classic graph implementations. There are fancier variations, and once we've seen randomized data structures, we could revisit them to make some of these operations faster. Each of the three has its place

- **Edge list** - Great for storing graphs on disk, since reading and writing it is as fast as reading and writing the file
- **Adjacency matrix** - Great when we mostly care about `areAdjacent`, and pretty good as long as the set of vertices doesn't change
- **Adjacency list** - The best choice in almost every case, with really only one weakness: `areAdjacent`

From now on, unless we say otherwise, we'll assume a graph is stored as an adjacency list

Operation	**Edge List**	**Adjacency Matrix**	**Adjacency List**
Space	$O(n+m)$	$O(n^2)$	$O(n+m)$
`insertVertex`	$O(1)$	$O(n)$	$O(1)$
`removeVertex`	$O(m)$	$O(n)$	$O(\deg(v))$
`insertEdge`	$O(1)$	$O(1)$	$O(1)$
`removeEdge`	$O(m)$	$O(1)$	$O(1)$
`incidentEdges`	$O(m)$	$O(n)$	$O(\deg(v))$
`areAdjacent`	$O(m)$	$O(1)$	$O(\min(\deg(v_1), \deg(v_2)))$

The adjacency list's $O(1)$ `removeEdge` assumes we already have the edge. If we only know its endpoints, finding it first takes $O(\min(\deg(v_1), \deg(v_2)))$

24 Graph Traversals

In MP Traversals, we've been traversing an image: start at a pixel, visit its neighbors, and if we've already been somewhere, don't go there again. We pretended the image was basically a tree, with a patch for "don't go back where you've been". But that patch is exactly what a real graph traversal does. Now let's write the actual graph algorithms, **breadth-first search** (BFS) and **depth-first search** (DFS)

Just like traversing a list or an AVL tree, traversing a graph means visiting all of its data, and a graph's data is its vertices **and** its edges. So a graph traversal looks at every vertex and every edge

24.1 Labeling the Graph

The graph versions of these algorithms do more than visit vertices: they also **label** every edge with how the traversal used it. So we keep a label on every vertex and every edge

- Every vertex starts **unexplored**, and becomes **visited** when the traversal reaches it. This is the "have I been here before?" structure you built in MP Traversals
- Every edge starts **unexplored**. If the traversal uses an edge to reach a new vertex, it becomes a **discovery edge**. If the edge leads somewhere the traversal had already reached, it becomes a **cross edge** in BFS, or a **back edge** in DFS (we'll see why they get different names)

Do we need the edge labels? Not to traverse: without them, it's the same algorithm, since we can always just check whether we've been to a vertex before. We keep them because the labeled graph is the **output**, and the labels tell us a lot about the graph. In MP Traversals we didn't care about the edges at all, only the vertices, and there's nothing wrong with that either

24.1.1 Traversing Every Component

A flood fill in MP Traversals has a start location, and it only reaches what it can get to from there. A traversal of the **whole** graph might have to start from every vertex, since the graph might not be connected. Think of a graph with 100 vertices and no edges at all. It's not a very exciting graph, but it could happen, and the only way to explore it is to start from vertex 0, then start from vertex 1, and so on. So a full traversal is a wrapper around a helper

1. Label every vertex and every edge as unexplored
2. For each vertex `v`, if `v` is still unexplored, run the traversal starting from `v`

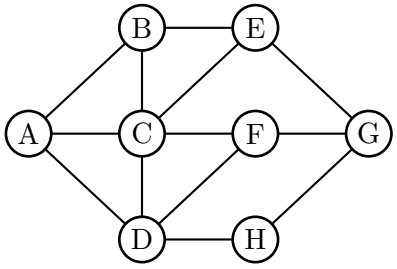
The helper explores everything it can reach from `v`, which is `v`'s whole connected component, while the wrapper makes sure every component gets explored. We call both of these "BFS" (or "DFS"), to be as confusing as possible, given that they're doing quite different jobs

24.2 Breadth-First Search

BFS uses a **queue** as its working data structure. Instead of just vertices, the queue holds pairs: a vertex, and the edge we found it by. That way, when we pull a vertex out, we know which edge to label as its discovery edge. The starting vertex goes in with no edge, since we got there from nowhere

24.2.1 An Example

Figure 24.1 shows the graph we'll use for the rest of this chapter. We'll assume each adjacency list is in alphabetical order, so that's the order we look at neighbors in



Vertex	Adjacent Vertices
A	B, C, D
B	A, C, E
C	A, B, D, E, F
D	A, C, F, H
E	B, C, G
F	C, D, G
G	E, F, H
H	D, G

Figure 24.1: Example: the graph we'll traverse in this chapter (left), and its adjacency lists in alphabetical order (right)

Along with the labels, we'll keep track of two things for every vertex

- d, the **distance**: how many edges we took to get from the start to this vertex
- p, the **predecessor**: the vertex we came from right before reaching this one

Let's run BFS starting from A. In the queue, we'll write the pair "vertex C, found by the edge from B" as $B \to C$. We start by putting A in the queue (with no edge), and then

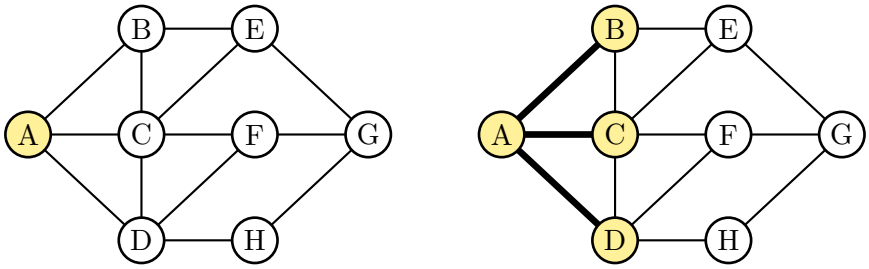
- Dequeue A. It's unexplored, so we visit it, with $d = 0$ and no predecessor. All of its neighbors, B, C, and D, are unexplored, so they all go in the queue
- Dequeue $A \to B$. B is unexplored, so we visit it, and AB is the edge that discovered it, so AB is a discovery edge. B's predecessor is A, and its distance is one more than A's, so $d = 1$. Now we look at B's neighbors. A has already been visited, so we skip it. C hasn't been visited yet (it's in the queue, but nobody has pulled it out), so $B \to C$ goes in, and so does $B \to E$
- C and D come out next, the same way, with $d = 1$ and $p = A$
- Dequeue $B \to C$. We've already visited C, so we don't do anything with it. The edge BC led somewhere we'd already been, so it's a cross edge

The table shows every step. Each row is one dequeue, and the queue is written front first

Dequeued	**What Happens**	**Queue Afterwards**
A	Visit A: $d = 0$, no predecessor	$A \to B$, $A \to C$, $A \to D$
$A \to B$	Visit B: $d = 1$, $p = A$, AB is a discovery edge	$A \to C$, $A \to D$, $B \to C$, $B \to E$
$A \to C$	Visit C: $d = 1$, $p = A$, AC is a discovery edge	$A \to D$, $B \to C$, $B \to E$, $C \to D$, $C \to E$, $C \to F$
$A \to D$	Visit D: $d = 1$, $p = A$, AD is a discovery edge	$B \to C$, $B \to E$, $C \to D$, $C \to E$, $C \to F$, $D \to F$, $D \to H$
$B \to C$	C was already visited, so BC is a cross edge	$B \to E$, $C \to D$, $C \to E$, $C \to F$, $D \to F$, $D \to H$
$B \to E$	Visit E: $d = 2$, $p = B$, BE is a discovery edge	$C \to D$, $C \to E$, $C \to F$, $D \to F$, $D \to H$, $E \to G$
$C \to D$	CD is a cross edge	$C \to E$, $C \to F$, $D \to F$, $D \to H$, $E \to G$
$C \to E$	CE is a cross edge	$C \to F$, $D \to F$, $D \to H$, $E \to G$
$C \to F$	Visit F: $d = 2$, $p = C$, CF is a discovery edge	$D \to F$, $D \to H$, $E \to G$, $F \to G$
$D \to F$	DF is a cross edge	$D \to H$, $E \to G$, $F \to G$
$D \to H$	Visit H: $d = 2$, $p = D$, DH is a discovery edge	$E \to G$, $F \to G$, $H \to G$
$E \to G$	Visit G: $d = 3$, $p = E$, EG is a discovery edge	$F \to G$, $H \to G$
$F \to G$	FG is a cross edge	$H \to G$
$H \to G$	GH is a cross edge	Empty, so we're done

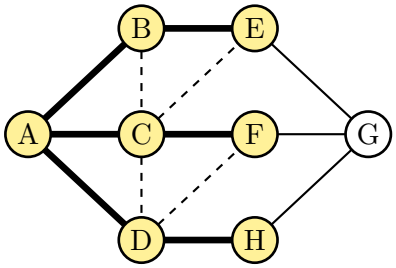
Notice how G ended up in the queue three times, once for each way we found it. Only the first copy to come out visits it, and the other two just label their edges as cross edges

Figure 24.2 shows the same run as pictures. Visited vertices are shaded, discovery edges are bold, and cross edges are dashed. BFS visits A, then everything one edge away from A, then everything two edges away, and finally G, three edges away. It spreads out from the start in **layers**, which is where the name breadth-first comes from

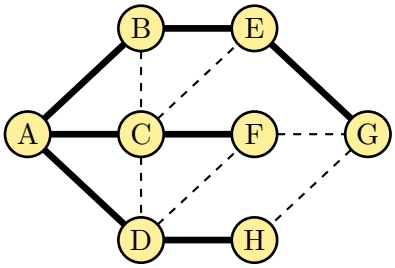


(a) Visit A ($d = 0$)

(b) Visit B, C, and D ($d = 1$)



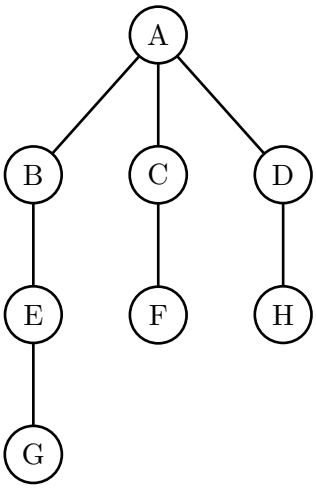
(c) Visit E, F, and H ($d = 2$)



(d) Visit G ($d = 3$)

Figure 24.2: BFS from A on the graph in **Figure 24.1**. Visited vertices are shaded, discovery edges are bold, and cross edges are dashed

When we're done, every vertex has been visited, and every edge has been labeled as either a discovery edge or a cross edge. **Figure 24.3** pulls out just the discovery edges, along with every vertex's d and p



Vertex	d	p
A	0	None
B	1	A
C	1	A
D	1	A
E	2	B
F	2	C
G	3	E
H	2	D

Figure 24.3: The discovery edges from **Figure 24.2** drawn as a tree rooted at A (left), and the distance and predecessor of every vertex (right). Each level of the tree is one distance from A

24.2.2 The Algorithm

To run BFS starting from a vertex `s`

1. Put `s` in an empty queue, with no edge
2. While the queue isn't empty, dequeue a vertex `v` and the edge `e` we found it by, and
   1. If `v` is unexplored, mark it visited and label `e` (if there is one) as a discovery edge. Then, for every edge `f` incident on `v`, if the vertex `w` on its other end is still unexplored, enqueue `w` along with `f`
   2. Otherwise, `v` was already visited, so `e` led somewhere we'd already been, and we label it as a cross edge

24.2.3 When to Mark a Vertex Visited

Many of you marked a pixel as visited in MP Traversals when you **added** it to the queue, not when you took it out. For BFS, that gives exactly the same result, and it's a nice optimization. The first time we find a vertex, it goes in the queue, and since a queue is first in, first out, that's the copy of it that comes out first anyway. Marking it early just keeps the later copies out of the queue

But that only works because BFS uses a queue. As we'll see, DFS is the same algorithm with a stack, and with a stack, the copy that comes out first is the **last** one that went in. So marking a vertex when it goes in changes which edge discovers it. We want one algorithm that works for both, so we mark vertices when they come **out**

24.2.4 The Code

In the code, the graph is an adjacency list, where `adj[v]` holds the numbers of the edges incident on `v`, and each edge knows its two endpoints. The labels live in two vectors indexed by vertex and edge number, and we use -1 for "no edge"

```
// BFS.cpp

enum class Label { UNEXPLORED, VISITED, DISCOVERY, CROSS, BACK };

struct Edge {
    int u;
    int v;
    int other(int w) const { return w == u ? v : u; }
};

struct Graph {
    std::vector<Edge> edges;
    std::vector<std::vector<int>> adj; // adj[v] holds the numbers of v's incident
edges
    int size() const { return adj.size(); }
};

std::vector<Label> vLabel;
std::vector<Label> eLabel;

/** Runs BFS from start, labeling everything it can reach
 * Runtime: O(n + m)
```

```
 */
void BFS(const Graph& g, int start) {
    std::queue<std::pair<int, int>> q; // (vertex, edge we found it by)
    q.push({start, -1});
    while (!q.empty()) {
        auto [v, e] = q.front();
        q.pop();
        if (vLabel[v] == Label::UNEXPLORED) {
            // mark v visited when it comes out, so the same code works for DFS
            vLabel[v] = Label::VISITED;
            if (e != -1) { eLabel[e] = Label::DISCOVERY; }
            for (int f : g.adj[v]) {
                int w = g.edges[f].other(v);
                // only queue neighbors we haven't visited yet
                if (vLabel[w] == Label::UNEXPLORED) { q.push({w, f}); }
            }
        } else {
            // we already reached v by another edge, so e leads somewhere we've
been
            eLabel[e] = Label::CROSS;
        }
    }
}

/** Runs BFS on every connected component of g
 * Runtime: O(n + m)
 */
void BFS(const Graph& g) {
    vLabel.assign(g.size(), Label::UNEXPLORED);
    eLabel.assign(g.edges.size(), Label::UNEXPLORED);
    for (int v = 0; v < g.size(); v++) {
        // start a new BFS from any vertex the earlier ones didn't reach
        if (vLabel[v] == Label::UNEXPLORED) { BFS(g, v); }
    }
}
```

This doesn't keep track of d and p, since we don't need them just to traverse. How would we find them? The predecessor of `v` is the other end of the edge we found it by, which is exactly why we store that edge in the queue. And its distance is one more than its predecessor's. So to keep track of them too, we add two lines right after marking `v` as visited

```
// we came from the other end of e
p[v] = (e == -1) ? -1 : g.edges[e].other(v);
// one more edge than our predecessor
d[v] = (e == -1) ? 0 : d[p[v]] + 1;
```

24.3 What BFS Tells Us

The labeled graph we get from BFS tells us a lot about the graph

24.3.1 Shortest Paths

What's the shortest path from A to H? BFS already found it. Starting at H, its predecessor is D, and D's predecessor is A. Reading that backwards, the path is A, D, H, which takes 2 edges, exactly H's distance d

```
// BFS.cpp

/** Returns a shortest path from the start of the BFS to end
 * Runtime: O(n)
 */
std::vector<int> pathTo(int end) {
    std::vector<int> path;
    // follow the predecessors back to the start, which has none
    for (int v = end; v != -1; v = p[v]) {
        path.push_back(v);
    }
    // we built the path backwards
    std::reverse(path.begin(), path.end());
    return path;
}
```

So if we store the predecessors (or even just the discovery edges), we can recover a shortest path from the start of the BFS to any vertex. This only works because it's a breadth-first search on an **unweighted** graph, where the shortest path is the one with the fewest edges

What about the shortest path from E to H? This BFS doesn't tell us anything about it. It only knows shortest paths from A to everything else (or, since the graph is undirected, from everything else to A). To learn about paths from E, we'd have to run another BFS starting from E

24.3.2 Cross Edges

Our cross edges are BC, CD, CE, DF, FG, and GH. Look at the distances of their endpoints: B and C are both 1, while DF goes from 1 to 2. In fact, the distances at the two ends of a cross edge **never** differ by more than one

Why? BFS finds everything that's one edge away from the start before anything that's two edges away, and everything that's two edges away before anything that's three away. Suppose some edge connected a vertex x to a vertex y with $d(y) \geq d(x) + 2$. When we visited x, y hadn't been found yet (it's farther away), so we'd have put y in the queue right then, and it would have come out at distance $d(x) + 1$. So that edge would have been the discovery edge for y, not a cross edge

For example, if we added an edge between A and G to the graph and ran BFS again, G would be discovered straight from A at distance 1, and EG would become a cross edge instead

24.3.3 Spanning Trees, Components, and Cycles

- **Spanning trees** - The vertices with just the discovery edges form a tree, like in **Figure 24.3**: every vertex other than the start was discovered exactly once, by one edge. If the graph is connected, that tree reaches every vertex, so it's a **spanning tree**. Our graph has 8 vertices and 7 discovery edges, which is exactly $n - 1$
- **Connected components** - The wrapper starts a new BFS once for every connected component, so counting how many times it starts one counts the components
- **Cycles** - If there's a cross edge, the graph has a cycle. A cross edge connects two vertices that were already connected through the tree, so adding it closes a loop. For example, cross edge BC closes the cycle A, B, C, A. If the graph is a tree, BFS never finds a cross edge, no matter where it starts
- **Distances** - d is the length of the shortest path from the start to every vertex

That's a lot of information from one algorithm!

24.4 Running Time of BFS

How long does BFS take? Let's start with the obvious analysis of the code

- Labeling every vertex and every edge as unexplored is $O(n)$ and $O(m)$
- The wrapper's loop runs n times, calling the helper each time
- In the helper, the `while` loop runs once per dequeue. Every vertex has to come out of the queue at least once, and some come out more than once, so let's say $O(n)$
- Inside it, the `for` loop runs once per incident edge, and a vertex can have up to $n - 1$ of those, so that's $O(n)$ too

Three nested loops that each run $O(n)$ times gives $O(n^3)$. That would be a terrible algorithm! Everything we said is true, but it misses something: this isn't some arbitrary data structure, it's a graph, and the loops limit each other

- The body of the `if` runs exactly **once per vertex** over the whole traversal, no matter how many times the wrapper calls the helper, since once a vertex is visited, it never becomes unexplored again
- So the `for` loop runs $\deg(v)$ times for each vertex v, and only once per vertex. In total, it runs $\sum_{v \in V} \deg(v) = 2m$ times
- A vertex only goes in the queue when one of its neighbors is visited and looks at the edge between them, so it can go in at most once per incident edge (plus once if it's a starting point). In total, the queue gets at most $n + 2m$ things, so the `while` loop runs at most $n + 2m$ times, across **every** call of the helper

So the total is $O(n) + O(m) + O(n) + O(n + 2m) + O(2m) = O(n + m)$, not $O(n^3)$. In the worst case, a graph where every vertex is connected to every other vertex has $m \approx \frac{n^2}{2}$, so that's $O(n^2)$. But $O(n + m)$ is optimal: a traversal wants to visit every vertex and every edge, and that's exactly how much work it does

The trick is to step back from the code and think about what the algorithm does. It goes to every vertex once and looks at every edge incident on it. Even if we could teleport straight to every vertex, that's n vertices plus every edge twice (once from each end), $n + 2m$. The number of times

one loop runs affects how many times the other loops can run, even though they're nested, because the structure of the graph doesn't change

This also answers what graph implementation BFS wants. The analysis assumes `incidentEdges(v)` takes $O(\deg(v))$, which is what an adjacency list gives us. With an adjacency matrix, finding the neighbors of each vertex means checking its whole row, which is $O(n)$ per vertex and $O(n^2)$ in total, no matter how few edges there are. With an edge list, it's $O(m)$ per vertex, or $O(nm)$ in total

24.5 Depth-First Search

DFS explores the same graph a different way. The easiest way to think about it is recursively: when we visit a vertex, we go through its neighbors, and for each one, we immediately run DFS on it, before moving on to the next neighbor

24.5.1 An Example

Let's run DFS from A on the graph in **Figure 24.1**, again looking at neighbors in alphabetical order

- Visit A. Its first neighbor is B, so AB is a discovery edge and we call DFS on B
- B's first neighbor is A, but we came from there along AB, which is already labeled. So we move on to C, and BC is a discovery edge
- C's first neighbor is A, which has already been visited, so AC is a **back edge**. Then we skip B (we came from there), and CD is a discovery edge
- From D, AD is a back edge, and DF is a discovery edge
- From F, CF is a back edge, and FG is a discovery edge
- From G, EG is a discovery edge
- From E, BE and CE are both back edges, and E has nothing else to explore, so we return to G
- G still has one more neighbor to try, so GH is a discovery edge. From H, DH is a back edge
- Now everything returns, all the way back to A. Every time we return to a vertex, we check the rest of its neighbors, but all of their edges have been labeled by now, so there's nothing left to do

So DFS visits A, B, C, D, F, G, E, H. It goes as far (as **deep**) as it can before it starts working its way back up, which is where the name depth-first comes from. The recursive calls form a stack, and **Figure 24.4** shows the graph and the call stack at the moment DFS first reaches E

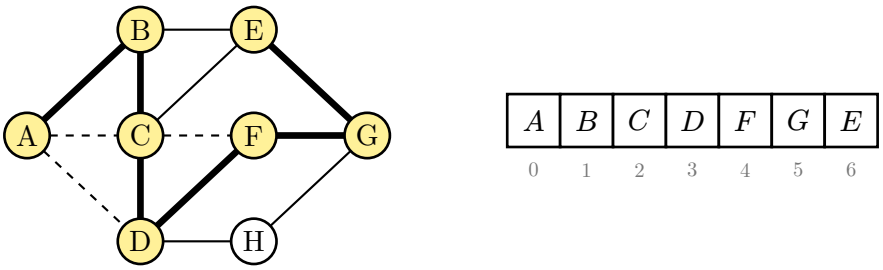


Figure 24.4: DFS from A at the moment it first reaches E (left), and the call stack at that moment, with A at the bottom (right). The number under each vertex is its distance from A in the DFS tree

Figure 24.5 shows the final labeling. Just like BFS, every vertex has been visited, and every edge is either a discovery edge or a back edge

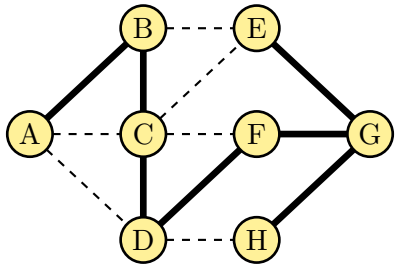
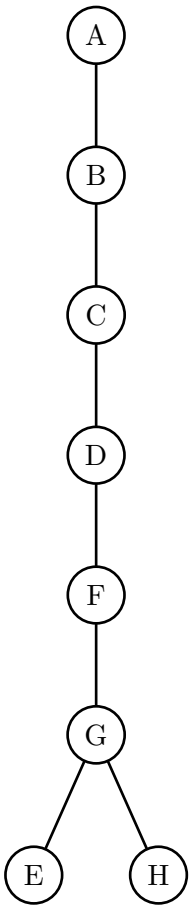


Figure 24.5: The result of DFS from A. Discovery edges are bold, and back edges are dashed

24.5.2 Back Edges

DFS also produces a tree out of its discovery edges, shown in **Figure 24.6**. If we keep track of the distance from A in that tree, we get $A = 0$, $B = 1$, $C = 2$, $D = 3$, $F = 4$, $G = 5$, and $E = H = 6$



Vertex	d	p
A	0	None
B	1	A
C	2	B
D	3	C
E	6	G
F	4	D
G	5	F
H	6	G

Figure 24.6: The discovery edges from **Figure 24.5** drawn as a tree rooted at A (left), and every vertex's distance and predecessor in that tree (right)

Those distances don't have much to do with shortest paths. E is 6 edges from A in the DFS tree, but only 2 edges away in the graph (A, B, E). Compare the long, skinny DFS tree in **Figure 24.6** with the short, bushy BFS tree in **Figure 24.3**: the DFS distances only describe paths in the DFS tree, not in the whole graph. Back edges don't have the nice "at most one apart" property of cross edges either. The back edge BE connects distances 1 and 6, a difference of 5

That's why these get a different name. A cross edge goes **across**, to a vertex at roughly the same level. A back edge always goes **back up**, to an ancestor of the current vertex in the DFS tree, so the vertex on the other end always has a smaller distance. Notice in **Figure 24.4** that when E finds its back edges BE and CE, both B and C are still on the call stack. That's always true. If a back edge led to a vertex that had already finished and returned, then that vertex, while it was still exploring, would have followed the same edge and discovered us first

How far back a back edge goes isn't something we can predict, so it doesn't tell us about shortest paths. But one thing works just like a cross edge: if DFS finds a back edge, the graph has a cycle. If the graph is a tree, there's no cycle, so neither BFS nor DFS will ever find a cross edge or a back edge, no matter where they start. And just like BFS, the discovery edges form a spanning tree if the graph is connected, and the wrapper starts DFS once per connected component

24.5.3 From BFS to DFS

Here's the code for DFS

```
// DFS.cpp

/** Runs DFS from start, labeling everything it can reach
 * Runtime: O(n + m)
 */
void DFS(const Graph& g, int start) {
    // a stack instead of a queue
    std::stack<std::pair<int, int>> s; // (vertex, edge we found it by)
    s.push({start, -1});
    while (!s.empty()) {
        auto [v, e] = s.top();
        s.pop();
        if (vLabel[v] == Label::UNEXPLORED) {
            vLabel[v] = Label::VISITED;
            if (e != -1) { eLabel[e] = Label::DISCOVERY; }
            for (int f : g.adj[v]) {
                int w = g.edges[f].other(v);
                if (vLabel[w] == Label::UNEXPLORED) { s.push({w, f}); }
            }
        } else {
            // a back edge instead of a cross edge
            eLabel[e] = Label::BACK;
        }
    }
}
```

How could we possibly understand such a massive change? The queue became a stack, `front` became `top`, and cross edges became back edges. That's it! The wrapper is the same too, just calling `DFS` instead of `BFS`. This is the heart of MP Traversals: you're really writing one algorithm, and plugging in either a stack or a queue

There's one small difference from the example. A stack pops whatever was pushed **last**, so this version tries a vertex's neighbors in reverse order. From A, it goes to D first, and visits A, D, H, G, F, C, E, B. That's still a depth-first search, it just tries neighbors in the opposite order from the run in **Figure 24.5**. If we wanted the same order as the example, we'd push the neighbors in reverse

24.5.4 The Recursive Version

What about the recursive version we ran in the example? Start from the stack version, and

- Remove the stack, the loop, and the pop, since the recursion does all of that for us
- Instead of pushing a vertex and its edge onto the stack, make a recursive call on them, which uses the **call stack** instead

```
// DFS.cpp

/** Runs DFS from v, which we reached by the edge e
 * Runtime: O(n + m)
 */
void DFS(const Graph& g, int v, int e) {
    if (vLabel[v] == Label::UNEXPLORED) {
        vLabel[v] = Label::VISITED;
        if (e != -1) { eLabel[e] = Label::DISCOVERY; }
        for (int f : g.adj[v]) {
            // check the edge, not the vertex, so the call can find back edges
            if (eLabel[f] == Label::UNEXPLORED) { DFS(g, g.edges[f].other(v),
f); }
        }
    } else {
        // we already visited v, so e goes back to an ancestor
        eLabel[e] = Label::BACK;
    }
}
```

The wrapper calls `DFS(g, v, -1)` for every vertex that's still unexplored

The body is the same, except for one detail: the check before the call looks at the **edge**, not the vertex. In the stack version, we pushed `w` while it was still unexplored, and by the time it came back out, it might have been visited along another path, which is how back edges got labeled. A recursive call happens **right away**, so if we only called on unexplored vertices, we'd never reach the `else`, and back edges would never get labeled. So instead, we call on every edge we haven't labeled yet, and let the call find out that its vertex was already visited. The edge we came in on is already a discovery edge, so we never walk straight back along it

Going in this direction makes it look easy. Going the other way, from a recursive DFS to an iterative one, is trickier, since we have to figure out exactly what needs to go on the stack, and when we've actually "gotten" somewhere. But it's interesting that the same algorithm can turn from one into the other with so few lines of code

24.5.5 Running Time of DFS

We don't need a new analysis. Every operation on a stack is $O(1)$, just like on a queue, and we put exactly the same number of things in and take the same number out. Every vertex gets visited once, and every edge gets looked at once from each end. So DFS is also $O(n + m)$, with an adjacency list

And that's how we traverse a graph: with either breadth-first or depth-first search, both in $O(n + m)$ time

25 MST: Minimum Spanning Tree (Kruskal's Algorithm)

We're finally moving into the last chunk of graphs. We've done the vocabulary, the implementations, and the traversals, so now let's look at graph algorithms that **aren't** traversals. The first one is the **minimum spanning tree**

25.1 Why Minimum Spanning Trees?

Here's a town where every road is a dirt road. The town just got some money, and it's going to pave some of them. Paving is expensive, so every road has a cost (in millions of dollars), and the question is: what's the **cheapest** set of roads we can pave so that everybody can get from anywhere to anywhere driving only on paved roads? **Figure 25.1** shows the town as a weighted graph, with a vertex for every house and an edge for every road

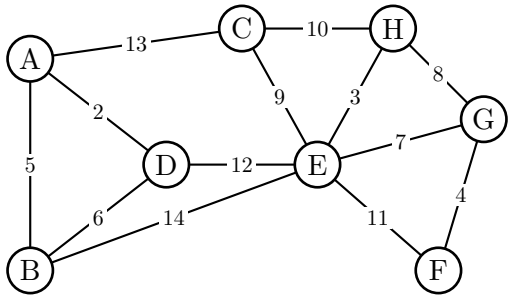


Figure 25.1: Example: a town with 8 houses and 13 dirt roads. Each edge's weight is the cost (in millions) of paving that road

This is a really useful question to be able to answer. It's the same as a networking question: which network links have to be working for me to get a message to every place on my network?

There are a couple of greedy approaches

- **Start from my house** - I live at A, and I'm greedy, so I start paving roads that make it easier for **me** to get places. Every time, I pave the cheapest road that reaches a house I can't drive to yet. That's AD for 2, then AB for 5, then DE for 12 (the cheapest way out of $\{A, B, D\}$), and so on, growing a paved area outwards from my house
- **Cheapest road anywhere** - Someone less selfish ignores where they live. Every time, they pave the cheapest road **anywhere in town** that connects two places that weren't connected yet. That's AD for 2, EH for 3, FG for 4, and so on, with little paved patches all over town that slowly join together

If we carry both of them out, they end up paving exactly the same roads, the 7 bold edges on the left of **Figure 25.2**, for a total of $2 + 3 + 4 + 5 + 7 + 9 + 12 = 42$ million either way. These two strategies are the two algorithms we'll study for this problem. We didn't come up with them, of course: the first is **Prim's algorithm** (next chapter), and the second is **Kruskal's algorithm**, which is this chapter

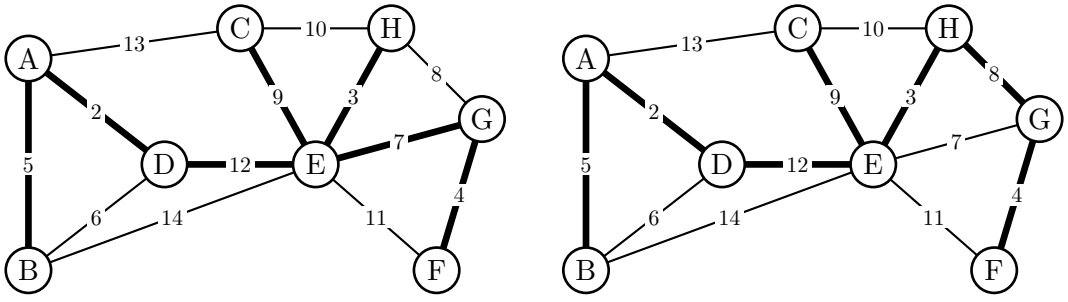
The obvious question is how we keep track of all of this. It's easy when we're looking at a picture and pointing at roads, but in code, how do we know whether a road "connects two places that weren't connected yet"?

25.2 The MST Problem

Formally, the **minimum spanning tree** (MST) problem takes a **connected**, **undirected**, **weighted** graph $G = (V, E)$ and returns a graph $G' = (V, E')$ such that

- G' is a **spanning tree** of G: recall that means it includes every vertex, is connected, and has no cycles, so it has exactly $n - 1$ edges
- G' has the **minimum total edge weight** among all possible spanning trees of G

There are usually a **lot** of spanning trees. From any spanning tree, we can remove one edge, which splits it into two pieces, and add a different edge that reconnects them. In **Figure 25.2**, swapping GE (7) for GH (8) gives another perfectly good spanning tree, but it costs 43 instead of 42



The MST: total weight 42 Another spanning tree: total weight 43

Figure 25.2: Two spanning trees of the town from **Figure 25.1**, shown as the bold edges. Swapping GE (7) for GH (8) still connects everything without a cycle, but it costs more

Notice we said **a** minimum spanning tree. If several edges have the same weight, there can be more than one spanning tree with the minimum total, and any of them is a correct answer. When every edge weight is different, like in our town, there's exactly one MST

25.3 Kruskal's Algorithm

Kruskal's algorithm is the "cheapest road anywhere" strategy: look at the edges from **cheapest to most expensive**, and add an edge to the tree as long as it connects two pieces that weren't already connected. To actually do that, we need answers to two questions

- How do we get the edges in order from cheapest to most expensive?

- How do we know whether an edge connects two things that weren't already connected?

25.3.1 Getting the Edges in Order

What we need is a **priority queue** of edges, ordered by weight, where `removeMin` always hands us the cheapest edge we haven't looked at yet. There are two good ways to build one

- **Sort the edge list** by weight. A sorted list is a perfectly good priority queue, as long as we never `insert` anything into it, and we won't: we just keep track of where we are and move forward one edge at a time
- **Build a heap**. Recall that `buildHeap` turns an array into a heap in linear time, and then we `removeMin` the edges one at a time

We're sorting (or heaping) the **edges**, not the vertices, so these cost $O(m \log m)$ and $O(m)$ to build. Building the heap looks better, since we might not have to remove every edge, but we'll come back to which one is actually better once we look at the whole running time. For the example, we'll use the sorted list, since keeping a heap up to date while also running an example by hand gets messy

25.3.2 Is the Edge Useful?

When we pull an edge (u, v) out of the priority queue, we need to know whether u and v can already reach each other through the edges we've picked. What data structure stores whether two things belong to the same set or not? A **disjoint set**!

We keep a disjoint set of the **vertices**, where each set is one connected piece of the tree we're building. At the start, no edges have been picked, so every vertex is in a set by itself. For every edge (u, v)

- If `find(u) != find(v)`, then u and v are in different pieces, so the edge connects two things that weren't connected. We add it to the tree and `union` the two sets, since those two pieces are now one
- If `find(u) == find(v)`, then u and v are already connected. The edge isn't just useless, it's actively **bad**: there's already a path from u to v in the tree, so adding the edge would make a cycle, and then we wouldn't have a tree anymore. We skip it

This is the "really cool" algorithm we were promised back when we built disjoint sets. Each check is just two `find`s, so with smart union and path compression, it's effectively $O(1)$

25.3.3 An Example

Let's run Kruskal's algorithm on the town from **Figure 25.1**. The disjoint set uses union by size and path compression, and when two sets are the same size, the first root points at the second, like `unionBySize` does. **Figure 25.3** lists the edges in sorted order, along with what the two `find`s return and what we do with each edge

Edge	Weight	`find(u)`	`find(v)`	Result
AD	2	A	D	Add, union
EH	3	E	H	Add, union
FG	4	F	G	Add, union
AB	5	D	B	Add, union
BD	6	D	D	Skip: cycle
GE	7	G	H	Add, union
GH	8	H	H	Skip: cycle
EC	9	H	C	Add, union
CH	10	H	H	Skip: cycle
EF	11	H	H	Skip: cycle
DE	12	D	H	Add, union: done
AC	13			Never looked at
BE	14			Never looked at

Figure 25.3: Kruskal's algorithm on the town from **Figure 25.1**, one edge at a time in order of weight

The first three edges are easy. `find(A)` returns A and `find(D)` returns D, so they're in different sets: we add AD to the tree and union A and D, which points A at D. The same thing happens for EH and FG. Now we have three little paved patches, which you can see at the top of **Figure 25.4**

Next is AB. `find(A)` now returns D, and `find(B)` returns B, so that's still fine: we add the edge and union. D's set is bigger, so B ends up pointing at D too (with union by size, height, or rank, we'd get the same thing here)

Then we get to BD, which is really cheap. But `find(B)` returns D, and `find(D)` returns D. Look at the bottom of **Figure 25.4**: we can already get from B to D through A, so BD would close the cycle B, A, D. We skip it, which is shown with a dashed edge

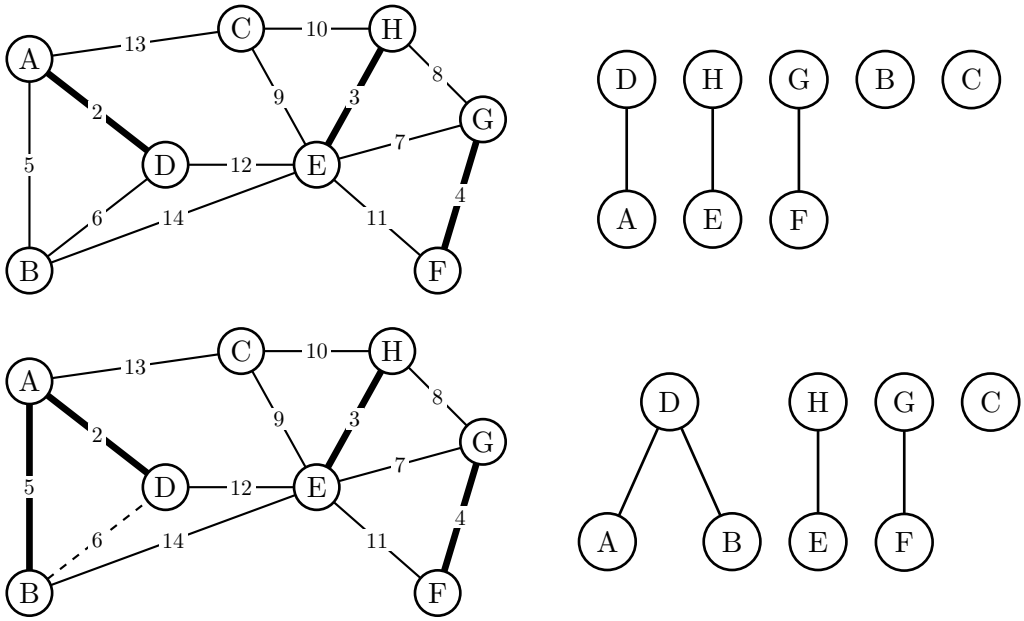


Figure 25.4: Kruskal's algorithm after AD, EH, and FG (top), then after adding AB and skipping BD (bottom). The bold edges are in the tree, the dashed edge was skipped, and the uptrees on the right are the disjoint set

Next, GE: `find(G)` returns G and `find(E)` returns H, so we add it. Both sets have 2 vertices, so G points at H. That makes GH a cycle, since `find(G)` and `find(H)` both return H, so we skip it

EC is fine (`find(C)` is just C), and C joins H's set. But then CH and EF are both cycles: everything on the right side of town is already connected. Notice what path compression does during `find(F)`: F points at G, which points at H, so on the way back, F gets updated to point straight at H. You can see F move in **Figure 25.5**

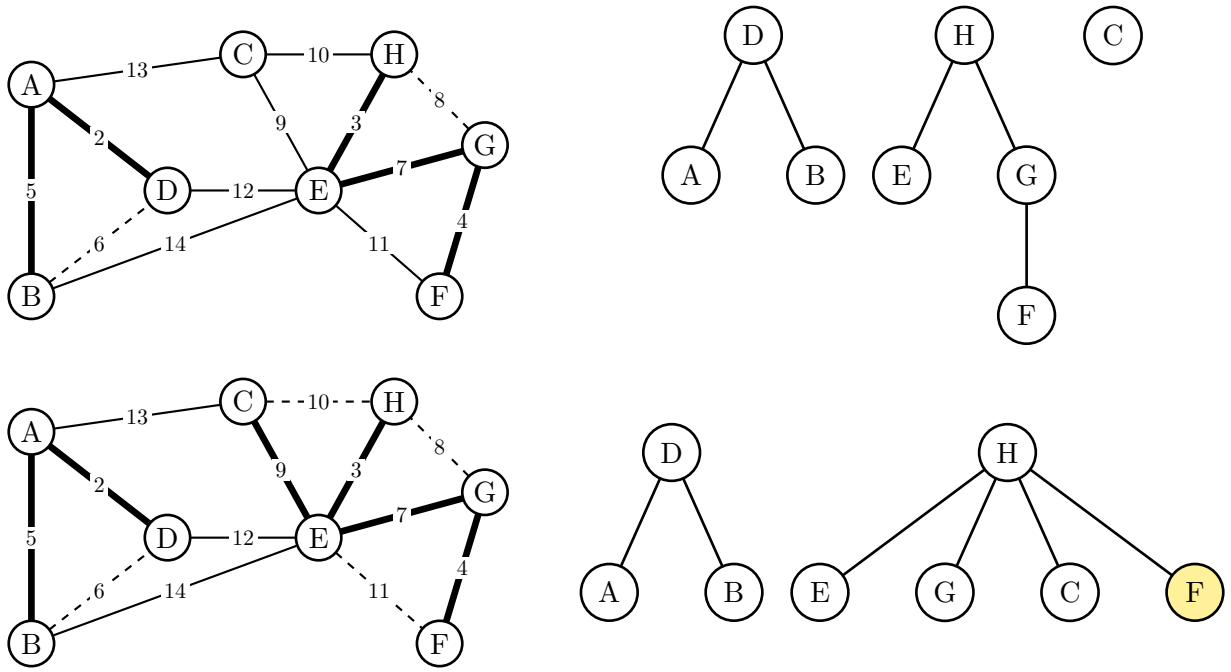


Figure 25.5: Kruskal's algorithm after adding GE and skipping GH (top), then after adding EC and skipping CH and EF (bottom). Path compression during `find(F)` moved F (highlighted) to point straight at the root, H

Finally, DE: `find(D)` returns D and `find(E)` returns H, so we add it, and D's set (3 vertices) gets pointed at H's (5 vertices). Now every vertex is in one set, and we have the minimum spanning tree in **Figure 25.6**. We never even had to look at AC or BE

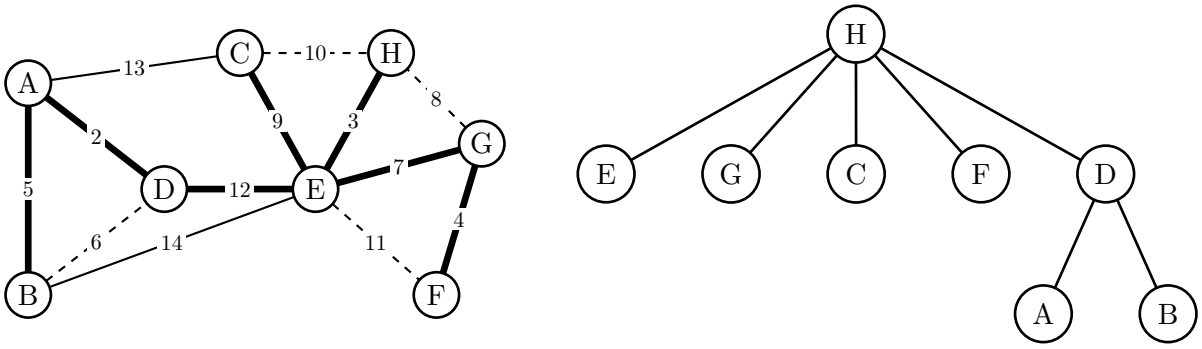


Figure 25.6: After adding DE, the 7 bold edges are the MST, with total weight 42. Every vertex is now in one set, rooted at H

25.3.4 When Are We Done?

We could just keep going until the priority queue is empty, since every remaining edge would be skipped anyway. But it's clearly better to stop as soon as the tree is finished. There are a few ways to notice that

- **Everything is in the same set**. We could call `find` on every vertex and check they all return the same root, but that's $O(n)$ every time we check
- **The set is as big as the graph**. If we union by size, the root already stores the size of its set, so after each `union`, we can check whether the new set has all n vertices
- **Count the edges**. A spanning tree always has exactly $n - 1$ edges, so we keep a counter of how many edges we've added, and stop when it reaches $n - 1$

The last one is the simplest, and it's what our code will do

All of this works because of data structures we already have, and the disjoint set in particular is super fast. Honestly, that's why Kruskal's algorithm is considered efficient and is used so often: without the disjoint set, keeping track of which vertices are already connected would be really hard, and nobody would use it. The disjoint set is what fundamentally enables this algorithm

25.4 The Algorithm

Given a connected, undirected, weighted graph with n vertices and m edges

1. Make a disjoint set with every vertex in its own set
2. Put every edge into a priority queue ordered by weight, either by sorting the edge list or with `buildHeap`
3. Start the tree T with all n vertices and no edges
4. While T has fewer than $n - 1$ edges, remove the cheapest edge (u, v) from the priority queue
5. If `find(u) != find(v)`, add (u, v) to T and `union` the two sets. Otherwise, adding it would make a cycle, so skip it
6. Return T

25.5 The Code

We'll use the `DisjointSets` from the disjoint sets chapter, with `find` doing path compression and `unionBySize` taking two roots. Each edge stores its two endpoints (as vertex numbers 0 to $n - 1$) and its weight, and we return the edges of the tree

```
struct Edge {
    int u;
    int v;
    int weight;
};

/** Builds a minimum spanning tree from the cheapest edges that skip cycles
 * Runtime: O(m log m)
 */
std::vector<Edge> kruskal(size_t n, std::vector<Edge> edges) {
    DisjointSets forest;
    forest.addElements(n);

    // a sorted list is a priority queue, as long as we never insert
```

```
    std::sort(edges.begin(), edges.end(), [](const Edge& a, const Edge& b) {
        return a.weight < b.weight;
    });

    std::vector<Edge> tree;
    size_t next = 0;

    // stop once the tree has n - 1 edges or we run out of edges
    while(tree.size() < n - 1 && next < edges.size()) {
        Edge e = edges[next++];
        int ru = forest.find(e.u);
        int rv = forest.find(e.v);

        // different roots: the edge joins two trees, so it can't make a cycle
        if(ru != rv) {
            tree.push_back(e);
            forest.unionBySize(ru, rv);
        }
    }
    return tree;
}
```

`next` is the "current top" of our sorted priority queue: removing the minimum just moves it forward one edge. The `next < edges.size()` check only matters if the graph isn't connected, in which case we run out of edges before we ever get $n - 1$ of them

To use a heap instead, we swap the sort for a `std::priority_queue`. It's a max-heap by default, so we give it a comparison that puts the **lightest** edge on top. Building it from the whole vector at once runs `buildHeap` (`std::make_heap`), which is $O(m)$

```
struct HeavierThan {
    // max-heap by default, so heavier edges count as smaller (lightest on top)
    bool operator()(const Edge& a, const Edge& b) const {
        return a.weight > b.weight;
    }
};

using EdgeHeap = std::priority_queue<Edge, std::vector<Edge>, HeavierThan>;

// building from the whole vector at once runs buildHeap in O(m)
EdgeHeap pq(HeavierThan(), std::move(edges));
```

Then, instead of `edges[next++]`, each step of the loop takes `pq.top()` and calls `pq.pop()`, and the loop stops when `pq` is empty

25.6 Running Time

Let's go through the algorithm one piece at a time

- **Making the disjoint set** - We put n things into the disjoint set, so we have to touch n things in memory no matter what: $O(n)$
- **Building the priority queue** - This depends on our choice. `buildHeap` (heapify down, starting from the last non-leaf and working back to the root) is $O(m)$, and sorting is $O(m \log m)$. Remember, these are **edges**, so it's m, not n
- **Making T** - A graph with n vertices and no edges is $O(n)$
- **Each `find` and `union`** - Technically this is $O(\alpha(m, n))$, the inverse Ackermann function, but we'll just call it constant because it makes our lives easier
- **Each `removeMin`** - Also depends on our choice. In a heap, we swap the last element to the root and heapify down, which is $O(\log m)$. In the sorted list, we just move `next` forward, which is $O(1)$

25.6.1 How Many Times Does the Loop Run?

It's tempting to say $n - 1$ times, since we stop as soon as the tree has $n - 1$ edges. If that were true, we'd get

- **Sorted list**: $O(m \log m + n)$
- **Heap**: $O(m + n \log m)$

and the heap would clearly win. But that's not what's happening: not every trip through the loop adds an edge. If we always got to add the $n - 1$ cheapest edges, that would be awesome, but there might be one vertex whose only road is the most expensive road in town, and then we have to look at **every other edge** before we can connect it. So the loop runs $O(m)$ times, and the totals are

- **Sorted list**: $O(m \log m)$ to build, plus m removals at $O(1)$ each, for $O(m \log m + m) = O(m \log m)$
- **Heap**: $O(m)$ to build, plus m removals at $O(\log m)$ each, for $O(m + m \log m) = O(m \log m)$

They're the same! The $O(n)$ parts don't matter, since a connected graph has $m \geq n - 1$ edges

Step	**Sorted list**	**Heap**
Make the disjoint set	$O(n)$	$O(n)$
Build the priority queue	$O(m \log m)$	$O(m)$
Make T with no edges	$O(n)$	$O(n)$
$O(m)$ `removeMin`s	$O(1)$ each	$O(\log m)$ each
$O(m)$ `find`s and `union`s	$O(\alpha(m, n))$ each	$O(\alpha(m, n))$ each
Total	$O(m \log m)$	$O(m \log m)$

25.6.2 In Terms of n

What's the worst case for m? In a simple graph, m is at most $\frac{n(n-1)}{2}$, so $m = O(n^2)$. Replacing every m with n^2, the sorted list is $O(n^2 \log n^2 + n^2)$ and the heap is $O(n^2 + n^2 \log n^2)$, and since $\log n^2 = 2 \log n$, both are $O(n^2 \log n)$ for a **dense** graph

More generally, $\log m \leq \log n^2 = 2 \log n$, so $O(m \log m)$ is the same as $O(m \log n)$, which is how Kruskal's algorithm is usually written. For a **sparse** graph, where m is about n, that's $O(n \log n)$

25.7 Sorted List or Heap?

The running times are the same, so which should we use? A more precise answer is that the heap is better when we only have to **examine** a small number of edges, and it doesn't matter when we have to examine most of them

If we're going to run the algorithm once, return the tree, and keep nothing else around, a heap is a fine choice. It's never worse, and sometimes it's a little better: we're hoping we won't have to remove every edge, and that's fairly likely. In fact, the **denser** the graph gets, the less likely it is that we have to wait for the very last edge to connect everything, since for that to happen, there can't be any other edge to that vertex. Plus, we already have heap code everywhere, so it isn't any harder to write

So why would we ever pick the sorted list? A heap gets **destroyed** as we use it, since every `removeMin` takes an edge out. A sorted list doesn't change at all: our priority queue is just a position in the list, and we move it along. So if we're going to do a lot of things with this graph, a list of the edges sorted by weight is a really useful thing to keep around, and we can use it again and again

That's a real design decision

- If we're writing Kruskal's algorithm as a **library**, separate from the rest of the program, we'd probably use a heap
- If we're building a **whole program** that analyzes one big graph, like loading a million-vertex social network graph and running all sorts of analyses and visualizations on it, we might just store the graph's edges in sorted order all the time. We aren't changing the edges, just analyzing them, so the sort only has to happen once

This is where we, as programmers, can do much better than something that can't step back and look at the bigger picture. Graph algorithms aren't particularly fast, and the graphs we want to analyze can be huge, so getting the answer in three hours instead of nine actually matters. These design opportunities are where we can really use our deeper understanding of the data structures to make a difference

25.8 The Partition Property

Why does the "start from my house" strategy work? Being greedy and always paving the cheapest road out of the paved area doesn't sound like it should give the best answer overall. The reason is the **partition property**

Split the vertices of a connected graph into two pieces, U and $V - U$. Some set of edges **crosses** the partition, with one endpoint on each side. It might be one edge or many, but since the graph is connected, it's at least one. The partition property says that the **minimum weight edge crossing the partition** must be part of **some** minimum spanning tree

In **Figure 25.7**, $U = \{A, B, D\}$, and three edges cross the partition: DE (12), AC (13), and BE (14). The cheapest one, DE, is in the MST

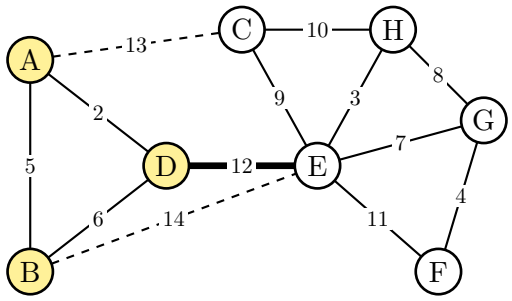


Figure 25.7: Example: partitioning the town into $U = \{A, B, D\}$ (highlighted) and everything else. Three edges cross the partition, and the cheapest one, DE (bold), must be in some MST. The other two are dashed

It's fairly easy to see why. Every spanning tree has to include at least one of the crossing edges, since otherwise nothing in U could reach anything outside it. If a spanning tree doesn't use the cheapest crossing edge, adding it makes a cycle. That cycle crosses the partition, so it has to cross back through some other crossing edge, which is at least as expensive. Removing that edge breaks the cycle and leaves a spanning tree that costs no more. Going from "must be part of **some** MST" to "must be part of the MST this algorithm builds" takes a bit more work as a proof

That's what makes the greedy strategy legal. If U is the set of houses we can already reach, then the cheapest road leaving U is the minimum edge crossing a partition, so every road we pave is one that belongs in a minimum spanning tree. We start from some vertex, add the cheapest edge leaving it, and keep going

26 MST: Minimum Spanning Tree (Prim's Algorithm)

In the previous chapter, we built a minimum spanning tree with Kruskal's algorithm, which looks at the edges from cheapest to most expensive and keeps every edge that doesn't make a cycle. Along the way it grows a whole **forest** of little trees that slowly merge into one. Prim's algorithm takes a different approach: it grows **one** tree, starting from a single vertex, and adds one vertex to it at a time

Both algorithms rely on the same fact, the **partition property** from the end of the previous chapter (**Figure 25.7**): if we split the vertices of a graph into two groups, the minimum weight edge crossing between the two groups is part of some minimum spanning tree. Kruskal's uses it indirectly, but Prim's is basically the partition property turned directly into an algorithm

26.1 The Idea: Grow One Side of the Partition

We pick a starting vertex, say a, and partition the graph into $\{a\}$ and everything else. That's a great place to start, since the minimum spanning tree of just $\{a\}$ is easy: it's a with no edges

Now we look at every edge crossing the partition and add the cheapest one. By the partition property, that edge must be part of some minimum spanning tree. Its far endpoint joins our side, so we **repartition** the graph, look at the edges crossing the new boundary, add the cheapest one, and keep going until every vertex is on our side

Figure 26.1 is the weighted graph we'll use for the rest of this chapter. It has 8 vertices and 15 edges, all with different weights

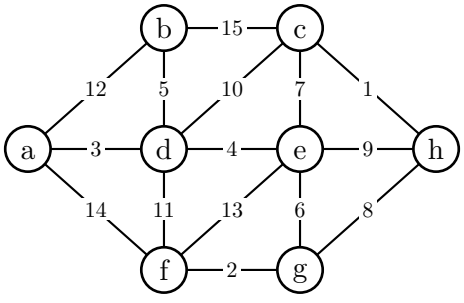
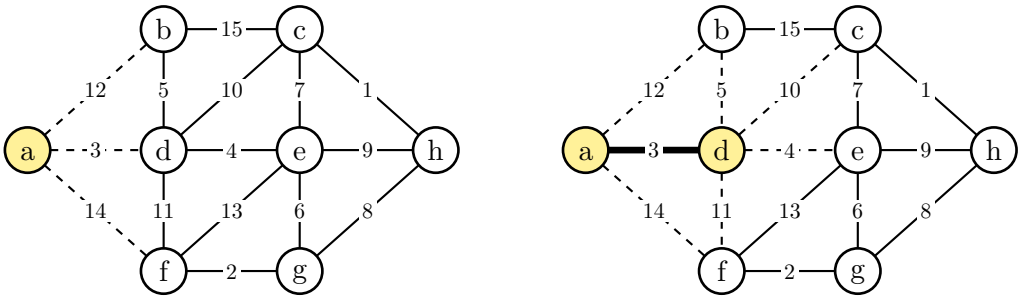


Figure 26.1: Example: the weighted graph we'll find a minimum spanning tree of, starting from a

Figure 26.2 shows the first two partitions. In each one, the highlighted vertices are the tree we've grown so far, the bold edges are the edges in that tree, and the dashed edges are the ones crossing the partition. At the start, the crossing edges are (a, d), (a, b), and (a, f), with weights 3, 12, and

14, so we add (a, d). Then d joins our side, and suddenly there are six crossing edges: 4, 5, 10, 11, 12, and 14



Start: the tree is just a Add the cheapest crossing edge, (a, d)

Figure 26.2: The first two partitions of **Figure 26.1**. Highlighted vertices are in the tree, bold edges are tree edges, and dashed edges cross the partition

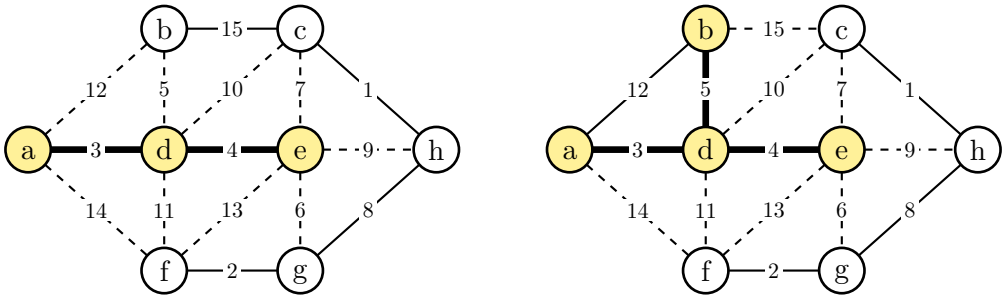
26.1.1 Why It's Correct

Every time we add an edge, we're adding the cheapest edge crossing **some** partition of the graph, and the partition property says that edge belongs in a minimum spanning tree. All of the edges we picked earlier live entirely inside our side of the partition, so none of them cross it, and the new edge never conflicts with them. So by induction, after every step, the edges we have are part of some minimum spanning tree, and after $n - 1$ edges, they **are** one

What about ties? If two crossing edges have the same weight, it doesn't matter which one we take: both are part of **some** minimum spanning tree. We might end up with a different tree than someone who broke the tie the other way, but both trees have the same total weight. That's the reason for the word "some": a graph with repeated weights can have more than one minimum spanning tree

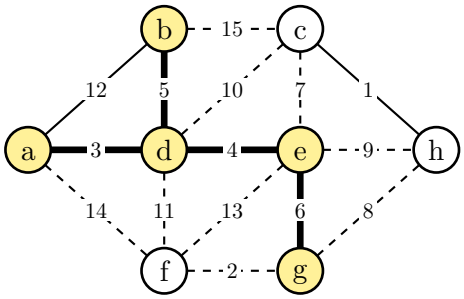
26.2 Running Prim's by Hand

Figure 26.3 continues from **Figure 26.2**, adding one vertex at a time. Each panel shows the tree right after the new edge is added, with the edges that now cross the partition dashed

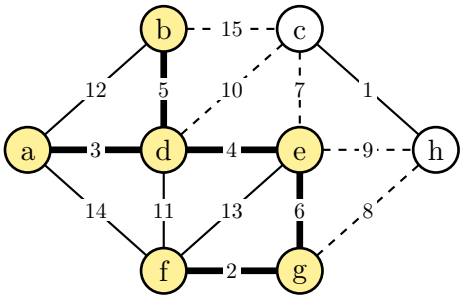


Add (d, e) with weight 4

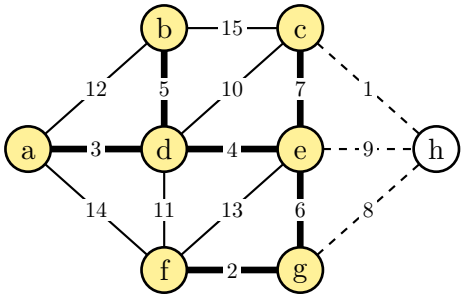
Add (b, d) with weight 5



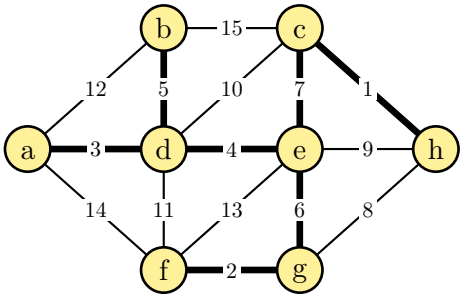
Add (e, g) with weight 6



Add (f, g) with weight 2



Add (c, e) with weight 7



Add (c, h) with weight 1: done

Figure 26.3: Prim's algorithm on **Figure 26.1**, continuing from **Figure 26.2**. The last panel is the minimum spanning tree, with total weight 28

Step by step, the crossing edges and the edge we pick are

1. $\{a, d\}$: crossing 4, 5, 10, 11, 12, 14. Add (d, e) with weight 4
2. $\{a, d, e\}$: crossing 5, 6, 7, 9, 10, 11, 12, 13, 14. Add (b, d) with weight 5
3. $\{a, b, d, e\}$: crossing 6, 7, 9, 10, 11, 13, 14, 15. Add (e, g) with weight 6
4. $\{a, b, d, e, g\}$: crossing 2, 7, 8, 9, 10, 11, 13, 14, 15. Add (f, g) with weight 2
5. $\{a, b, d, e, f, g\}$: crossing 7, 8, 9, 10, 15. Add (c, e) with weight 7
6. $\{a, b, c, d, e, f, g\}$: crossing 1, 8, 9. Add (c, h) with weight 1

The final tree is (a, d), (d, e), (b, d), (e, g), (f, g), (c, e), and (c, h), with total weight $3 + 4 + 5 + 6 + 2 + 7 + 1 = 28$. Since every weight in **Figure 26.1** is different, this is the **only** minimum spanning tree, so Kruskal's algorithm finds exactly the same tree. It just adds the edges in a different order: Kruskal's takes (c, h) first and (f, g) second, while Prim's can't take either one until the tree reaches them. Notice how Prim's adds 2 after 6 and ends with 1: it doesn't add edges in sorted order, it only ever picks the cheapest edge that touches the tree

26.2.1 Only the Cheapest Edge to Each Vertex Matters

Running this on paper is pretty easy: you just walk along the boundary and look at what crosses it. But it's not at all clear how a program keeps track of this, since every time we repartition the graph, some edges stop crossing and new ones start

The key observation is that for any vertex outside the tree, the cheapest crossing edge **to that vertex** only ever goes down. The tree only grows, so an edge from the tree to v keeps crossing the partition until v itself joins the tree, and then it never crosses again. In the meantime, new edges to v can show up as their other endpoints join the tree, and each one can only make v's cheapest option cheaper

Look at h in **Figure 26.3**. At first there's no edge from the tree to h at all, so its cost is ∞. When e joins, the edge (e, h) gives it a cost of 9. When g joins, (g, h) drops it to 8. When c joins, (c, h) drops it to 1, and now it's the cheapest crossing edge in the whole graph, so we pick it. Every vertex follows the same pattern: it starts at ∞ (or with some edge from the start vertex), its cost goes down over time, and then it's chosen

That means we never care about a more expensive edge to the same vertex. Once we've seen the edge of weight 8 into h, the edge of weight 9 will **never** be part of the spanning tree, so we can just forget about it. Instead of tracking every crossing edge, each vertex v outside the tree only needs to remember two things

- $d[v]$, the weight of the cheapest edge from the tree to v that we know of (or ∞ if there isn't one yet)
- $p[v]$, the **predecessor** of v: the tree vertex on the other end of that edge. The edge is defined by where we came from and where we are, so $(p[v], v)$ is the edge we'd add to reach v

Then picking the next edge is just picking the vertex outside the tree with the smallest $d[v]$. **Figure 26.4** shows these values after each vertex joins the tree

Add	a	b	c	d	e	f	g	h
Start	**0**	∞	∞	∞	∞	∞	∞	∞
a		12 (a)	∞	**3 (a)**	∞	14 (a)	∞	∞
d		5 (d)	10 (d)		**4 (d)**	11 (d)	∞	∞
e		**5 (d)**	7 (e)			11 (d)	6 (e)	9 (e)
b			7 (e)			11 (d)	**6 (e)**	9 (e)
g			7 (e)			**2 (g)**		8 (g)
f			**7 (e)**					8 (g)
c								**1 (c)**

Figure 26.4: The cost $d[v]$ and predecessor $p[v]$ of every vertex after each vertex joins the tree in **Figure 26.3**. Shaded vertices are already in the tree, and the bold entry is the smallest cost outside the tree, which is the vertex that joins next

Reading down a column of **Figure 26.4** shows the "only goes down" pattern: f goes from 14 to 11 to 2, c goes from ∞ to 10 to 7, and h goes from ∞ to 9 to 8 to 1

26.3 The Algorithm

To find a minimum spanning tree of a connected graph starting from a vertex s

1. Set every vertex's cost $d[v]$ to ∞ and its predecessor $p[v]$ to nothing, since we don't know how to get anywhere yet. Then set $d[s] = 0$: we start at s, so we didn't come from anywhere
2. Put **every** vertex into a min priority queue, ordered by its cost $d[v]$
3. Repeat n times: remove the vertex m with the smallest cost from the priority queue and add it to the tree (along with the edge $(p[m], m)$, unless m is s)
4. For each neighbor v of m that isn't in the tree yet, if the edge (m, v) is cheaper than $d[v]$, we've found a better way to reach v: set $d[v]$ to the weight of (m, v), set $p[v] = m$, and **update v's position in the priority queue**
5. When the priority queue is empty, the edges $(p[v], v)$ for every $v \neq s$ are a minimum spanning tree

26.4 The Code

Here, the graph is stored as an adjacency list, where `g[m]` holds the incident edges of vertex `m`, and each edge stores the vertex on its other end and its weight. Instead of building a separate graph for the tree, we just mark which vertices are in it, and return the predecessors, which describe every tree edge

```
struct Edge {
    int to;
    int weight;
};

using Graph = std::vector<std::vector<Edge>>; // g[v] holds v's incident edges

/** Builds a minimum spanning tree from `s` and returns each vertex's parent
 * Runtime: O(m log n)
 */
std::vector<int> prim(const Graph& g, int s) {
    int n = g.size();
    std::vector<int> d(n, INT_MAX); // INT_MAX stands in for infinity
    std::vector<int> p(n, -1);      // -1 means no predecessor
    std::vector<bool> inTree(n, false);
    d[s] = 0;

    VertexHeap q(d); // buildHeap on all n vertices, ordered by d
    for(int i = 0; i < n; i++) {
        // the cheapest vertex left is the next one added to the tree
        int m = q.removeMin();
        inTree[m] = true;
```

```
        for(const Edge& e : g[m]) {
            int v = e.to;
            // found a cheaper edge connecting `v` to the tree
            if(!inTree[v] && e.weight < d[v]) {
                d[v] = e.weight;
                p[v] = m;
                q.decreaseKey(v, d[v]);
            }
        }
    }
    return p;
}
```

If this looks familiar, it should: it's almost exactly breadth-first search. The one really important change is that where BFS used a **queue**, Prim's uses a **priority queue**. Notice the priority queue holds **vertices**, not edges like the one in Kruskal's, and that every vertex goes in at the start. So instead of adding neighbors to the queue, we **update** the neighbors that are already there

26.4.1 Updating the Priority Queue

The subtle part is `d[v] = e.weight`. It looks like a constant time assignment to an array, but the priority queue is ordered by d, so whenever a cost changes, v might be in the wrong place in the queue. For the code to be correct, that line also has to reorder the priority queue, which is why we call `decreaseKey` right after it. This is very different from Kruskal's priority queue, which is ordered by edge weights that never change

How expensive is that? If the priority queue is a **sorted** array, v might have to move all the way to the front, shifting everything in between, so each update is $O(n)$

With a heap, we can do much better. If a key in a heap suddenly has the wrong priority, we can fix it, and here we have a big advantage: the only way a priority ever changes is by getting **smaller**. A smaller key can only be out of place relative to its parent, never its children, so all we have to do is **heapify up** from that node, which is $O(\log n)$ since the heap holds at most n vertices

There's one catch the code hides: to heapify up from v, we need to know **where** v is in the heap. Searching the heap for it would be $O(n)$, which defeats the point. So the heap also keeps an array `pos_`, where `pos_[v]` is v's index in the heap, and every swap updates it

```
// VertexHeap.cpp

/** Lowers the cost of `v` and heapifies it up to its new place
 * Runtime: O(log n)
 */
void VertexHeap::decreaseKey(int v, int newCost) {
    cost_[v] = newCost;

    // a smaller cost can only be out of place relative to its parent
    _heapifyUp(pos_[v]);
```

```
}

/** Swaps the vertex at `index` up until its parent's cost is no larger
 * Runtime: O(log n)
 */
void VertexHeap::_heapifyUp(size_t index) {
    if(index > 1) {
        size_t parent = _parent(index);
        if(cost_[heap_[index]] < cost_[heap_[parent]]) {
            _swap(index, parent);
            _heapifyUp(parent);
        }
    }
}

/** Swaps two heap slots and updates `pos_` for both vertices
 * Runtime: O(1)
 */
void VertexHeap::_swap(size_t i, size_t j) {
    std::swap(heap_[i], heap_[j]);
    pos_[heap_[i]] = i; // keep track of where each vertex moved
    pos_[heap_[j]] = j;
}
```

This is the same `_heapifyUp` from the heaps chapter, except the heap stores vertex numbers, compares them by their costs, and swaps through `_swap` so `pos_` always stays correct. `removeMin` and `_heapifyDown` use `_swap` the same way

26.5 Running Time

Let's break `prim` into pieces and see how each one depends on how we store the priority queue and the graph

1. **Initialization** - Setting up `d`, `p`, and `inTree` touches every vertex once, which is $O(n)$
2. **Building the priority queue** - With a heap, `buildHeap` is $O(n)$. With a sorted array, we'd have to sort, which is $O(n \log n)$. With an unsorted array, we just copy the vertices in, which is $O(n)$
3. **Removing the minimum** - This happens exactly n times. Each `removeMin` is $O(\log n)$ with a heap, so that's $O(n \log n)$. With an unsorted array, we have to look through the whole array for the smallest cost, which is $O(n)$ each and $O(n^2)$ total
4. **Looping over the neighbors** - Over the whole algorithm, we take every vertex out of the queue once and look at all of its neighbors, which is $\sum_{v \in V} \deg(v) = 2m$ neighbor checks. The question is how long it takes to **find** the neighbors
5. **Updating a cost** - This can happen at most once per neighbor check, so $O(m)$ times. Each update is $O(\log n)$ with a heap, $O(n)$ with a sorted array, and $O(1)$ with an unsorted array, where we just change the cost and leave the vertex where it is

26.5.1 Adjacency List vs. Adjacency Matrix

The neighbor loop is where the graph implementation matters. With an **adjacency list**, `incidentEdges(m)` takes $O(\deg(m))$, which is exactly as much time as the loop body runs anyway. So we can **charge** the cost of finding each neighbor to the work we do on that neighbor, and finding all of them over the whole algorithm only adds $O(m)$

With an **adjacency matrix**, finding the neighbors of m means checking its whole row, which takes $O(n)$ no matter how many neighbors m actually has. We do that for all n vertices, so that's $O(n^2)$. Notice this doesn't change how many times the loop body runs (still once per neighbor), only how long it takes to get to it

Adding up the pieces for each combination (the initialization and building the queue never dominate)

- **Heap and adjacency list** - $O(n \log n)$ to remove, $O(m)$ to find neighbors, and $O(m \log n)$ to update, so $O(n \log n + m \log n)$
- **Heap and adjacency matrix** - $O(n \log n)$ to remove, $O(n^2)$ to find neighbors, and $O(m \log n)$ to update, so $O(n^2 + m \log n)$
- **Unsorted array and either graph** - $O(n^2)$ to remove, at most $O(n^2)$ to find neighbors, and only $O(m) = O(n^2)$ to update, since updating is free, so $O(n^2)$

Priority queue	**Adjacency matrix**	**Adjacency list**
Binary heap	$O(n^2 + m \log n)$	$O(n \log n + m \log n)$
Unsorted array	$O(n^2)$	$O(n^2)$

There's a reason we picked an **unsorted** array here, even though a sorted array seems like the natural priority queue. A sorted array makes `removeMin` cheap, but every update costs $O(n)$, and we might do $O(m)$ of them, which is $O(nm)$, or $O(n^3)$ for a graph with $m \approx n^2$ edges. Since Prim's updates far more often than it removes, cheap updates win

The graph is connected, so $m \geq n - 1$, which means $n \log n$ is never bigger than $m \log n$. So with a heap and an adjacency list, we can simplify the runtime to just $O(m \log n)$

26.5.2 Sparse vs. Dense Graphs

The number of edges in a connected graph can be anywhere from $n - 1$ to about n^2. A **sparse** graph has very few edges, $m \approx n$, and a **dense** graph has close to the most edges possible, $m \approx n^2$. Plugging those in

Implementation	**Sparse ($m \approx n$)**	**Dense ($m \approx n^2$)**
Heap and adjacency list	$O(n \log n)$	$O(n^2 \log n)$
Heap and adjacency matrix	$O(n^2)$	$O(n^2 \log n)$
Unsorted array (either graph)	$O(n^2)$	$O(n^2)$

So which one is best depends on how dense the graph is. On a sparse graph, the heap and adjacency list win by a lot. On a dense graph, the unsorted array wins, because we might update a cost almost every time we look at an edge, and with a heap, every one of those updates costs $O(\log n)$

Notice that on a dense graph, the adjacency matrix stops mattering. The only problem with the matrix is that finding the neighbors of v takes $O(n)$ instead of $O(\deg(v))$. But if $m \approx n^2$, then $\deg(v)$ is close to n anyway. If a graph has a huge number of edges, checking every **possible** edge costs about the same as checking every **real** edge, so the matrix is fine. It's only when there are few edges that checking every real edge is much faster than checking every possible one

26.5.3 Fibonacci Heaps

If you look up Prim's algorithm on Wikipedia, you'll see a different bound: $O(m + n \log n)$. That comes from a different heap, the **Fibonacci heap**, invented by Fredman and Tarjan in 1984. We won't cover how it works: it's complicated to code, and it has bad enough constant factors that it doesn't really help until the graph is very large. What matters is that it's a priority queue with a special operation, **decrease key**, which increases a key's priority (makes it smaller, in our case) in amortized $O(1)$ time, while `removeMin` is still amortized $O(\log n)$

With a Fibonacci heap and an adjacency list, every update becomes constant time, so the $O(m \log n)$ updates turn into $O(m)$, and the total is $O(n \log n + m)$. That's the bound Wikipedia gives, and it assumes both a Fibonacci heap and an adjacency list

Why can't we get rid of the $n \log n$ too? It comes from calling `removeMin` n times, and the best heaps we know still take $O(\log n)$ per `removeMin`. That isn't just a gap in our knowledge. Prim's builds a priority queue of n vertices and then takes them all out in sorted order, so if a priority queue that only compares keys could do that in less than $O(n \log n)$ total, we could use it to sort n values faster than $O(n \log n)$, which is impossible for a comparison sort

26.6 Prim's vs. Kruskal's

Kruskal's algorithm runs in $O(n + m \log m)$, since its priority queue holds edges. So why do people usually write it as $O(m \log n)$? Because m isn't an arbitrary number: in a simple graph, it's at most about n^2. So for some constant c, $m \leq cn^2$, and

$$\log m \leq \log(cn^2) = \log c + 2 \log n$$

The constant $\log c$ and the factor of 2 fall out under big O, so $O(\log m)$ and $O(\log n)$ are the same thing, and $O(m \log m) = O(m \log n)$. We can also drop the n: Kruskal's only produces a spanning tree if the graph is connected, so $m \geq n - 1$, and n is smaller than $m \log n$. You'll see it written both with and without the n

Algorithm	**Sparse ($m \approx n$)**	**Dense ($m \approx n^2$)**
Kruskal's: $O(m \log n)$	$O(n \log n)$	$O(n^2 \log n)$
Prim's, binary heap: $O(n \log n + m \log n)$	$O(n \log n)$	$O(n^2 \log n)$
Prim's, Fibonacci heap: $O(n \log n + m)$	$O(n \log n)$	$O(n^2)$

Prim's bound has an extra $n \log n$ term, so it looks like it should be worse, but on both sparse and dense graphs, Kruskal's and Prim's with a binary heap come out **exactly the same**. A Fibonacci heap doesn't help on a sparse graph either. It only pulls ahead on dense graphs, and there, the unsorted array version of Prim's already gets $O(n^2)$ without any fancy heap

So which should you choose? There isn't really a right answer: pick whichever one you'd rather code. The two algorithms approach the problem completely differently (one sorts all the edges and merges a forest, the other grows a single tree outward), so they'll behave differently on particular graphs, but their running times end up about the same. That suggests there might be something fundamental about how much work finding a minimum spanning tree takes, though nobody has proven a lower bound like the one for comparison sorting

> In fact, $O(m \log n)$ isn't the best possible. Chazelle's **soft heap** is a priority queue that is allowed to be **wrong** sometimes: it can "corrupt" (raise) the keys of a limited fraction of its items, and in exchange, for a fixed error rate, its operations run in amortized constant time. An algorithm that knows how many errors it could have gotten can then correct for them, and this leads to an MST algorithm that runs in $O(m \cdot \alpha(m, n))$, the same inverse Ackermann function from disjoint sets. We won't cover it in this class, but it's a good reminder that algorithms are never really done

27 Dijkstra's Algorithm

So far, we've only used edge weights to build minimum spanning trees. Now we'll use them for what most people think of first when they see a weighted graph: finding **shortest paths**. The algorithm we'll use, Dijkstra's algorithm, turns out to be Prim's algorithm with one tiny change, which makes it really easy to understand and to analyze

27.1 Weighted Graphs

From a purely theoretical point of view, a **weight** is just a number attached to every edge, and a **weighted graph** is a graph where every edge has one. That's not very exciting on its own, but weights are what let a graph represent something useful. Without them, the shortest path is just the one with the fewest edges, which breadth-first search already finds for us, since it visits vertices in order of how many edges away they are. With weights, the shortest path can be a completely different path, with more edges but less total weight, so BFS isn't enough anymore

Picture a map of Illinois: the towns (Champaign-Urbana, Kankakee, Bloomington, Joliet, Naperville, Chicago) are vertices, and the roads between them are edges. What could the weights be?

- **Distance** - The obvious choice: how many miles long each road is
- **Travel time** - What map apps actually use. It's distance divided by speed, adjusted for traffic, since an interstate and a one-lane dirt road of the same length shouldn't be weighted the same
- **Cost** - If we're taking an Uber, the weight could be what each leg costs, so the shortest path is the cheapest trip

Costs can even be **negative**. Airlines are required to run a certain number of flights a day out of a small airport like Champaign-Urbana to keep flying there, so when hardly anyone is traveling, they sometimes price the flight from Champaign-Urbana to Chicago so that flying CU to Chicago to Europe is cheaper than flying straight from Chicago to Europe. The leg out of CU effectively has a negative cost. Negative weights are a special case, but they can make real-world sense, and we'll see that they matter a lot to our algorithms

A graph can also store several weights at once (cost, time, distance), and the weight we actually run an algorithm on might combine them, like a heuristic that balances cost against time. From an algorithmic point of view, we don't care what the weights mean. What we do care about is the **conditions** an algorithm runs under: is the graph directed? Can a weight be negative? Some algorithms work in one case and not the other

27.2 Single-Source Shortest Paths

The most basic version of the problem asks: given a weighted graph, what is the shortest path from A to B, where the **length** of a path is the **sum of the weights** of its edges? That exact question

is very important in AI, but in this class we'll solve a slightly bigger problem, the **single-source shortest path** problem: given one starting vertex s, find the shortest path from s to **every** vertex that can be reached from it

Why solve the bigger problem? Because it costs the same. The algorithms we know find shortest paths in order of length, so they find the shortest path to every closer vertex on the way to the farthest one. If B happens to be the farthest vertex from A, we'll find every other shortest path before we find the one to B, and since we do worst-case analysis, asking only about A to B saves us nothing

The answer is a **shortest path tree**. Just like a spanning tree, every vertex v stores a **predecessor** $p[v]$, the vertex right before it on its shortest path from s, so following the predecessors back from any vertex traces its shortest path in reverse

27.3 From Prim's to Dijkstra's

Recall Prim's algorithm: we keep a priority queue of vertices keyed on $d[v]$, start with $d[s] = 0$ and every other $d[v] = \infty$, and repeatedly remove the vertex with the smallest d, add it to the tree, and update its neighbors. Dijkstra's algorithm is **exactly the same algorithm**. The only thing that changes is what $d[v]$ means

- **Prim's** - $d[v]$ is the weight of the cheapest single **edge** from the tree to v
- **Dijkstra's** - $d[v]$ is the length of the cheapest **path** we know of from s to v

So when we remove a vertex u and look at an edge from u to v with weight $w(u, v)$, Prim's compares $w(u, v)$ with $d[v]$, but Dijkstra's compares $d[u] + w(u, v)$ with $d[v]$. Getting to v through u means first getting to u, which costs $d[u]$, and then taking the edge

There's one more difference. A spanning tree only makes sense in an undirected graph, so Prim's only works on undirected graphs, but Dijkstra's works on **directed** graphs with no problem at all. **Figure 27.1** shows the directed graph we'll use for the rest of the chapter, with A as the source

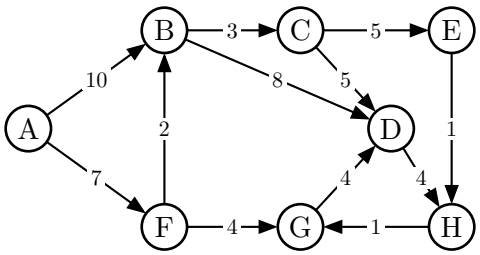


Figure 27.1: Example: a directed weighted graph with 8 vertices and 12 edges. We'll find the shortest paths from A

To see why the change matters, imagine treating the edges as undirected and running Prim's from A. After A and F join the tree, G's key is 4, since the edge from F to G weighs 4. But it doesn't cost 4 to get from A to G: it costs 7 to get to F and then 4 more, so 11. **Figure 27.2** compares the two at that moment

Vertex	Prim's key	Dijkstra's d
B	2, the edge $F \to B$	9, the path $A \to F \to B$
G	4, the edge $F \to G$	11, the path $A \to F \to G$

Figure 27.2: The values of B and G once A and F are done. Prim's only cares about the last edge, while Dijkstra's stores the whole path's length from A

27.4 Running Dijkstra's Algorithm

We start with $d[A] = 0$ and every other vertex at ∞ with no predecessor. Every round, we remove the vertex u with the smallest d from the priority queue. That vertex is now **finished**: $d[u]$ is its true shortest distance, and the edge from its predecessor joins the tree. Then for every neighbor v of u that isn't finished yet, we check whether going through u is better, and if $d[u] + w(u, v) < d[v]$, we lower $d[v]$ to that and set $p[v] = u$

1. **Remove A (0)** - B becomes 10 and F becomes 7, both through A
2. **Remove F (7)** - F is the smallest. Going through F to B costs $7 + 2 = 9$, which beats 10, so $d[B]$ drops to 9 and its predecessor changes to F. Notice that the direct edge from A to B lost to a path with **more** edges. G becomes $7 + 4 = 11$
3. **Remove B (9)** - C becomes 12 and D becomes 17, both through B
4. **Remove G (11)** - Going through G to D costs $11 + 4 = 15$, which beats 17, so D's predecessor changes to G
5. **Remove C (12)** - Going through C to D costs $12 + 5 = 17$, which is **not** better than 15, so D doesn't change. E becomes 17
6. **Remove D (15)** - H becomes 19
7. **Remove E (17)** - Going through E to H costs $17 + 1 = 18$, which beats 19, so H's predecessor changes to E
8. **Remove H (18)** - H's only neighbor is G, which is already finished, so there's nothing to do

Figure 27.3 draws each round: finished vertices are shaded, bold edges are in the shortest path tree, and dashed edges are each unfinished vertex's current best predecessor. **Figure 27.4** lists d and p for every vertex after each round

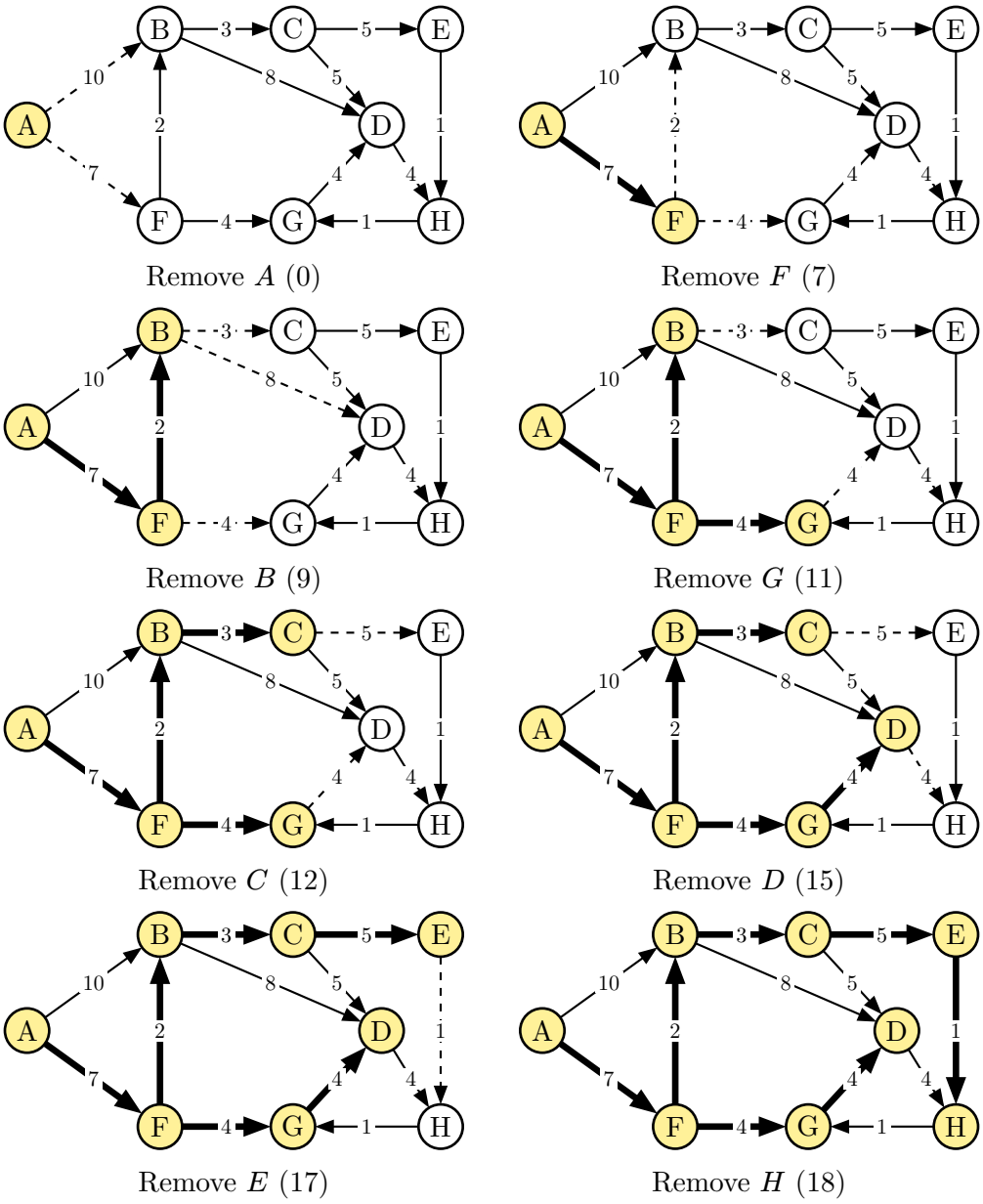


Figure 27.3: Dijkstra's algorithm on **Figure 27.1** from A. Finished vertices are shaded, bold edges are in the shortest path tree, and dashed edges point from each unfinished vertex's current predecessor

Round	A	B	C	D	E	F	G	H
Start	0	∞	∞	∞	∞	∞	∞	∞
Remove A	0	**10 (A)**	∞	∞	∞	**7 (A)**	∞	∞
Remove F	0	**9 (F)**	∞	∞	∞	7 (A)	**11 (F)**	∞
Remove B	0	9 (F)	**12 (B)**	**17 (B)**	∞	7 (A)	11 (F)	∞
Remove G	0	9 (F)	12 (B)	**15 (G)**	∞	7 (A)	11 (F)	∞
Remove C	0	9 (F)	12 (B)	15 (G)	**17 (C)**	7 (A)	11 (F)	∞
Remove D	0	9 (F)	12 (B)	15 (G)	17 (C)	7 (A)	11 (F)	**19 (D)**
Remove E	0	9 (F)	12 (B)	15 (G)	17 (C)	7 (A)	11 (F)	**18 (E)**
Remove H	0	9 (F)	12 (B)	15 (G)	17 (C)	7 (A)	11 (F)	18 (E)

Figure 27.4: $d[v]$ and, in parentheses, $p[v]$ for every vertex after each round. Shaded entries are finished, and bold entries changed that round

The final shortest distances from A are $A = 0$, $F = 7$, $B = 9$, $G = 11$, $C = 12$, $D = 15$, $E = 17$, and $H = 18$. To get an actual path, we follow predecessors backwards: H came from E, which came from C, then B, then F, then A, so the shortest path to H is $A \to F \to B \to C \to E \to H$, with length $7 + 2 + 3 + 5 + 1 = 18$

27.5 The Algorithm

1. Set $d[v] = \infty$ and $p[v] =$ none for every vertex, then set $d[s] = 0$
2. Build a priority queue of every vertex, keyed on d
3. Repeat n times
   1. Remove the vertex u with the smallest $d[u]$, and add it to the tree T along with the edge from $p[u]$
   2. For every neighbor v of u that isn't in T yet, if $d[u] + w(u, v) < d[v]$, set $d[v] = d[u] + w(u, v)$ and $p[v] = u$ (which updates v's priority in the queue)
4. Return T, the shortest path tree

Here it is as pseudocode. Put it next to Prim's and the body is the same, except for the `d[u]` added in the comparison and the update, and that `T.add` records the edge from `p[u]` so we get back the shortest path tree

```
DijkstraSSSP(G, s):
    foreach (Vertex v : G):
        d[v] = +infinity
        p[v] = NULL
    d[s] = 0

    PriorityQueue Q // min distance, defined by d[v]
    Q.buildHeap(G.vertices())
```

```
    Graph T // "labeled set"

    repeat n times:
        Vertex u = Q.removeMin()
        T.add(u, p[u])
        foreach (Vertex v : neighbors of u not in T):
            if cost(u, v) + d[u] < d[v]:
                d[v] = cost(u, v) + d[u]
                p[v] = u

    return T
```

27.6 Why It Works

A very reasonable worry is: what if there's some other path we didn't consider? What if a path through lots of light edges beats one heavy edge that we saw early on? **Figure 27.3** already shows what happens for B: the direct edge of weight 10 sat in the queue, and before it was ever the smallest thing in there, we found the cheaper path through F. The number of edges on a path doesn't matter, only its total weight

Here's the intuition for why it always works. Just like Prim's grows a tree outwards, Dijkstra's grows a **frontier** of finished vertices around s, and every round it adds whichever vertex is the shortest distance away from s across that frontier. Suppose we're about to remove u, and imagine any path from s to u. It starts inside the frontier at s, so at some point it has to **cross** the frontier, at some first unfinished vertex x. The part of the path up to x is already accounted for in $d[x]$, since x's predecessor on it is finished and updated x when it was removed. And u is the smallest thing in the queue, so $d[u] \leq d[x]$. The rest of the path from x to u can only **add** weight, so the whole path is at least $d[x] \geq d[u]$. No path can beat $d[u]$, so it's safe to finish u

27.6.1 Ties

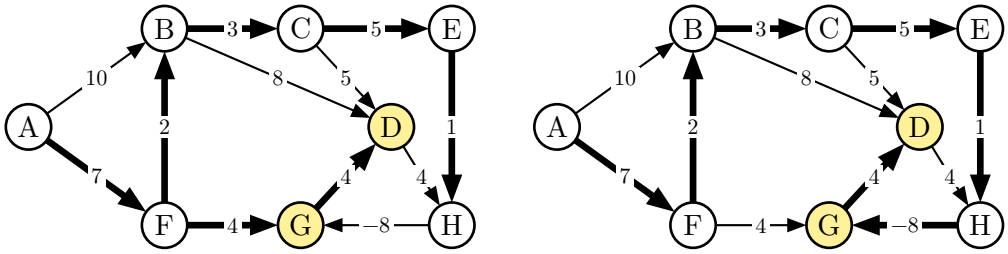
Sometimes two vertices are tied for the smallest d. Which one comes out first depends on how our priority queue happens to break ties, and that can change which tree we get. That's fine: we only want **a** shortest path tree, and when there's a tie, both trees have the same distances. The same goes for paths of equal length to one vertex. If the edge from F to B weighed 3 instead of 2, going through F would cost $7 + 3 = 10$, which isn't **less** than 10, so B would keep A as its predecessor, and both paths are equally short

27.7 Negative Edge Weights

There's one assumption hiding in that argument: "the rest of the path can only **add** weight." That's only true if no weight is negative. With a negative edge, we might have to go **farther** in order to go **shorter**, and Dijkstra's has no way to find that path

Let's change the weight of the edge from H to G from 1 to -8. There is no negative cycle: the cycle $G \to D \to H \to G$ costs $4 + 4 - 8 = 0$. Dijkstra's runs exactly like **Figure 27.3**, since the

first seven rounds never look at that edge. G is finished at 11 and D at 15, and only in the very last round do we see that going through H to G costs $18 - 8 = 10$, but G is already finished, so it's skipped. The real shortest path to G is $A \to F \to B \to C \to E \to H \to G$, with length $7 + 2 + 3 + 5 + 1 - 8 = 10$, which goes all the way out to the farthest vertex and comes back. Even worse, D was finished using the wrong $d[G]$, so its distance is wrong too: it should be 14. **Figure 27.5** compares the two trees



Dijkstra's: $d[G] = 11$, $d[D] = 15$ True shortest paths: $G = 10$, $D = 14$

Figure 27.5: With the edge from H to G changed to -8, Dijkstra's (left) gets the shaded vertices wrong. The true shortest path tree (right) reaches G through H

So standard Dijkstra's works by always choosing the shortest thing, knowing that any shorter path would already have been found, and with negative edges that's just not true anymore

27.7.1 Modified Dijkstra's

We can fix this with a very small change, which we'll call **modified Dijkstra's**

- Whenever we find a shorter path to **any** vertex, even one that's already been removed, update it and put it back in the priority queue
- Instead of removing exactly n vertices, keep going until the priority queue is empty

On the graph from **Figure 27.5**, the first eight removals are exactly the same. But when we remove H (18), we now update G to $18 - 8 = 10$ with predecessor H and put it back in the queue. G (10) is the smallest thing left, so it comes out again and updates D to $10 + 4 = 14$, which goes back in too. Removing D (14) gives $14 + 4 = 18$ for H, which isn't better, so nothing else changes and the queue runs dry. That's the correct tree on the right of **Figure 27.5**

The catch is the running time. A vertex can now come out of the queue many times, once for every time we find a better path to it, so we can no longer say we only remove n things. Usually it does fine, but we can't prove it does: on some graphs with negative edges it takes exponentially long. That's why everyone analyzes and writes about Dijkstra's, and hardly anyone talks about modified Dijkstra's

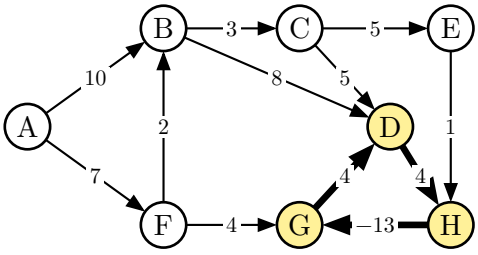
27.7.2 Relaxation

Another way to describe modified Dijkstra's is as a **relaxation** algorithm. Every vertex starts in a tense, high-energy state of ∞, and every time we find a better path to it, we **relax** it down to the better value. We keep going until nothing can be relaxed anymore

Seen this way, the priority queue only decides the **order** we try things in. We could make it a plain queue and it would still work (as long as there is an answer), but then we may have to pull a vertex out once for every path to it, which is potentially every edge. CS 374 goes into this basic idea much more, since a huge number of algorithms work by starting everything at ∞ and improving it bit by bit

27.7.3 Negative Cycles

Now let's make that same edge from H to G weigh -13, like in **Figure 27.6**. The cycle $G \to D \to H \to G$ now costs $4 + 4 - 13 = -5$, so every lap around it **saves** 5. What is the shortest path from A to H? There isn't one: whatever length you claim, I can find a cheaper path by going around the cycle one more time



Lap	*G*	*D*	*H*
Start	11	15	18
1	5	9	13
2	0	4	8
3	−5	−1	3
4	−10	−6	−2

Figure 27.6: With the edge from H to G changed to -13, the bold cycle costs -5. Modified Dijkstra's lowers every vertex on it by 5 per lap, forever

So shortest paths simply aren't defined when a **negative weight cycle** can be reached on the way from the start to the destination. Here, A, B, C, E, and F still have perfectly good shortest paths, since you can't get back to them from the cycle, but G, D, and H don't. Modified Dijkstra's has no way to recognize this, so it just keeps saying "hey, I can make it better" and never stops. That's another reason it rarely gets discussed: if there **is** an answer, modified Dijkstra's finds it, but if there isn't, it runs forever

27.8 Implementing Dijkstra's

The pseudocode quietly assumes our priority queue can **update** a vertex's priority when $d[v]$ goes down (**decrease key**). Our heap can do it with the same extra bookkeeping as Prim's (the `pos_` array that tracks where each vertex is in the heap), but the STL's `std::priority_queue` can't do it at all. There are two easy ways around this

- **Use an unsorted array as the priority queue**, just like we did for Prim's. Updating $d[v]$ is just changing a number, and `removeMin` scans every unfinished vertex. The scan takes time, but we only ever remove n things, and we save all the work of building and maintaining a heap
- **Always do modified Dijkstra's**. Instead of updating a vertex in the heap, just push a **new** entry with the better distance. When an entry comes out that's worse than the vertex's current

d, it's stale, so we throw it out. The heap can now hold up to m entries instead of n, since every update adds one

If you had to implement Dijkstra's yourself, the unsorted array version is probably the simplest, but sticking entries back into the STL heap is a perfectly good and reasonable approach. Both versions here use an adjacency list, where `adj[u]` holds the edges leaving `u`

```
struct Edge {
    int to;
    int weight;
};

/** Finds shortest distances from `s` with a lazy heap, skipping stale entries
 * Runtime: O(m log n)
 */
std::vector<int> dijkstra(const std::vector<std::vector<Edge>>& adj, int s,
std::vector<int>& pred) {
    const int INF = std::numeric_limits<int>::max();
    std::vector<int> dist(adj.size(), INF);
    pred.assign(adj.size(), -1);
    dist[s] = 0;

    // (distance, vertex) pairs, with the smallest distance on top
    std::priority_queue<std::pair<int, int>, std::vector<std::pair<int, int>>,
std::greater<>> pq;
    pq.push({0, s});

    while(!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if(d > dist[u]) { continue; } // stale: we already found a cheaper path to
u

        for(const Edge& e : adj[u]) {
            // relax: going through `u` is a shorter path to `e.to`
            if(dist[u] + e.weight < dist[e.to]) {
                dist[e.to] = dist[u] + e.weight;
                pred[e.to] = u;
                pq.push({dist[e.to], e.to});
            }
        }
    }
    return dist;
}
```

- `std::priority_queue` is a max-heap by default, so `std::greater<>` flips it into a min-heap. Pairs compare by their first element (the distance) first
- Unreachable vertices keep `dist` at `INF` and `pred` at -1

- There's no "finished" set: a vertex gets processed again whenever its distance improves. With non-negative weights, that never happens after it first comes out, so each vertex is processed exactly once. With negative weights (but no negative cycle), this is exactly modified Dijkstra's, so it still gets the right answer

```
/** Finds shortest distances from `s`, using an unsorted array as the queue
 * Runtime: O(n^2)
 */
std::vector<int> dijkstraArray(const std::vector<std::vector<Edge>>& adj, int s,
std::vector<int>& pred) {
    const int INF = std::numeric_limits<int>::max();
    int n = adj.size();
    std::vector<int> dist(n, INF);
    std::vector<bool> done(n, false);
    pred.assign(n, -1);
    dist[s] = 0;

    for(int step = 0; step < n; step++) {
        // removeMin: scan every vertex we haven't finished
        int u = -1;
        for(int v = 0; v < n; v++) {
            if(!done[v] && dist[v] != INF && (u == -1 || dist[v] < dist[u])) { u =
v; }
        }
        if(u == -1) { break; } // everything left is unreachable
        done[u] = true;

        for(const Edge& e : adj[u]) {
            // relax: going through `u` is a shorter path to `e.to`
            if(!done[e.to] && dist[u] + e.weight < dist[e.to]) {
                dist[e.to] = dist[u] + e.weight;
                pred[e.to] = u;
            }
        }
    }
    return dist;
}
```

This one is standard Dijkstra's, with `done` playing the role of T: it removes each vertex exactly once and never looks back at finished vertices, so it needs non-negative weights. If the smallest thing left is ∞, the rest of the graph can't be reached from `s`, so we stop early

27.9 Running Time

Since Dijkstra's is Prim's with one extra addition, the analysis of Prim's carries over directly. We remove each of the n vertices once, and when we remove u, we look at every edge leaving it. Over the whole algorithm, that's every edge exactly once (handshaking again), so we do at most m updates

- **Binary heap and adjacency list** - `buildHeap` is $O(n)$, the n calls to `removeMin` are $O(n \log n)$, and the up to m priority updates are $O(m \log n)$. That's $O((n+m) \log n)$. If every vertex can be reached from s, then $m \geq n-1$, so this is $O(m \log n)$
- **Binary heap and adjacency matrix** - Finding u's neighbors means scanning its whole row, which is $O(n)$ per vertex and $O(n^2)$ total, so we get $O(n^2 + m \log n)$
- **Unsorted array** - Each `removeMin` is an $O(n)$ scan, so that's $O(n^2)$, and each update is $O(1)$, so the total is $O(n^2 + m) = O(n^2)$, with either graph implementation. For a **dense** graph, where m is close to n^2, that's actually better than the heap's $O(n^2 \log n)$
- **Lazy binary heap (modified Dijkstra's)** - Up to m pushes and pops, each $O(\log m)$. Since $m < n^2$, $\log m < 2 \log n$, so this is still $O(m \log n)$, it just uses $O(m)$ space for the heap
- **Fibonacci heap** - A fancier heap where updating a priority is $O(1)$ (amortized), which brings the total down to $O(m + n \log n)$. It's the best on paper, but a Fibonacci heap is a giant mess to implement, and it's not in the STL

Priority queue	**Graph**	**Running time**
Binary heap	Adjacency list	$O(m \log n)$
Binary heap	Adjacency matrix	$O(n^2 + m \log n)$
Unsorted array	Either	$O(n^2)$
Lazy binary heap	Adjacency list	$O(m \log n)$
Fibonacci heap	Adjacency list	$O(m + n \log n)$

Dijkstra's is a great example of reusing the entire structure of an algorithm to do something honestly quite different, and a huge number of graph algorithms turn out to be "Dijkstra's, but then we do something else." Both Prim's and Dijkstra's explore a graph the way a person standing **on** the graph would: start somewhere and walk around, one neighbor at a time. Next, we'll look at an **all-pairs** shortest path algorithm, which can't give an answer for a graph like **Figure 27.6** (there isn't one), but can at least tell us right away that there isn't one

28 Floyd Warshall's Algorithm

The Floyd-Warshall algorithm is our last graph algorithm in lecture, and it's another **shortest path** algorithm. It's also our first real example of **dynamic programming**. You don't need to know what that means yet, but once we see how the algorithm works, it'll be interesting to come back to it

28.1 Why Another Shortest Path Algorithm?

The classic answer is that Floyd-Warshall works with **negative weight edges**. That's true, but recall from the previous chapter that **modified** Dijkstra's (which puts a vertex back into the priority queue whenever it finds a cheaper path to it) also handles negative edges. So Floyd-Warshall has two other things going for it

- **It detects negative weight cycles** - Modified Dijkstra's just loops forever on a negative weight cycle, saying "hey, I can make it better" every time it goes around. Floyd-Warshall finishes, and its answer tells us there's a negative weight cycle, and even which pairs of vertices have no shortest path because of it
- **It finds every shortest path** - Dijkstra's is a **single source** shortest path algorithm: when it finishes, we know the shortest path from one starting vertex to every other vertex. Floyd-Warshall solves **all pairs** shortest paths: when it finishes, we know the shortest path from **every** vertex to **every** vertex

That second point is why Floyd-Warshall shows up so much. Just like a lot of algorithms start with Dijkstra's and build on it, there's a large family of graph analysis algorithms that start by running Floyd-Warshall, since analyzing the structure of a graph often needs every shortest path, and Floyd-Warshall gets us all of them directly

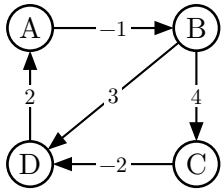
28.2 The Distance Matrix

Floyd-Warshall keeps an $n \times n$ matrix `d`, where `d[u][v]` is the cost of the shortest path we know so far from `u` to `v`. When the algorithm finishes, `d[u][v]` will be the true shortest path distance from `u` to `v`, which is why it's called `d`. This is **not** the adjacency matrix from earlier: it starts out looking a lot like one, but it ends up storing distances between vertices that aren't connected by an edge at all

We start by filling in everything we know directly from the graph, without computing any longer paths

- The shortest path from any vertex to itself costs 0, so the diagonal is all 0
- If there's an edge from `u` to `v`, the best path we know from `u` to `v` is that edge, so `d[u][v]` is its weight
- Everything else is ∞, since we don't know any path there yet

We'll use the weighted, directed graph in **Figure 28.1** for the rest of the chapter. It has two negative edges, $A \to B$ and $C \to D$, but no negative weight cycles (the cheapest cycle, $A \to B \to C \to D \to A$, costs $-1 + 4 - 2 + 2 = 3$). The edges have directions, so even though there's an edge from D to A, there's no edge from A to D: entry (A, D) starts at ∞, while entry (D, A) starts at 2



	A	**B**	**C**	**D**
A	0	-1	∞	∞
B	∞	0	4	3
C	∞	∞	0	-2
D	2	∞	∞	0

Figure 28.1: Example: a weighted, directed graph (left) and its initial distance matrix (right). Row u, column v is the cost of the best path we know from u to v

Notice what this matrix already is: the shortest paths that use **at most one edge**. Everything that's immediately true from the graph is stored, and that's a hint about how the algorithm works. We're going to compute longer paths by building on what we've already computed, which is basically what every dynamic programming algorithm does

28.3 Allowing One Vertex at a Time

The idea is to grow the set of vertices that paths are allowed to pass **through** (their **intermediate** vertices), one vertex at a time. Right now, our matrix has the shortest paths that go through no other vertices at all. After the first step, it will have the shortest paths that may go through A. After the next, the shortest paths that may go through A and B, and so on. Once every vertex is allowed, the matrix holds the true shortest paths

When we allow a new vertex k, the shortest path from u to v either doesn't use k, in which case we already know it, or it goes from u to k and then from k to v. Both of those halves only use vertices we've **already** allowed, so their best costs are already in the matrix! So for every pair, we just check

$$d[u][v] > d[u][k] + d[k][v]$$

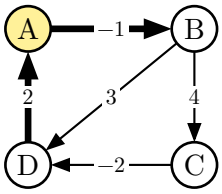
and if it's true, going through k is cheaper, so we update `d[u][v]`. If either half is ∞, the path through k doesn't exist, so it can't help, and we can stop right there

28.3.1 Through A

Figure 28.2 allows A as an intermediate vertex. Nothing can get **to** A except D, so the only candidates are paths from D, through A, and onward

- $B \to C$ costs 4, and $B \to A \to C$ is ∞ (there's no path from B to A at all), so no change. The same goes for every other pair starting at B or C

- $D \to B$ is ∞, but $D \to A \to B$ costs $2 + (-1) = 1$, which is an improvement, so `d[D][B]` becomes 1
- $D \to C$ is ∞, and $D \to A \to C$ is also ∞, since A has no path to C yet, so no change



	A	B	C	D
A	0	−1	∞	∞
B	∞	0	4	3
C	∞	∞	0	−2
D	2	1	∞	0

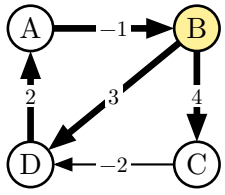
Figure 28.2: After allowing A (highlighted). The only improvement is $D \to A \to B$, which costs $2 + (-1) = 1$

So we now have every shortest path that uses zero vertices in between, plus every shortest path that goes through A and nothing else. We haven't changed the graph: we've just found a **virtual path** from D to B that costs 1, which really means "I know I can get from D to B for 1, by way of A"

28.3.2 Through B

Now **Figure 28.3** allows B, so paths can go through A, B, or both. This is where building on old answers pays off: `d[D][B]` is already 1 from the last step, so we reuse the path through A without recomputing it

- $A \to B \to C$ costs $-1 + 4 = 3$, better than ∞
- $A \to B \to D$ costs $-1 + 3 = 2$, better than ∞
- $C \to B$ is ∞, so nothing from C can go through B. Once one half is ∞, we can stop
- $D \to B \to A$ needs a path from B to A, which we still don't have, so no change
- $D \to B \to C$ costs $1 + 4 = 5$, using the virtual path $D \to A \to B$ we found through A, so `d[D][C]` becomes 5

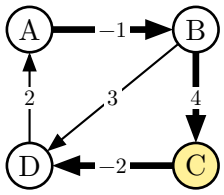


	A	B	C	D
A	0	−1	3	2
B	∞	0	4	3
C	∞	∞	0	−2
D	2	1	5	0

Figure 28.3: After allowing B. The new paths are $A \to B \to C$ (3), $A \to B \to D$ (2), and $D \to A \to B \to C$ (5)

28.3.3 Through C and D

The last two steps work exactly the same way. Allowing C in **Figure 28.4**, the only edge out of C is the -2 edge to D, and it improves two paths that we **already** had: $A \to D$ drops from 2 to $3 + (-2) = 1$, and $B \to D$ drops from 3 to $4 + (-2) = 2$. Notice how finding a path isn't the end of the story: a longer path with a negative edge can beat a shorter one

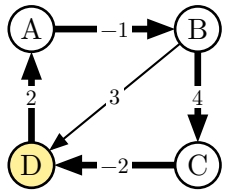


	A	B	C	D
A	0	−1	3	1
B	∞	0	4	2
C	∞	∞	0	−2
D	2	1	5	0

Figure 28.4: After allowing C. Going through C and its -2 edge improves $A \to D$ to 1 and $B \to D$ to 2

Finally, allowing D in **Figure 28.5** gives B and C a way back to A, since $D \to A$ is the only edge into A

- $B \to D \to A$ costs $2 + 2 = 4$, using the $B \to C \to D$ path we just found through C
- $C \to D \to A$ costs $-2 + 2 = 0$
- $C \to D \to B$ costs $-2 + 1 = -1$, which is really $C \to D \to A \to B$



	A	B	C	D
A	0	−1	3	1
B	4	0	4	2
C	0	−1	0	−2
D	2	1	5	0

Figure 28.5: After allowing D, every vertex is allowed, so this is the final distance matrix: the shortest path from every vertex to every vertex

Every vertex has now been allowed, so **Figure 28.5** holds the shortest path from every vertex to every other vertex. For example, the cheapest way from B to A is $B \to C \to D \to A$ for 4, even though $B \to D \to A$ uses fewer edges and costs 5

There's one more thing to notice in each step: row k and column k never change while we're allowing k. Going from u to k through k costs $d[u][k] + d[k][k] = d[u][k] + 0$, which is no better than what we had. That's why we can update the matrix **in place** without worrying that we're reading a value we just changed (as long as there are no negative weight cycles, which make $d[k][k]$ negative, as we'll see)

28.4 The Algorithm

1. Create an $n \times n$ matrix `d`, with 0 on the diagonal, the edge weight wherever there's an edge, and ∞ everywhere else
2. For each vertex k, in any fixed order, allow k as an intermediate vertex: for every pair of vertices u and v, if $d[u][k] + d[k][v] < d[u][v]$, set $d[u][v] = d[u][k] + d[k][v]$
3. When every vertex has been allowed, `d[u][v]` is the shortest path distance from `u` to `v`

Written as a recurrence, if $d_k(u, v)$ is the cheapest path from u to v whose intermediate vertices are all among the first k vertices, then

$$d_k(u, v) = \min(d_{k-1}(u, v), d_{k-1}(u, k) + d_{k-1}(k, v))$$

That recurrence is how we'd show the algorithm is correct. The loops just compute it from the bottom up, in an order where everything on the right-hand side is always ready

28.5 The Code

We number the vertices 0 to $n - 1$ as usual, and use the largest `int` as ∞. Before adding two distances, we check that neither is ∞: without that check, ∞ plus a positive weight would overflow into a huge **negative** number, which would look like an amazing shortcut

```
const int INF = std::numeric_limits<int>::max();

struct Edge {
    int u;
    int v;
    int weight;
};

using Matrix = std::vector<std::vector<int>>;

/** Finds the shortest distance between every pair of vertices
 * Runtime: O(n^3)
 */
Matrix floydWarshall(int n, const std::vector<Edge>& edges) {
    Matrix d(n, std::vector<int>(n, INF));
    // a vertex reaches itself for 0, and each edge is the best path known yet
    for(int v = 0; v < n; v++) {
        d[v][v] = 0;
    }
    for(const Edge& e : edges) {
        d[e.u][e.v] = e.weight;
    }

    // allow one more middle vertex `k` per pass, so `k` must be the outer loop
    for(int k = 0; k < n; k++) {
        for(int u = 0; u < n; u++) {
            for(int v = 0; v < n; v++) {
```

```
                // no known path through `k` (adding to INF would overflow)
                if(d[u][k] == INF || d[k][v] == INF) { continue; }
                if(d[u][k] + d[k][v] < d[u][v]) {
                    d[u][v] = d[u][k] + d[k][v];
                }
            }
        }
    }
    return d;
}
```

28.5.1 Does the Order of the Loops Matter?

Yes! The **outer** loop has to be the vertex we're going through, and the two inner loops are the source and the destination. Swapping which variable plays which role breaks the algorithm. The order we visit the vertices in, on the other hand, doesn't matter at all: we could allow D first and A last, as long as every iteration of the outer loop finishes all of its pairs before we move to the next k

To see it break, put `k` in the innermost loop instead. Now each pair (u, v) gets looked at exactly once, checking every possible middle vertex, and then never again. On our example, that computes $B \to A$ as 5 instead of 4: when it looks at (B, A), the best path it knows from B to D is still the edge that costs 3, since the cheaper $B \to C \to D$ path for 2 doesn't get found until it looks at (B, D) later. With `k` on the outside, every path through the earlier vertices is finished before anything tries to build on it

28.6 Recovering the Paths

From a theoretician's point of view, the shortest path problem is done once we have the number. But if your car's GPS just said "the road to Chicago is 196 miles" and left it at that, it wouldn't be very helpful. In practice, we want the **path**, not just its length

So we add a second $n \times n$ matrix, which we'll call `next`, where `next[u][v]` is the vertex we should go to **first** to get from `u` to `v` along the best path we know. The distance matrix changes exactly the same way as before

- At the start, `next[v][v]` is `v` (to get to yourself, you go to yourself), and for every edge, `next[u][v]` is `v`, since the best path we know is that edge. Everywhere else is empty, since we don't know any path there yet
- Whenever going through k improves `d[u][v]`, the first step from `u` to `v` is now the first step from `u` to `k`, so we set `next[u][v] = next[u][k]`

On our example, allowing A found $D \to A \to B$, so `next[D][B]` becomes `next[D][A]`, which is A. Then allowing B found $D \to B \to C$ for 5, so `next[D][C]` becomes `next[D][B]`, which is also A: to get from D to C, we go to A first. **Figure 28.6** shows the `next` matrix before and after the whole algorithm

	A	B	C	D
A	A	B		
B		B	C	D
C			C	D
D	A			D

Before

	A	B	C	D
A	A	B	B	B
B	C	B	C	C
C	D	D	C	D
D	A	A	A	D

After

Figure 28.6: The `next` matrix for **Figure 28.1**. Row u, column v is the first vertex on the best path from u to v, and the highlighted entries changed while the algorithm ran

To recover a path, start at the source and keep asking "where do I go next to reach the destination?" until we get there. From D to C, `next[D][C]` says go to A. From A, `next[A][C]` says go to B (we filled that in when we allowed B and found $A \to B \to C$ for 3). From B, `next[B][C]` says go to C, and we're done. So the path is $D \to A \to B \to C$, with cost $2 - 1 + 4 = 5$, exactly `d[D][C]`, as **Figure 28.7** shows

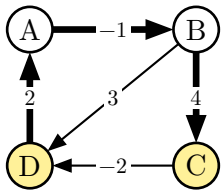


Figure 28.7: Following `next` from D to C gives the path $D \to A \to B \to C$, which costs $2 + (-1) + 4 = 5$

```
Matrix d;
Matrix next;

/** Fills `d` with all-pairs shortest distances and `next` with first steps
 * Runtime: O(n^3)
 */
void floydWarshallWithPaths(int n, const std::vector<Edge>& edges) {
    d.assign(n, std::vector<int>(n, INF));
    next.assign(n, std::vector<int>(n, -1));
    for(int v = 0; v < n; v++) {
        d[v][v] = 0;
        next[v][v] = v;
    }
    for(const Edge& e : edges) {
        d[e.u][e.v] = e.weight;
        next[e.u][e.v] = e.v;
    }
```

```
    // allow one more middle vertex `k` per pass, so `k` must be the outer loop
    for(int k = 0; k < n; k++) {
        for(int u = 0; u < n; u++) {
            for(int v = 0; v < n; v++) {
                // no known path through `k` (adding to INF would overflow)
                if(d[u][k] == INF || d[k][v] == INF) { continue; }
                if(d[u][k] + d[k][v] < d[u][v]) {
                    d[u][v] = d[u][k] + d[k][v];
                    // first step toward `v` is now the first step toward `k`
                    next[u][v] = next[u][k];
                }
            }
        }
    }
}

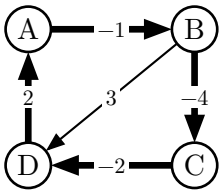
/** Follows `next` from `u` to rebuild the shortest path to `v`
 * Runtime: O(n) (one step per edge on the path)
 */
std::vector<int> path(int u, int v) {
    // -1 means there is no path from `u` to `v`
    if(next[u][v] == -1) { return {}; }
    std::vector<int> p = {u};
    while(u != v) {
        u = next[u][v];
        p.push_back(u);
    }
    return p;
}
```

An empty entry is -1, so `path` returns an empty vector when there's no path at all. Storing paths only costs one more $n \times n$ matrix, and following one takes time proportional to its number of edges

28.7 Detecting Negative Weight Cycles

When we walked through the example, we skipped pairs like A to A, since a path from a vertex to itself can never beat 0. But to detect a negative weight cycle, those are exactly the pairs we need. The loops in the code do check them, and on a normal graph they're unexciting: the diagonal stays 0

Now suppose the edge $B \to C$ cost -4 instead of 4, like in **Figure 28.8**. Then $A \to B \to C \to D \to A$ costs $-1 - 4 - 2 + 2 = -5$, so it's a negative weight cycle. As the algorithm runs, it finds that there's a cheaper way to get from A to A than staying put: go around the cycle, for -5. `d[A][A]` becomes negative, and `next[A][A]` becomes B, meaning "the cheapest way to get to A from A is to leave and go to B". That makes no sense for a real shortest path, and it's exactly the sign that something's wrong



	A	B	C	D
A	−5	−6	−10	−12
B	−4	−5	−9	−11
C	0	−1	−5	−7
D	−3	−4	−8	−10

Figure 28.8: Example: changing $B \to C$ to -4 makes $A \to B \to C \to D \to A$ a negative weight cycle that costs -5. Floyd-Warshall still finishes, and the negative entries on the diagonal (highlighted) give the cycle away

So the rule is

- If there are no negative weight cycles, every diagonal entry stays 0
- If some `d[x][x]` ends up negative, then x can leave and come back to itself for a negative total cost, so there's a negative weight cycle

```
/** Returns whether any vertex can reach itself for a negative cost
 * Runtime: O(n)
 */
bool hasNegativeCycle() {
    for(size_t v = 0; v < d.size(); v++) {
        // a negative diagonal entry means `v` sits on a negative weight cycle
        if(d[v][v] < 0) { return true; }
    }
    return false;
}
```

It also tells us **which** shortest paths don't exist. If `u` can reach some vertex `x` with a negative `d[x][x]`, and `x` can reach `v`, then a path from `u` to `v` can go around the cycle as many times as it likes, getting cheaper every time, so there is no shortest path from `u` to `v`. Pairs that can't get to the cycle (or can't get from it to their destination) still have perfectly good shortest paths. In **Figure 28.8**, every vertex is on the cycle, so **none** of the numbers in the matrix mean anything: they're just how many times the algorithm happened to go around the cycle. That also means that with a negative weight cycle, the numbers can keep shrinking, so a big graph might need a wider type like `long long` to avoid overflowing in the negative direction

28.8 Running Time

This is the most screamingly obvious runtime we've seen in this class. Initializing the diagonal is $O(n)$, copying in the edges is $O(m)$, and the three nested loops each run n times, so the whole thing is $O(n^3)$. It doesn't matter whether the graph is sparse or dense: we just go through the loops n^3 times no matter what. The distance matrix takes $O(n^2)$ space, and adding the `next` matrix for paths is another $O(n^2)$

Is that good? To get all pairs from Dijkstra's, we'd run it once from every vertex. Recall that Dijkstra's is $O(m \log n)$ with a binary heap, and $O(m + n \log n)$ with a Fibonacci heap, so running it n times gives

Algorithm	**Any Graph**	**Sparse ($m \approx n$)**	**Dense ($m \approx n^2$)**
$n \times$ Dijkstra's (binary heap)	$O(nm \log n)$	$O(n^2 \log n)$	$O(n^3 \log n)$
$n \times$ Dijkstra's (Fibonacci heap)	$O(nm + n^2 \log n)$	$O(n^2 \log n)$	$O(n^3)$
Floyd-Warshall	$O(n^3)$	$O(n^3)$	$O(n^3)$

So in pure theory, running Dijkstra's from everywhere wins on sparse graphs, and with a Fibonacci heap, it ties on dense ones. But in practice, it's not that simple

- **Fibonacci heaps have bad constants**, which is part of why nobody spends much time on them (and they're not in the standard template library)
- **Floyd-Warshall is very cache friendly**. Your computer's hardware tries to guess what memory you'll want next, and its biggest guess is that if you just used something in memory, you'll want the thing right after it. The inner loop walks straight across rows of a matrix, which is exactly that. Cache is somewhere around 100 to 1,000 times faster than main memory, so an algorithm that keeps hitting cache has a tremendous real-world advantage. Dijkstra's jumps all over memory following edges and heap entries
- **Negative edges**. Plain Dijkstra's doesn't work at all with negative edges, and modified Dijkstra's loses the guarantee that each vertex comes out of the priority queue only once, so it can end up redoing a lot of work for every source. Floyd-Warshall is $O(n^3)$ no matter what the weights are

So if we need every shortest path, especially with negative edges, Floyd-Warshall is a very good solution, and in practice it'll usually win

28.9 Dynamic Programming

The bigger reason to see Floyd-Warshall is to start thinking about **dynamic programming**, one of the most powerful techniques for designing fast algorithms. The idea is

1. Describe the problem in terms of **recursive subproblems**. Here, a subproblem is "the shortest path from u to v using only the first k vertices in between", and the recurrence shows it only needs subproblems with $k - 1$
2. Figure out an order where every subproblem's pieces are solved before we need them. Here, that's the outer loop over k
3. Solve each subproblem **once**, iteratively, storing its answer instead of recomputing it every time

We can show the algorithm is correct by describing it recursively, and make it fast by doing each subproblem exactly once in a loop instead of recursing. Running a search from every vertex would rediscover the same pieces of paths over and over, while Floyd-Warshall stores those partial solutions in the matrix and reuses them, like how `d[D][B]` from allowing A got reused to find `d[D][C]` when we allowed B

The advantage here is limited, because Dijkstra's is already a really good algorithm that does some of the same trick. It says that to get anywhere, we must take the shortest path to the vertex one hop before it, and it finds shortest paths in order, shortest first, so it never has to recompute one. That's also exactly why basic Dijkstra's only works with non-negative edges, and why modified Dijkstra's gets so much worse: once negative edges can make an old answer wrong, the "I've already computed it" advantage goes away. Most optimizations come down to the same question: how do I avoid doing something I've already done?

29 Probability in Computer Science

30 Hashing, Hash Collisions, and Hash Tables

Hashing is the data structure you've all heard of before, but that we've been studiously, intentionally ignoring until now. It's also our first **probabilistic** data structure: instead of promising that every operation is fast, it promises that operations are fast **in expectation**

Let's start with the problem it solves. Say I have a library full of books and I want to store them in a dictionary (recall the dictionary ADT: `insert(key, value)`, `remove(key)`, and `find(key)`). So far we've seen three real dictionaries

Dictionary	**`find`**	**`insert`**	**`remove`**
Sorted array	$O(\log n)$	$O(n)$*	$O(n)$
BST	$O(h)$, worst $O(n)$	$O(h)$, worst $O(n)$	$O(h)$, worst $O(n)$
Balanced BST (AVL tree)	$O(\log n)$	$O(\log n)$	$O(\log n)$

`find` in a sorted array is just binary search. `insert` is a binary search to find where the new key goes, and then $O(n)$ to shift everything over to make space. The * is a reminder that it's also amortized, since sometimes the array is full and we double it, which only costs a constant amount of extra work per insert

A BST runs in the height of the tree, since there's only one place a key can go and finding it means walking down from the root. Strictly that's $O(n)$, since a "tree" shaped like a linked list is still a BST, which is why we built AVL trees to force the height back down to $O(\log n)$. So as long as we're doing any edits, the balanced BST is our best dictionary. It also has one bonus feature: an in-order traversal lists the keys in sorted order in $O(n)$, which is what made range searches (and KD trees) possible. Not every dictionary can do that, so keep it in mind

But $O(\log n)$ isn't constant. For any one lookup, $\log n$ is fast enough and I don't care. But what if my program's main loop is "look this up, look this up, look this up" millions of times? Then those $\log n$s add up, and we'd like to do better

Here's the plan: instead of a guarantee that **every** operation is fast, we'll settle for operations that are **usually** fast. Just like amortization let us charge an expensive resize across many cheap inserts, if we do a huge number of operations whose **expected** runtime is constant, the law of large numbers makes it very likely that the total really is about (expected runtime) $\times$ (number of operations)

30.1 Hash Functions

What we want is a function that takes **any key that could possibly exist** and quickly turns it into an integer in some range: an index into an array. If we had that, finding a key would be easy: "where is it? It's at this index, we just have to run the function." That's exactly what a **hash function** is

$$h : K \to \{0, 1, ..., m-1\}$$

where K is the **keyspace** (every possible key) and m is the size of our array

We've actually already used one. Back when we wanted to remember which pixels of an image we'd visited, we turned pixel (x, y) into the number `y * width + x`. That number is unique for every pixel and really fast to compute, so it's a hash function over a small, finite keyspace. When the array is at least as big as the keyspace like this, we can have a **perfect hash function**, where every key gets its own unique index (it's one-to-one). From a data structures point of view, the **ideal** hash function is both one-to-one and onto, so we can go both ways: given a key we can compute its index, and given an index we can compute which key lives there

The problem is that most keyspaces are enormous. Even plain integers on our machines have 2^{64} possible values, and I don't have the memory for an array with 2^{64} entries. The number of books that could ever exist is way bigger than the number of spots on my library's shelves, and my function has to work for any book that could ever exist, even though the library only stores a finite number of them. So in general, our hash function has to squeeze a huge keyspace into a much smaller array

30.1.1 What a Hash Function Must Do

A hash function has three requirements

1. **Deterministic**: if $k_1 = k_2$, then $h(k_1) = h(k_2)$. Hashing the same key twice must give the same index, or we could never find anything again. Notice it's this direction only: two **different** keys are allowed to land on the same index, and we just argued that with a huge keyspace they have to. Rolling dice is a great way to get a random integer, but it's a terrible hash function, since if I roll dice again for the same key I get a different number
2. **Efficient**: the whole point is to beat $O(\log n)$, so the hash function should run in $O(1)$. If it takes $O(\log n)$, just use a balanced BST and stop
3. **Defined with respect to m**: our array is a real object with a real size, so the output has to land in $\{0, ..., m-1\}$, not just anywhere in the natural numbers

The last one is the easy one. If we have a function that produces any natural number, we just take the remainder: `% m`. So in practice, a hash function is built in two stages

1. A **hash**, which turns a key of whatever type into a (usually huge) integer
2. A **compression**, which squeezes that integer into the range of the table, almost always with `% m`

Compression can still break a hash that was fine on its own. If our hash only ever produced multiples of 8, say $8k$, then on a table of size 8 every single key would compress to index 0. Mod is

what we'll use every single time, but it's worth remembering that a badly built hash can interact badly with it

The short answer to "how do I write a good hash function?" is: **don't come up with your own, you'll do it wrong**. For the rest of this chapter, we'll assume someone else hands us a good hash function, and focus on what to do with it

30.1.2 Example: Professors and Their Courses

Say we want to store professors as keys and the course they're teaching as values, in a table of size 8, using

$$h(k) = (k[0] - \texttt{'A'}) \bmod 8$$

That is, we take the first character of the name and subtract `'A'`. Since characters are encoded in order in both ASCII and Unicode, `'A' - 'A'` is 0, `'B' - 'A'` is 1, and so on

Professor (key)	**Course (value)**	$k[0]$	$h(k)$
Angrave	CS 341	`'A'`	0
Beckman	CS 101	`'B'`	1
Challen	CS 124	`'C'`	2
Evans	CS 225	`'E'`	4
Alawini	CS 411	`'A'`	0

The first four go in beautifully, like in **Figure 30.1**. It looks like an amazing hash function, until we try to add Alawini, who also starts with an `'A'`. Index 0 is already taken by Angrave, so we have a conflict

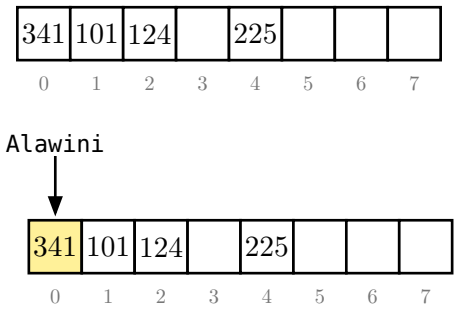


Figure 30.1: Example: storing courses in a table of size 8 with $h(k) = (k[0] - \texttt{'A'}) \bmod 8$. Angrave, Beckman, Challen, and Evans each get their own index, but Alawini also hashes to 0, which Angrave already holds

Notice that doubling the table to 16 and switching to `mod 16` doesn't help at all: both names still start with `'A'`, so they'd still both go to 0. The real problem is that this hash only looks at one character, and the first letters of names aren't evenly spread out. It's like an early version of Java's string hash, which only looked at a handful of a string's characters. On the web, most strings were

addresses that all start with the same `https://`, so huge numbers of them hashed to the same value. The hash depends entirely on what the user gives it, and users don't give us random data

30.1.3 Good and Bad Hash Functions

Let's try some hash functions for our library, where the key is a book, m is the size of the table, and letters are numbered $A = 1$ through $Z = 26$

Hash function	**Verdict**
(first initial + last initial of the author) $\bmod m$	Valid: deterministic and $O(1)$ (two lookups and one addition). Not great, since names aren't evenly distributed and every book by the same author lands in the same place, but it's something
(`rand()` + number of pages) $\bmod m$	Useless: not deterministic, since looking up the same book twice can give two different indices
(number of times I've read the book) $\bmod m$	Useless: that number can't be derived from the book itself (we'd need a dictionary of books just to build our dictionary of books), and it changes every time I reread one

A few questions came up about these. Isn't the first one non-deterministic if one author wrote several books? No: all of their books get the same hash value, which isn't ideal, but the **same book** always gets the same answer, and that's all deterministic means. The output must be determined entirely by the input. What about two different authors who share initials? Also fine, for the same reason: since there are more possible books than slots in the table, **something** has to share a slot. And isn't `rand()` usually pseudorandom and seeded, so technically deterministic? Maybe, but the function as a whole doesn't give the same answer for the same book, which is what we care about. (There are even truly random sources, like Cloudflare's LavaRand, which films a wall of lava lamps.)

30.2 Hash Collisions

So the first hash function is our best one. Let's try it

- **The Hobbit** by J.R.R. Tolkien: $J + T = 10 + 20 = 30$
- **Goosebumps** by R.L. Stine: $R + S = 18 + 19 = 37$
- **Aardvarks Anonymous** by Jim Truth: $J + T = 10 + 20 = 30$

Uh oh. **The Hobbit** and **Aardvarks Anonymous** both hash to 30, before we've even compressed anything. With $m = 16$, they both go to $30 \bmod 16 = 14$, like in **Figure 30.2**. If a hash table were just "hash the key, go to that index, store it there", it would break right here, because index 14 is one box and we want to put two things in it

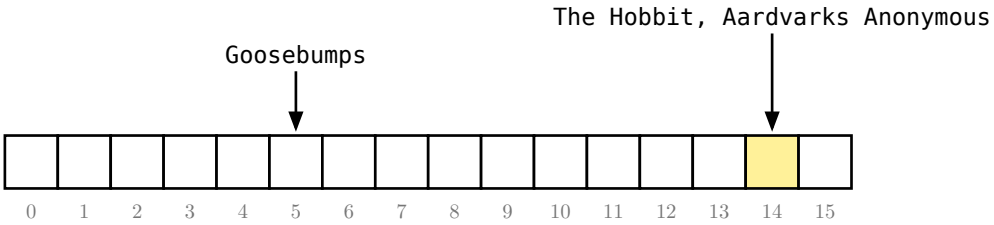


Figure 30.2: Example: hashing books by the author's initials with $m = 16$. Goosebumps goes to $37 \bmod 16 = 5$, but Tolkien and Truth share the initials J and T, so their books collide at index 14

When two different keys hash to the same index, we have a **hash collision**. You've seen these all semester: when the autograder prints a text version of a PNG, it's printing a **hash** of the image. That hash ignores the order of the pixels, so any two PNGs with the same set of pixels hash to the same value, even when they look completely different. That's why "my hashes match but the test says the images don't!" comes up every semester. It isn't a bug: an arbitrarily large image squeezed down to a single 64-bit number has to lose information somewhere

In fact, collisions are unavoidable. If the table is at least as large as the keyspace, we can use a perfect hash function, but normally the keyspace is far bigger than any table we'd choose. By the **pigeonhole principle**, if we have m slots (pigeonholes) and more than m possible keys, at least one slot must get more than one key. And since our hash function has to be fixed (if it changed based on the data coming in, it wouldn't be deterministic), some collisions are baked in

So a **hash table** is built out of three things

1. A **hash function** (let somebody else come up with it)
2. A **data storage structure**, usually an array
3. A **method of handling collisions**

The biggest difference between different kinds of hash tables is the third one, and that's what the rest of this chapter is about

30.3 Open vs. Closed Hashing

There are two families of collision handling, and they're named after where the data lives

- **Open hashing**: the array holds **pointers**, and the key-value pairs themselves live somewhere else in memory
- **Closed hashing**: the key-value pairs are stored **directly in the array**

Watch out for the confusing names: closed hashing is also called **open addressing** (the address is "open", since a key might end up at an index other than its hash), so open addressing is closed hashing. People are also a bit sloppy in practice, since a closed hashing table sometimes stores pointers to the data anyway. What really matters is the collision algorithm: if it conceptually stores each item **in** the table, we call it closed hashing

That sloppiness has a reason. A closed hashing table needs a way to say "there's nothing in this slot", so the stored type needs a value that can never be real data. A pointer gets that for free with `nullptr`, but for an arbitrary key-value type, the easiest way to guarantee it is often to store

a pointer. The fastest closed hashing tables really do reserve some never-valid value of the data type instead

Open hashing is the most common, has the fewest constraints, and degrades gracefully when things go a bit wrong. Closed hashing is faster when all our ducks are in a row: the fastest hash tables in the world use closed hashing, but they get complicated. We'll start with the easier one

30.4 Separate Chaining

The only kind of open hashing we'll talk about (and honestly, practically the only kind anyone uses, so the names are nearly synonyms) is **separate chaining**. Every slot of the array, called a **bucket**, holds the head of a **linked list** of key-value pairs, and every bucket starts out as `nullptr` (an empty list). To insert, we hash the key and `push_front` the pair onto that bucket's list. A collision is no problem at all: putting two items in a linked list is easy

Let's insert our running example keys $16, 8, 4, 13, 29, 11, 22$ (in that order) into a table of size $m = 7$, using

$$h(k) = k \bmod 7$$

1. $16 \bmod 7 = 2$, so bucket 2 becomes the list [16]
2. $8 \bmod 7 = 1$, so bucket 1 becomes [8]
3. $4 \bmod 7 = 4$ and $13 \bmod 7 = 6$, so they start buckets 4 and 6
4. $29 \bmod 7 = 1$: our first collision! We just put 29 at the front of bucket 1's list: $[29, 8]$
5. $11 \bmod 7 = 4$, so bucket 4 becomes $[11, 4]$
6. $22 \bmod 7 = 1$, so bucket 1 becomes $[22, 29, 8]$

Figure 30.3 shows the final table. Each node really stores a key **and** its value as a pair (otherwise, once two keys share a bucket, how would we know which value belongs to which key?), but just like with BSTs, the drawings only show the keys

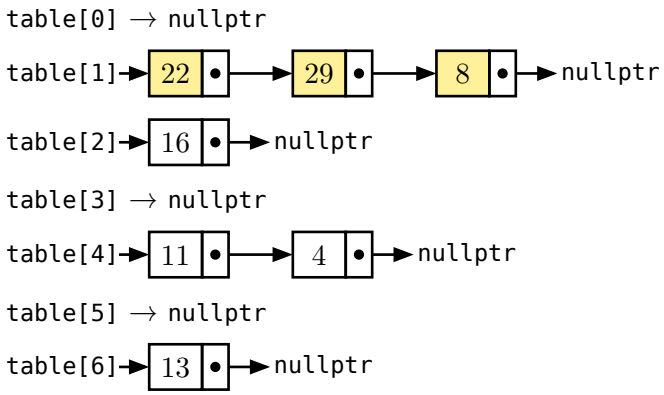


Figure 30.3: Example: separate chaining with $h(k) = k \bmod 7$ after inserting 16, 8, 4, 13, 29, 11, 22. Colliding keys are pushed onto the front of their bucket's list. `find(8)` and `find(15)` both walk the highlighted chain

There's no order to a chain. It happens to be in reverse insertion order, but all we know is that every key in bucket 1 hashes to 1

30.4.1 Insert

Computing the hash is $O(1)$, and inserting at the front of a linked list is $O(1)$, so `insert` is $O(1)$ in the **worst case**. That's great! We could insert at the tail instead, but that either costs a walk down the list or a tail pointer, and buys us nothing. (This assumes we're told the key isn't already in the table. If we have to check for duplicates first, `insert` costs a `find`.)

30.4.2 Find

To find a key, we hash it, go to its bucket, and walk the chain comparing keys. In **Figure 30.3**, `find(8)` hashes to 1, then checks 22 (not 8), 29 (not 8), and 8, which is a match. `find(15)` also hashes to 1 and checks 22, 29, and 8, and then hits `nullptr`, so now we know 15 isn't in the table

So `find` runs in the length of the chain, and in the worst case (say, a hash function that returns 1 for every key) that's all n keys, so $O(n)$. If the hash function is perfect, every chain has at most one key. The real answer depends on how long we **expect** the chains to be, and we'll work that out once we've seen the other collision strategies

30.4.3 Remove

`remove` is the same as `find`: walk the chain to the key, and then unlinking a node from a linked list is $O(1)$. **Figure 30.4** shows `remove(29)`, which just points 22's `next` at 8. So `remove` also runs in the length of the chain: $O(n)$ in the worst case

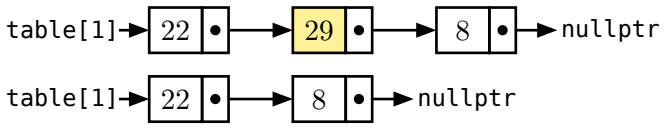


Figure 30.4: Example: `remove(29)` from **Figure 30.3** walks bucket 1's chain to 29 and unlinks it

30.4.4 The Code

This basically codes itself with `std::list` and `std::pair`. Keys are assumed to be non-negative integers so that `key % m` works, and since this version never resizes, its expected runtimes are in terms of the load factor α, which we'll define soon

```
// ChainedTable.h

#pragma once

#include <list>
#include <string>
#include <utility>
#include <vector>
```

```
class ChainedTable {
public:
    /** Inserts a key-value pair at the front of the key's chain
     * Runtime: O(1)
     */
    void insert(int key, const std::string& value);

    /** Returns a pointer to the key's value, or nullptr if it's missing
     * Runtime: O(1 + alpha) expected (under SUHA), O(n) worst case
     */
    std::string* find(int key);

    /** Removes the key and its value from the table, if it's there
     * Runtime: O(1 + alpha) expected (under SUHA), O(n) worst case
     */
    void remove(int key);

private:
    /** Compresses a key into a bucket index
     * Runtime: O(1)
     */
    size_t hash(int key) const { return key % table_.size(); }

    // every bucket starts as an empty list (a nullptr head)
    std::vector<std::list<std::pair<int, std::string>>> table_ =
        std::vector<std::list<std::pair<int, std::string>>>(7);
};
```

```
// ChainedTable.cpp

#include "ChainedTable.h"

/** Inserts a key-value pair at the front of the key's chain
 * Runtime: O(1)
 */
void ChainedTable::insert(int key, const std::string& value) {
    // colliding keys just share the list, so no probing is needed
    table_[hash(key)].push_front({key, value});
}

/** Returns a pointer to the key's value, or nullptr if it's missing
 * Runtime: O(1 + alpha) expected (under SUHA), O(n) worst case
 */
std::string* ChainedTable::find(int key) {
    // only this bucket can hold the key, so walk its chain comparing keys
    for (auto& entry : table_[hash(key)]) {
        if (entry.first == key) { return &entry.second; }
    }
```

```
    return nullptr;
}

/** Removes the key and its value from the table, if it's there
 * Runtime: O(1 + alpha) expected (under SUHA), O(n) worst case
 */
void ChainedTable::remove(int key) {
    // a reference, so erasing changes the chain inside the table
    auto& chain = table_[hash(key)];
    for (auto it = chain.begin(); it != chain.end(); ++it) {
        if (it->first == key) {
            // unlinking a node from a linked list is O(1)
            chain.erase(it);
            return;
        }
    }
}
```

`find` returns a pointer to the value, or `nullptr` if the key isn't there

Operation	Worst case	In terms of chains
`insert`	$O(1)$	$O(1)$
`find`	$O(n)$	$O(\text{length of the chain})$
`remove`	$O(n)$	$O(\text{length of the chain})$

This feels a lot like when we first built a BST and realized the worst case was still $O(n)$. But it's not really n, it's the length of a chain. Intuitively, if the hash function acts like rolling an m-sided die for every key, then each bucket expects about $\frac{n}{m}$ of the n keys, just like if I roll 100 six-sided dice, I expect $\frac{100}{6} \approx 16.7$ of them to land on any particular face. We'll make this precise later

30.5 Closed Hashing

Linked lists are easy, but closed hashing is often faster for a reason we haven't talked about much in this course: the **cache**. (You'll see the details in CS 233 or CS 340, or ECE 391 and 411.) Modern computers really want data packed densely into contiguous memory, and following pointers to linked list nodes scattered all over memory is slow in comparison. Storing everything directly in one array doesn't improve the big O, but it improves the constant factors on a lot of operations, which is why anyone bothers with the more complicated scheme

But now each slot holds exactly one key-value pair. If a key hashes to a slot that's already taken, it has to go **somewhere else in the array**, so we need a rule for which other slots to try. Checking a slot is called a **probe**

30.5.1 Linear Probing

The simplest rule is **linear probing**: if the slot is full, try the next one, and keep going until we find an empty slot. If we fall off the end of the array, we wrap around to the start. Formally, the ith probe for key k looks at

$$h(k, i) = (h(k) + i) \bmod m$$

for $i = 0, 1, 2$, and so on. The `mod m` at the end is what wraps us around

Some people write linear probing as one big hash function like that, and others write it as a hash function plus a probing rule. They're the same thing, but it's easiest to think of the hash function and the probing separately, since we analyze them separately

Let's insert the same keys $16, 8, 4, 13, 29, 11, 22$ with the same $h(k) = k \bmod 7$, like in **Figure 30.5**

1. 16, 8, 4, and 13 hash to 2, 1, 4, and 6, which are all empty, so they go right in
2. $29 \bmod 7 = 1$, but 8 is there. Try 2: 16 is there. Try 3: empty, so 29 goes in slot 3
3. $11 \bmod 7 = 4$, but 4 is there. Try 5: empty, so 11 goes in slot 5
4. $22 \bmod 7 = 1$, and slots 1 through 6 are all full, so we wrap around to slot 0 after **seven** probes

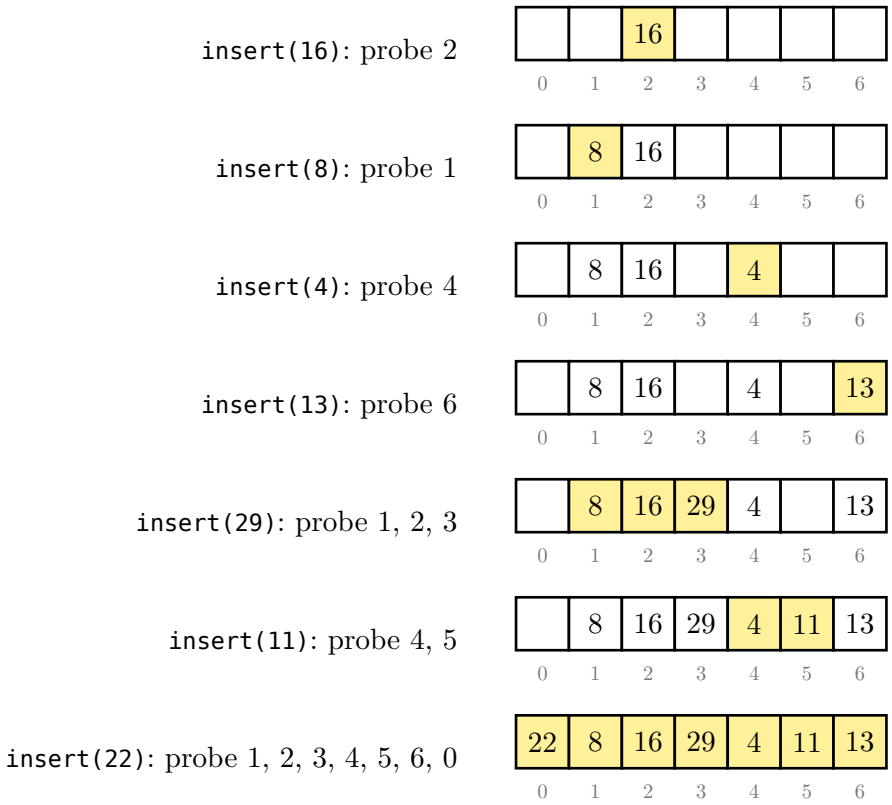


Figure 30.5: Example: linear probing with $h(k) = k \bmod 7$. Each row shows the table after one insert, with every slot that insert probed highlighted. 22 probes the whole table before wrapping around to slot 0

Insert is easy to picture, as long as we can tell when a slot is empty

30.5.2 Find

To find a key, we start at its hash and probe forward until we either find it or reach an **empty** slot. Let's use the table from right after `insert(29)` in **Figure 30.5**

- `find(29)`: start at 1. 8 is there, which isn't 29, but **something** is there, so 29 could have been pushed further down. Try 2: 16, keep going. Try 3: found it
- `find(22)`: start at 1 and probe 8, 16, 29, and 4, which are all not 22. Slot 5 is empty, so we stop: 22 isn't in the table, like in **Figure 30.6**

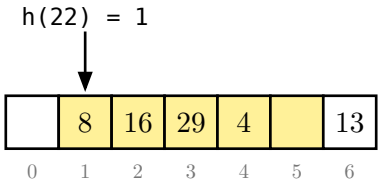


Figure 30.6: Example: an unsuccessful `find(22)` on the table after inserting 16, 8, 4, 13, 29. It probes from slot 1 until it hits the empty slot 5

Why is it safe to stop at an empty slot? If 22 were in the table, then when it was inserted it would have probed this same sequence of slots and taken the first empty one it found. So every key sits in a **contiguous** run of full slots starting at its hash, and an empty slot means it isn't there. On a completely full table, there's no empty slot to stop at, so we also stop after checking all m slots

30.5.3 Remove and Tombstones

Now let's go back to the full table from **Figure 30.5**, `remove(16)`, and then `find(29)`. If removing 16 just empties slot 2, then `find(29)` starts at 1, sees 8, moves to 2, finds it empty, and declares that 29 isn't in the table. But it is! It's sitting in slot 3

We can't fix this by searching the whole table every time, because then `find` is $O(m)$, which is terrible. Instead, we need to tell the difference between **"there's nothing here"** and **"there has never been anything here"**. If a slot has never held anything, we can stop. If a slot used to hold something, we have to keep going, since the key we want could have been pushed past it before it was removed

So when we remove a key, we don't empty its slot, we mark it with a **tombstone** that means "something was here, but it's gone". `find` treats a tombstone as full (keep probing), and `insert` treats it as empty (we're allowed to reuse it). **Figure 30.7** compares the two. It's the same trick we saw for removing from graph formats, where we marked an entry as missing instead of reshuffling a whole vector

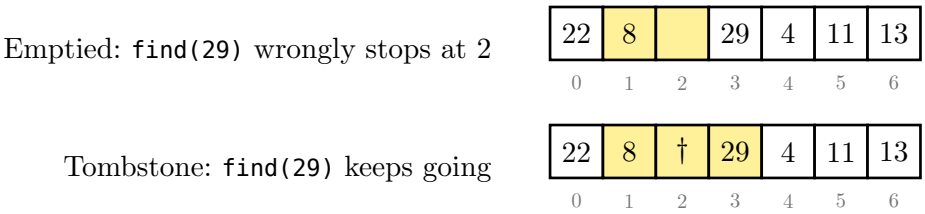


Figure 30.7: Example: `remove(16)` then `find(29)` on the full table from **Figure 30.5**. Emptying slot 2 cuts 29 off from its hash, while a tombstone (†) tells `find` to keep probing

So every slot is really in one of **three** states. We can store an extra bit or two per slot, or reserve two special values of the key type (one for "never used" and one for "deleted"), and real implementations do both

```
// ProbingTable.h

#pragma once

#include <string>
#include <utility>
#include <vector>

// never used, holding a pair, or holding a pair that was removed
enum class State { Empty, Full, Tombstone };

struct Slot {
    State state = State::Empty;
    int key = 0;
    std::string value;
};

class ProbingTable {
public:
    /** Inserts a key-value pair into the first open slot it probes
     * Runtime: O(1) amortized expected (under SUHA), O(n) worst case
     */
    void insert(int key, const std::string& value);

    /** Returns a pointer to the key's value, or nullptr if it's missing
     * Runtime: O(1) expected (under SUHA), O(n) worst case
     */
    std::string* find(int key);

    /** Removes the key from the table by leaving a tombstone in its slot
     * Runtime: O(1) expected (under SUHA), O(n) worst case
     */
    void remove(int key);

private:
```

```
    /** Returns the slot to check on the ith probe for a key (linear probing)
     * Runtime: O(1)
     */
    size_t probe(int key, size_t i) const { return (key + i) % table_.size(); }

    /** Doubles the table and rehashes every key into it
     * Runtime: O(n)
     */
    void resize();

    std::vector<Slot> table_ = std::vector<Slot>(7);

    // the number of Full slots (tombstones don't count)
    size_t size_ = 0;

    // the load factor that triggers a resize
    double maxLoad_ = 0.7;
};
```

```
// ProbingTable.cpp

#include "ProbingTable.h"

/** Returns a pointer to the key's value, or nullptr if it's missing
 * Runtime: O(1) expected (under SUHA), O(n) worst case
 */
std::string* ProbingTable::find(int key) {
    // stop after m probes, so a full table can't loop forever
    for (size_t i = 0; i < table_.size(); i++) {
        Slot& slot = table_[probe(key, i)];
        // a never-used slot ends the run, so the key isn't in the table
        if (slot.state == State::Empty) { return nullptr; }
        // a tombstone or another key: ours could be further down
        if (slot.state == State::Full && slot.key == key) { return &slot.value; }
    }
    return nullptr;
}

/** Removes the key from the table by leaving a tombstone in its slot
 * Runtime: O(1) expected (under SUHA), O(n) worst case
 */
void ProbingTable::remove(int key) {
    for (size_t i = 0; i < table_.size(); i++) {
        Slot& slot = table_[probe(key, i)];
        if (slot.state == State::Empty) { return; }
        if (slot.state == State::Full && slot.key == key) {
            // not Empty, so later finds keep probing past it
            slot.state = State::Tombstone;
```

```
            size_--;
            return;
        }
    }
}

/** Inserts a key-value pair into the first open slot it probes
 * Runtime: O(1) amortized expected (under SUHA), O(n) worst case
 */
void ProbingTable::insert(int key, const std::string& value) {
    // resize first, so there's always an open slot to land in
    if (size_ + 1 > maxLoad_ * table_.size()) { resize(); }
    for (size_t i = 0; i < table_.size(); i++) {
        Slot& slot = table_[probe(key, i)];
        // both Empty slots and tombstones can be reused
        if (slot.state != State::Full) {
            slot = {State::Full, key, value};
            size_++;
            return;
        }
    }
}
```

Like the chaining version, `insert` assumes the key isn't already in the table. The `resize` call is there because a closed hashing table can fill up, which we'll deal with soon

How fast is this? An unsuccessful `find` probes from the key's hash to the end of the run of full slots it lands in, so in the worst case it walks the **longest run of full slots, plus one** for the empty slot that stops it. Everything depends on how long those runs get

30.5.4 Clustering

And that's the problem with linear probing: runs of full slots (**clusters**) love to grow. Say we have a table of size 10 with $h(k) = k \bmod 10$, and insert 1, 11, and 21 (which all hash to 1) and then 3 and 13 (which hash to 3). We're only using half of the table, but all five keys are in one clump from slot 1 to slot 5, like in **Figure 30.8**

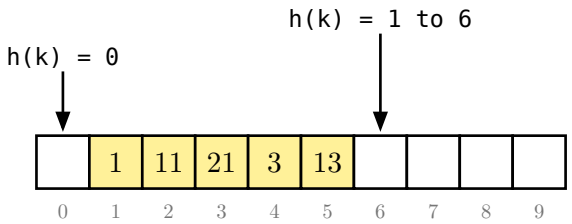


Figure 30.8: Example: linear probing with $h(k) = k \bmod 10$ after inserting 1, 11, 21, 3, 13. Any new key that hashes to 0 through 6 lands next to the cluster and grows it

If our hash function acts like a fair 10-sided die, what's the chance the **next** insert makes this cluster bigger? Any key hashing to 1 through 5 probes to the end and lands in slot 6, a key hashing to 6 lands in 6 directly, and a key hashing to 0 lands in 0, right in front of the cluster. That's 7 out of 10 hash values, so there's a **70%** chance of growing the cluster, even though the table is only half full! And each time it grows, the chance of growing it again goes up

Even starting from a single key, any later key that collides with it extends it, and every collision makes the clump bigger and easier to hit. A cluster never really goes away either: even after removing keys from it, `find` still has to probe through the tombstones. Analyzing this formally takes more probability than we're going to do here, but it's clear that big clusters get bigger. This is called **primary clustering**

The fix: instead of always trying the very next slot, let's **pick better**

30.6 Double Hashing

Every closed hashing scheme has collisions, and the only thing it can do is choose another slot to try. The trick is to choose it in a way that doesn't feed clusters, while staying cheap: whatever work we do to pick the next slot gets charged to every insert

Double hashing uses **two** hash functions: h_1 picks the starting slot, and h_2 picks the **step size**, how far to jump on each probe

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

For our running example, we'll use

$$h_1(k) = k \bmod 7 \qquad h_2(k) = 5 - (k \bmod 5)$$

Notice $h_2(k)$ is always between 1 and 5, so it's never 0 (a step size of 0 would just probe the same slot forever). Two keys that collide at the same starting slot will usually have different step sizes, so they spread out instead of piling up next to each other. Also notice that h_2 isn't compressed by `mod 7` until it's combined with h_1

Key	$h_1(k)$	$h_2(k)$	Linear probes	Double hashing probes
16	2	4	2	2
8	1	2	1	1
4	4	1	4	4
13	6	2	6	6
29	1	1	1, 2, 3	1, 2, 3
11	4	4	4, 5	4, 1, 5
22	1	3	1, 2, 3, 4, 5, 6, 0	1, 4, 0

29 happens to get a step size of 1, so it does the same thing as linear probing, but 22 jumps by 3 and finds a slot after 3 probes instead of 7, like in **Figure 30.9**. In total that's 13 probes instead of 16 for the same seven keys

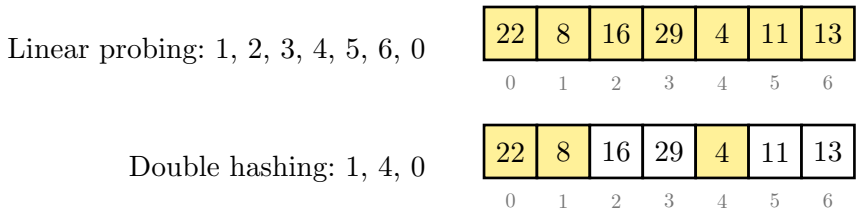


Figure 30.9: Example: the probes for `insert(22)`, the last of our seven keys. Linear probing walks the whole cluster, while double hashing jumps by $h_2(22) = 3$

To use double hashing, the probing code only needs a different `probe` function in place of the linear one

```
// ProbingTable.cpp

/** Returns the slot to check on the ith probe for a key (double hashing)
 * Runtime: O(1)
 */
size_t ProbingTable::probe(int key, size_t i) const {
    // h2(k) = 5 - (k mod 5) is always between 1 and 5, so it's never 0
    size_t step = 5 - key % 5;
    // start at h1(k) = k mod m and jump by the step on every probe
    return (key % table_.size() + i * step) % table_.size();
}
```

30.6.1 Step Sizes Must Be Relatively Prime to m

Double hashing introduces a new problem. What if our table had 8 slots instead of 7, using $h_1(k) = k \bmod 8$ and the same h_2? Say slots 1, 3, 5, and 7 are full, and we insert 23. $h_1(23) = 7$, which is full, and $h_2(23) = 5 - 3 = 2$, so we probe 7, 1, 3, 5, 7, 1, … forever, like in **Figure 30.10**. Half the table is empty, but a step of 2 from an odd slot only ever visits odd slots

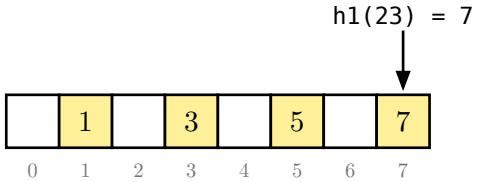


Figure 30.10: Example: double hashing into a table of size 8. With a step size of $h_2(23) = 2$, the probes cycle through 7, 1, 3, 5 and never reach the empty even slots

So $h_2(k)$ must be **relatively prime** to m: the two can share no common divisor besides 1. They don't have to be prime themselves, they just act like primes with respect to each other. Then the probe sequence visits every slot before repeating, so if there's an empty slot, we'll find it

The easiest way to guarantee that is to make the table size **prime**, since every number from 1 to $m - 1$ is relatively prime to a prime m. That's why our table has 7 slots, and since h_2 is always

between 1 and 5, every step size works. Finding big primes is inconvenient, though, so in practice implementations keep a list of primes that roughly double (up to around 100,000), and past that they double and subtract one, since odd numbers are relatively prime to more numbers than even ones. That isn't a guarantee, but once a table is that big, it's unlikely to matter

There are hundreds of other closed hashing schemes, almost all variations on this idea. One fun one is **cuckoo hashing**, where every key has two possible slots and a new key can kick an old one out to its other slot. Going further takes a lot more math than we have, but we'll stick with three strategies: separate chaining, linear probing, and double hashing

30.7 Resizing and Rehashing

So far our tables have a fixed size, and that's a real problem for all three strategies. With closed hashing, once we've stored m keys, the table is 100% full and we can't store anything else, whether we use linear probing or double hashing. Separate chaining never fills up, but it gets worse forever: even with a perfect hash function that spreads keys evenly, the chains have length $\frac{n}{m}$, and if m never changes, that's just $O(n)$

To get the constant time we want, the table has to grow. We use the same strategy from very early in the course: when the table gets too full, allocate a new table about **double** the size (rounded up to a prime, if we're using double hashing) and move everything over

But we can't just copy the old array into the front of the new one. Suppose our full linear probing table from **Figure 30.5** doubles to 14 slots, and we copy every slot straight over. Now the hash function is $k \bmod 14$, so `find(8)` looks in slot $8 \bmod 14 = 8$, which is empty, and decides 8 isn't in the table! The index a key belongs at depends on m, so every key has to be inserted again with the new hash function. This is called **rehashing**. Going through the old table in order

- $22 \bmod 14 = 8$, so 22 goes in slot 8
- $8 \bmod 14 = 8$, which 22 just took, so linear probing puts 8 in slot 9
- $16 \bmod 14 = 2$, $29 \bmod 14 = 1$, and 4, 11, and 13 hash to themselves

Figure 30.11 shows all three tables. Since we're using linear probing here, the new size doesn't need to be prime

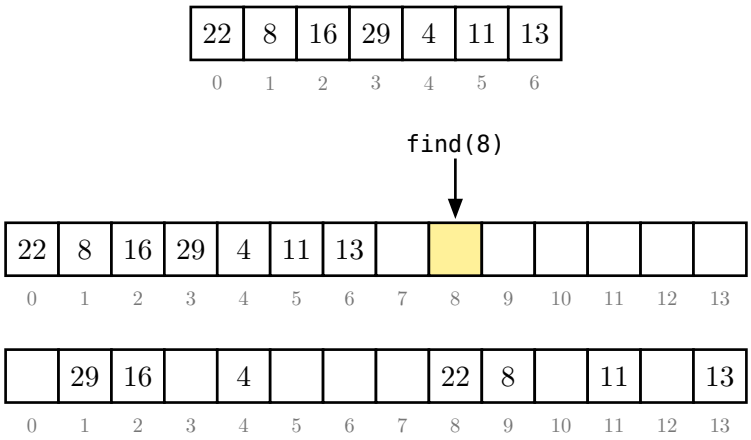


Figure 30.11: Example: growing the full linear probing table from **Figure 30.5** (top) to 14 slots. Copying the slots straight over (middle) breaks `find(8)`, which now looks in slot $8 \bmod 14 = 8$. Rehashing every key with $k \bmod 14$ (bottom) puts each key where `find` will look

The code is easy, since we already wrote `insert`: walk through the old table and insert every key into the new one. Tombstones don't get copied, so rehashing also cleans them out

```
// ProbingTable.cpp

/** Doubles the table and rehashes every key into it
 * Runtime: O(n)
 */
void ProbingTable::resize() {
    // take the old slots out, leaving table_ free to be replaced
    std::vector<Slot> old = std::move(table_);
    table_ = std::vector<Slot>(2 * old.size());
    size_ = 0;
    for (const Slot& slot : old) {
        // rehash into the new size, dropping tombstones
        if (slot.state == State::Full) { insert(slot.key, slot.value); }
    }
}
```

Rehashing is $O(n)$, but it has exactly the same analysis as the array doubling in **Figure 5.2**: as long as we (roughly) double, every insert pays a constant amount toward the next resize, so `insert` stays amortized $O(1)$. Doubling and then rounding up to a prime only changes things by a small factor, which doesn't matter

While we're rehashing everything anyway, there's one more thing we should do: **pick a whole new hash function**, not just a new compression. Remember the one big worry about hash tables: if someone knows our hash function, they can deliberately insert a bunch of keys that all collide, and nothing we do helps. Since the old hash function doesn't matter anymore after a rehash, choosing a fresh one forces an attacker to figure out our function all over again every time we resize

30.8 The Simple Uniform Hashing Assumption

Here's where we are: we **want** to say that hash table operations are $O(1)$ expected (and `insert` is $O(1)$ amortized expected). But an expected runtime needs randomness from somewhere. In many real hash table implementations, the only randomness is **the data itself**, which gives us an average case, and if someone knows what our table is built on, they can hand it garbage. That really has been done in practice

So to analyze hash tables, we make an assumption, the **simple uniform hashing assumption** (**SUHA**). For a table of size m, a simple uniform hash h satisfies

$$\forall k_1 \neq k_2, \quad P(h(k_1) = h(k_2)) = \frac{1}{m}$$

In words: any two different keys collide with probability $\frac{1}{m}$, as if each key's index were an independent roll of a fair m-sided die. That really assumes two things: that keys are **independent** of each other, and that every index is **equally likely**

Under SUHA, the expected length of a chain is easy. Take any key k in a separate chaining table with n keys. Each of the other $n-1$ keys lands in k's bucket with probability $\frac{1}{m}$, so by linearity of expectation, the expected number of keys sharing its chain is $\frac{n-1}{m}$. Similarly, any particular bucket expects $\frac{n}{m}$ keys. This ratio is called the **load factor**

$$\alpha = \frac{n}{m}$$

which is just how full the table is. The longest chain in the whole table is somewhat longer than α, but `find` only walks the chain its own key lands in

So far that only gets us $O(n)$, since α grows with n if m is fixed. But now we combine it with resizing: we pick a fixed maximum load factor, and **whenever $\frac{n}{m}$ reaches it, we resize**. We still resize by doubling, and the amortization still works out (the math is a little more involved since we resize when the table is a fixed fraction full instead of completely full). Since α never exceeds a constant,

$$E[\text{find}] = O(1 + \alpha) = O(1)$$

That's it: under SUHA, with a bounded load factor, hash tables really do give expected $O(1)$ operations, and you can push this further to show they're fast with high probability

30.8.1 Why SUHA Is Suspicious

The whole weakness of hash tables is the hash function. No **fixed** function can satisfy SUHA for every input: if I know your function, I can always pick a set of keys that all hash to the same index. In fact, for any fixed function, at least $\frac{|K|}{m}$ keys share some index, and I can just keep choosing those

Here's a hint that something is off. If we really assume the keys are independent and uniform, then the identity function `k % m` is a **perfect** hash function, and we've already seen how badly simple functions like that do on real data. The assumption is pushing the randomness onto the data

The way to make this honest is to put the randomness in the **hash function** instead of the data: every time we build or resize the table, choose a hash function uniformly at random from a whole family of functions that spread keys out uniformly. If the keyspace is infinite, there are infinitely many functions to choose from, and a program can't really choose uniformly from all of them. In practice, though, our inputs are finite (strings of at most 10,000 characters, say), so we can choose from a finite family, which is doable, if tricky to get right

A few places really do this. The routers that handle internet backbone traffic have to look up addresses incredibly fast, so they use hash tables that

- keep their load factor bounded by simply refusing to store more than a certain number of entries (once full, they start throwing old data out, since they mostly care about recent traffic)
- generate a random number the size of an address (64 or 128 bits), and hash each address by XORing it with that random number and then taking `mod m`
- pick a fresh random number every time they rehash

That works because every address is exactly the same size, and the XOR is done in custom hardware. Strings are a lot harder, since they vary in length, and XORing a 1,000-character string with a 1,000-character random number takes real time

A couple of questions came up about this. Can we predict when a hash function will go bad? For a specific hash function, if you can efficiently predict its collisions, it's a degenerate, terrible hash function to begin with. If it really is uniform, you can't do better than chance, **unless you know the function itself**, at which point you can always defeat it. And if we do everything right, won't it never go bad? No: a fixed function can always be attacked by someone who knows it, which is exactly why we keep the specific function (like the router's random number) secret and change it on every rehash. There's also a huge amount of research on hash functions for different goals: cryptographic hashes, hashes for fast tables like ours, and even hashes for **similarity**, where we want similar keys to collide, which is the exact opposite of what we want here

In practice, even a hash table with a kind of garbage hash function is usually fast. But sometimes it isn't, and that's what's going on when a website that mostly works suddenly starts acting horribly for no particular reason: it's hit a hash function that conflicts with the data it's being given. A business can dump the table and restart. A military or medical system can't. This is also why theoreticians treat algorithms that need a hash table as much worse than ones that don't (in CS 374, you'll often be told you aren't allowed to use one)

30.8.2 Choosing a Load Factor

So how full should we let the table get? Analyzing linear probing and double hashing fully gives these expected numbers of probes per search under SUHA (you don't need to memorize them, but you should know what the curves look like)

Strategy	**Search**	**Expected probes**	$\alpha = 0.5$	**0.7**	**0.8**	**0.9**	**0.95**
Separate chaining	Found	$1 + \frac{\alpha}{2}$	1.25	1.35	1.4	1.45	1.48
	Not found	$1 + \alpha$	1.5	1.7	1.8	1.9	1.95
Linear probing	Found	$\frac{1}{2}\left(1 + \frac{1}{1-\alpha}\right)$	1.5	2.2	3	5.5	10.5

Strategy	Search	Expected probes	$\alpha = 0.5$	0.7	0.8	0.9	0.95
	Not found	$\frac{1}{2}\left(1 + \frac{1}{(1-\alpha)^2}\right)$	2.5	6.1	13	50.5	200.5
Double hashing	Found	$\frac{1}{\alpha}\ln\left(\frac{1}{1-\alpha}\right)$	1.39	1.72	2.01	2.56	3.15
	Not found	$\frac{1}{1-\alpha}$	2	3.3	5	10	20

Up to around 50% full, everything is really good. Double hashing stays good until roughly 80% and then gets worse fast, and linear probing gets worse sooner, since clusters give it more long runs early. An unsuccessful search at 95% is already dozens to hundreds of probes. The elbows on these curves are **sharp**, so as long as we keep the table reasonably empty, things are fine

Separate chaining never blows up, since chains can grow past $\alpha = 1$: its cost just grows linearly with α, so we can trade space for speed at almost any point we want. By probe count it actually wins at every load factor, but each step along a chain is a pointer jump somewhere else in memory, while probing walks through neighboring array cells, which the cache loves. That's why closed hashing (especially double hashing) is often faster in practice at a sensible load factor. And chaining's space savings are mostly an illusion: it uses the full array just like closed hashing **and** allocates a new node for every key, so the more collisions, the more extra space

So in practice, people resize somewhere between 60% and 90% full, and around 70% to 80% is common. Think about why: doubling a table that's only 50% full barely helps, but doubling one that's 90% full takes it down to 45%, which is a huge difference. If memory is really cheap, a load factor of 10% makes collisions almost nonexistent, and an infinitely large table would never collide at all. But a table that's only as big as the number of things we're storing is just as bad a trade-off in the other direction

30.9 Which Hash Table?

If the records are **very large** (say, each value is a 10,000 by 10,000 image), separate chaining is clearly better, since closed hashing stores whole records in the array, and every empty slot wastes a record's worth of memory. For reasonably sized records, closed hashing, at least double hashing, tends to be better, mostly for structural reasons like the cache, and its expected probe counts are a little better too

In C++, `std::unordered_map` and `std::unordered_set` are hash tables that use separate chaining. That lets them hash many more kinds of things, since closed hashing needs two extra values per slot (never used and deleted) that we can identify efficiently. You can see the chaining in its interface: its default `max_load_factor()` is 1.0, which only makes sense if chains can hold more than one key. (`std::map`, on the other hand, is a red-black tree: a balanced BST.)

30.10 Hash Tables vs. BSTs

A hash table is yet another implementation of the dictionary ADT. So why did we spend so much time on BSTs?

First, the key type has to be **hashable**. If you try to put your own type in a `std::unordered_set`, it'll refuse to compile, because it has no idea how to hash it. C++ comes with hash functions for the built-in types, so we can build one for our type out of those, along with `==` for checking whether two keys match

```
// Point.h

#pragma once

#include <cstddef>
#include <functional>

struct Point {
    int x;
    int y;

    // the table uses == to tell keys in the same bucket apart
    bool operator==(const Point& other) const {
        return x == other.x && y == other.y;
    }
};

namespace std {
template <>
struct hash<Point> {
    /** Hashes a point by combining the built-in hashes of its coordinates
     * Runtime: O(1)
     */
    size_t operator()(const Point& p) const {
        // multiplying by 31 makes (1, 2) and (2, 1) hash differently
        return hash<int>()(p.x) * 31 + hash<int>()(p.y);
    }
};
}
```

With that, a `std::unordered_set<Point>` works just like a `std::unordered_set<int>`

Second, a hash table's runtimes are **amortized** (sometimes we have to rehash) and **probabilistic** (we don't have a guarantee, only an expectation). A balanced BST always runs in $O(\log n)$: that's the real worst case, with no amortization and no "sometimes it's bad". If we need a hard deadline, we want the BST

Third, a hash table can't do **range** or **nearest** searches. It can tell us whether a key is in the table, but it can't tell us the key that's one bigger, so the only way to find the nearest key is to search the whole table. A BST does a `find` and then walks to the neighbors, and in multiple dimensions a KD tree does the same thing

Operation	Hash table	AVL tree	Unsorted linked list
`find`	$O(1)$ expected, worst $O(n)$	$O(\log n)$	$O(n)$
`insert`	$O(1)^*$ expected	$O(\log n)$	$O(1)$
`remove`	$O(1)$ expected, worst $O(n)$	$O(\log n)$	$O(n)$
Storage	$O(n)$	$O(n)$	$O(n)$
Range find	No	Yes	No

The hash table numbers assume SUHA, a bounded load factor, and amortized resizing (the $*$). Its storage is still $O(n)$, since the load factor is a constant ratio. An unsorted linked list only makes sense as a dictionary if `insert` is most of what we do, since it's just an insert at the front. We can also add B-trees next to AVL trees: they have the same $O(\log n)$ guarantees, but are shorter and do more work per level, which is perfect when the data doesn't fit in memory and every seek to disk (or another machine) is slow. Every one of these is the right dictionary in some situation, depending on which operations we do most and which guarantees we need

Honestly, hash tables are one of the most frustrating data structures, because there isn't a simple rule for when they'll work. Most of the time they're great, and they've made slow, cumbersome projects fast. But sometimes they bite, and they've taken entire server infrastructures down for weeks. So use them when you can't think of something better (which happens a lot), but know that a hash table can't give you a real worst-case guarantee. They're also an active research area right now: since SUHA is basically always a lie, people are designing hash tables that need weaker guarantees from their hash functions, and they're making progress, though the analysis is well beyond this course

31 Bloom Filters

Hash tables were all about **speed**: we wanted `find` and `insert` in expected $O(1)$ time, and we were happy to spend $O(n)$ space to get it. But what if the problem isn't time? What if we just don't have enough **space**?

Some data sets are so big that even an index of them doesn't fit anywhere. Back in 2013, Google estimated its index at about 60 billion web pages, and the whole web at about 130 trillion pages. The Sequence Read Archive, which holds the genetic sequences people have collected, is around 60 petabytes (10^{15} bytes). Sky surveys have grown from a few terabytes to hundreds of petabytes, and the Square Kilometre Array is expected to produce about 4.6 exabytes. We're not even asking where to store the data itself. We just want an index that can answer "is this thing in there?"

Space is also speed. Reading all of the cache takes less than a second, reading all of RAM takes hours to days, reading all of the disks takes months, and reading the whole network takes years. So the less space our index takes, the closer to the CPU it can live

Our first instinct might be compression, like the Huffman encoding we saw earlier. But compression only buys a **constant factor**: 10 times smaller turns 4.6 exabytes into 0.46 exabytes, which still takes years to read. And it can't do much better than that, because if we want to tell n different things apart **exactly**, we need n different representations, so storing them exactly always takes at least n space

So we're going to give up on exact answers. That's the same move we made with probability: we accept a small chance of being wrong in exchange for something much better

31.1 From a Hash Table to a Bit Vector

Recall separate chaining from the previous chapter: an array of pointers, where each pointer leads to a linked list of everything that hashed to that index. With the hash function $h(x) = x \bmod 7$, inserting 16, 8, 4, 13, 29, 11, and 22 gives the hash table on the left of **Figure 31.1**

Now let's throw almost all of it away. We don't keep the keys, and we don't keep the lists. We only keep **whether anything ever hashed to each index**. That's a yes or no question, so instead of an 8-byte pointer, each index only needs a **single bit**: 1 if something hashed there, 0 if nothing did. That's the bit vector on the right of **Figure 31.1**

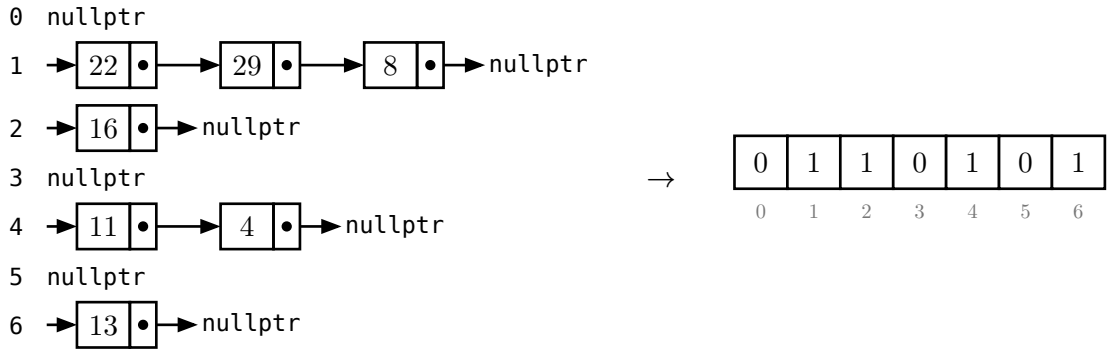


Figure 31.1: Example: a separate chaining hash table with $h(x) = x \bmod 7$ after inserting 16, 8, 4, 13, 29, 11, and 22, and the bit vector we get by keeping only whether each index is used

This is the most primitive version of a **Bloom filter**. It's a bit vector, initialized to all 0s, and a hash function

- To insert a key, hash it and set that bit to 1
- To find a key, hash it and look at that bit

We have to do a little bit of fancy arithmetic to get at a single bit inside a chunk of memory, but we'll see how to do that later in this chapter

31.1.1 Insert

Figure 31.2 inserts the same seven keys into a 7-bit filter with $h(x) = x \bmod 7$. Notice that most of the inserts don't change anything: 29 and 22 both hash to 1, which 8 already set, and 11 hashes to 4, which 4 already set. Setting a bit that's already 1 just leaves it 1

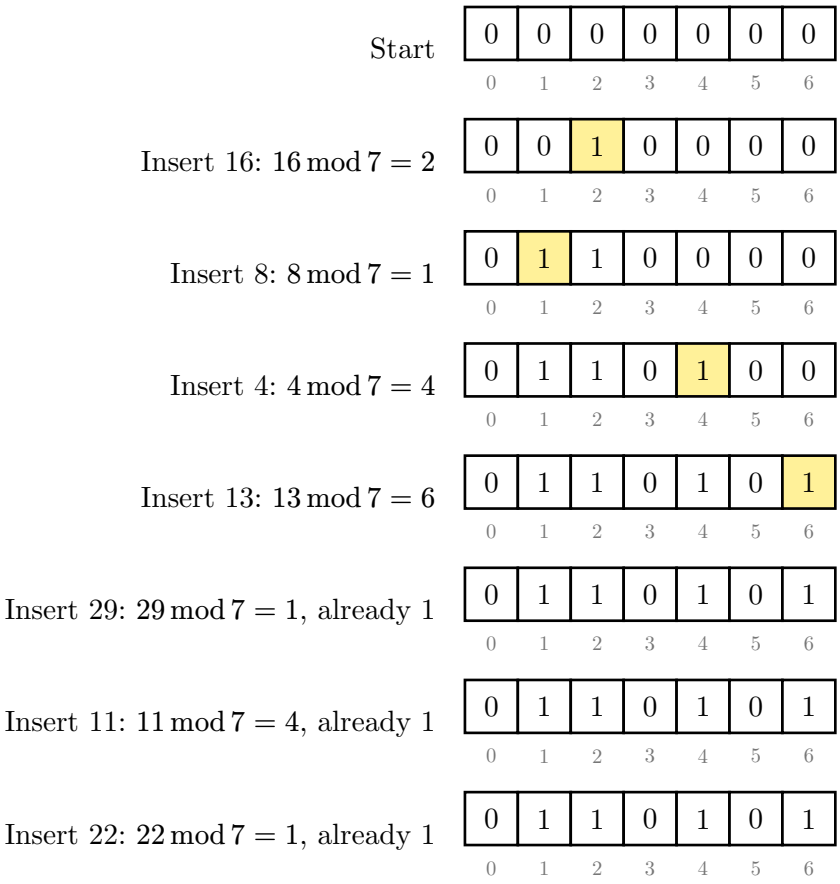


Figure 31.2: Example: inserting 16, 8, 4, 13, 29, 11, and 22 into a 7-bit Bloom filter with $h(x) = x \bmod 7$. The highlighted cells are the bits that changed

Seven `int`s take $7 \cdot 4 = 28$ bytes, and our filter takes 7 **bits**, less than a single byte. Who cares about 28 bytes, but the same idea lets a filter of a few megabytes stand in for an index of exabytes

31.1.2 Find

Finding is where things get interesting. In **Figure 31.3**, we query three keys on the finished filter from **Figure 31.2**

- `find(16)`: $16 \bmod 7 = 2$, and bit 2 is 1, so we say **yes**. 16 really is in the set, so this is a **true positive**
- `find(20)`: $20 \bmod 7 = 6$, and bit 6 is 1, so we say **yes**. But 20 was never inserted! Bit 6 is 1 because of 13. This is a **false positive**
- `find(3)`: $3 \bmod 7 = 3$, and bit 3 is 0, so we say **no**. 3 is not in the set, and we're definitely right

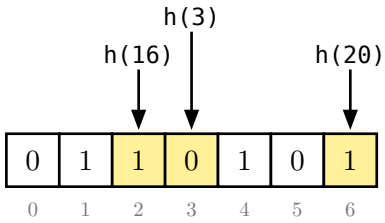


Figure 31.3: Example: `find(16)` (a true positive), `find(20)` (a false positive), and `find(3)` (a true negative) on the filter from **Figure 31.2**

A 0 means **nothing** in the whole set ever hashed there, so a "no" is always correct. A 1 only means **something** hashed there, and we have no idea what. In **Figure 31.3**, `find(2)` would also say yes, for the same reason as 20

That's the key thing to understand about `find` on a Bloom filter: a "yes" really means "**maybe**". From the point of view of someone using the filter, a true positive and a false positive look **identical**. Both are just a 1, and the filter has literally no data to tell them apart, since we threw out the keys. So the filter is 100% right when it says no, and right with some probability when it says yes

- If the bit is 0, the key is **definitely not** in the set
- If the bit is 1, the key **might** be in the set

Even a filter that says "maybe" to half of the keys that aren't in the set is useful. Every "no" is a search through exabytes that we never have to do, so we've cut the number of expensive searches in half, using a filter small enough to keep in memory all the time. The more bits we give the filter, the fewer collisions it has, and the less often it says maybe when the answer is no

31.2 One-Sided Error

Any yes-or-no test, like a disease test, has four possible outcomes, depending on what it says and what's actually true

	Actually in the set	**Actually not in the set**
Filter says yes	True positive	False positive
Filter says no	False negative (impossible)	True negative

Most real tests can get any of the four. A Bloom filter can **never** give a false negative: if a key was inserted, its bit was set to 1, and bits never go back to 0. This is **one-sided error**, just like the primality test we saw when we talked about probability

Here's a concrete use: a **malicious website detector** in a browser. Checking every URL with an authoritative server would double the network traffic. Instead, the browser occasionally downloads a small Bloom filter with every known malicious site inserted into it, and checks each URL against the filter first

- If the filter says **safe** (no), the site is definitely not on the list, and we never had to ask anyone

- If the filter says **malicious** (maybe), we ask the real service whether the site is actually malicious

The one outcome we can't afford is calling a malicious site safe, and that's exactly the outcome a Bloom filter can never give. Calling a safe site malicious only costs us one extra request

Another way to see it: a Bloom filter is a filter in the literal sense. If we pass all our data through it, **everything** in the set makes it through, along with a few things that aren't. The filter never loses anything we're looking for, it just doesn't remove everything we aren't. And because it never loses anything, we can pass what's left through a **second**, different filter to remove even more of the wrong things

31.3 Repeated Trials

How do we make the filter more accurate? One idea is to store a **count** at each index instead of a single bit. That gives us a hint, but it uses more space and gets complicated, so let's put it aside

Instead, recall the primality test: one random trial could wrongly say "prime", but we could make a mistake extremely unlikely by running **more** random trials, since we only have to be wrong once to catch it. A hash function is also fundamentally a random selection (that's the whole point of SUHA), so we can do the same thing here. Note that hashing the same key with the same function again isn't a new trial: it gives the same answer every time. We need **different** hash functions

31.3.1 k Separate Filters

The most straightforward way is to build k completely separate Bloom filters, each with its own hash function $h_1, ..., h_k$. To insert a key, we insert it into every filter. To find a key, we check every filter, and say yes only if **all** of them say yes. If any of them says no, the key is definitely not in the set

For a false positive, **every** filter has to be wrong. If the filters are independent and each one is wrong half the time, then

- one filter gives a false positive rate of $\frac{1}{2}$
- two filters give $\frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$
- three filters give $\frac{1}{8}$

and in general, if one filter has false positive rate p, then k filters have a false positive rate of p^k

For an example, we'll switch to 10-bit filters and three hash functions

$$h_1(x) = x \bmod 10 \qquad h_2(x) = 2x \bmod 10 \qquad h_3(x) = (3x + 5) \bmod 10$$

Figure 31.4 inserts 6, 8, and 4 into three separate filters. For instance, 6 sets bit $h_1(6) = 6$ in the first filter, bit $h_2(6) = 12 \bmod 10 = 2$ in the second, and bit $h_3(6) = 23 \bmod 10 = 3$ in the third. Querying 9 checks bit 9 of the first filter, which is 0, so we can say no right away

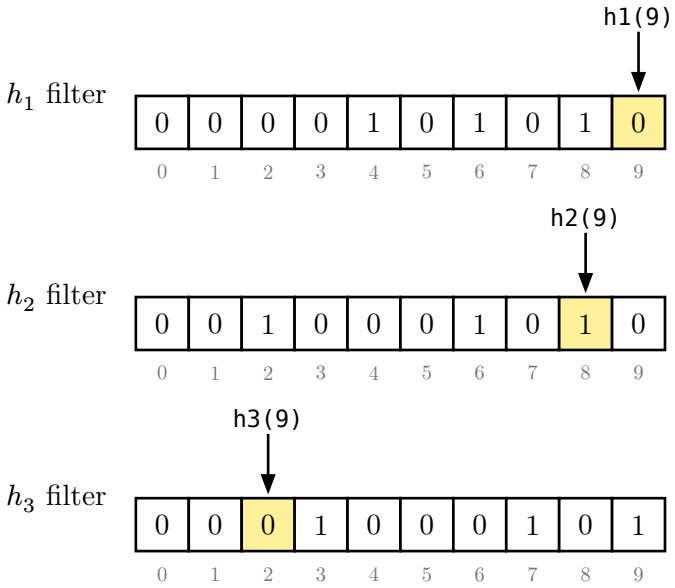


Figure 31.4: Example: 6, 8, and 4 inserted into three separate 10-bit filters. `find(9)` checks one bit in each filter, and the 0 in the h_1 filter means 9 is not in the set

Notice that we can stop as soon as we see a 0: once one filter says no, the rest can't change the answer. That's a nice optimization for time, but it doesn't help with space, and space is the problem. We started with a hash table and threw out everything except one bit per index, and now we're storing ***k* bits** per index. If k gets anywhere close to the size of a key, we've built something almost as big as the data we were trying to avoid storing

31.3.2 One Shared Bit Vector

So let's throw something out again. Instead of k filters, we keep **one** bit vector of m bits, and all k hash functions set and check bits in that same vector. This is what people actually mean by a **Bloom filter**

To insert a key x

1. Compute $h_1(x), h_2(x), ..., h_k(x)$
2. Set every one of those bits to 1 (if a bit is already 1, it stays 1)

To find a key x

1. Compute $h_1(x), h_2(x), ..., h_k(x)$
2. If **any** of those bits is 0, the key is definitely not in the set, so return false (we can stop at the first 0)
3. If **all** of them are 1, the key might be in the set, so return true

Figure 31.5 inserts 6, 8, and 4 into one 10-bit vector using the same three hash functions. Each insert sets up to three bits, and 8 only sets two new ones, since $h_2(8) = 6$ was already set by $h_1(6)$

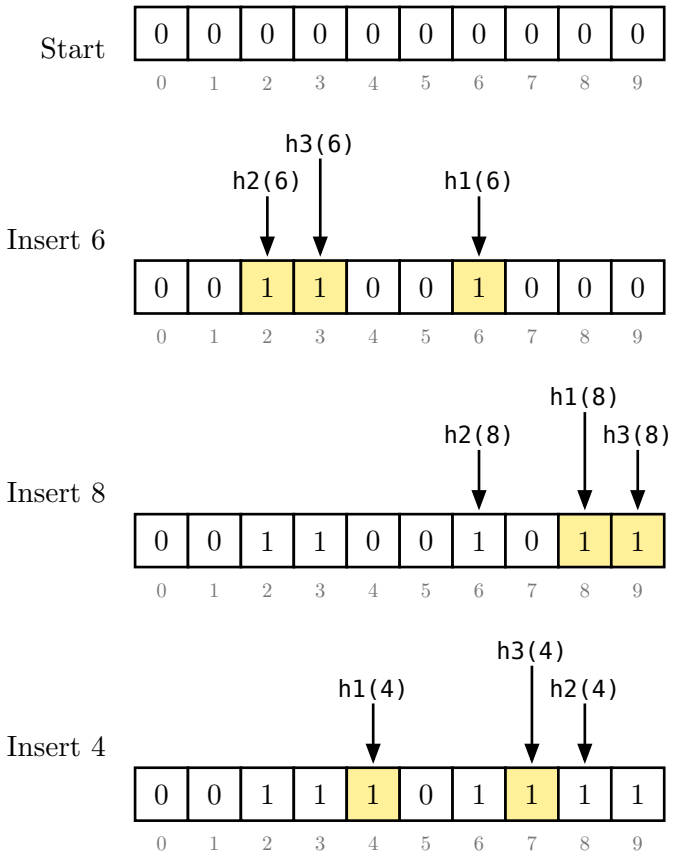


Figure 31.5: Example: inserting 6, 8, and 4 into one 10-bit Bloom filter with $k = 3$ hash functions. The highlighted cells are the bits that changed

This still never gives a false negative: if a key was inserted, **all** of its bits were set to 1, so all of them will still be 1 when we look it up. **Figure 31.6** runs four queries on the finished filter, where only bits 0, 1, and 5 are still 0

- `find(6)`: bits 6, 2, and 3 are all 1, so **yes**, a true positive
- `find(1)`: $h_1(1) = 1$, and bit 1 is 0, so **no** right away
- `find(16)`: bits 6, 2, and 3 are all 1, so **yes**, but 16 was never inserted, so this is a false positive
- `find(9)`: bits 9, 8, and 2 are all 1, so **yes**, another false positive

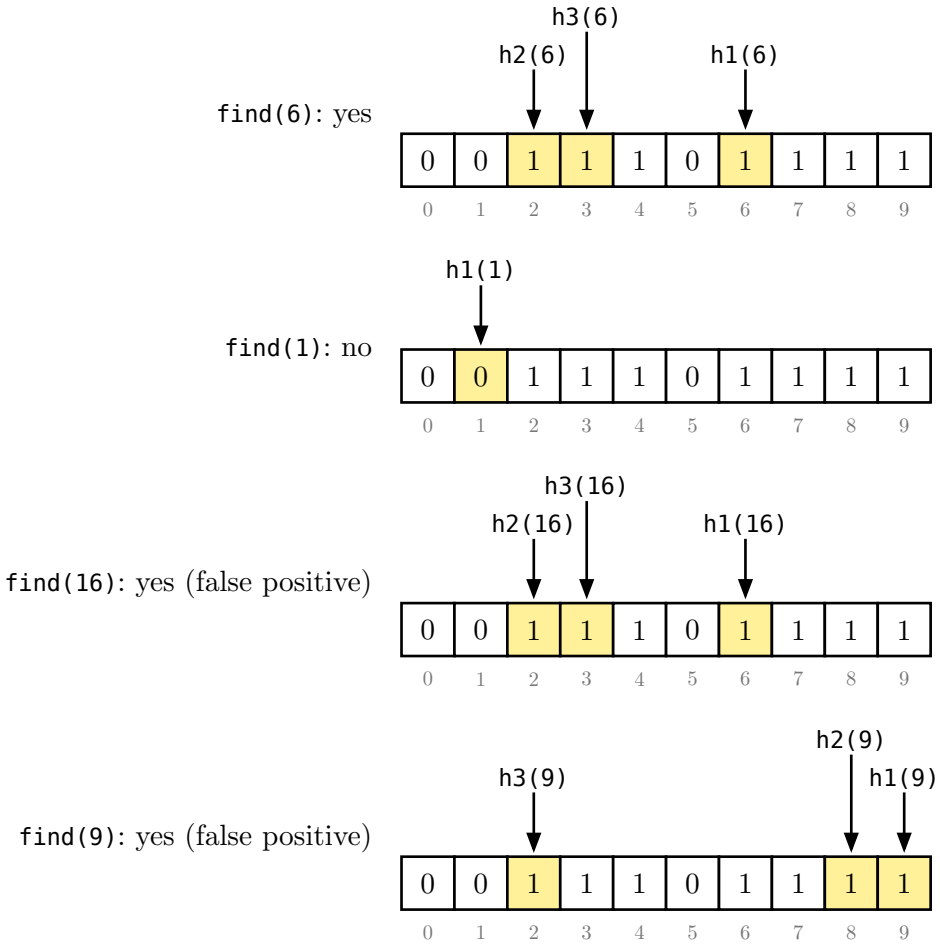


Figure 31.6: Example: four queries on the filter from **Figure 31.5**. 6 is a true positive, 1 is a true negative, and 16 and 9 are false positives

Two things are worth noticing here. First, 16 checks **exactly** the same bits as 6. That's because all three of our toy hash functions only depend on the last digit of the key, so they aren't independent at all, and no number of them could ever tell 6 and 16 apart. Real hash functions need to be independent for repeated trials to help

Second, look at 9. In **Figure 31.4**, the separate filters said no to 9, because bit 9 of the h_1 filter was 0. In the shared vector, bit 9 is 1, but it was set by $h_3(8)$, not by any h_1. That's the price of sharing: when h_2 looks at a bit, it might see a 1 that was put there by h_1 of a **different** key. So the number of 1s in the filter isn't just the number of keys we inserted, and a shared filter fills up faster than k separate ones. Our simple p^k analysis doesn't work anymore

We can also take this to the extreme. A "Bloom filter" whose only hash function sends every key to bit 0 is technically correct: it never gives a false negative. But as soon as anything is inserted, it says yes to everything, which is a 100% false positive rate. Being correct is easy; what we care about is **how often** we're wrong

31.4 Analysis

31.4.1 Running Time and Space

Both `insert` and `find` compute k hash functions and touch k bits, so they're $O(k)$. But k is fixed when we create the filter and never changes for that filter, so it's also fair to call them $O(1)$, the same way $O(100000)$ is still $O(1)$. $O(k)$ is just the more precise answer

The filter stores a bit vector of m bits and the k hash functions, so its space is $O(m)$, or $O(m+k)$ if we count the functions as data too, since we need to know them to use the filter. Interestingly, that doesn't depend on n at all: no matter how much we insert, the filter never gets bigger. But that's a little bit of a lie, since a filter that's too small for its n fills up with 1s and becomes useless. To pick m, we need to know how often we're wrong

The size has to be fixed at creation for another reason too. A hash table can resize by rehashing, because it still has all of its keys. A Bloom filter threw its keys away, so there's nothing to rehash: to make it bigger, or to change k, we'd have to rebuild it from the original data

For the same reason, a Bloom filter has **no remove**. To remove a key, we'd want to set its bits back to 0, but every one of those bits might also belong to other keys. In **Figure 31.5**, removing 8 by clearing bits 8, 6, and 9 would also clear bit 6 for 6 and bit 8 for 4, and suddenly `find(6)` says no: a false negative, which is the one thing a Bloom filter promises never to do. (Storing a count instead of a bit, the idea we put aside earlier, is how variations of the Bloom filter support removing)

31.4.2 False Positive Rate

Let's work out the probability of a false positive for a filter of m bits, with k SUHA hash functions, after inserting n keys. With k separate filters, the answer was just p^k. With one shared vector, it's tempting to guess that one key sets $\frac{k}{m}$ of the bits, but that's wrong, since some of the k hash functions might hit the same bit

Computing the probability that a bit is 1 is hard. But a bit is either 0 or 1, so $P(\text{bit is 1}) = 1 - P(\text{bit is 0})$, and the probability that a bit is **0** is much easier. (This is the same trick as with AVL trees, where we couldn't compute the height directly, but we could bound it by counting nodes)

1. By SUHA, one hash function hits a specific bit with probability $\frac{1}{m}$, so it **misses** that bit with probability $1 - \frac{1}{m}$
2. Inserting one key uses k independent hash functions, which is just like inserting k keys with one function, so the bit is still 0 with probability $\left(1 - \frac{1}{m}\right)^k$
3. Inserting n keys uses nk hash functions in total, so after all of them, the bit is still 0 with probability $\left(1 - \frac{1}{m}\right)^{nk}$
4. So a bit is 1 with probability $1 - \left(1 - \frac{1}{m}\right)^{nk}$

That's the chance that **one** check of a key that isn't in the set lands on a 1 when it should have found a 0. For `find` to give a false positive, **all k** of its checks have to land on 1s, so

$$P(\text{false positive}) = \left(1 - \left(1 - \frac{1}{m}\right)^{nk}\right)^k$$

Let's see how each variable affects this

- A bigger m makes $\left(1 - \frac{1}{m}\right)^{nk}$ closer to 1, so fewer bits are 1, which is **better**: a bigger filter makes fewer mistakes
- A bigger n puts more 1s in the filter, which is **worse**
- A bigger k does both. The k in the exponent inside means every key sets more bits, so the filter fills up faster, which is worse. The k outside means we need **more** 1s to be wrong, which is better

31.4.3 The Optimal Number of Hash Functions

So for a given m and n, what's the best k? It turns out to be

$$k^* = \left(\frac{m}{n}\right) \ln 2 \approx 0.7 \cdot \frac{m}{n}$$

Getting there takes a bit of calculus, which you won't be asked to do, but here's the sketch. First, we rewrite the inside using $x = e^{\ln x}$ and $\ln(x^y) = y \ln x$

$$\left(1 - \frac{1}{m}\right)^{nk} = e^{nk \ln\left(1 - \frac{1}{m}\right)}$$

Next, recall the series $\ln(1 + x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + ...$, with $x = -\frac{1}{m}$. Since m is a number of bits, it's big (at least a thousand or so), so $\frac{1}{m}$ is tiny, the squared and higher terms are tinier, and $\ln\left(1 - \frac{1}{m}\right) \approx -\frac{1}{m}$. Plugging that back in

$$P(\text{false positive}) \approx \left(1 - e^{-\frac{nk}{m}}\right)^k$$

To find the k that minimizes this, we can minimize its natural log instead, since ln is increasing and has its minimum at the same place. Taking the derivative of $k \ln\left(1 - e^{-\frac{nk}{m}}\right)$ with respect to k and setting it to 0 gives $e^{-\frac{nk}{m}} = \frac{1}{2}$, which is $k = \left(\frac{m}{n}\right) \ln 2$. At that point, exactly half of the bits are 1, and the false positive rate is about $\left(\frac{1}{2}\right)^k$

The table shows the approximate false positive rate for a filter with 10 bits per key ($\frac{m}{n} = 10$). It drops quickly, bottoms out around $k = 7$ (since $k^* = 10 \ln 2 \approx 6.93$), and then climbs again as the filter gets too full. If we measure it with real data instead, the curve isn't perfectly smooth, since we can only ever have a whole number of hash functions, but it has the same shape

k	False positive rate for $\frac{m}{n} = 10$
1	9.52%
2	3.29%
3	1.74%
4	1.18%
5	0.94%
6	0.84%
7	**0.82%**

k	**False positive rate for $\frac{m}{n} = 10$**
8	0.85%
9	0.91%
10	1.02%

Notice that $\frac{m}{n}$ is the **inverse** of the load factor from hash tables: it's bits per key, not keys per slot. And notice how small these numbers are. Our early examples assumed a filter that's wrong half the time, but with 10 bits per key and 7 hash functions, a Bloom filter is wrong less than 1% of the time. That's still an error, but it's a very small one

We can solve the same equation for whichever variable we don't know

- **100 keys and 3 hash functions**: $m = \frac{kn}{\ln 2} = \frac{300}{\ln 2} \approx 433$ bits
- **100 bits and 20 keys**: $k = (\ln 2) \cdot \frac{100}{20} \approx 3.47$, so 3 or 4 hash functions. There's no such thing as 3.47 hash functions, so we'd plug both 3 and 4 into the formula and see which is better
- **100 bits and 2 hash functions**: $n = (\ln 2) \cdot \frac{100}{2} \approx 34.7$ keys

31.5 Using Bloom Filters

Here's an important catch: for the optimal error rate, $m = \frac{kn}{\ln 2}$, which is still **linear in n**. We get a big constant factor improvement (a few bits per key instead of the whole key), but it's still $O(n)$ space. There are two ways to think about that

- We can combine it with other tricks, like first reducing the data in other ways, so the Bloom filter is a constant factor improvement on top of another one
- We can just pick a budget. "You get one megabyte, how accurate can you be?" A 100,000-bit filter might have a fairly high error rate for our data, but still filter out most of what we don't need

Either way, there's a lot of engineering here without hard answers. What you need to remember is that when we create a Bloom filter, we fix

- the number of hash functions, k
- the size of the filter, m
- and ideally, an estimate of the number of keys, n

That last one is new. Every other data structure we've seen lets us keep inserting forever, but a Bloom filter can't grow. If we insert fewer keys than we planned for, we've wasted space, which defeats the point. If we insert too many, the filter fills with 1s and stops telling us anything

One place most current web browsers use a Bloom filter is their **cache**. Most resources a browser loads are only ever visited **once**, so caching all of them wastes disk activity on things we'll never look at again. Instead, the browser keeps a Bloom filter of resources it has seen

- When it loads a resource, it checks the filter. If the bits are 0, it has definitely never seen this resource, so it just inserts it into the filter and doesn't cache it
- If the bits are already 1, it has **probably** seen it before, so this is the second visit, and it caches it

A false positive just means we cache something we've only seen once, which is what the browser did for **everything** before the Bloom filter. When this was turned on, disk activity dropped a lot with no real impact on performance, which also tells us how many resources really are only visited once. The filter can stay small, too, since it only needs to hold what we've seen since the browser started, and when it gets too full, we can just throw it out and start a new empty one: anything we visit constantly is already in the cache

31.6 Working With Bits

Throughout this chapter, we've been talking about bits, but C++ doesn't give us an obvious way to work with a single bit. What it does have are the **bitwise operators**, which treat a number as a sequence of bits and work on every bit at once. With A = `10110` and B = `01110`

Operator	Meaning	Example
`~B`	Not: flips every bit	`10001`
`A & B`	And: 1 only where both are 1	`00110`
`A | B`	Or: 1 where either is 1	`11110`
`A >> 2`	Right shift: moves every bit 2 places right, dropping the bits that fall off and filling with 0s	`00101`
`B << 2`	Left shift: moves every bit 2 places left, filling with 0s	`11000`

Shifts are only well-defined when the value is **unsigned**. In our computers, a negative number has a 1 in its most significant bit, and right shifting a signed value usually fills with 1s instead of 0s to keep it negative, but the C++ standard doesn't promise that, so different compilers and hardware can do different things. That's why we use types like `uint64_t`: an unsigned 64-bit integer, which is the size of a register on our computers. Each of these operators works on all 64 bits in a **single instruction**, so with a vector of `uint64_t`, we handle 64 bits at a time instead of one

To check a single bit, we build a **mask**: a number with only that bit set. If we want bit b (counting from 0 on the right), the mask is `1 << b`. And-ing the mask with our bits zeroes out everything except bit b, so the result is nonzero exactly when bit b is 1. For example, to check the third bit (index 2), the mask is `1 << 2 = 00100`. Or-ing with the mask sets the bit instead

```
/** Returns whether bit `b` of `bits` is 1
 * Runtime: O(1)
 */
bool isSet(uint64_t bits, unsigned b) {
    uint64_t mask = uint64_t(1) << b;
    return (bits & mask) != 0;
}

/** Returns `bits` with bit `b` set to 1
 * Runtime: O(1)
 */
```

```
uint64_t setBit(uint64_t bits, unsigned b) {
    uint64_t mask = uint64_t(1) << b;
    return bits | mask;
}
```

Notice we shift `uint64_t(1)`, not just `1`. A plain `1` is an `int`, which is usually only 32 bits, so shifting it 40 places wouldn't work. Code like this used to be written everywhere for performance, because compilers weren't good enough to do it for us. Nowadays, compilers do a lot of this on their own, and your time is usually worth more than the computer's, but for a structure that's **all** bits, it's exactly what we need

31.6.1 Implementation

Putting it together, bit i of the filter lives in word `i / 64` of the vector, at position `i % 64` within that word

```
// BloomFilter.h

class BloomFilter {
  public:
    using HashFunction = std::function<size_t(int)>;

    BloomFilter(size_t m, std::vector<HashFunction> hashes)
        : m_(m), bits_((m + 63) / 64, 0), hashes_(hashes) { }

    /** Sets the bit each hash function picks for `key`
     * Runtime: O(k)
     */
    void insert(int key) {
        for(const HashFunction& h : hashes_) {
            size_t i = h(key) % m_;
            // bit `i` lives in word `i / 64`, at position `i % 64` within it
            bits_[i / 64] |= uint64_t(1) << (i % 64);
        }
    }

    /** Returns whether `key` might be in the filter (no false negatives)
     * Runtime: O(k)
     */
    bool contains(int key) const {
        for(const HashFunction& h : hashes_) {
            size_t i = h(key) % m_;
            // one 0 means the key was never inserted
            if((bits_[i / 64] & (uint64_t(1) << (i % 64))) == 0) {
                return false;
            }
        }
        return true;
```

```
    }

    /** Turns this filter into the union of itself and `other`
     * Runtime: O(m)
     */
    void unionWith(const BloomFilter& other);

  private:
    // the number of bits in the filter
    size_t m_;

    // the m bits, packed 64 to a word
    std::vector<uint64_t> bits_;

    // the k hash functions
    std::vector<HashFunction> hashes_;
};
```

31.7 Union and Intersection

Here's where working with bits really pays off. Suppose we have two Bloom filters with the same size m and the **same** hash functions (not just the same number of them, the exact same functions). One holds the set S_1 and the other holds S_2. What do we get if we bitwise or them?

A bit is 1 in the result exactly when some key from S_1 or S_2 set it, which is exactly the filter we'd get by inserting every key of $S_1 \cup S_2$. So **bitwise or is union**, and it's exact. In **Figure 31.7**, $S_1 = \{6, 4\}$ and $S_2 = \{8, 4\}$, and their or is the same filter we built for $\{6, 8, 4\}$ in **Figure 31.5**

That's a big deal for huge data sets. Every computer holding part of the data can build a Bloom filter for its own part, and then we or them all together to get the filter for everything, which gives us massive parallelism

```
// BloomFilter.cpp

/** Turns this filter into the union of itself and `other`
 * Runtime: O(m)
 */
void BloomFilter::unionWith(const BloomFilter& other) {
    // a bit is 1 in the union exactly when it is 1 in either filter
    for(size_t w = 0; w < bits_.size(); w++) {
        bits_[w] |= other.bits_[w];
    }
}
```

What about bitwise and? If a key is in both sets, all of its bits are 1 in both filters, so they're still 1 in the result, and `find` will say yes. So it still never gives a false negative for $S_1 \cap S_2$. But it's **not** the same as the filter for $S_1 \cap S_2$. In **Figure 31.7**, $S_1 \cap S_2 = \{4\}$, whose filter only has bits 4, 7, and 8 set. The and also has bit 6 set, because 6 set it in S_1 and 8 happened to set it in S_2,

even though neither 6 nor 8 is in the intersection. So and-ing gives us something that still works as a filter, but with extra 1s, and a different error rate that our analysis doesn't cover

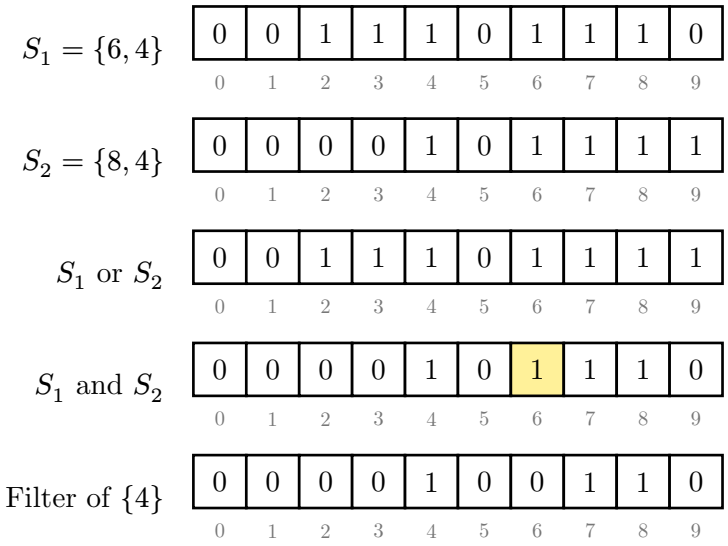


Figure 31.7: Example: the bitwise or and the bitwise and of two Bloom filters with the hash functions from **Figure 31.5**. The or is exactly the filter of $S_1 \cup S_2$, but the and has an extra 1 (highlighted) that the filter of $S_1 \cap S_2 = \{4\}$ doesn't have

32 Cardinality

Bloom filters let us store a set in far less space than anything else we've seen, at the cost of the occasional false positive. But there's a catch we ran into when we optimized one: to pick the filter size m and the number of hash functions k, we need to know n, the number of **unique** items going in, before we ever start. A bloom filter can't grow, so if we guess too small it fills up with ones and tells us nothing, and if we guess too big we waste the space we were trying to save. So how do we figure out how many unique things are in a data set that's too big to store?

32.1 Counting Unique Items

Let's start by asking what a bloom filter, exactly as we've studied it, can and can't do

Can a bloom filter...	**Answer**
Find the closest matching item to a query?	No
Check if a query exists in a data set?	Yes
Compare the similarity between two data sets?	Kind of
Count the number of unique items in a data set?	Kind of

- **Closest match: no.** A bloom filter never finds anything. It only says "probably here" or "definitely not here". Worse, a good hash function sends two very similar keys to completely unrelated places (we've been using `mod` because it's easy to compute by hand, but a real hash function scrambles its keys), so similar hashes tell us nothing about similar keys
- **Membership: yes.** That's exactly what it's built for: either the item is definitely not in the set, or it probably is
- **Similarity: kind of.** If two filters use the same m and the same hash functions and have exactly the same bits set, they were very likely built from the same data, since every query would give the same answer. But that's not really a measure of **how** similar two sets are
- **Unique items: kind of.** With $k = 1$, counting the ones in the filter is close to the number of unique items, but every collision makes two items look like one, so the count comes up short

The last two aren't what a bloom filter does, but the same probabilistic idea (hash everything, then reason about where the hashes land) leads to data structures that do solve them efficiently. This chapter is about the last one, which is exactly what we needed to size our bloom filter

The size of a set is called its **cardinality**, and the key word is **unique**. If we stream every picture we can find on the internet, we don't want to know how many pictures went by (a single counter would do that); we want to know how many **different** cat photos there are, no matter how many times each one shows up. To count that exactly, we'd have to store every unique photo we've seen so far and check each new one against all of them. With something like 130 billion photos, we can't

store them all, so we'll settle for a **cardinality estimate**: a good guess at the number of unique items, made in one pass over the data while storing almost nothing

32.2 The Hat Problem

Before we bring hashing in, let's play a game with a hat. Take a deck of cards numbered 0 to 999. I put some of them in a hat, and you want to know how many. I let you pull out one card at random, and it's 95. That tells you almost nothing: there's at least one card in the hat, and one of them is 95

Now suppose I tell you something different: the card I handed you is the **minimum** card in the hat. That tells you a lot more. Cards 0 through 94 definitely aren't in the hat, and every other card is above 95. But if I chose which cards went in, I could have put in exactly 95 through 999, so the minimum alone still can't tell you how many there are

So let's make one more change: the cards in the hat are chosen **uniformly at random**. Now a minimum of 95 really means something. If the hat had 900 cards, every single one would have to avoid 0 through 94, and the chance of that is about 10^{-127}, so it basically never happens. With 9 cards, the chance is about 41%. A big minimum means there aren't many cards in the hat, and a small minimum means there are a lot

How do we turn that into a number? If we scatter n cards uniformly at random across the range 0 to 999, they cut it into $n+1$ gaps (one before the first card, one between each pair, and one after the last). No gap is special, so on average each one is about $\frac{1000}{n+1}$ long, and that includes the very first gap, from 0 up to the minimum. **Figure 32.1** shows a hat with 9 cards: the gaps aren't all exactly 100, but they're all close, and the first one is 95

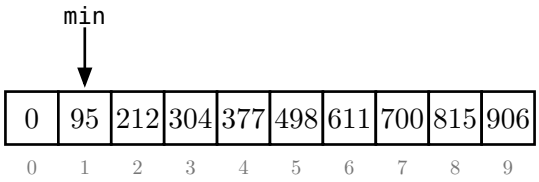


Figure 32.1: Example: 9 cards drawn at random from 0 to 999, in sorted order, after a 0 marking the start of the range. The 10 gaps (95, 117, 92, 73, 121, 113, 89, 115, 91, and 94 from 906 up to 1000) average out to $\frac{1000}{10} = 100$

So if the minimum is 95, our best guess is that the first gap is average, and we solve for n

$$95 \approx \frac{1000}{n+1} \quad \Rightarrow \quad n+1 \approx \frac{1000}{95} \approx 10.5 \quad \Rightarrow \quad n \approx 9.5$$

Our estimate is 9.5 unique cards. It's not an integer, but it's an estimate, not a count. The same reasoning ranks any number of hats by their minimums: the smaller the minimum, the more cards we expect

Minimum card	Estimate $\frac{1000}{\text{min}} - 1$
10	99
95	9.5
200	4
500	1

32.3 From Cards to Hashes

Real data isn't a hat of uniformly random numbers. Photos, genomes, and URLs are anything but uniform, and if we just looked at the smallest photo, we'd learn nothing. This is where hashing comes in. Under **SUHA**, a hash function sends every key to a uniformly random value in its range, independently of every other key. So hashing an item is the same as putting a card with a random number into the hat, and the minimum hash value we've seen plays the role of the minimum card

Hashing also handles the "unique" part for free. If the same item shows up twice, it hashes to the same value twice, which can't change the minimum. So the minimum hash depends only on the **set** of unique items, no matter how many duplicates are mixed in

To make the math clean, we turn every hash into a number between 0 and 1 by dividing it by the size of the hash range. A hash function produces an integer in some fixed range (a 64-bit hash, for example, is always below 2^{64}), so we divide by the size of that range and get a real number in $[0, 1)$, where 0 means the smallest possible hash and values close to 1 mean the largest. That should start to look a lot like a probability

32.3.1 The Expected Minimum

Let $X_1, X_2, ..., X_n$ be the hashes of n unique items, each uniform in $[0, 1)$, and let M be their minimum. We claim

$$E[M] = \frac{1}{n+1}$$

To see why, imagine hashing one more unique item, X_{n+1}, and ask: what's the chance it lands below M, becoming the new minimum? We can answer that two ways

1. If the minimum is some value m, then X_{n+1} is uniform, so it lands below m with probability exactly m and above with probability $1 - m$. Averaging over every possible minimum, the chance is $E[M]$
2. Landing below M means X_{n+1} is the smallest of all $n + 1$ values. All $n + 1$ are uniform and independent, so each one is equally likely to be the smallest, and the chance is $\frac{1}{n+1}$

Both answers describe the same event, so $E[M] = \frac{1}{n+1}$. This is the hat problem again, just with the range scaled down from 1000 to 1: n values cut $[0, 1)$ into $n + 1$ gaps that average $\frac{1}{n+1}$ each

32.3.2 The Estimator

Flipping that around gives us our estimate. If $M \approx \frac{1}{n+1}$, then

$$n \approx \frac{1}{M} - 1$$

So to estimate the number of unique items in a stream

1. Start with the minimum at 1, the top of the range
2. For every item in the stream, hash it, divide by the size of the hash range, and keep the result if it's smaller than the current minimum
3. At the end, estimate $\frac{1}{M} - 1$ unique items

```
// MinSketch.h

class MinSketch {
  public:
    /** Hashes `item` into [0, 1] and keeps it if it is the new minimum
     * Runtime: O(1)
     */
    void insert(const std::string & item) {
        // divide by the size of the hash range to land in [0, 1]
        double h = hash_(item) / maxHash_;
        min_ = std::min(min_, h);
    }

    /** Estimates the number of unique items as 1 / min - 1
     * Runtime: O(1)
     */
    double estimate() const {
        return 1 / min_ - 1;
    }

  private:
    double min_ = 1;
    std::hash<std::string> hash_;
    static constexpr double maxHash_ = std::numeric_limits<size_t>::max();
};
```

Notice that we still look at every single item, so this is still a full pass over the data. The win is **space**: instead of storing every unique item to compare against, we store one `double`. Each `insert` is $O(1)$, and the whole sketch is $O(1)$ space no matter how big the stream is. An empty stream even gives a sensible answer, since $\frac{1}{1} - 1 = 0$

Let's try it. To keep the numbers small, say our hash function produces values 0 to 999, so we divide by 1000. **Figure 32.2** shows three different streams. In the first one, the same item appears twice (both hash to 920), so the stream has 4 unique items, while the other two have 5

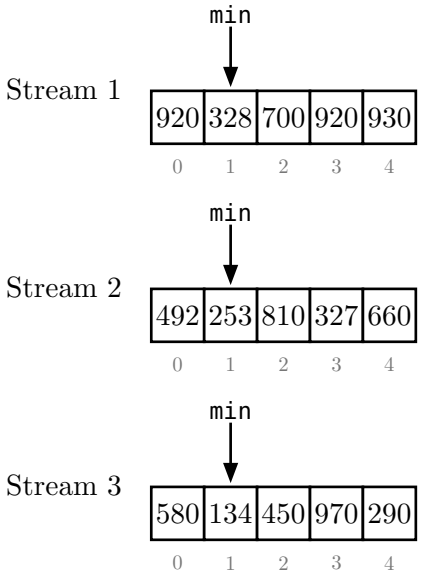


Figure 32.2: Example: the hashes (out of 1000) of three streams. Stream 1 has a repeated item (920 at indices 0 and 3) and 4 unique items, while streams 2 and 3 have 5 unique items each

Stream	**Minimum M**	**Estimate $\frac{1}{M} - 1$**	**Actual unique items**
1	0.328	2.05	4
2	0.253	2.95	5
3	0.134	6.46	5

The repeated 920 in stream 1 didn't matter at all, which is exactly what we want. But honestly, these guesses aren't great: 2.05 for 4 items, 2.95 for 5, then 6.46 for 5. A single minimum is a single sample, and one lucky (or unlucky) hash throws the whole estimate off. A small minimum suggests lots of items and a big minimum suggests few, but one number is a pretty noisy way to tell those apart

32.4 Keeping the k Smallest

How do we do better? We want more samples. Right now the whole sketch is one number, and we can afford a lot more than that. So instead of keeping only the smallest hash, we keep the ***k* smallest** hashes: the smallest, the second smallest, and so on up to the k-th smallest, which we call M_k

The same reasoning about gaps still works. The n hashes cut $[0, 1)$ into $n + 1$ gaps of about $\frac{1}{n+1}$ each, and the k-th smallest hash sits at the end of the first k gaps, so

$$E[M_k] = \frac{k}{n+1} \quad \Rightarrow \quad n \approx \frac{k}{M_k} - 1$$

With $k = 1$, this is exactly the estimator from before. The difference is that M_k depends on k gaps instead of just one, so a single unusually short or long gap gets averaged out with the others.

Figure 32.3 shows stream 2 from **Figure 32.2** in sorted order. With 5 items, the ideal spacing would be $\frac{1000}{6} \approx 167$, and the first gap alone (253) is a lot further off from that than the first three together (492, versus an ideal $3 \times 167 = 500$)

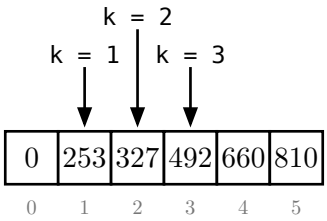


Figure 32.3: Stream 2 from **Figure 32.2**, sorted, after a 0 marking the start of the range. The k-th smallest hash sits at index k

k	M_k	Estimate $\frac{k}{M_k} - 1$
1	0.253	$\frac{1}{0.253} - 1 \approx 2.95$
2	0.327	$\frac{2}{0.327} - 1 \approx 5.12$
3	0.492	$\frac{3}{0.492} - 1 \approx 5.10$

Just going from $k = 1$ to $k = 2$ took us from 2.95 to 5.1, almost exactly the right answer, and $k = 3$ stays at 5.1. Using all of the values doesn't magically make the estimate perfect: k is about getting enough samples, not about looking at everything

32.4.1 Building the Sketch

We maintain the k smallest hashes as the stream goes by

1. Hash the item and divide by the size of the hash range
2. If that value is already in the sketch, it's a repeated item, so do nothing
3. Otherwise, add it. If the sketch now has $k + 1$ values, throw away the largest one
4. At the end, if the sketch has fewer than k values, the stream had fewer than k unique items and we know the count exactly. Otherwise, estimate $\frac{k}{M_k} - 1$, where M_k is the largest value left in the sketch

Figure 32.4 builds the $k = 3$ sketch for stream 2. The first three hashes all fit, then 327 pushes out 810, and 660 is bigger than everything we're keeping, so it changes nothing

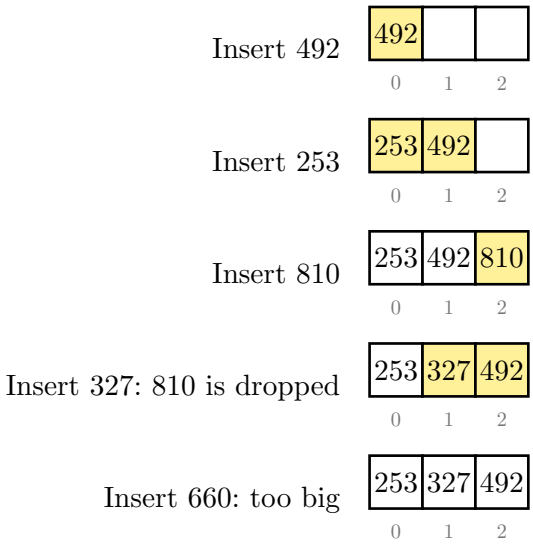


Figure 32.4: Building the $k = 3$ sketch for stream 2 from **Figure 32.2**, keeping the 3 smallest hashes in sorted order. The highlighted cells are the ones that changed. At the end, $M_3 = 0.492$

In C++, a `std::set` does almost all of the work for us: it keeps the values sorted, so the largest is at the end, and it ignores a value it already has, which takes care of repeated items

```
// CardinalitySketch.h

class CardinalitySketch {
  public:
    CardinalitySketch(size_t k) : k_(k) { }

    /** Hashes `item` into [0, 1] and keeps it if it is among the k smallest
     * Runtime: O(log k)
     */
    void insert(const std::string & item) {
        // divide by the size of the hash range to land in [0, 1]
        double h = hash_(item) / maxHash_;
        smallest_.insert(h);

        // keep only the k smallest hashes by dropping the largest one
        if(smallest_.size() > k_) {
            smallest_.erase(std::prev(smallest_.end()));
        }
    }

    /** Estimates the number of unique items as k / (kth smallest hash) - 1
     * Runtime: O(1)
     */
    double estimate() const {
        // fewer than k unique items so far, so we know the exact count
        if(smallest_.size() < k_) {
```

```
            return smallest_.size();
        }
        double kthSmallest = *smallest_.rbegin();
        return k_ / kthSmallest - 1;
    }

  private:
    size_t k_;
    std::set<double> smallest_;
    std::hash<std::string> hash_;
    static constexpr double maxHash_ = std::numeric_limits<size_t>::max();
};
```

The set never holds more than $k+1$ values, so each `insert` is $O(\log k)$ and the sketch takes $O(k)$ space. This is our **cardinality sketch**. Even with $k = 100$, that's 100 `doubles`, or 800 bytes, to estimate the number of unique items in a stream of billions

32.5 Accuracy

How much does a bigger k actually help? The full statistical analysis is well outside this course (even with magic SUHA hash functions it gets messy fast), but we can just try it. The table shows `CardinalitySketch` run over thousands of random streams, each with every item repeated twice, for two different numbers of unique items n. The typical error is the median of $|\text{estimate} - n|/n$, and the last two columns give the error that 90% of the trials stayed under

k	Typical error, $n = 1000$	Typical error, $n = 100{,}000$	90% within, $n = 1000$	90% within, $n = 100{,}000$
1	67%	65%	830%	670%
10	21%	21%	58%	58%
100	6%	7%	16%	17%

A few things stand out

- **With $k = 1$, the estimate is close to useless.** Typically it's off by two thirds, and in one trial out of ten it's off by a factor of 7 or more. Most of those huge errors are overestimates: one hash that happens to land very close to 0 makes $\frac{1}{M}$ enormous. If the hash range is small, like 10,000 values, a single unlucky stream can be estimated at nearly 10,000 items, as many as the hash range can even tell apart
- **$k = 10$ is already a lot better**, and **$k = 100$ is good**: typically within about 6%, and 9 times out of 10 within about 17%. In our streams of 1000 items, $k = 100$ is a tenth of the set, but in the streams of 100,000 items it's a thousandth, and it does just as well. We don't need k to be any particular fraction of n
- **The returns diminish.** Making k 10 times bigger made the typical error only about 3 times smaller (roughly $\sqrt{10}$), so the jump from $k = 1$ to $k = 10$ is much bigger than the jump from $k = 10$ to $k = 100$

- **It's k that matters, not n.** It's tempting to think a bigger stream would make the estimate better on its own, but with k fixed, $n = 100{,}000$ is no more accurate than $n = 1000$. Since n is so much bigger than k, the error depends on how many gaps we average, which is k. To get a better estimate, we keep more hashes

So we choose k for the accuracy we want, the same way we chose m and the number of hash functions for a bloom filter. Every estimate here depends on the hash function behaving like SUHA, though: if real hashes aren't close to uniform, none of these numbers hold

33 MinHash Sketch Strategy

Cardinality is a useful measure of a set, but it's a fairly limited one. A question we often want to ask instead is **how similar are two sets?** If I have a huge collection of images and you have one too, and we're deciding whether to merge them, we want to know how much they overlap. If they're very similar, we wouldn't gain much by joining them, but if they're very different, we would. Or if we're studying a disease and have sequenced its genome, we might want to know which diseases we already understand have a similar genome, so we can use what we know about them

So we want to measure the similarity between sets that are potentially very large, and often between **lots** of sets that are potentially very large

33.1 Measuring Similarity

Which pair of sets is more similar? If both pairs have sets of the same size, that's easy: the pair that shares more is more similar. But in **Figure 33.1**, both pairs share exactly the same items (2, 3, and 4), so how much they share can't be the whole story

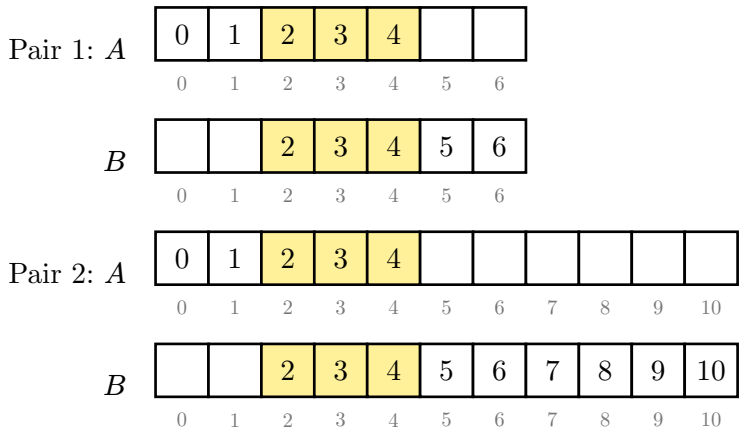


Figure 33.1: Example: two pairs of sets, with each item drawn in its own column. Both pairs share the same three items (highlighted), but in pair 2, B also has lots of items that A doesn't

Pair 1 is clearly more similar: in pair 2, B has a whole bunch of stuff that A doesn't. So when we measure similarity, we care about what the sets share, but **also** about what they don't share. The standard measure is the **Jaccard similarity** (or **Jaccard coefficient**): the size of the intersection divided by the size of the union

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

For pair 1, that's $\frac{3}{7} \approx 0.43$, and for pair 2, it's $\frac{3}{11} \approx 0.27$, which matches our intuition. If the sets share nothing, the intersection is empty and $J = 0$. If they share everything, the intersection **is** the union and $J = 1$. So the Jaccard similarity is always a number between 0 and 1, with no units

Now say I have 600 trillion cat pictures and you have 476 trillion, and we want to know if we'd get more cat pictures by putting them together, or if they're really all the same cat pictures. We can't go through every picture and compare them exactly. We couldn't even count the unique pictures in **one** collection exactly (that's why we needed cardinality estimates in the first place), so computing the intersection and union exactly is hopeless

33.2 Estimating Similarity From Sketches

It turns out we can build on top of our cardinality estimate. At some point, I streamed through my whole collection once (probably as I gathered it from the internet), and to count the unique pictures, I kept the k smallest hash values I saw, which is exactly the cardinality sketch from the previous chapter. Since it holds the k minimum hashes, we'll call it a **k-min sketch** of the set. It's tiny, maybe 100 bytes for 600 trillion cat pictures, so storing it costs nothing. If you did the same thing for your collection, then we each have a sketch, and we want to estimate our similarity using **only** the two sketches

33.2.1 A First Example

To keep things readable, let's write every item as its hash value, out of a range of 100. **Figure 33.2** shows two full sets, sorted by hash value. They share 5 items, and their union has 12, so their real similarity is $J = \frac{5}{12} \approx 0.42$

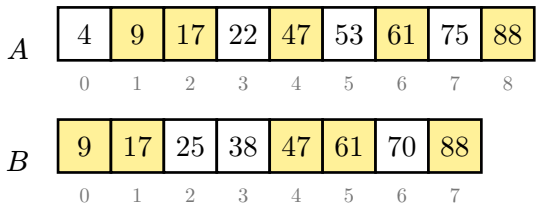


Figure 33.2: Example: two sets, with every item written as its hash value and sorted from smallest to largest. The 5 shared items are highlighted, and the union has 12 items, so $J(A, B) = \frac{5}{12}$

But all we actually stored is the $k = 4$ smallest values of each set, shown in **Figure 33.3**. If we combine the two sketches and take the 4 smallest values overall, we get 4, 9, 17, and 22, which are the 4 smallest hash values in all of $A \cup B$. Two of those four (9 and 17) are in both sketches, so we estimate that the sets are $\frac{2}{4} = \frac{1}{2}$ similar. The real answer was $\frac{5}{12}$, so we did okay

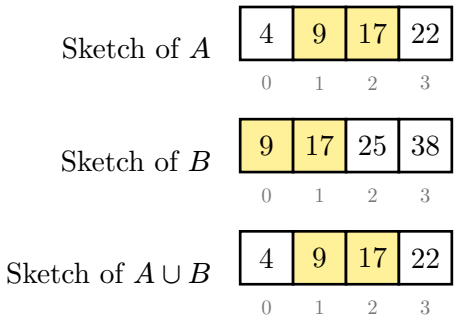


Figure 33.3: Example: the $k = 4$ sketches of the sets from **Figure 33.2**. The 4 smallest values of the union include two shared values (highlighted), so we estimate $J \approx \frac{2}{4}$

Notice we never needed the full sets to tell whether 22 is in B. If a value is one of the k smallest in $A \cup B$ and it belongs to B, then at most $k-1$ values of B can be smaller than it, so it has to be one of the k smallest in B too. Since 22 isn't in B's sketch, it isn't in B at all

33.2.2 Why This Works

Let's start with just the single smallest hash value in $A \cup B$. Since we assume SUHA, every item in $A \cup B$ is equally likely to be the one with the smallest hash. The smallest hash in A equals the smallest hash in B exactly when that item is in **both** sets, and since it's equally likely to be any item of the union, the chance it lands in the intersection is the fraction of the union that's shared

$$P(\min h(A) = \min h(B)) = \frac{|A \cap B|}{|A \cup B|} = J(A, B)$$

This is where the name **MinHash** comes from. But one sample is a coin flip: it always says either 0 or 1. In **Figure 33.2**, the smallest value in the union is 4, which is only in A, so with $k = 1$ we would have guessed that the sets share nothing. Keeping the k smallest values of the union gives us k samples instead of one, and the fraction of them that are shared is our estimate of J, just like keeping more values made our cardinality estimate better

33.3 Building a k-Min Sketch

Let's slow down and work through this more carefully, starting with how we compute the sketch in the first place. As we stream through the data, we keep the smallest hash values we've seen so far in a sorted array of size k

1. Hash the next item
2. If its hash is already in the sketch, we've seen this item before, so skip it (we only care about **unique** items)
3. If the sketch has fewer than k values, insert the hash in sorted order
4. Otherwise, if the hash is smaller than the largest value in the sketch, insert it in sorted order and drop the largest value
5. Otherwise, ignore it: it can't be one of the k smallest

We store the hash values themselves, not the items, so the sketch is k numbers no matter how big the set is. **Figure 33.4** streams the data 16, 8, 18, 13, 15 into a $k = 3$ sketch, using the toy hash function $h(x) = x \bmod 7$

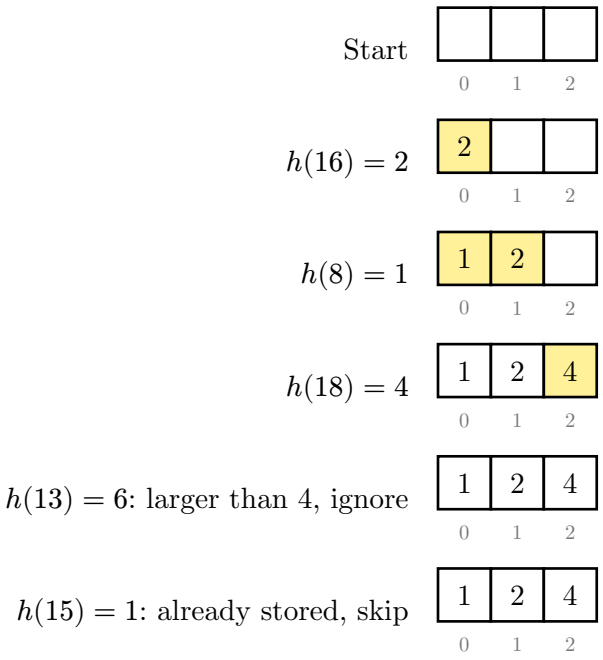


Figure 33.4: Example: building a $k = 3$ sketch of 16, 8, 18, 13, 15 with $h(x) = x \bmod 7$. The highlighted cells are the ones that changed

When 1 arrives, it's the new smallest value, so it goes at index 0 and 2 shifts over. In the worst case, a new value goes at the front and we shift all k values, so each item takes $O(k)$ time, which is a constant if k is fixed

Our toy hash function is terrible on purpose: 8 and 15 are different items with the same hash, so 15 gets skipped even though it's new. With a real hash function (say, 64 bits) that we treat as SUHA, a repeated hash value really does mean a repeated item, and skipping it is exactly how we ignore duplicates

33.4 The Union Sketch

We run that process over both of our huge data sets and end up with one sketch each. **Figure 33.5** shows $k = 8$ sketches for two sets A and B, with hash values out of a range of 100. These sixteen numbers are all we have left of our cat picture collections

We could build the sketch of $A \cup B$ by feeding A's values and then B's values through the same process, but that does a lot of shifting (all of B's small values have to squeeze in front of A's). Since both sketches are already sorted, it's easier to **merge** them, just like the merge step of merge sort

1. Look at the front of each sketch and take the smaller value

2. If both fronts are equal, it's the same item in both sets, so write it down once and move past it in both sketches
3. Stop once we've written k values

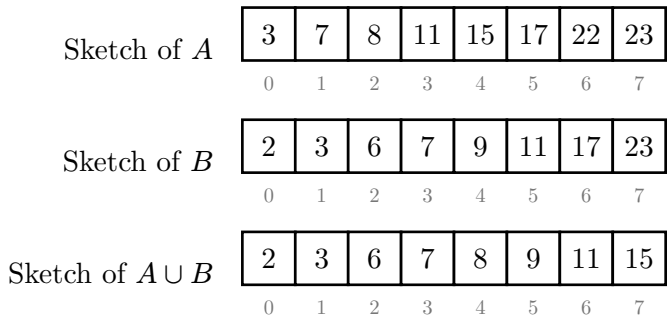


Figure 33.5: Example: merging two $k = 8$ sketches. Taking the smallest remaining front value each time gives 2, 3, 6, 7, 8, 9, 11, 15, the 8 smallest hash values in $A \cup B$

If we kept merging past k, we'd get 17, 22, and 23 too, but we only need 8. The result is **exactly** the sketch we would have gotten by streaming through all of $A \cup B$: each of the k smallest values of the union is in A or in B, so by the same argument as before, it's among the k smallest of whichever set it came from. So from just the two sketches, we get a full-accuracy sketch of the union

Now we can estimate the size of the union exactly like we did for any other set. Recall from the previous chapter that the k-th smallest hash value M_k (as a fraction of the hash range) has expected value $\frac{k}{n+1}$. The 8th smallest value is 15 out of 100, so

$$\frac{15}{100} = \frac{8}{n+1} \quad \Rightarrow \quad n = \frac{800}{15} - 1 \approx 52.3$$

So our estimate is that there are about 52.3 unique items in $A \cup B$

33.5 The Intersection

We'd like to do the same thing for the intersection, so let's walk through both sketches and only write down the values that appear in both. 2 is in B but not A, 3 is in both, 6 is only in one, 7 is in both, and so on, which leaves the five values in **Figure 33.6**

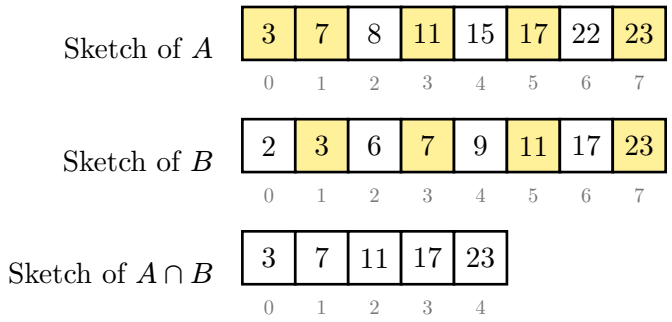


Figure 33.6: Example: the values in both sketches (highlighted) from **Figure 33.5**. Only 5 of them survive, so the intersection gets a smaller $k = 5$ sketch

And that's all we can compute. Those five values really are the five smallest hashes in $A \cap B$ (any shared item with a hash of 23 or less is in both sketches), but there's no sixth: a shared item with a bigger hash never made it into either sketch, and we have no more data. To get an equally accurate sketch of the intersection, we'd need bigger sketches. The only way the intersection keeps all k values is if the two sketches are identical. So we have a couple of options

33.5.1 Option 1: Use the Smaller Sketch

We could just say, this is all we've got, and treat the intersection as a $k = 5$ sketch. Its 5th smallest value is 23, so

$$\frac{23}{100} = \frac{5}{n+1} \quad \Rightarrow \quad n = \frac{500}{23} - 1 \approx 20.7$$

which gives $J \approx \frac{20.7}{52.3} \approx 0.40$. That works, but the intersection estimate is less accurate than the union estimate, since it's built from fewer values

33.5.2 Option 2: Go Through the Union

If we know the size of A, the size of B, and the size of $A \cup B$, we can compute the size of the intersection directly. Adding $|A|$ and $|B|$ counts every shared item twice, and counts everything else once, so

$$|A \cap B| = |A| + |B| - |A \cup B|$$

In pair 1 of **Figure 33.1**, that's $5 + 5 - 7 = 3$. The nice thing is that all three sizes on the right come from full $k = 8$ sketches. Both A's and B's 8th smallest values are 23, so

$$|A| \approx |B| \approx \frac{800}{23} - 1 \approx 33.8$$

$$|A \cap B| \approx 33.8 + 33.8 - 52.3 = 15.2$$

$$J(A, B) \approx \frac{15.2}{52.3} \approx 0.29$$

So from a k-min sketch of each set, we can compute two potentially valuable things: the **cardinality** of each set, and the **Jaccard similarity** between any two of them. And if k is around 100, working with just k values per set takes almost no time compared to comparing the full sets

Notice the three estimates we've seen for this example (0.40, 0.29, and the $\frac{3}{8} = 0.375$ we'd get from the shared fraction of the union's sketch, as in **Figure 33.3**) don't agree. That's not a mistake: $k = 8$ is a tiny sketch, and every one of these is only an estimate

33.6 How Accurate Is It?

Just like with cardinality, the answer depends on k. The statistical analysis is well outside this course, so let's run an experiment instead. Take two sets of 1000 items that share 500, so $J = \frac{500}{1500} = \frac{1}{3}$, and estimate J from their sketches 500 times, with a different hash function each time

k	Average error: shared fraction	Average error: through the union
10	0.12	0.20
50	0.05	0.10
100	0.035	0.066
500	0.013	0.029

Both estimates get better as k grows, and just like with cardinality, the improvement slows down: going from 10 to 100 helps a lot more than going from 100 to 500. Going through the union has a weakness worth knowing about: it **subtracts** estimates, and subtracting doesn't cancel their errors, it adds them. With $k = 10$, it estimated a **negative** intersection in 35 of the 500 trials

33.7 MinHash in Practice

Here's a real use of this. Researchers estimated the size and similarity of all the reference microbial genomes, where each genome is treated as the set of its **k-mers** (short, fixed-length pieces of its DNA sequence). There are tens of thousands of genomes, and comparing every pair of them directly would be an enormous amount of work. But once someone generated the sketches, estimating the similarity of every pair, which is really the valuable part (which organisms are related to which), could be done on a desktop

Storing a small sketch for each of tens of thousands of genomes is easy, while working with hundreds of thousands of k-mers for each of them starts to get really hard. Even as computers keep getting faster, this kind of estimation is an important tool for doing real science

33.8 Sets of Very Different Sizes

Can we always use this technique? Let's look at a case where it doesn't go very well: one set is very small, and the other is very large. In **Figure 33.7**, the small set S and the large set L actually share 59, 82, and 101 (hash values out of 1000 now), three of S's four smallest values

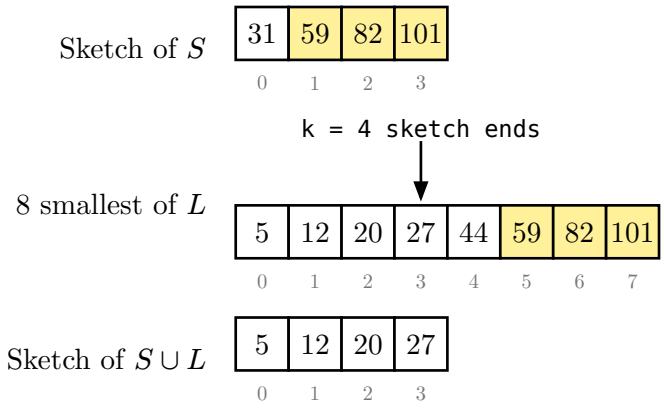


Figure 33.7: Example: a small set S and a large set L with $k = 4$. The shared values (highlighted) are only the 6th through 8th smallest in L, so L's sketch never sees them, and the union's sketch is entirely L's values

With $k = 4$, L's sketch stops at 27, so we never see that the sets share anything. The union's sketch is made entirely of L's values, and neither estimate notices the overlap. For S's first four values to be recognized, we'd have needed L's first **eight**. When the sets are similar in size, the data we lose past the end of the sketch is fairly small, but when one set is much bigger, the lost data becomes significant

Our estimates assume we see everything that overlaps, but here we don't, so the union looks different than it really is: it looks like less is shared. So even though S shares almost everything with L (while L has lots of other items), our estimate of how much they share is going to be off. To see how bad this gets, take $|S| = 100$ and $|L| = 10000$, with 90 of S's items inside L, so $J \approx 0.009$. With $k = 100$, the shared-fraction estimate came out as exactly 0 in 118 of 300 trials, and was off by 87% of the true value on average

So this technique doesn't work well when the sizes of the two sets are very far apart. The error isn't bounded by some nice statistical guarantee: as the cardinalities of the two sets drift farther apart, the error in the similarity gets larger. For some problems, that's fine. Whole families of genomes are known to be about the same size, and the genome comparisons from the previous section actually group genomes by size and relation for exactly this reason

33.9 k-Partitions

We won't completely defeat this problem, but we can do something easy to help. Instead of keeping the k smallest values of the whole hash range, we **partition** the range into chunks and keep the smallest values in **each** chunk. With hash values between 0 and 1, three partitions would be $[0, 0.3)$, $[0.3, 0.6)$, and $[0.6, 1]$ (non-overlapping, so nothing gets counted twice). Now our samples are spread evenly across the whole hash range, instead of all coming from the very bottom of it

In practice, a hash value is just an integer stored in binary. We've been treating it as a number between 0 and 1, but that's only for the analysis, and working with integers is faster. So we pick partitions using the **most significant bits**: with two bits, the partitions are the values starting

with `00`, `01`, `10`, and `11`. **Figure 33.8** keeps the $k = 2$ smallest values in each partition of a set of 6-bit hash values

Top bits	Hash values in binary	Partition
00	000101, 001001, 001110	5 (0), 9 (1), 14 (2)
01	011000	24 (0)
10	100011, 101010, 101101	35 (0), 42 (1), 45 (2)
11	111001	57 (0)

Figure 33.8: Example: partitioning 6-bit hash values by their two most significant bits and keeping the $k = 2$ smallest in each partition (highlighted). Partitions `01` and `11` only have one value, so they keep just that one

We keep two values in partition `00`, one in `01`, two in `10`, and one in `11`, so the sketch is an even subsample of the whole set. To compare two sets, we compare their sketches one partition at a time, exactly like before, and then combine the results

What if a partition is empty for one of the sets? That's okay: it just says the set has nothing in that range. An empty set is a perfectly valid set, so that partition has zero similarity, and it counts for $\frac{1}{4}$ (or $\frac{1}{8}$, $\frac{1}{16}$, and so on, depending on how many partitions we use) of the whole comparison. Since we assume SUHA, this shouldn't happen unless the partitions are very small compared to the number of items in the set

Partitioning softens the size problem, but it doesn't remove it. In the experiment from the previous section, 50 partitions of 2 values each did only a little better than a single $k = 100$ sketch (off by 78% on average instead of 87%), so the safest approach is still to compare sets of similar sizes

And all of this depends on having magic SUHA hash functions. That's the biggest fundamental flaw in this whole family of techniques: every estimate in this chapter is only as good as our hash function

34 Wrapping up CS 225!

This last chapter is a tour back through the whole course: the C++ ideas we leaned on, and every data structure and algorithm we built, with how they compare. It's a good place to start reviewing for the final

34.1 C++ as a Vehicle

This course is **in** C++, but it isn't **about** C++. We chose C++ because it puts two ideas front and center that matter for data structures in any language

- **Templates** separate the **kind of data** from the **data structure** that stores it, which is exactly how we want to think about data structures. Almost every MP abstracted the stored type with templates
- **Memory management** is fundamental to how a data structure performs, and C++ makes us handle it ourselves. Almost every assignment had tests checking that memory was handled correctly

34.1.1 Iterators

Iterators are how we access a data structure without knowing how it's built. An iterator needs

- `operator++` (pre-increment) to move to the next element
- `operator*` to access the current element
- `operator!=` to compare against another iterator

and the data structure provides `begin()` and `end()`. The thing that catches people the most is that **`end()` is one past the last element**, not the last element itself (like in **Figure 7.1**). It's unintuitive until you get used to it, but it matches how C-style loops were always written

34.1.2 Functions as Data

A **lambda** defines a function inline, usually stored in a variable of type `auto`. It has three parts: the **capture** (variables from the surrounding code it can use), the **parameters**, and the **body**. A common use is a custom comparator, like turning `std::priority_queue` (which is a **max**-heap by default) into a min-heap

```
auto greater = [](int a, int b) { return a > b; };
std::priority_queue<int, std::vector<int>, decltype(greater)> pq(greater);

int threshold = 5;
auto isBig = [threshold](int x) { return x > threshold; }; // captures threshold
```

A **function pointer** is just the address of a function's code in memory. A function's name **is** that address, which makes sense: to call a function, the compiler inserts a call to that location. The syntax for declaring a function pointer type is weird, but with `auto` it's easy to use one

```
int max(int x, int y) {
    return x > y ? x : y;
}

auto myMax = max;   // myMax points to max
int result = myMax(4, 7); // calls max, so result is 7
```

That's how a hash table can store its hash function, for example: as a function pointer it calls whenever it needs a hash

34.2 Lists

Lists come in two flavors, **linked** and **array**, each sorted or unsorted

Operation	**Linked list**	**Array list**
Access by index	$O(n)$	$O(1)$
Insert or remove given a pointer to the spot	$O(1)$	$O(n)$
Insert or remove at an index	$O(n)$ to get there, $O(1)$ to change	$O(1)$ to get there, $O(n)$ to shift
Search for a value	$O(n)$	$O(n)$, or $O(\log n)$ with binary search if sorted

An array is just a mathematical offset from its start, so indexing is $O(1)$, but keeping it contiguous makes inserting and removing $O(n)$. A linked list is the opposite. Each one plays to the other's weakness: either it takes time to **find** the spot, or it takes time to **do** the change. Whenever we need an array to grow, we **double** it, for amortized $O(1)$ inserts (see **Figure 5.2**)

34.2.1 Stacks and Queues

- **Stack** - Last in, first out (LIFO). Every operation is $O(1)$ with a linked list, and amortized $O(1)$ with an array
- **Queue** - First in, first out (FIFO). Also $O(1)$ (amortized for an array). With an array, the natural implementation is a **circular queue** (like in **Figure 6.5**)

A circular buffer between two pieces of a program that run at different or uneven speeds is one of the most common data structures in real code. It can even be built in hardware for incredibly fast performance

34.3 Trees

We spent almost half of the course on trees, and we've still only scratched the surface. Some things to remember

- An **empty** tree is a perfectly good tree, with height -1. Most theory courses skip it, but data structures start out empty
- Our trees are almost always **rooted** (we need a place to attach the data structure) and **binary** (it's good enough, and much less complicated)

Traversals visit everything in the tree. **Pre-order**, **in-order**, and **post-order** are all recursive, and only differ in whether we visit a node the first, middle, or last time we reach it. They're all like depth-first search. **Level order** visits everything at distance 0 from the root, then 1, then 2, which is breadth-first search. Written the general way, the only difference between the two is a **stack** versus a **queue**

34.3.1 Trees as Dictionaries

Tree	`find` / `insert` / `remove`	Notes
BST	$O(h)$, which is $O(n)$ in the worst case	An in-order traversal gives the keys in sorted order
AVL tree (BBST)	$O(\log n)$	Rotations keep it balanced
B-tree of order m	$O(\log_m n)$ seeks	Each node holds up to $m-1$ keys

A plain BST is only as good as its height, and a tree with only right children is perfectly legal. An AVL tree uses rotations to keep itself balanced, which is like pretending the keys were inserted in a better order. A B-tree makes the base of the log bigger, so the tree is very short (**Figure 17.1**), which matters when following a pointer is slow, like on a disk or over a network (a disk is roughly a million times slower than cache, and a network can be tens to hundreds of millions of instructions)

That idea goes way beyond B-trees: lots of performance models say "if it's smaller than X, it's basically free, and if it's bigger, it's slow". If we can reduce how often we go outside of that small, fast space, we make a huge impact

34.3.2 Special Trees

- **KD trees** - Every balanced BST can do approximate searches: if a key isn't in the tree, the nearest key is its in-order predecessor or successor, and we can find every key in a range efficiently. A KD tree does this in k dimensions by splitting the data evenly in a different dimension at every level (**Figure 16.4**)
- **Huffman trees** - If we know how often each character appears (either predicted, like for English, or counted from the data), we can build a binary tree with the **least** frequent characters deepest. Each character's code is its path from the root (left is 0, right is 1), which is the most efficient way to encode the data, so it's used for **compression**. In **Figure 34.1**, the frequent `A` gets a 1-bit code, so the 12 characters take 21 bits instead of 24 with fixed 2-bit codes
- **Heaps** - A complete tree, usually stored as an array (**Figure 19.3**), that implements a **priority queue**. `insert` adds at the bottom and heapifies up, and `removeMin` swaps the last

value to the root and heapifies down. Theoreticians all say a min-heap is the default, but `std::priority_queue` is a max-heap by default

- **Disjoint sets** - A forest of uptrees supporting add, `union`, and `find`. With smart union and path compression (**Figure 21.9**), operations are amortized $O(\alpha(m, n))$: the inverse Ackermann function, which is single digits for anything that could fit in the universe, so effectively constant

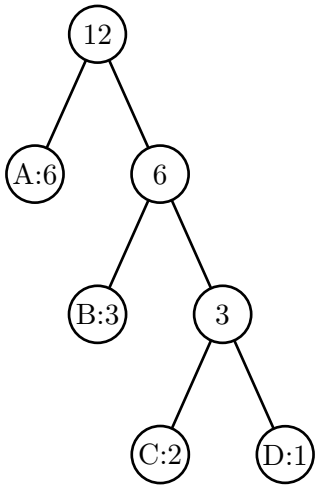


Figure 34.1: Example: a Huffman tree for `A` (6 times), `B` (3), `C` (2), and `D` (1). The codes are `A` = 0, `B` = 10, `C` = 110, `D` = 111

34.4 Graphs

There are three ways to represent a graph

- **Edge list** - Super simple, with no overhead for saving or loading, so it's the standard file format, but slow for almost everything else
- **Adjacency matrix** - Easy to write, with $O(1)$ `areAdjacent`, but it uses $O(n^2)$ space and usually isn't the best choice. It shows up a lot because it takes no time to think about
- **Adjacency list** - Pointers everywhere, so it's harder to code, but it's essentially optimal for everything except `areAdjacent`, and it matches how almost every real graph algorithm works

34.4.1 Traversals

BFS and DFS on a graph work like they do on a tree, except we **don't visit a vertex twice**. Watch out for one optimization: in BFS, the first time a vertex goes into the queue is when it will be visited, so we can mark it as visited as soon as we **see** it. That works great for BFS, but if you switch the queue to a stack, it breaks DFS, since in DFS a vertex we've seen might be reached by another path before we get to it

34.4.2 Graph Algorithms

- **Kruskal's algorithm** (minimum spanning tree) - Put every edge in a priority queue by weight and every vertex in its own disjoint set. Repeatedly take the cheapest edge, and if it connects two different sets, keep it and union them, until we have $n - 1$ edges

- **Prim's algorithm** (minimum spanning tree) - Start from one vertex and repeatedly add the cheapest edge from the tree to a vertex not yet in it, until every vertex is reached
- **Dijkstra's algorithm** (shortest paths from one vertex to all others) - Exactly like Prim's, except the cost of reaching a new vertex is the cost to get to the vertex we're coming from **plus** the edge. Since that's the only change, the analysis is the same as Prim's
- **Floyd-Warshall** (shortest paths between all pairs) - A dynamic programming algorithm: first the shortest paths using no intermediate vertices, then allowing one more vertex at a time. The code is just a triple nested loop, so it's $O(n^3)$

Kruskal's and Prim's runtimes look different on paper, but for connected graphs both come out to $O(m \log n)$ with a binary heap, and analyzing them on sparse versus dense graphs shows they're about the same. There's no clear winner, though Prim's leads directly into Dijkstra's

34.5 Hashing

Hashing is really cool, but it has requirements and downsides

- **A good hash function.** For our analysis, it has to satisfy **SUHA**, the simple uniform hashing assumption: the chance that any particular key hashes to any particular slot is exactly $\frac{1}{N}$, where N is the number of slots. The only real way to get that is to pick the hash function at random from a family of hash functions. That one fact is the trick behind all of our analysis. In practice, hash functions often aren't truly uniform, which is why hash tables usually perform great but sometimes perform terribly
- **Storage**, which is an array
- **A way to handle collisions.** The universe of keys is bigger than the number of slots, so collisions will happen
  - **Separate chaining** (open hashing) stores a list at every slot
  - **Probing** (closed hashing, or open addressing) stores everything inside the table, and on a collision goes somewhere farther down: the next empty slot (**linear probing**), or a step size from a second hash function (**double hashing**). Closed hashing also needs to tell whether a slot is empty **now** versus whether it was **ever** used, since a `find` has to keep probing past slots that used to be full

Altogether, a hash table gets **expected amortized $O(1)$** `insert` and `find` with $O(n)$ space for n items. With separate chaining, `insert` is actually $O(1)$ (just add to the front of the list), but `find` and `remove` can be $O(n)$ in the worst case

Dictionary	**`find`**	**`insert`**
Hash table	expected $O(1)$	expected amortized $O(1)$
AVL tree	$O(\log n)$	$O(\log n)$
Sorted array	$O(\log n)$	$O(n)$
Unsorted list	$O(n)$	$O(1)$

The big takeaway: hash tables are really useful, but for a **specific** problem, there's usually a better solution, because we usually know something about the data that lets us organize it more

efficiently. Think of a hash table as "I don't have a good solution yet, so let's put this here for now". If nothing better exists, hash tables are pretty good, but sometimes they'll slow down (when rehashing, or when the hash function turns out to be bad)

34.6 Bloom Filters

A bloom filter is a hash table that **doesn't store the data**: just a bit vector of m bits, with one hash function or k of them. Inserting sets the hashed bits to 1, and a lookup checks whether all of them are 1. Both take $O(1)$ time, every time, not just in expectation

Of the four possible outcomes, only three can happen

	Filter says yes	**Filter says no**
Was inserted	True positive	Can't happen
Wasn't inserted	False positive	True negative

So a bloom filter is very small and very fast, as long as we're okay with **sometimes** saying yes when the answer is no. "Is this in the cache?" is a great use: a false positive just means a cache miss, which isn't a big deal. "Is this safe to touch?" would be a terrible one

The false positive rate goes up as we insert more items, and there's a best number of hash functions k for a given size: too few and each item sets too few bits, too many and the bit vector fills up. Finding that sweet spot takes work and isn't always practical. Bloom filters also can't **delete** (clearing a bit might erase another item) and can't **resize** (we don't have the original items to rehash)

34.7 Sketching: Cardinality and Similarity

Cardinality asks how many **unique** items there are. Counting all the items is easy, but telling whether an item is unique means checking whether we've seen it before, which means storing everything. With data sets of 60 billion to 130 trillion items, that's impossible

Instead, we hash every item to a number between 0 and 1 (divide the hash by the largest possible hash value) and only keep the **minimum**. If n unique items hash uniformly, the minimum is expected to be about $\frac{1}{n+1}$, so if the smallest value we saw was 0.095, we'd estimate $n \approx \frac{1}{0.095} - 1 \approx 9.5$. Keeping the k smallest values instead of just one makes the estimate much more reliable: the k-th smallest is about $\frac{k}{n+1}$

The same k smallest values let us estimate **similarity**: comparing two sets through their unions and intersections (Jaccard similarity) using only the k items we stored. We didn't analyze the accuracy, since that probability is beyond this course (and leans even harder on SUHA), but with just k values per set we get reasonably accurate predictions

34.8 The Final Exam

- **Coding (about half the points).** Expect around three coding questions

  - One will be **recursion on trees**: build a tree with BST inserts or some kind of splitting, compute heights or count leaves, mirror a tree, and so on. These follow the usual recursive pattern; they won't need a clever or non-obvious algorithm, just comfort with recursion
  - The others will be straightforward C++ on data structures we've built, like a single AVL rotation in an otherwise complete AVL tree, some heap operations, some hash table operations, or a bloom filter given its hash functions, possibly followed by using it (like counting words with your hash table). **Not** linked lists, and **not** graphs
- **Multiple choice and short answer.** Everyone will get questions on **sketching** (bloom filters, cardinality, and Jaccard similarity). Beyond that, questions are pulled from pools covering the whole semester, so not everyone sees every topic, but everyone sees a selection of all of them
- **Studying.** The practice exams are the best resource, since the questions are pulled from (or are like) them. The C++ documentation is linked from the exam itself

Appendix A: Midterm Reviews

A.1 Midterm 0: Review of Prerequisite Material

Midterm 0 is CS 225′s formal introduction to UIUC's Computer Based Testing Facility (CBTF)

Covers the info discussed in course prerequisites, including Graph Theory from **CS 173/MATH 213** and C++ principles from **CS 128/ECE 220**

A.1.1 Topics Covered

- Course Policies
- Mathematical Foundations
  - Be able to give a closed form solution to a recurrence relation
  - Be able to indicate the big-O running time of common algorithms (eg: binary search, merge sort, etc)
  - Be able to understand basic graph terminology
  - Be familiar with Trees
  - Be able to understand a standard proof and proof by induction
- C++ Topics
  - Classes in C++
    - Public member functions
    - Private helper functions
    - Private variables
    - Constructors
      - Automatic default constructor
      - Custom constructors (default and non-default)
    - Copy constructor
      - Automatic copy constructor
      - Custom copy constructor
    - Inheritance
  - Namespaces in C++
    - Creating a class that is part of a namespace (eg: `Cube` is part of the `cs225` namespace)
    - Using a class from a namespace (eg: `cs225::Cube`)
    - Purpose and usefulness of namespaces
  - Variables
    - Four properties: name, type, location (in memory), and value
    - Primitive vs. user-defined
  - Memory
    - Indirection in C++:
      - Reference variables
      - Pointers

    - Differences and trade-offs between each type
  - Stack memory
  - Heap memory
- Functions: Calling and Returning
  - Pass by value, by reference, and by pointer
  - Return by value, by reference, and by pointer

A.1.2 Recursive Definitions

- **Recursive definitions** contain formulas where each term is based on one or more preceding terms
  - Many recursive numerical formulas have something called a **closed form**, which is a closed expression that doesn't involve computing each term in the formula/sequence (similar to a summation)
    - Deriving this closed form is an act known as **unrolling**
      - When unrolling, we substitute a recursive definition into itself until there is an explicit pattern suggesting the closed form expression

For example with the expression,

$$\begin{aligned} T(n) &= 2T(n-1) + 3 \\ &= 2(2T(n-2) + 3) + 3 \\ &= 2(2(2T(n-3) + 3) + 3) + 3 \\ &= 2^3 T(n-3) + 2^2 \cdot 3 + 2 \cdot 3 + 3 \\ &= 2^4 T(n-4) + 2^3 \cdot 3 + 2^2 \cdot 3 + 2 \cdot 3 + 3 \\ &\vdots \\ &= 2^k T(n-k) + 2^{k-1} \cdot 3 + ... + 2^2 \cdot 3 + 2 \cdot 3 + 3 \end{aligned}$$

we can use summation notation to represent the kth unrolling step, where

$$\begin{aligned} T(n) &= 2^k T(n-k) + 2^{k-1} \cdot 3 + ... + 2^2 \cdot 3 + 2 \cdot 3 + 3 \\ &= 2^k T(n-k) + 3\big(2^{k-1} + ... + 2^2 + 2 + 1\big) \\ &= 2^k T(n-k) + 3 \sum_{i=0}^{k-1} (2^i) \end{aligned}$$

The input of function T hits the base case when $n - k = 1$, or when $k = n - 1$. Recall that $T(1) = 1$ and substituting $n - 1$ value for k gives

$$
\begin{aligned}
T(n) &= 2^k T(n-k) + 3\sum_{i=0}^{k-1}(2^i) \\
&= 2^{n-1}T(1) + 3\sum_{i=0}^{n-2}(2^i) \\
&= 2^{n-1} + 3\sum_{k=0}^{n-2}(2^k) \\
&= 2^{n-1} + 3(2^{n-1} - 1) \\
&= 4(2^{n-1}) - 3 \\
&= 2^{n+1} - 3
\end{aligned}
$$

therefore making the closed form for function T, $2^{n+1} - 3$

1. Given that $T(0) = 5$, find the closed form solution of

$$T(n) = T(n-1) + 2$$

   a. Unroll the first couple of terms

$$
\begin{aligned}
T(n) &= T(n-1) + 2 \\
&= (T(n-2) + 2) + 2 \\
&= ((T(n-3) + 2) + 2) + 2 \\
&\vdots \\
&= T(n-k) + 2k
\end{aligned}
$$

   b. where $k = n$ and $T(n) = T(0) + 2n = 5 + 2n$
   c. Recall that $T(0)$ can represent any sort of numeric value, where in this case is 5. This numeric value of 5 is known as the **base case** and $2n$ is known as the **recursive case**

2. Given that $T(1) = 3$, find the closed form solution of

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

   a. Unroll the first couple terms of the sequence

$$
\begin{aligned}
T(n) &= 2T\left(\frac{n}{2}\right) + n \\
&= 2\left(2T\left(\frac{n}{4}\right) + \frac{n}{2}\right) + n = 4T\left(\frac{n}{4}\right) + 2n \\
&= 2\left(2\left(2T\left(\frac{n}{8}\right) + \frac{n}{4}\right) + \frac{n}{2}\right) + n = 8T\left(\frac{n}{8}\right) + 3n \\
&\vdots \\
&= 2^k T\left(\frac{n}{2^k}\right) + kn
\end{aligned}
$$

   b. where $\frac{n}{2^k} = 1 \rightarrow n = 2^k \rightarrow k = \log_2 n$ and

$$
\begin{aligned}
T(n) &= 2^{\log_2 n} \cdot T(1) + n \cdot \log_2 n \\
&= n \cdot T(1) + n \log_2 n \\
&= 3n + n \log_2 n
\end{aligned}
$$

A.1.3 Proofs by Induction

Induction is a technique used to prove recursive definitions/formulas, indicating that it does match its respective closed form solution

- Includes a **base case**, **inductive hypothesis**, and **inductive step**
  - **Base Case** - The starting point, where you prove that it holds true for the smallest value, typically $n = 0$ or $n = 1$
  - **Inductive Hypothesis** - An assumption that the statement being proven is true for some value k
  - **Inductive Step** - Prove the inductive hypothesis with the next step $k + 1$

1. Prove that $1 + 2 + 3 + ... + n = \frac{n(n+1)}{2}$ for $n \geq 1$
   a. Let $n = 1$, then $\frac{1(1+1)}{2} = 1$, where $1 = 1$ (both sides are equal)
   b. Assume the formula holds true for $k \geq 1$, where $1 + 2 + 3 + ... + k = \frac{k(k+1)}{2}$
   c. Prove the inductive hypothesis from b. for $k + 1$, the next recurring step

$$
\begin{aligned}
1 + 2 + ... + k + (k + 1) &= \frac{(k + 1)(k + 2)}{2} \\
(1 + 2 + ... + k) + (k + 1) &= \frac{k(k + 1)}{2} + (k + 1) \\
\frac{k(k + 1) + 2(k + 1)}{2} &= \frac{(k + 2)(k + 1)}{2}
\end{aligned}
$$

   where the inductive step does indeed hold

2. Prove $2^n \geq n + 1$
   a. Check if true for $n = 0$

$$
\begin{aligned}
2^0 &\geq 0 + 1 \\
1 &\geq 1
\end{aligned}
$$

   The base case does hold
   b. Assume the statement holds for $n = k$, where

$$
2^k \geq k + 1
$$

   c. Using this assumption, prove it holds for $n = k + 1$

$$
\begin{aligned}
2^{k+1} &\geq (k + 1) + 1 \\
2^{k+1} &\geq k + 2 \\
2^k \cdot 2 &\geq 2(k + 1) \geq k + 2
\end{aligned}
$$

A.1.4 Big-O

Big-O describes how fast a function grows as its input size, n, increases. It observes the asymptotic behavior as n gets large.

- measures the efficiency of algorithms
- compares runtime/space usage based on large inputs
- describes the worst case runtime of an algorithm
  - or the **upper bound** of an algorithm, which indicates how bad the algorithm's runtime can get

Big-O Notation	Name	Typical Algorithmic Behavior
$O(1)$	Constant Time	Accessing a single array element
$O(\log n)$	Logarithmic Time	Binary Search; algorithm halves the input per iteration
$O(n)$	Linear Time	Linear Search; looping over an array once
$O(n \log n)$	Log-Linear Time	Merge Sort, Quick Sort, Heap Sort; Divide and Conquer, where problems are broken into subproblems that are solved individually
$O(n^2)$	Quadratic Time	Bubble Sort, Selection Sort, Insertion Sort; Anything involving nested for loops
$O(2^n)$	Exponential Time	Typically many recursive algorithms, including brute force (attempts all possible solutions to a problem until one satisfies the problem's conditions)
$O(n!)$	Factorial Time	Permutation-Based Solutions

A.1.4.1 Big-O of Common Algorithms

Algorithm	Big-O Notation	Description
Linear Search	$O(n)$	Checks each element one by one from beginning to end looking for the given element
Binary Search	$O(\log n)$	Takes in a sorted array and iteratively halves the array that may contain the given value
Bubble Sort	$O(n^2)$	Repeatedly steps through an array and compares adjacent elements, swapping them if they are in the wrong order
Insertion Sort	$O(n^2)$	Iteratively inserts each element of an unsorted list into its correct position in a sorted portion of the list
Selection Sort	$O(n^2)$	Sorts an array by repeatedly selecting the **smallest** element from the

Algorithm	Big-O Notation	Description
		unsorted portion and swapping it with the first unsorted element
Merge Sort	$O(n \log n)$	Recursively divides an array/list into smaller sublists, where each sublist is sorted and then merged back together into one big list
Quick Sort	$O(n^2)$ with average case of $O(n \log n)$	Selects a pivot element from the array and partitions the other elements into two sub-arrays, based on whether they are less than or greater than the pivot
Heap Sort	$O(n \log n)$	Organizes elements in an array using a binary heap, by repeatedly swapping the root node (whether it's the maximum or minimum element for max/min heap respectively) with the last element of the heap

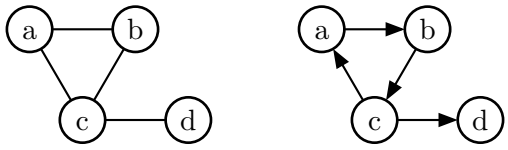
A.1.5 Graphs

Graphs consist of a set of nodes (points/vertices), V, and a set of edges, E, where the graph is commonly referred to as a pair of sets (V, E).

Each edge in E joins two nodes in V, where two nodes connected by an edge are **adjacent** or **neighbors**

Edges can have direction, where graphs with these edges are known as **directed graphs**. This is useful when discussing relations between vertices with contexts to their edges. In contrast, edges that can be traversed in both directions are known as **undirected graphs**

Figure A.1.1 draws the same four nodes both ways: on the left the edges have no direction, and on the right each edge has an arrow



Undirected Directed

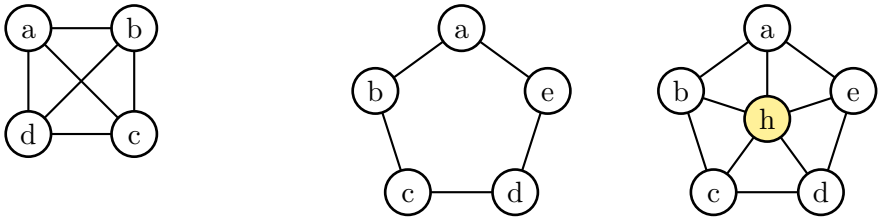
Figure A.1.1: Example: $V = \{a, b, c, d\}$ with four edges; `a` and `b` are adjacent, `a` and `d` are not

A.1.5.1 Additional Graph terminology

- **Loops** - a node where an edge connects to itself
- **Multiple Edges** - two nodes are connected by multiple edges

- **Complete Graphs** - Graphs where each node is connected to every other node. They have $\frac{n(n-1)}{2}$ edges
- **Walks** - A sequence of nodes and edges, where each edge connects the node before and after it and nodes and edges can repeat
- **Paths** - Walks that **cannot repeat nodes and edges**
- **Cycles** - A path that **starts** and **ends** at the same node
  - **Cyclic Graphs** consist of graphs where nodes are connected in a cycle (non cyclic graphs are called **acyclic**)
- **Wheels** - Cycle Graphs where there exists a **single central node** that connects to every other node in the cycle graph
- **Subgraphs** - A smaller graph made from a subset of vertices and edges of a bigger graph

You can see a complete graph, a cycle, and a wheel side by side in **Figure A.1.2**. Notice how the wheel is just the cycle plus a center `h` connected to every node



Complete ($4 \cdot 3/2 = 6$ edges) Cycle Wheel

Figure A.1.2: Example: a complete graph, a cycle graph, and a wheel with center `h`

A graph is **simple** if it does not contain **loops** or **multiple edges**

A.1.5.2 Degree

The degree of a node v, notated as $\deg(v)$, is the number of edges that are connected to v. Loops count **twice** towards the degree of a node.

Each edge contributes to two degrees of a node. Therefore, the **sum of the degrees of all nodes in a graph** is **twice** the number of edges, which can be mathematically expressed as

$$\sum_{v \in V} \deg(v) = 2|E|$$

or

$$\sum_{k=1}^{n} \deg(v_k) = 2|E|, \quad v_k \in V$$

This is known as the **Handshaking Theorem**. Let's check it on the graph in **Figure A.1.3**: the degrees add up to 10, which is twice its 5 edges

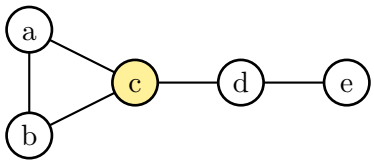


Figure A.1.3: Example: $\deg(c) = 3$, and the degrees $2 + 2 + 3 + 2 + 1 = 10 = 2 \cdot 5$ match the 5 edges

A.1.5.3 Isomorphism

Isomorphism exists when two graphs have the same structure, albeit the drawings may be different

Two graphs, G_1 and G_2 are isomorphic when there exists a bijection of

$$f : V_1 \to V_2$$

Or simply,

- The two graphs must have the same number of nodes and the same number of edges
- For any node degree k, the two graphs must have the same number of nodes of degree k. For example, they must have the same number of nodes with degree 3
- Given a subgraph of G_1, that subgraph must also exist in G_2

In **Figure A.1.4**, G_1 and G_2 are drawn differently but are isomorphic

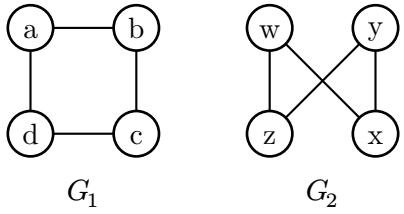


Figure A.1.4: Example: isomorphic graphs; $f(a) = w$, $f(b) = x$, $f(c) = y$, $f(d) = z$ maps every edge to an edge

A.1.5.4 Connectivity

A graph is **connected** when there exists a walk between every pair of nodes in G, or simply you can get from **any node to any other node**, as in the left graph of **Figure A.1.5**

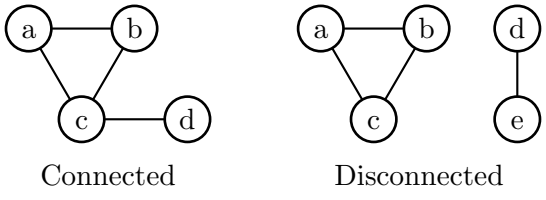


Figure A.1.5: Example: on the right there is no walk from `a` to `d`

Sometimes removing an edge of a graph will completely lose its connectivity, where such edge is known as a **cut edge**. If removed, the graph would be disconnected. Try removing the edge between `c` and `d` in **Figure A.1.6**, and you're left with two separate pieces

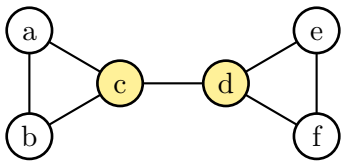


Figure A.1.6: Example: `c-d` is a cut edge; removing it leaves $\{a, b, c\}$ and $\{d, e, f\}$ disconnected

An **Euler Circuit** exists when there is a closed walk (circuit) that uses every edge of the graph once. This can only occur when **all degrees of the nodes are even**. This is because in order to complete the circuit, you must leave each node that you enter. This means that the graph is **connected** and therefore exists a closed walk that uses every edge of the graph once. In **Figure A.1.7**, for example, every node has an even degree, so a circuit starting and ending at a covers each edge exactly once.

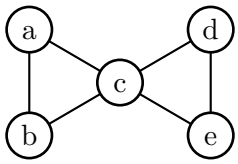


Figure A.1.7: Example: every degree is even ($\deg(c) = 4$, the rest are 2), so the circuit a, b, c, d, e, c, a uses each edge once

A.1.5.5 Bipartite Graph

A graph $G = (V, E)$ is **bipartite**, such that V can be split into two nonoverlapping subsets V_1 and V_2, where every edge connects to an element of V_1 in V_2.

What this simply means is that **a graph can only be bipartite when** $\chi(G) = 2$. This means that a graph can only be colored with **two colors**, like **Figure A.1.8**, where the shaded nodes a, b, c get one color and x, y, z get the other

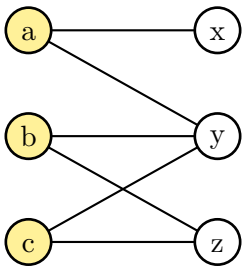


Figure A.1.8: Example: $V_1 = \{a, b, c\}$ (shaded) and $V_2 = \{x, y, z\}$; every edge crosses sides, so two colors suffice

A.1.6 Trees

Trees are undirected graphs with a special node called the **root**, where every node is connected to the root via one exact **path**

They are also **acyclic**, **connected**, and have n **vertices** with $n-1$ **edges**

A.1.6.1 Tree Vocabulary

- **Parent** - A node that is directly above another node in a tree
- **Child** - A node that is directly below another node in a tree
- **Sibling** - Nodes that **share the same parent**
- **Leaf** - A node that has no children
- **Internal** - A node that has at least one child
- **Level** - The number of edges from the **root** to the node (the root has a level of 0)
- **Height** - The longest path from a node to a leaf (in terms of edges)
- **Ancestor** - The ancestor of a node is one that lies on the path from the root to such node
- **Descendant** - The descendant of a node is one that lies on the path from such node to a leaf node

Try matching these terms to the small tree in **Figure A.1.9**. For example, `B` is an internal node at level 1, and `A` is an ancestor of every other node

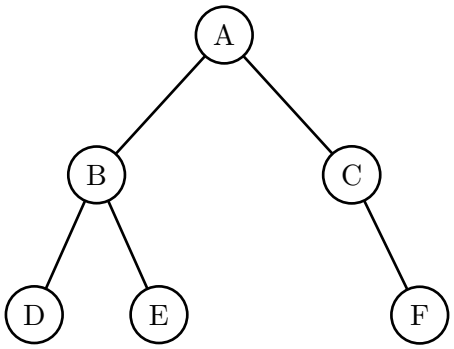


Figure A.1.9: `A` is the root and the parent of siblings `B` and `C`; `D`, `E`, and `F` are leaves; the tree has height 2

A.1.6.2 m-ary Trees

m-ary trees are trees that can have at most m children. A common type of tree is a **Binary Search Tree**, where each node has at most 2 children.

- **Full** trees exist when each node has either 0 or m children
- **Complete** trees exist when all levels are completely filled, except for the lowest one, which is filled from the left
- A tree is **full** and **complete** when the whole bottom level is fully populated with leaves
- **Balanced** trees exist when the heights of its left and right subtrees **differ by at most** 1

Going left to right in **Figure A.1.10**, the first tree is full, the second is complete, and the third is both. For balance, compare the two trees in **Figure A.1.11**: node `1` on the right has a left subtree that is two levels taller than its right

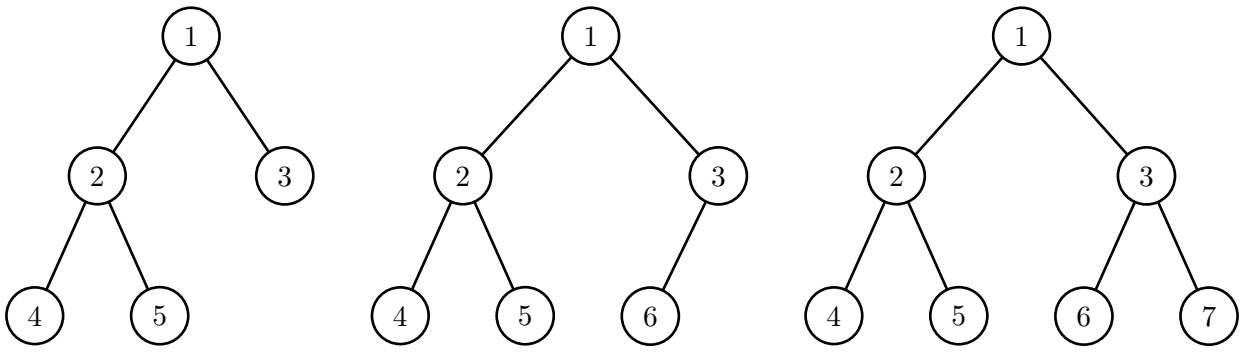
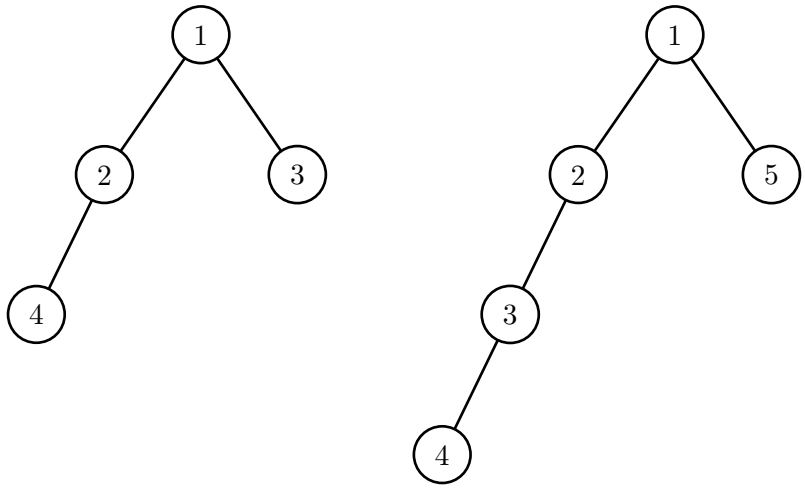


Figure A.1.10: Binary trees that are full, complete, and both full and complete (left to right)



Balanced Not balanced

Figure A.1.11: Example: on the right, node `1` has a left subtree of height 2 and a right subtree of height 0

There are $mi + 1$ nodes total in a m-ary tree. This means a binary tree has $2i + 1$ nodes total. You can count this yourself on the full 3-ary tree in **Figure A.1.12**, which has 2 internal nodes and $3 \cdot 2 + 1 = 7$ nodes

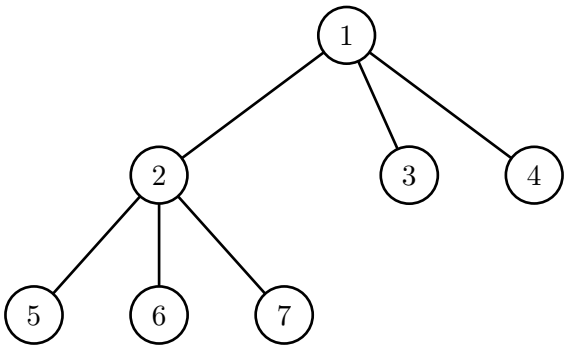


Figure A.1.12: Example: a full 3-ary tree with $i = 2$ internal nodes (`1` and `2`) has $3 \cdot 2 + 1 = 7$ nodes

A.1.6.3 Height and number of nodes

There is no exact formula to find the number of nodes and leaves in an m-ary tree. This is because some trees are more bushier (have more children) than others.

However, there are some patterns **(assuming that a given tree is binary)**

- The number of nodes are maximized when a tree, with height h, is full and complete is 2^h
  - where the number of nodes at level L are 2^L and the total nodes are $\sum_{L=0}^{h} 2^L$, with a closed form of $2^{h+1} - 1$

A.1.6.4 Public vs Private

Each member of a class in C++ has a specific **access level** that determines how that member can be accessed

A member can be one of three types: **public**, **private**, or **protected**

- `private` - Members declared as private can only be accessed from **within the same class**
- `protected` - Members declared as protected can only be accessed by derived classes (subclasses)
- `public` - Members declared as public can be accessed **anywhere** (inside the class, from other classes, or even outside the class)

Access Specifier	Access within Class	Access in Derived Class	Access Outside Class
`public`	Yes	Yes	Yes
`protected`	Yes	Yes	No
`private`	Yes	No	No

A.1.6.4.1 Classes vs Structs

> Members of a `struct` are `public` by default

> Members of a `class` are `private` by default

That is the simple difference between a `class` and a `struct`

The keywords `public protected` and `private` can explicitly define access levels of functions and variables for both a struct or class.

However, it is **best practice** for a struct to not use any access specifiers (all members will anyway default to `public`) and for a class to make member variables `private` (or `protected`) and methods `public`

A `struct` is typically used when
- There is a simple collection of data that doesn't benefit from restricting access
- Aggregate initialization is sufficient (no need for object)
- Structs should be aggregates by convention, where direct initialization of members are allowed through an initializer list `{ }` and there are no **constructors**, no **private/protected access members**, no **virtual functions**, and no **base classes**

Otherwise, just use classes

A.1.6.4.2 Member Functions

Member functions are functions that belong to a class type. This means that they can be called on an instance of that class type (e.g. `e.getData()`, where `getData()` is a member function of object type `e`)

- This means they are declared in the class definition!

> C# and Java refer to these as methods, since they are object oriented languages at the core. They aren't referred to as methods in C++, but the idea still remains the same

Functions that don't belong to a class are **non-member functions** or simply **free functions**.

With free functions, the object must be explicitly passed into the function, while with member functions, the object is implicitly passed and accessed through that object with the dot operator (`.`)

Member functions can sometimes create more clarity on what is doing the action!

A.1.6.5 Constructors

Constructors are special functions automatically called after a non-aggregate class type is created

- If an accessible matching constructor is found, then memory for the object is allocated and the constructor function is called. Otherwise, a compilation error is generated

They can also be **overloaded**, like functions

By convention, constructors **initialize member variables** (via a member initialization list) and perform **other setup functions** (error checking initialization values, opening a file or database, etc.)

Constructors must **have the same name as the class** and have **no return type** (not even `void`)

Class members are initialized in a constructor as follows via an initializer list, which initializes the member variable to the parameter variable

```
class Foo {
    int m_x;
    int m_y;
    public:
        Foo(int x, int y) : m_x{ x }, m_y{ y } { }
};
```

This is the proper way to initialize member variables.

The code below does the same thing, but its technically not initializing, but rather assigning. This means that `m_x` and `m_y` may go uninitialized

```
class Foo {
    int m_x;
    int m_y;

public:
    Foo(int x, int y) {
        m_x = x;
        m_y = y;
    }
};
```

A **default constructor** is one that accepts no arguments, meaning that it has been defined with no parameters. Suppose, class `Foo` exists where the only constructor that exists is the default constructor

```
class Foo {
public:
    Foo() { }
};
```

When creating an object, like `Foo f;` or `Foo f2{};` the **default constructor** is called, which doesn't do anything in this example

If no constructor is given, then the compiler will generate a public default constructor that does nothing, equivalent to the example above. This is known as an **implicit default constructor**

Instead of explicitly stating a default constructor, like `Foo() { }`, we can use the default keyword to have the compiler explicitly generate a default constructor for us. This is shown in the example below

```
class Foo {
public:
    Foo() = default;
};
```

With a user defined constructor, member variables are not automatically initialized (you need to do it yourself). With compiler provided constructors (either `= default` or implicitly provided), the members are also left uninitialized for `Foo f;`. The compiler only zero initializes them when the object is value initialized, like `Foo f{};`

Therefore, only create a default constructor when it makes sense for objects of a class type to be created using all default values

A.1.6.6 Namespaces

Namespaces group together related code (variables, functions, classes, etc.) under a name to avoid naming conflicts.

- The built in `cout` function for console output belongs to the standard `std` namespace, which is why we namespace it with `std::cout`
- This avoids naming conflicts in case we decided to make our own `cout` function (this actually was a thing back in old C++, where `cout` didn't belong to the `std` namespace)
- There is a way to get around this with `using namespace std`, but if we end up defining our own `cout` function, this causes a **namespacing collision** and our code won't compile!

A.1.7 References and Pointers

A **reference** is an alias of an existing object. It refers to the same object in memory with a different **name**. References are notated with an `&`

```
int x = 10;
int& xRef = x;
xRef = 20;

// After x will also become 20 because they refer to the same memory
```

After running the code above, `x` will also become 20 because `x` and `xRef` refer to the same address in memory, which is why **Figure A.1.13** has just one box with both names on it

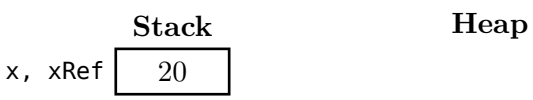


Figure A.1.13: `xRef` is not a new box: it is a second name for the memory of `x`

Note that this is called an lvalue reference in modern C++

A **pointer** is a variable that **stores the memory address** of another variable, thereby "pointing" to that variable's address in memory

- * is the syntax used after a type to declare a pointer type and it is used in conjunction with & (the address of operator) to get a variable's address in memory!
- * (dereferencing operator) is also used to dereference a pointer to access the value that it points to
- Pointers can be **null** (meaning they point to nothing!)
- Dereferencing these pointers can lead to undefined behavior (segmentation fault!)
- `nullptr` keyword can be used to check if a pointer is null! This also avoids dangling pointers!
- `NULL` and `0` can also reference null pointers. `NULL` comes from `C`, while `0` is an address that typically represents a null pointer. However `0` and `NULL` should be avoided in modern C++ in favor of `nullptr`

```
int x = 7;
int* xPtr = &x;
std::cout << *xPtr << std::endl;
```

The code snippet above still prints `7` to the console because it is dereferencing the pointer's value! Unlike the reference above, `xPtr` gets its own box in **Figure A.1.14**, and that box holds the address of `x`

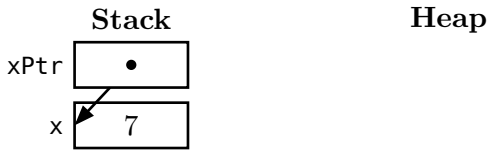


Figure A.1.14: `xPtr` is its own variable holding the address of `x`, so `*xPtr` reads 7

Since a pointer points to the memory address of another variable, it is important to initialize a proper memory address to it. Leaving a pointer **uninitialized** can cause problems!

A pointer is **very similar** to a reference. However, a **reference** is an alias that always refers to one fixed variable, while a **pointer** stores the **memory address** of any variable (or nothing/null). Try to use **references** over pointers because pointers can be dangerous

Dangling Pointers and References are those that happen when they refer to memory that's no longer valid (usually because the object was destroyed or went out of scope)

- **Dangling pointer:** A pointer points to memory that has been freed or deleted
- **Dangling reference:** A reference refers to an object that no longer exists (e.g., a local variable that went out of scope)

These can be **dangerous** because they can lead to **undefined behavior**, **crashes**, or **data corruption** (see **Figure A.1.15** for a pointer left holding freed memory)

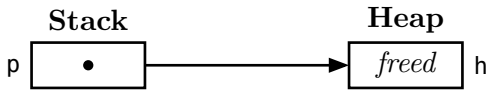


Figure A.1.15: Example: after `delete p;`, the dangling pointer `p` still holds the address of freed memory

A.1.8 Dynamic Memory Allocation

Dynamic Memory Allocation assigns memory space during a program's execution rather than compile time. This memory that the program uses is generally divided into the **stack** and the **heap**.

A.1.8.1 `new` and `delete`

C++ supports static and automatic memory allocation. **Automatic allocation** creates variables that live only while the function runs (on the stack), while **static allocation** creates variables that stay alive for the whole program (in static memory). The period in which a variable stays alive is known as its **lifetime**

Lifetime based on allocation type

- **Automatic objects**: lifetime ends when they go **out of scope** (e.g., when a function/block ends)
- **Static objects**: lifetime ends when the **program terminates**

With these two methods, the size of the variable/array must be known at compile time, and memory allocation and deallocation happen automatically (when a variable is instantiated/destroyed)

However, there may be situations where declaring a fixed size memory may lead to a waste of unused memory or there are more allocated objects of memory than what was assigned.

To fix this problem, memory can be manually managed dynamically, which is known as **dynamic memory allocation**. This is done using two keywords `new` and `delete`

`new` - **Dynamically allocate memory at runtime on the heap for an object/array**. Also calls the **constructor** to initialize the object and returns a typed pointer to the allocated memory

- This is very similar to `malloc` in C (both allocate memory on the heap), but `malloc` does not call a constructor. Also, `malloc` returns a void pointer (`void*`) and needs to be casted to the appropriate pointer type. Therefore in modern C++, `new` is highly recommended.

`delete` - **Frees memory allocated by** `new`. Also calls the **destructor** to clean up the object. Must match the type of allocation (`delete` for single objects and `delete[]` for arrays)

- Ensure that the pointer points to a valid object (has a valid address of memory!). Otherwise, unpredictable behavior can happen!
- Similar to `free` in C, but since there are no destructors, `free` won't call the destructor if there is any
- **Ends the lifetime of the object the pointer points to!**

Note that when you use `delete`, even though the object is freed from memory, its pointer now points to **invalid memory**. This is a **dangling pointer**. Therefore, it is best practice to set the pointer to `nullptr` (null) after using `delete`, as suggested in the code snippet below, which avoids the dangling pointer.

```
int* xPtr = new int(5);
delete xPtr;
xPtr = nullptr;
```

Comparing the two sides of **Figure A.1.16**, the `int` on the heap is gone after the `delete`, and `xPtr` now safely holds `nullptr`

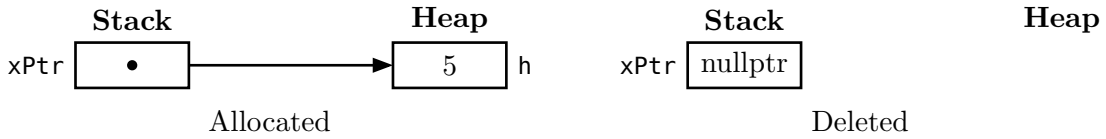


Figure A.1.16: After `new int(5)` (left) and after `delete xPtr; xPtr = nullptr;` (right): the pointer lives on the stack, the `int` on the heap

While static and automatic memory management free variables from memory its lifetime ends or out of scope respectively, memory must be explicitly freed with `delete` when dynamically allocated.

> In other words, when creating memory calls with `new` they **must** be freed with `delete`.

Otherwise, a **memory leak** happens. **Memory leaks** occur when a program allocates memory via `new`, but never frees it via `delete`. This causes unused memory, which can eventually exhaust available memory if it happens repeatedly. In **Figure A.1.17**, for example, `p` gets pointed at a new `int`, so nothing points to the old block `h1` anymore

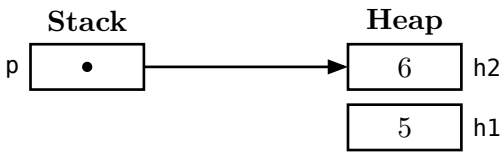


Figure A.1.17: Example: after `int* p = new int(5); p = new int(6);` nothing points to `h1`, so it can never be deleted

A.1.8.2 Destructors

A **destructor** is another special kind of class member function that runs when an object of that class is **destroyed**, using `delete`. While constructors initialize instances of a class, destructors help clean them up

When objects go out of scope or a dynamically allocated object is explicitly deleted using `delete`, the class destructor (if it exists) is automatically called to do any necessary clean up of the object before it is freed from memory

Like constructors, destructors also have specific naming rules

1. Must have **same name** as the class (like a constructor)
2. Cannot have a **return type** (like a constructor)
3. Cannot take **any arguments**

Suppose `Foo` is a class with a default constructor (`Foo() = default;`). The destructor for `Foo` is like the one below

```
~Foo() {

}
```

A.1.8.3 RAII

Resource Acquisition Is Initialization, or RAII, is a C++ programming design pattern that explains how to manage resources safely in a class through a constructor and a destructor. RAII is used heavily in the C++ STL (standard template library) for efficient resource management in objects.

> **RAII** - You acquire a resource (memory, file, lock, etc.) via an object's **constructor** and you automatically release that resource via the object's **destructor**

In other words, an object **acquires** a resource in its **constructor** and **releases** it in its **destructor**

These resources can include anything the class owns or manages
- A file handle
- A pointer to dynamic memory
- A mutex lock
- A network connection

RAII is a basic technique that explains best practices towards writing a class by **tying an object's resources to its lifetime**

- An object acquires a resource in its constructor, and releases it via its destructor when its lifetime ends

A.1.8.4 Heap

The heap, also known as the **free store**, **tracks memory used for dynamic memory allocation**

Recall that `new` allocates memory and returns a pointer that points to that allocated memory. In contrast, `delete` frees the allocated memory towards the operating system to decide what to do with that memory next.

This dynamic memory allocation behavior is all managed via the heap. This means that `new` will allocate memory onto the heap and `delete` will return the allocated memory from the heap, where the operating system can decide what to do with this memory later.

Heap Advantages and Disadvantages

- Allocating memory on the heap is slow
- Allocated memory stays allocated until the programmer deallocates the memory themselves or the application ends (OS cleans it up)
- Even though an OS will clean up dynamically allocated memory at the end of the program, **memory leaks** are still bad practice and you should always `delete` dynamically allocated memory
- This is a problem for bigger programs that handle lots of data, where they can possibly slow down, crash, or cause system instability
- Dynamically allocated memory must be accessed through a pointer. Therefore, dereferencing a pointer is slower than accessing a variable directly
- Because the heap is a big pool of memory, large arrays, structures, or classes can be allocated here

A.1.8.5 Stack

The **stack**, formally referred to as the **call stack**, keeps track of all active functions (ones that have been called and not terminated) from the start of the program to the current point of execution. It also handles allocation of all function parameters and local variables

It stores

- **Local Variables** - function parameters and variables declared inside functions
- **Function call information** - where to return after a function finishes

When a function is called, a stack frame is created on top of the call stack that holds the function's local data. When the function ends, its stack frame is removed, and the memory is automatically reclaimed

Key Details

- The stack is **fast** because elements are accessed in a **LIFO** (Last In First Out) format. This means that both removal and insertion is $O(1)$
- Variables in the stack have automatic storage duration and only exist when the function is running
- Limited size (few MBs), so you can run out of stack space if you create too many local variables or deep recursion

All in all, the **stack** is memory for **local variables** that are automatically managed and exist only when a function is **running**. The **heap** is slower memory for **dynamically allocated variables** (`new`) that must be managed **manually** with `delete`

- **Stack data**: local variables inside functions, function parameters, pointer variables themselves (not what they point to)
  - Used for short-lived, simple variables
- **Heap data**: Any variable dynamically allocated via `new` or `malloc`
  - Use when a variable is needed to outlive the function or for large or dynamically-sized data

Looking at **Figure A.1.18**, notice that the pointer `p` itself is stack data, and only the `int` it points to is on the heap



Figure A.1.18: Example: `int a = 3; int* p = new int(4);` puts `a` and the pointer `p` on the stack and the `int` it points to on the heap

A.1.8.6 Passing Arguments

There are three ways to pass arguments into a function

1. **By Value** - A copy of the argument is made; changes inside the function do **not** affect the original

```
void foo(int x) {
    x = 10;
}
```

In the example above, assigning `x = 10` only changes the local variable in the function. It does not affect the original `x` parameter being passed into the function. Instead, it creates a copy of `x` and changes that, which is why `n` still holds 5 in **Figure A.1.19**.



Figure A.1.19: Example: after `foo(n)` with `n = 5`, only the copy `x` holds 10

2. **By Reference** - The function receives an alias to the original argument; changes affect the caller's variable

```
void foo(int& x) {
    x = 10;
}
```

In the example above, assigning `x = 10` changes the original `x` parameter to `10`. Notice in **Figure A.1.20** that `n` and `x` share a single box



Figure A.1.20: Example: inside `foo(n)`, `x` is another name for the caller's `n`, so `n` becomes 10

3. **By Pointer** - The function receives a pointer to the original variable, where you must dereference to modify

```
void foo(int* x) {
    *x = 10;
}
```

In the example above, the function receives a pointer to the original parameter of `x`, which must be dereferenced with `*` to modify the original parameter (see **Figure A.1.21**, where `x` points at `n`)



Figure A.1.21: Example: inside `foo(&n)`, `x` holds the address of `n`, so `*x = 10` sets `n` to 10

A.1.8.7 Returning Values

Same three ways variables are passed into a function, are the same three ways variables are returned in a function

1. **By Value** - Returns a copy of the object

```
int foo() {
    int x = 5;
    return x;
}
```

In the example above, `x` is returned as a copy of the original `x` variable

2. **By Reference** - Returns a reference (alias) to an existing variable, but doesn't refer to a local variable that goes out of scope!

```
int& foo(int& x) {
    return x;
}
```

In the example above, `x` is simply returned as a reference to an existing variable. Be careful! Referencing a local variable that goes out of scope can lead to undefined behaviors!

3. **By Pointer** - Returns a pointer to an object (can be null). The caller can modify the object via the pointer

```
int* foo(int* x) {
    return x;
}
```

In the example above, `x` is simply returned as a pointer to an existing variable. Be careful! Like references, pointing to a local variable that goes out of scope can lead to undefined behaviors!

A.1.9 Copy Semantics

Copy Semantics refers to how objects are copied when they are passed as arguments to functions, returned from functions, or assigned to other objects. Copy semantics dictate the rules for creating new objects that are copies of existing ones. Simply speaking, they **define** how objects are copied

A **copy constructor** is used to initialize an object with an existing object of the same type. After the copy constructor executes, the newly created object becomes a copy of the object passed in the initializer

- **Implicit Copy Constructor** - If a copy constructor is not provided for a class, the compiler will create a public **implicit copy constructor** for you

An explicit copy constructor can also be created, by taking in an referenced object type of that class

```
class Foo {
    public:
    Foo() = default;
    Foo(const Foo& foo) {

    }
};
```

where `Foo a; Foo b = a;` implicitly calls the copy constructor on `b`. Declaring any constructor, including a copy constructor, stops the compiler from generating the implicit default constructor, which is why `Foo() = default;` is needed for `Foo a;` to compile.

A.1.9.1 Shallow and Deep Copies

Shallow and **deep** copies define how objects are copied!

A **shallow copy** occurs where an object's **pointers** are copied, but **not the data they point to**

- Both the object and the copied object share the same underlying data, which means that changing one pointer affects both objects
- If one object deletes or modifies the pointer, the other may be left with invalid data, causing crashes and segmentation faults resulting in undefined behavior or unexpected data, which may contain varied data than what was ideal

A **deep copy** is a completely independent copy of all data, including an object's pointers and the memory they point to

- Unlike a shallow copy, the object and the copied object **do not share the same data**
- Instead, the copied object **allocates new memory** and copies the actual data, so they're fully independent

You can spot the difference in **Figure A.1.22**: with a shallow copy, `a` and `b` point to the same 42, but with a deep copy each one gets its own 42

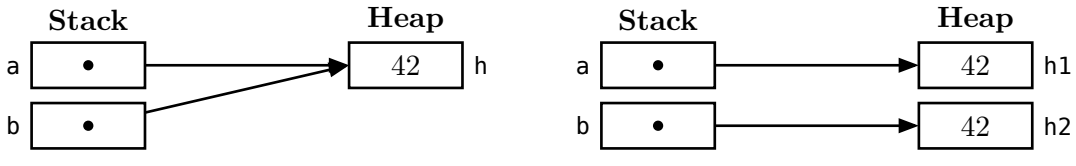


Figure A.1.22: Example: `a` and `b` are the `data` pointers of two objects after `Foo b = a;`

A.1.9.2 Copy Assignment Operator

The Copy Assignment Operator can be overloaded to define copy semantics **after** objects have been constructed. An example, for a `Foo` that owns a dynamically allocated `int* data` and an `int size`, looks like such code below

However, multiple checks that are typically implemented include

- Self assignment check (`this != &other`) to ensure that an object is not copying itself!
- Frees or cleans up resources that the current object owns
- Deep copies data from the source object
- Returns `*this`, a reference to the current object to allow assignment chaining (recall that `this` is an implicit pointer to the current object in a class)

```
Foo& operator=(const Foo& other) {
    // Self assignment check
    if (this != &other) {
        // Free the resource this object currently owns
        delete data;

        // Deep copy the data from the other object
        data = other.data ? new int(*other.data) : nullptr;
        size = other.size;
    }
    return *this;
}
```

Typically, you need to overload it when

- Your class owns dynamically allocated memory or other resources.
- You want to **avoid shallow copies** that lead to double frees or dangling pointers
- You want to implement **deep copying** semantics

A.1.9.3 Default and Delete on Copy constructors

Just like a regular constructor, we can have the compiler explicitly generate a `default` copy constructor, `Foo(const Foo& foo) = default`

- Note that this will perform a **shallow copy**

`= delete` on a copy constructor will prevent objects of a certain class from being copied

A.1.10 Move Semantics

Move semantics allow the resources owned by one object to be **moved** or **transferred** to another object **without copying** the underlying data. They simply define how objects are moved!

Deep copies can be expensive, especially when objects manage dynamic memory or other expensive resources. Therefore, move semantics come in handy and avoid a deep copy when necessary

- Simply use move semantics when you want to avoid copying expensive resources and instead transfer ownership to another object!

They are defined like copy semantics, including a **move constructor** and **move assignment operator**

A.1.10.1 rvalue references

An rvalue is a temporary object or value that **does not** have a persistent memory address. This is the opposite of an lvalue, which has a persistent memory address

`int x = 1`

In the code snippet above, `x` is an lvalue (left value) because it is stored in memory with a persistent memory address

- Recall that a pointer will store this memory address and an lvalue reference - sometimes simply called a reference - is a reference or alias to an existing value in memory

In contrast, `1` is an rvalue. It is a temporary value used to initialize `x`

rvalue references (right-value references) are references that bind to **rvalues**, temporary objects or literals that are about to expire. They enable **move semantics** by allowing the program to **take ownership** of resources from these temporary objects, avoiding expensive copies

Understanding these make understanding move semantics much more easier because ultimately what happens is the rvalue of one object is being moved to another object via a rvalue reference of the other object

A.1.10.2 Move Constructor

A **move constructor** defines how objects should be moved once initialized

The code below defines a move constructor. Suppose class `Foo` contains two variables `data` and `size` and we want to move it to an rvalue reference called `other`

- `data` is a dynamically allocated int and `size` is an int
- An example of moving the data from `other` to the current object `this` is shown below by using an object's rvalue reference, `other`

```
Foo(Foo&& other) {
    // Move the data of the other object to this object
    this->data = other.data;
    this->size = other.size;

    // Set data pointer to null and size to 0
```

```
    other.data = nullptr;
    other.size = 0;
}
```

A.1.10.3 Move Assignment Operator

The **move assignment operator** defines how objects should be moved after being initialized

The code below defines a move assignment operator, but typical operations that are implemented include

- **Self-assignment check** (`this != &other`) - ensures that an object is not moving into itself
- Free or clean up any resources that the current object owns (like deleting dynamically allocated memory)
- **Stealing resources from the source** - transfer the ownership of the resources from the `other` rvalue object to the current object by copying pointers or handles
- **Leave the source in a valid, but empty state** - Set the `other` object pointers or resource handles to `nullptr` or equivalent so that destructor won't free the resources again (double free - free what has already been freed)!
- Return `*this`, a reference to the current object!

```
Foo& operator=(Foo&& other) {
    // Self assignment check
    if (this != &other) {
        // Free the resource this object currently owns
        delete data;

        // Steal the resources from the other object
        data = other.data;
        size = other.size;

        // Leave the other object in a valid, but empty state
        other.data = nullptr;
        other.size = 0;
    }
    return *this;
}
```

`std::move()` will trigger move semantics! For example, in the code below

```
Foo a;
Foo b = std::move(a);
```

`b` is move constructed from `a`, effectively moving `a`'s resources into `b`. Looking at **Figure A.1.23**, the 42 on the heap never gets copied: `b` just takes over the pointer, and `a` is left with `nullptr`

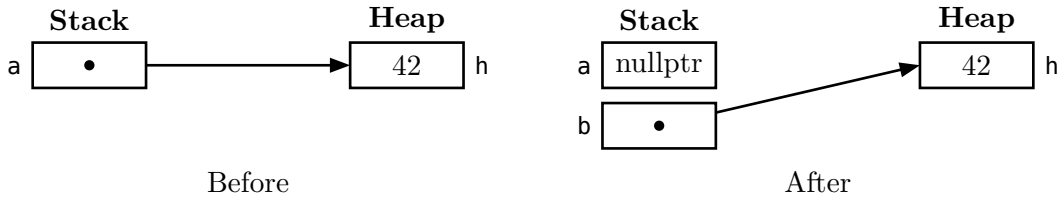


Figure A.1.23: Example: `Foo b = std::move(a);` hands the heap block of `a` to `b` without copying it and leaves `a.data` as `nullptr`

Simply speaking, move semantics are triggered when an object is treated as an rvalue (like the example above), either because it's a temporary (like `b = Foo();`, which move assigns into an existing `b`) or explicitly calling `std::move` (which converts an `lvalue` into an `rvalue`)

A.1.11 Rule of Zero/Three/Five

When a class manages resources (**dynamic memory**, file handles, etc.), it **must** define how it is **copied**, **moved**, and **destroyed** in order to avoid issues like memory leaks or double deletes

The rule of zero/three/five are rules widely adopted in C++ related to memory/resource management and ownership, specifically about when and how you should define or avoid defining special member functions in a class

Rule of Zero - If your class doesn't manage resources directly, **DO NOT** define the **big five**

- Destructor
- Copy Constructor
- Copy Assignment
- Move Constructor
- Move Assignment

Rule of Three - If you define one of a **destructor**, **copy constructor**, or **copy assignment operator**, **you should define ALL three of them!**

- This is because handling one implies your class is managing a resource that needs careful copying and cleanup

Rule of Five - Applies to modern C++, where if you plan on managing resources with move semantics, define **all 5 of the following** (this is the opposite of the **rule of zero**)

- Destructor
- Copy Constructor
- Copy Assignment
- Move Constructor
- Move Assignment

Use this rule when you need to **avoid deep copying** and allow your class to be efficiently moved

A.2 Midterm 1: Lists

Midterm 1 covers modules 1-7, which discuss mostly about Lists. The topics covered are listed below

A.2.1 Topics Covered

- Lists
  - ADT
  - Linked
  - Array

Use these assignments to reference when studying as well

- `lab_debug`
- `mp_stickers`

A.2.2 Lists

ADTs, or Abstract Data Types, are templates/descriptions used to explain how a data structure works and how any user should use it.

In a **`LinkedList`**, each node is stored independently in memory and contains
1. Data - the actual value stored in the node
2. Pointer - a pointer to the next node in the list (or previous in a doubly linked list)

From a visual standpoint a `LinkedList` looks like the one in **Figure A.2.1**

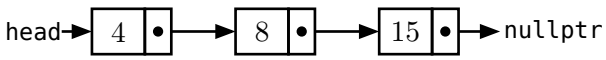


Figure A.2.1: A singly linked list: each node holds data and a pointer to the next node

Each node holds data and then a pointer to the next node!

The table below shows the core functionalities of a `LinkedList` and their runtimes

Operation	**Singly Linked List**
Insert/Remove at front	`O(1)`
Insert/Remove at back	`O(1)` for insert, `O(n)` for remove
Insert/Remove given a pointer	`O(1)`
Insert/Remove at a given index	`O(n)`
Find a value in a list	`O(n)`
Find a value in a sorted list	`O(n)`

- Inserting at the front of a `LinkedList` is `O(1)`. This is because we only have to do 3 steps. Create a new node with its given data, point its next to the original head, set the new node as `head`

You can see these steps in **Figure A.2.2**, where the new node 2 (shaded) points to the old head 4

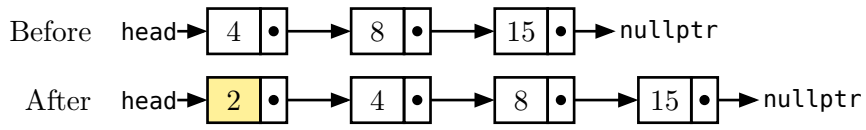


Figure A.2.2: Example: inserting 2 at the front only rewires the new node's `next` and `head`

- Removing at the front of a `LinkedList` is also `O(1)`. This is because to remove the front node, we only need to do 3 constant operations
  1. Create a temporary node pointing to the head
  2. set `head` to its next pointer (`head = head->next`)
  3. Free temp from memory (`delete temp`)

  Notice in **Figure A.2.3** how `head` moves on to 8 while `temp` hangs on to 4 so it can be deleted

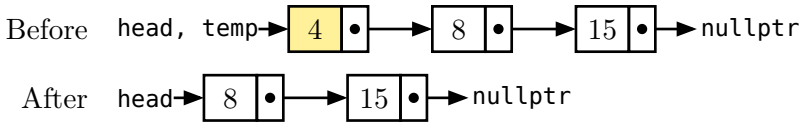


Figure A.2.3: Example: `temp` saves the node holding 4, `head` moves to 8, then `delete temp` frees 4

- Inserting at the back of a `LinkedList` is also `O(1)`, but this assumes that there is a `tail` pointer
  - Otherwise it would be `O(n)` because we would need to traverse `n` times through the entire list from `head` to find the last node
- Removing from the back of a `LinkedList` is `O(n)` because we have to traverse through the list `n` times in order to find the second to last node and set its `next` pointer to `nullptr`

  In **Figure A.2.4**, the walk from `head` stops at the second to last node, the shaded 8

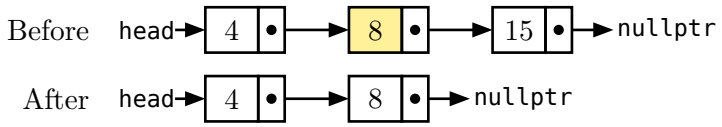


Figure A.2.4: Example: removing 15 means walking from `head` to 8, setting its `next` to `nullptr`, then deleting 15